



GeoLibre User Guide

A feature-by-feature tour of the workspace, menus, panels, and tools

Contents

Interface Overview	9
The top toolbar	9
Command palette and keyboard shortcuts	11
The side panels	13
The map	14
The status bar	14
Notifications	16
Theme	17
Desktop and browser	17
Rendering Engines	18
Switch the primary map	18
Mix engines in split panes	18
Credentials	19
Feature compatibility	19
Projects	20
New	20
Open	21
Save and Save As	22
Project history and crash recovery	22
Importing a QGIS project	23
Importing an ArcGIS Pro project	23
Templates	24
Share	24
Export as HTML	25
Layer styles	26
Collaborate	26
Offline basemap	26
Print	26
Story maps	27
The project format	27
Adding Data	28
Files	29
Web services	31
Cloud formats	32
3D layers	33
Databases	33
Drag and drop	34
The Browser panel	34
Basemaps	36

More data sources	37
Supported Data Formats	38
Vector and tabular files	38
Raster, tile archives, and multidimensional arrays	39
Web services and catalogs	39
3D, time-dependent data, and media	41
Databases and spatial query sources	41
Project formats	42
WCS raster subsets	42
Chrome Extension	44
Install the extension	44
Open datasets from a webpage	45
Use a different GeoLibre instance	45
Access and privacy	46
Managing Layers	47
Layer order and visibility	47
Hover tooltips	47
Blend modes	47
Per-layer actions	49
Layer groups	52
Refreshing live layers	53
DuckDB layers	53
The basemap	53
Styling Layers	55
Vector styling	55
Raster styling	60
Legends and colorbars	61
Attribute Table	62
The toolbar	62
Reading and navigating	63
Selecting features	63
Managing fields	64
Calculated fields	64
Joins	65
Attribute forms	66
Quick filters	67
Column explorer	67
Field statistics	68
Charts	68
Editing values	69
DuckDB layers	70
Exporting	70

Map Controls & Tools	71
Map controls	71
Component tools	72
Measuring distance, area, and heading	73
Right-click quick actions	74
Camera, overlay, and recording tools	76
Annotations and the Elements panel	77
Review comments	77
GPS tracking	78
Map navigation basics	78
GPS Tracking & Field Collection	79
GPS Tracking	79
Field Collection	81
Working offline	82
Print Layout	83
Title	83
Page	83
Map frame, scale, and extent	84
Map elements	84
Atlas (map series)	85
Exporting	86
Processing Tools	88
Two toolboxes in one menu	88
GeoLibre Toolbox	90
Whitebox Toolbox	99
Model Builder	102
Dashboard	103
Jupyter Notebook	105
Planetary Computer and Earth Engine	105
The Python sidecar	105
Model Builder	106
The title bar	106
Building a model	107
Running a model	108
Saving and sharing models	108
Python export	109
Building a model with the AI Assistant	109
AI Segmentation	110
How it works	110
Setup	110
Using it	111
Notes & limitations	112

Interactive SamGeo plugin	112
API (advanced)	113
Point Cloud Annotation	114
Start a session	114
Select and assign	114
Custom classes	115
Instances	115
Polygons to points	115
Pre-labelling with Whitebox	115
3D boxes (cuboids)	116
3D vectors	117
Saving labels with the project	117
Export	117
Limitations	118
SQL Workspace	119
Querying loaded layers	119
Reading files and URLs	119
Cloud URLs (s3://, gs://, az://)	120
Choosing a SQL engine	120
Sample queries and history	121
Using the results	121
Python Console	122
First run	122
Script editor	123
Driving the map	123
Async operations	124
Loading more packages	124
geolibre API reference	124
Notes & limitations	125
AI Assistant	127
Setup: choose an AI provider	127
Using it	131
Fast path for simple commands (optional)	132
Finding the right Whitebox tool (optional)	134
Voice commands	134
What it can do	135
Sample prompts	136
Safety and privacy	137
Limitations	138
Not the same as the agent skill	138
Data Integrations	139
Cloud catalogs	139

Self-hosted catalogs	140
Web Services	140
Imagery and street-level	141
Time series and comparison	141
AI analysis	141
Geocoding	141
Cloud Storage (S3)	144
Where S3 URLs work	144
S3 connections	144
Default S3 Browser location	146
S3 Browser	146
Where credentials are stored	147
CORS for private buckets (web app)	147
How it works	148
Limitations	148
Plugins & Marketplace	149
The Plugins menu	149
Built-in plugin reference	150
Open a plugin from a link	155
Manage Plugins	155
Where plugins come from	157
Writing your own plugin	157
USGS NLDI	157
Web Services	158
How the panels behave	158
At a glance	159
FEMA NFHL	160
NASA Earthdata	161
US EPA EnviroAtlas	162
USGS National Map	162
USGS NLDI	163
USGS 3DEP	163
USGS LiDAR	163
Vantor Open Data	163
Planet Open Data	164
Portolan	164
Earthdata GIS	164
OpenAerialMap	164
OSM Downloader	165
IGN LiDAR HD	165
ArcGIS Portal	165
ArcGIS Hub	166
Tennessee GIS	167

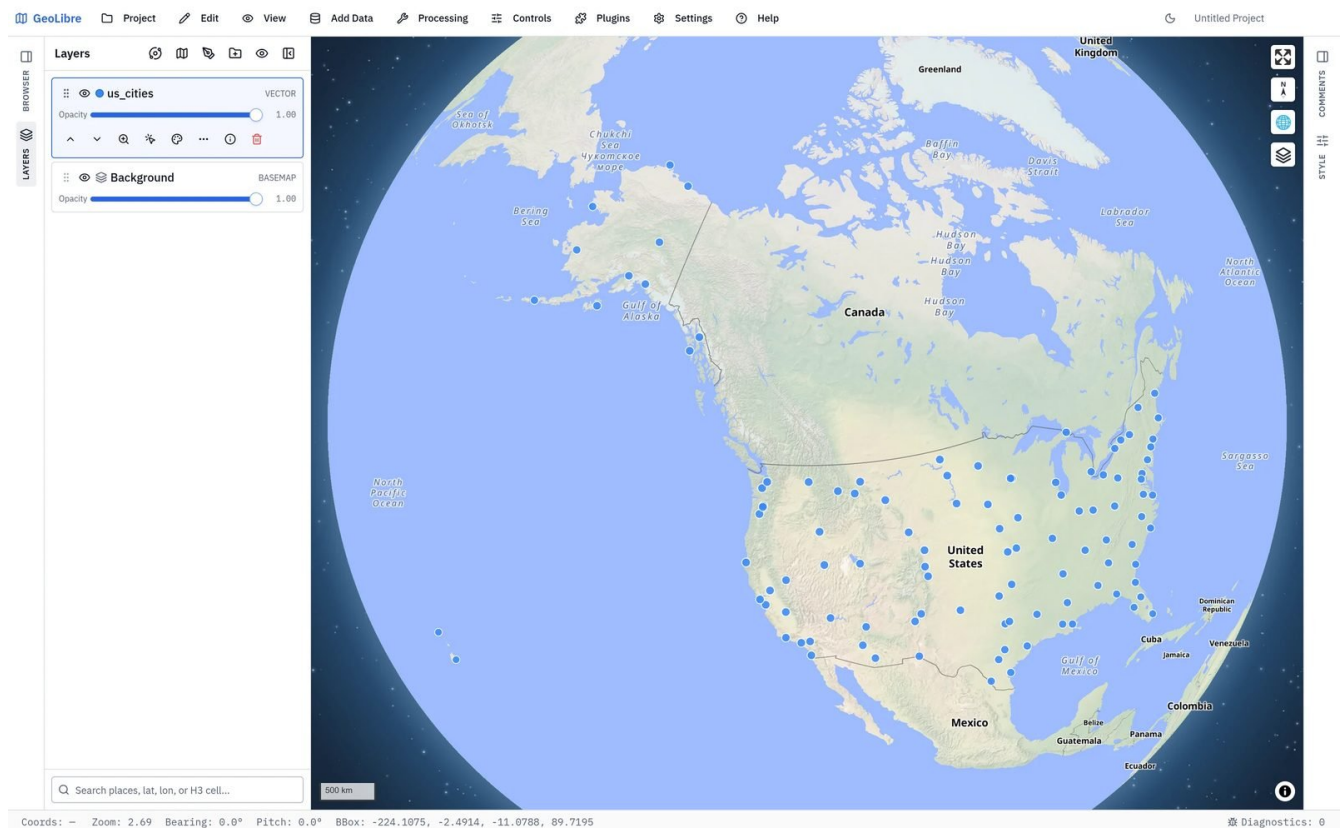
US Federal GIS	167
US State GIS	168
US Local GIS	168
Socrata.....	169
CKAN	169
STAC Catalogs.....	169
Source Cooperative	170
S3 Browser	170
Natural Earth	171
Hugging Face	171
Satellite Embeddings.....	171
Fields of the World.....	173
Sentinel-2 Explorer.....	173
Ocean Data Platform.....	175
GeoLens.....	175
Related pages.....	176
Settings & Preferences	177
Language	178
Map Preferences	178
Layout	180
Appearance.....	180
Interface	180
Geocoding.....	180
AI Providers.....	181
Environment Variables	181
Cloud Storage.....	182
Project name and file	182
Startup.....	182
Updates	183
Style Manager.....	184
Manage Plugins	184
Story Maps	185
Open the builder	185
Story settings	186
Chapters	186
Import and export chapters.....	186
Present.....	187
Handout (PDF)	188
Export to HTML.....	188
Embedding & Sharing	189
The live viewer	189
URL parameters	190
Waiting for a screenshot.....	192

Embedding in a page	193
Open remote data	194
Open a STAC catalog	197
Talking to the map at runtime	198
What works in an embed	203
Switching renderers.....	203
Troubleshooting	204
Stale browser cache.....	204
The Diagnostics dialog	204
Reporting a problem.....	205
Common errors	206
The map is blank or renders incorrectly.....	207
Plugins do not appear to activate	208
The Processing menu shows the same category twice	208
Desktop-only features in the browser	208
Self-hosted deployments	209

<https://geolibre.app/user-guide/interface/>

Interface Overview

GeoLibre opens to a single workspace that is the same on desktop and in the browser. This page is a tour of that workspace so the rest of the User Guide can refer to its parts by name.



The top toolbar

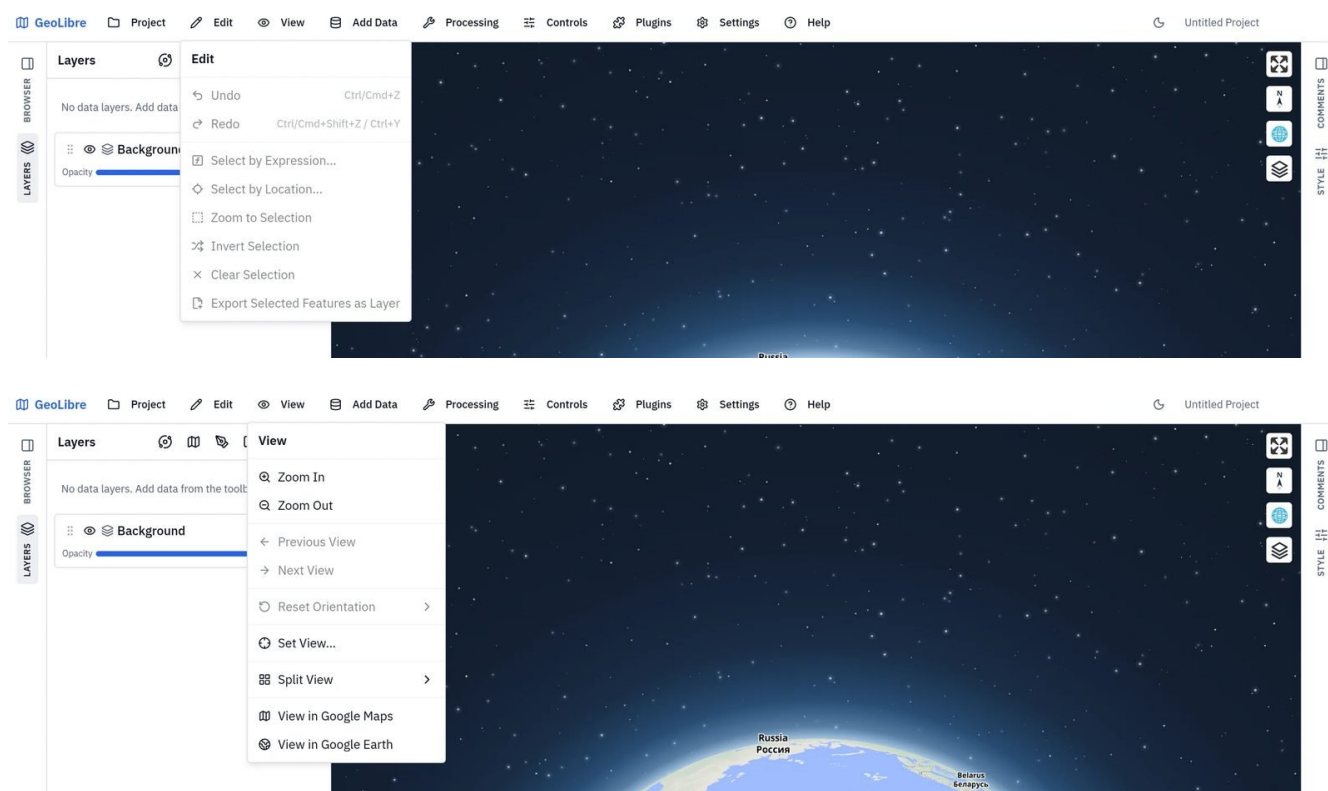
The toolbar across the top of the window groups every action into nine menus:

Menu	What it does
Project	Create, open, save, share, import, and print projects. See Projects .
Edit	Undo and redo, and the feature-selection tools: Select by Expression, Select by Location, zoom to / invert / clear the selection, and Export Selected Features as Layer.
View	Choose one of the four rendering engines , zoom in and out, step through viewport history, reset the camera orientation, set an exact view, create a split view, preview the map under color vision deficiency, or open the location in Google Maps / Google Earth.
Add Data	Add layers from files, web services, cloud formats, 3D data, and databases. See Adding Data .

Menu	What it does
Processing	Run vector, raster, conversion, Whitebox, and SQL tools, plus the AI Assistant . The menu holds two separate toolboxes , so some category names appear twice. See Processing Tools and SQL Workspace .
Controls	Toggle map controls and component panels (Measure, Bookmark, Minimap, and more). See Map Controls & Tools .
Plugins	Activate built-in plugins and set their on-map position. See Plugins & Marketplace .
Settings	Language, map preferences, layout, appearance, interface profile, geocoding, AI providers, environment variables, cloud storage, updates, startup project, Manage Plugins, and the Style Manager. See Settings & Preferences .
Help	The command palette, keyboard shortcuts, Simplify Interface... , links to the website and GitHub repository, diagnostics, feedback, update checks, and the About dialog.

On the right side of the toolbar are the light/dark theme toggle and the editable project name.

The **Edit** and **View** menus are the two most easily missed, because their contents live nowhere else:



View → Color vision preview shows the map as it looks with protanopia, deuteranopia, tritanopia, or achromatopsia, so you can check that a palette still reads. It filters only the map (every renderer and split-view pane, plus the on-map legend), not the panels and menus, and shows a badge on the map while it is on; click the badge's **x** or pick **Off** to end it. The preview lasts for the session only and is not saved in the project. Exports are not affected: Print Layout, map image export, and Record Video capture the map's real colors. Only a screenshot of the window shows the simulation, which is why the badge is there.

Toolbar labels

On narrow windows the toolbar collapses to icon-only buttons. You can also force icon-only buttons from **Settings** → **Layout**, or with the `toolbar=icons` URL parameter. See [Embedding & Sharing](#).

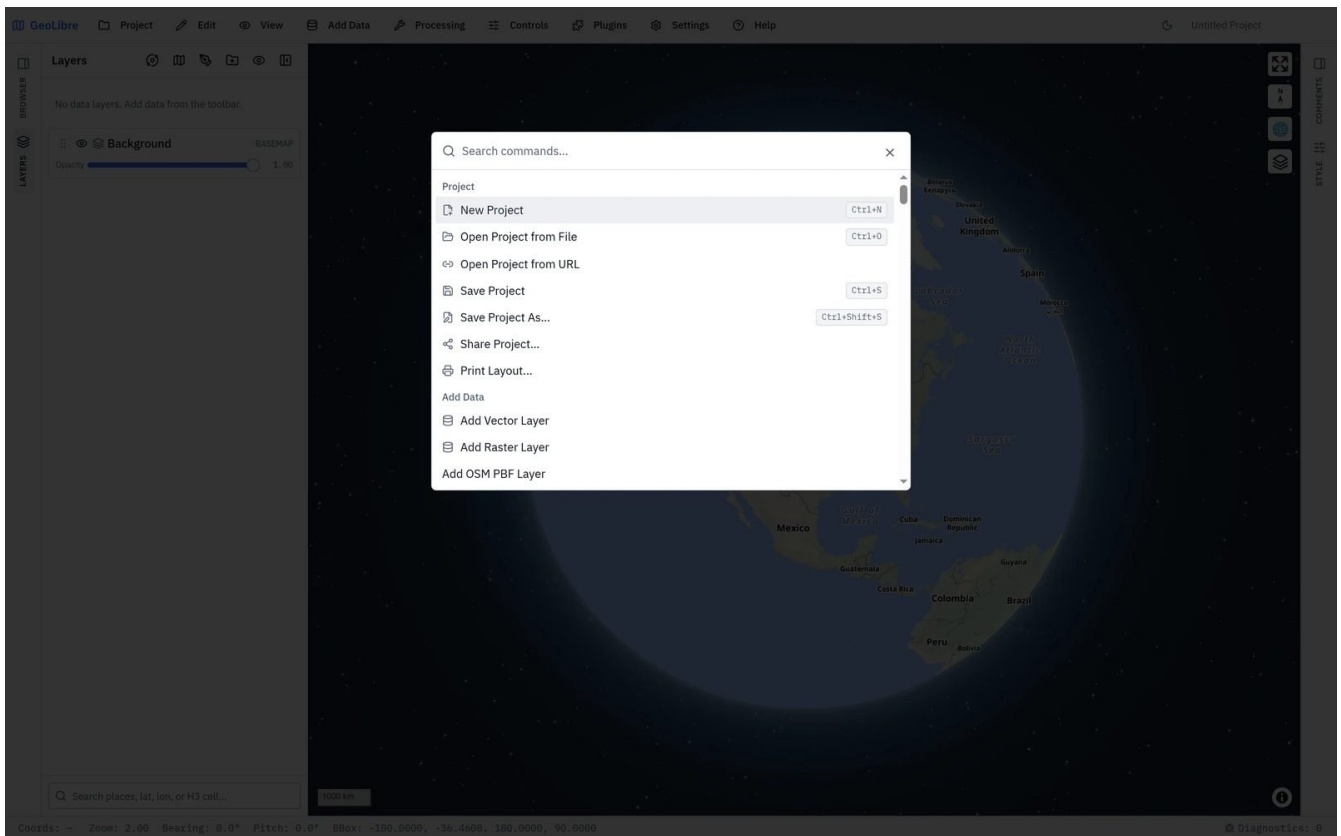
Simplify the interface

Help → **Simplify Interface...** (also in the command palette) opens **Settings** → **Interface**, where an **Experience level** of **Beginner**, **Intermediate**, or **Advanced** trims the Add Data sources, plugins, and menu items to match, and the **Data sources**, **Plugins**, and **Menus** checklists hide individual items (which switches the level to **Custom**). Nothing is removed: switch back to **Advanced**, or re-tick an item, to bring it back. The command palette follows the same choice. The Help entry itself can never be hidden, so there is always a way back; it is missing only where the deployment does not offer the Settings dialog, and an administrator can lock the profile. See [UI Profiles](#).

Command palette and keyboard shortcuts

Every menu and toolbar action is also reachable from the keyboard, so you don't have to hunt through nested menus.

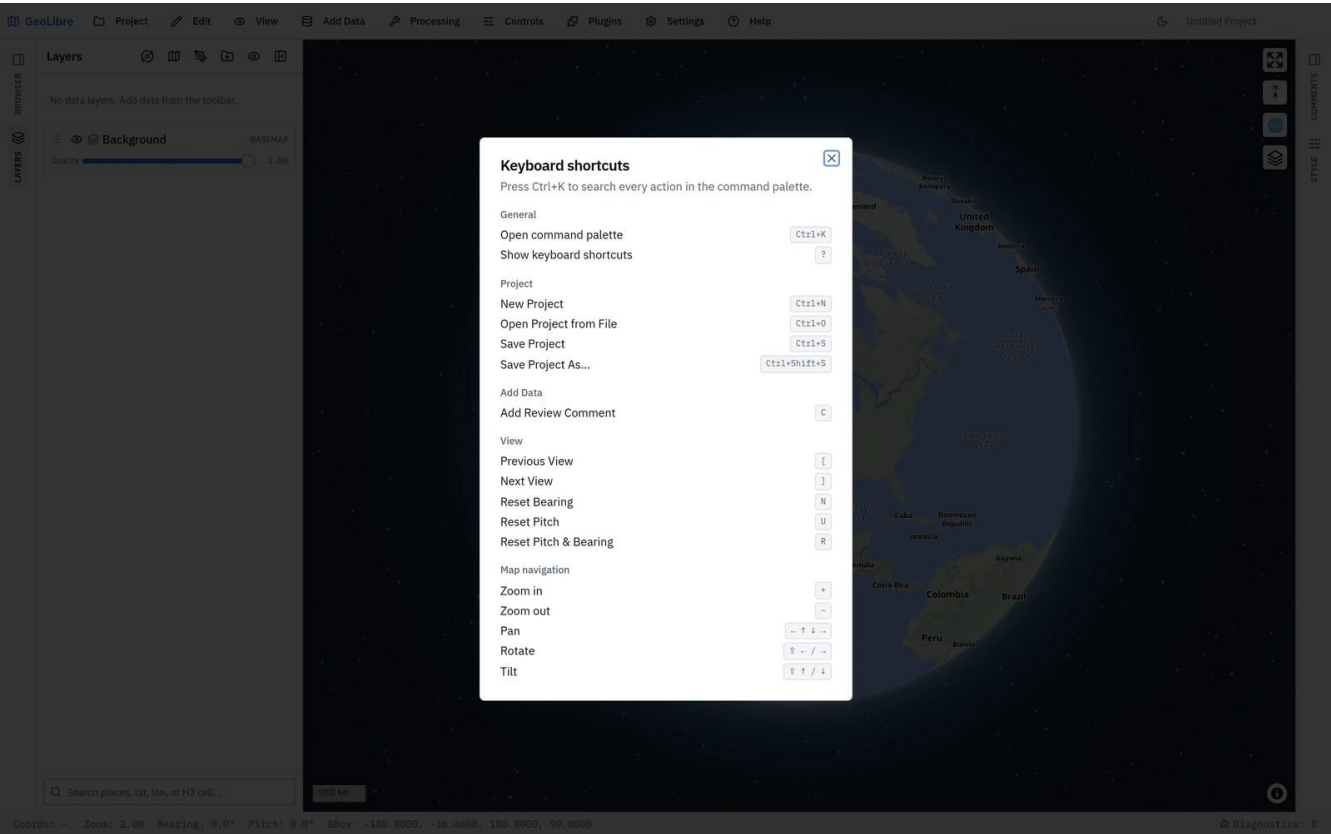
- **Command palette** — press `Ctrl / Cmd + K` (or **Help** → **Command Palette**) to open a searchable list of actions: Add Data sources, Processing tools, Controls, Plugins, and more. Type to filter, move the highlight with the arrow keys, and press `Enter` to run the highlighted command.
- **Individual tools** — start typing and the palette also lists every processing tool on its own: the GeoLibre Toolbox vector, network, and statistics tools under **Tools**, and each Whitebox tool under **Whitebox Toolbox**. Choosing one opens the processing dialog with that tool already selected, so `Ctrl / Cmd + K`, "slope", `Enter` reaches Whitebox's Slope without opening a menu. Tool names stay searchable in English in every language, an exact title match ranks first, and the palette hides whatever the active [interface profile](#) hides from the menus.
- **Keyboard shortcuts cheat sheet** — press `?` (or **Help** → **Keyboard Shortcuts**) to see the full list of global shortcuts.



The built-in global shortcuts are:

Shortcut	Action
Ctrl / Cmd + K	Open the command palette
?	Show the keyboard shortcuts
Ctrl / Cmd + N	New project
Ctrl / Cmd + O	Open project from file
Ctrl / Cmd + S	Save project
Ctrl / Cmd + Shift + S	Save project as...
Ctrl / Cmd + Z	Undo
Ctrl / Cmd + Shift + Z (or Ctrl + Y)	Redo
C	Toggle the review comment tool, then click the map to place the pin
N	Reset bearing (north up)
U	Reset pitch (top-down view)
R	Reset pitch and bearing
[	Previous view
]	Next view

The single-key view shortcuts mirror Google Earth Pro (**N** for north up, **U** for top-down, **R** to reset the view) and work anywhere in the app.



While the map has keyboard focus, MapLibre's own navigation keys are also available:

Key	Action
+ / -	Zoom in / out
Arrow keys	Pan
Shift + ← / →	Rotate
Shift + ↑ / ↓	Tilt

Shortcuts are ignored while you are typing in a text field, so they never interfere with search boxes or attribute editing. On macOS the `Cmd` key is used; on Windows and Linux the `Ctrl` key is used.

The side panels

Four dockable panels surround the map, plus the attribute table along the bottom. The left and right edges carry a vertical rail of tabs — **Browser** and **Layers** on the left, **Comments** and **Style** on the right — and clicking a tab expands that panel.

Panel	Where	What it holds
Layers	Left	The layer stack, including the basemap. Toggle visibility, change opacity, reorder layers, zoom to a layer, identify features, and open per-layer actions. See Managing Layers .

Panel	Where	What it holds
Browser	Left	A QGIS-style Data Source Manager: saved map services, database connections, recent items, and your personal My Data layer library, all addable without going through a menu. See Adding Data .
Style	Right	The styling controls for the selected layer, including data-driven styling for vector layers and image adjustments for rasters. See Styling Layers .
Comments	Right	Anchored review notes and their threads. See Review comments .
Attribute table	Bottom	The attributes of the selected vector or DuckDB layer, with its own explorer, statistics, chart, and export tools. Expand it from the status bar. See Attribute Table .

The Browser, Layers, Style, Comments, and Attribute panels can each be shown or hidden from **Settings** → **Layout**, and panels auto-hide on small screens. Resize the Layers and Style panels by dragging their inner edge, and the attribute table by dragging its top edge.

The map

The map fills the center of the workspace. It uses MapLibre GL JS for vector and raster rendering, with deck.gl for point clouds, 3D tiles, and other advanced overlays. Pan by dragging, zoom with the scroll wheel or the on-map zoom buttons, **rotate** by holding the right mouse button and dragging, **tilt** by holding **Ctrl** / **Cmd** and dragging, and reset north with the compass button.

On-map controls such as zoom, globe, fullscreen, and the Layer Control appear in the corners. Which controls are shown is set from the [Controls menu](#).

The status bar

The status bar along the bottom reports the live state of the map, from left to right:

Readout	Description
Coords	The coordinate under the pointer. Click it to switch notation — see below.
Elev	The ground elevation under the pointer. Off by default; see Elevation readout .
GPS	The current fix, while GPS tracking is running.
Zoom	The map zoom level, to two decimals.
Eye alt	The camera's altitude above sea level — the same quantity Google Earth Pro calls <i>Eye alt</i> .
Bearing / Pitch	The camera rotation and tilt, in degrees.
BBox	The bounding box of the current view (hidden on narrow windows).

It also holds a **Diagnostics: N** button (also under **Help → Diagnostics**) that counts the errors and warnings captured this session and opens the [Diagnostics dialog](#). It turns red when there are errors and amber when there are only warnings. While autosave is paused for a very large project, an **Autosave paused** notice appears here too.

Eye alt is scaled to the active celestial body, so it stays correct on a Mars or Moon basemap rather than reporting an Earth-derived height, and it follows the **Scale bar units** preference (metres/kilometres, feet/miles, or nautical miles). See [Settings → Map Preferences](#).

Coordinate format

GeoLibre can report the pointer coordinate in seven notations:

Format	Example
Decimal degrees (default)	-83.92074, 35.96064
Degrees, minutes, seconds	35°57'38.3"N 83°55'14.66"W
Degrees, decimal minutes	35°57.6384'N 83°55.2444'W
UTM (zone, easting/northing)	17S 236594mE 3983527mN
MGRS (Military Grid Reference System)	17SKV3659483527
USNG (US National Grid)	17S KV 36594 83527
Projected (EPSG:code)	-9342014.04, 4295206.85 (EPSG:3857)

Decimal degrees are written longitude-first, matching GeoJSON and the rest of the app; DMS and DDM lead with latitude, the way those notations are conventionally written.

Click the coordinates in the status bar to cycle through them, or set the notation in **Settings → Map Preferences → Coordinate format**. The choice is saved with the project.

The UTM readout uses the same projection that draws the [Gridlines](#) UTM grid. Its zone follows the same rule as MGRS, including the Norway and Svalbard exceptions (Bergen reads as zone 32V, Longyearbyen as 33X), so the UTM and MGRS readouts always name the same zone. Outside the UTM latitude band (below 80°S or above 84°N) there is no valid UTM coordinate, and the readout falls back to decimal degrees.

MGRS and USNG are the same grid reference at 1 m precision; USNG writes it with spaces. In the polar areas (below 80°S or above 84°N) the grid switches to UPS, which GeoLibre does not support, and the readout falls back to decimal degrees.

The projected format reports x/y in any coordinate reference system with an EPSG code. Choose it in **Settings → Map Preferences → Coordinate format**, then type a code (for example 3857 for Web Mercator or 32618 for UTM zone 18N) or pick one with **Browse**. The code is saved with the project alongside the format, and the readout falls back to decimal degrees for a code GeoLibre does not know or a point the projection cannot place.

The place search at the foot of the Layers panel accepts the grid notations as well as lat/lon: type an MGRS or USNG reference (`18SUJ2337106519` , `18S UJ 23371 06519` , or a coarser `18SUJ23370651`) or a UTM coordinate (`18N 323394 4307395` , or the readout's own `18S 323394mE 4307395mN`) and pick **Go to** to fly there. The **Paste coordinates** box in **View → Set View** accepts the same grid references. A UTM letter is read as a latitude band, so `S` means band S (32°N–40°N) when the northing fits it and the southern hemisphere otherwise.

Elevation readout

Controls → Elevation turns on the **Elev** readout. It is **off by default**, and it resolves the height under the pointer from one of two sources:

- **From the map's own 3D terrain**, whenever a usable sample is available there. This is instant, tracks the cursor live, and sends nothing off your device.
- **From the public [Open-Meteo elevation API](#)** when terrain returns no value for that point — because 3D terrain is off, but also when it is on and the terrain has no sample to give. The lookup waits until the pointer has been still for half a second, caches results per roughly 11 m cell, and runs only on Earth, never on a planetary basemap.

Because that fallback sends the coordinates under your pointer to a third-party service, GeoLibre asks for consent the first time you enable the readout. **Declining is what guarantees the readout never reaches the network** — turning 3D terrain on makes the remote lookup rare, but does not by itself rule it out. Decline and the readout still works wherever terrain can answer. While a lookup is in flight the readout is blank rather than showing the previous point's height.

Reading elevation along a line

For a profile rather than a single point, use the Elevation Profile plugin, or the [Measure tool](#), which reports terrain-aware 3D distances.

Notifications

Messages about what just happened (a layer that failed to load, a file that was skipped, a plugin that could not start) appear as notifications stacked in the bottom corner of the window, above the status bar:

- Success and information messages close themselves after a few seconds, and warnings after a little longer. Hovering over or focusing a notification pauses the timer.
- Errors stay until you dismiss them with **×** or **Esc** , and are also recorded in the Diagnostics dialog.
- A message that repeats collapses into one notification with a counter such as **×**3, and at most four show at once.
- Error notifications carry a **Report issue** button. See [Reporting a problem](#).

Theme

Use the sun/moon button on the toolbar to switch between light and dark themes, and **Settings** → **Appearance** to pick an accent color. The theme follows your operating system preference by default, and you can set it for embeds with the `theme=dark` or `theme=light` URL parameter. See [Embedding & Sharing](#).

Desktop and browser

The same UI runs as an installed desktop app (built with Tauri) and as a web app in the browser. The browser build covers most workflows, but features that need direct access to the local filesystem (native file dialogs, local MBTiles and raster reads, reopening referenced local files, and the Python sidecar tools, unless a self-hosted web deployment proxies the sidecar) require the desktop app. In the browser, projects open through the browser's file picker and save through its save dialog or as a download. Each affected page notes these differences. See [Getting Started](#) for installation and [Downloads](#) for installers.

<https://geolibre.app/user-guide/rendering-engines/>

Rendering Engines

GeoLibre can draw the same project with four rendering engines. **MapLibre GL JS** is the default, but the primary map and every pane in a split layout can independently use MapLibre, Mapbox GL JS, CesiumJS, or the ArcGIS Maps SDK for JavaScript.

Engine	Best suited to	Credentials and loading
MapLibre GL JS	The broadest GeoLibre feature and plugin support; vector styles, raster data, terrain, and lightweight 3D content	No credential required; bundled with GeoLibre
Mapbox GL JS	Mapbox styles and services, Mapbox globe and terrain, and a Style-Spec-compatible alternative to MapLibre	A Mapbox access token is required; the engine is loaded only when selected
CesiumJS	A native 3D globe, terrain, 3D Tiles, CZML, Cesium ion assets, I3S, and globe-oriented analysis	Works without a token; a Cesium ion token adds World Terrain and ion imagery and assets (the web version bundles a demo token, desktop and mobile need your own); loaded only when selected
ArcGIS Maps SDK for JavaScript	Native ArcGIS services, Esri basemaps, and ArcGIS 2D maps or 3D scenes	Works without a key; an ArcGIS API key adds Esri basemap styles; SDK modules load from Esri's CDN when first selected

Switch the primary map

Choose **View → Rendering engine**, then select **MapLibre**, **Mapbox**, **ArcGIS**, or **Cesium**. GeoLibre replaces the active canvas; it does not keep the previous engine hidden and consuming graphics resources.

The project store is independent of any renderer, so the camera, layers, groups, visibility, opacity, and most styling remain in place when you switch. The selected engine is saved in the project and restored when it is reopened. Some layer types and tools are engine-specific; unsupported layers remain in the project and return when you switch to a compatible engine.

Mix engines in split panes

Open **View → Split View** to create a multi-map layout. Use the rendering-engine menu in a pane to choose any of the four engines for that pane. Panes can share the primary camera or keep independent views, which makes it possible to compare 2D and 3D renderings, basemaps, or engine behavior side by side. Each pane's engine choice and view state are saved with the project.

Credentials

Open **Settings** → **Environment Variables** to enter a Mapbox access token, a Cesium ion token, or an ArcGIS API key. These credentials are stored on the current device and are not written to the shared project file.

- MapLibre needs no credential.
- Mapbox requires an access token and is subject to Mapbox's terms and usage pricing.
- Cesium runs without a token, using the project basemap as imagery. A Cesium ion token enables Cesium World Terrain and ion-hosted imagery and assets. The web version bundles a demo token; the desktop and mobile apps need your own.
- ArcGIS works without a key by translating the project basemap. A key enables Esri basemap styles. The renderer itself still needs access to `js.arcgis.com` on its first load because the SDK is not bundled.

See [Getting Started](#) for credential setup. The [MapLibre](#), [Mapbox](#), [Cesium](#), and [ArcGIS](#) renderer references cover engine-specific behavior, supported layers, licensing, and deployment details.

Feature compatibility

GeoLibre exposes only the tools and plugins supported by the active engine. MapLibre currently has the widest compatibility. Mapbox shares much of the MapLibre Style Specification surface, while Cesium and ArcGIS use native layer and scene adapters. A plugin also declares which engines it supports, so an unsupported plugin is suspended rather than allowed to fail silently.

For automation, the renderer names are `maplibre`, `mapbox`, `cesium`, and `arcgis`. They are accepted by the Python API, MCP tools, iframe API, project format, and plugin manifest. See [Embedding & Sharing](#), [Python](#), and the [project format](#) for examples.

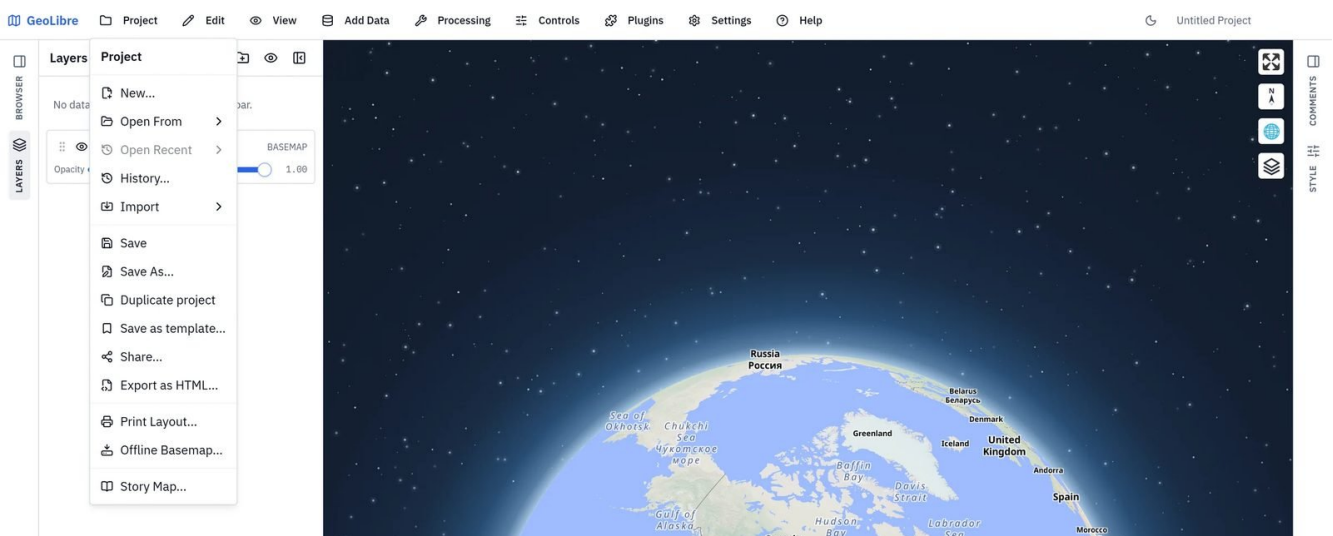
<https://geolibre.app/user-guide/projects/>

Projects

A GeoLibre project captures your whole workspace in a single `.geolibre.json` file: the map view, the basemap, every layer with its source and style, map preferences, plugin state, and environment variables. Everything in this section lives under the **Project** menu.

Some entries may be hidden

The Project menu is filtered by the active **UI profile**, and a few entries are desktop-only. If an item described below is missing, check the profile in use and whether you are running the browser build.

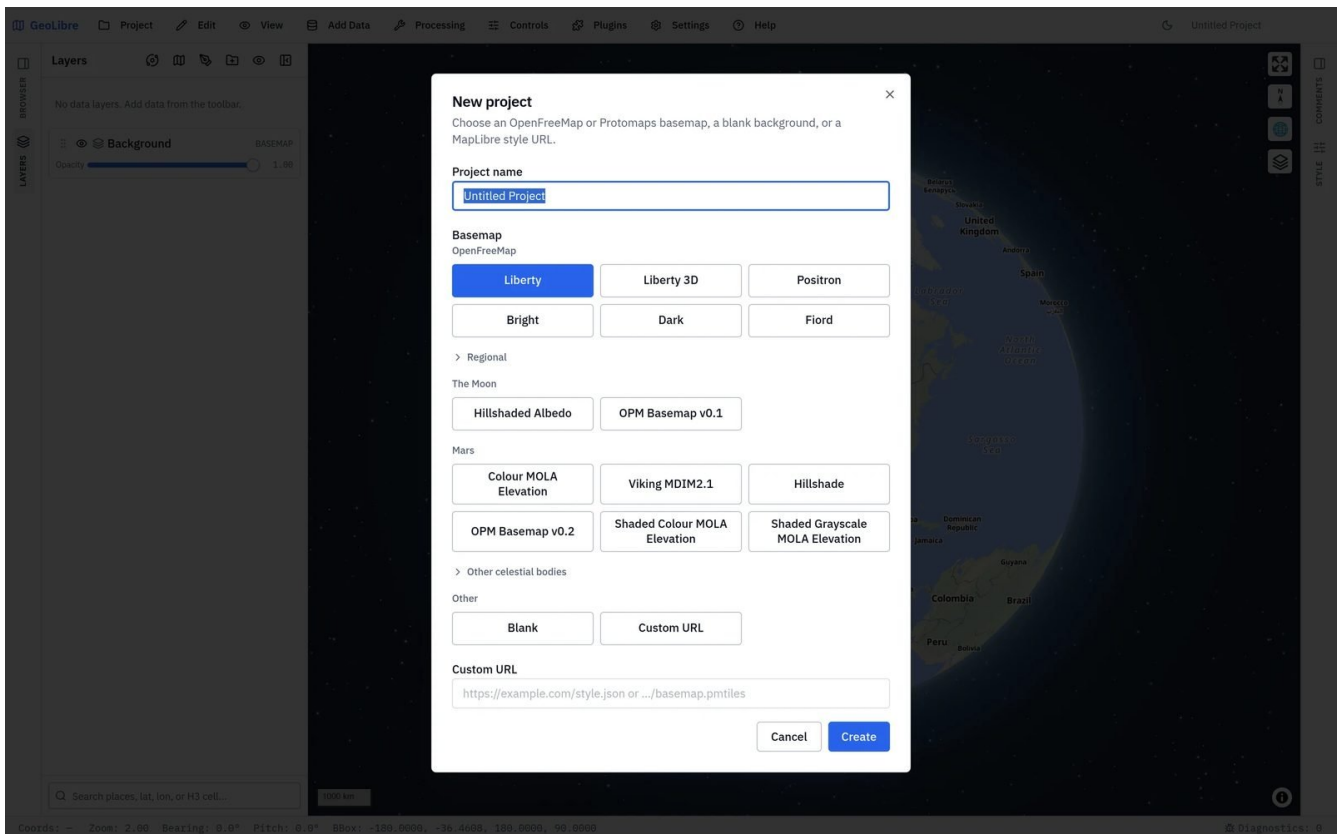


New

Project → New... starts a fresh project. GeoLibre offers to save the current project first, then resets the layers, map view, controls, and plugin state to defaults.

The **New project** dialog names the project and picks its starting basemap: the OpenFreeMap styles, the Protomaps styles (when the build offers them), a **Regional** group, sections for the Moon, Mars, and other celestial bodies, a blank background, or a custom MapLibre style or PMTiles URL.

Templates you saved with **Save as template...** appear above the basemaps under **Saved Templates**.



The collapsible **Examples** section lists a few curated starter projects from the [Gallery](#). Clicking one downloads it from `assets.geolibre.app` and opens it as an unsaved copy, the same way **Open From** → **URL** would. It needs an internet connection; if the download fails, the dialog stays open on your current project and says which example could not be opened.

Open

Project → **Open From** has three sources:

- **File...** opens a `.geolibre.json` file from disk. The browser build uses the browser's file picker.
- **URL...** loads a public `.geolibre.json` from an HTTP or HTTPS URL. This works in the browser too and adds the project to your recent list.
- **Gallery...** browses the shared project gallery and opens any entry with one click.

Project → **Open Recent** lists the projects you have opened before, each with its name, path, and the time you last opened it. Click an entry to reopen it, use the small remove button to drop a single entry, or choose **Clear Recent Projects** to empty the list. On the desktop app the recent list persists across sessions. In the browser, a URL-based entry reopens directly, while an entry for a local file asks you to pick the file again, because a browser cannot reopen a file on disk by itself.

Loading a project at startup

You can open a project directly by passing its URL with the `url` query parameter, for example `?url=https://share.geolibre.app/you/project.geolibre.json`. See [Embedding & Sharing](#).

On the desktop app you can also have GeoLibre reopen the last local project — or one specific local project — every time it launches; remote share links are never replayed on launch. See [Settings → Startup](#). A project URL in the address bar always takes precedence over that preference.

Save and Save As

- **Save** writes back to the project's existing file path.
- **Save As...** prompts for a new name and location.

Both capture the current map view, basemap, layers, styles, preferences, and plugin state at the moment you save. Projects that were opened from a URL have no writable local path, so both Save and Save As fall back to the save dialog. In the browser build, saving uses the browser's save dialog where it has one (Chromium-based browsers); elsewhere it asks for a file name and downloads the project to your downloads folder.

Project → Duplicate project copies the open project into a new, unsaved one, so you can branch off an experiment without touching the original file.

Project history and crash recovery

GeoLibre autosaves the project as you work. Three seconds after a change settles — a layer added, a style edited, the camera moved — it writes a snapshot to your browser's local IndexedDB storage. Autosaves never touch your `.geolibre.json` file; only **Save** does that.

Project → History... lists the snapshots for the current project, newest first, each summarized by its layer count and zoom level. **Restore** loads a snapshot back into the workspace, as an undoable step so you can back out of it. There is no manual delete here — snapshots age out on their own once a cap is hit.

Compare shows what changed between a snapshot and the current project, or between two snapshots (pick one under **Compare with**). Changes are grouped into collapsible sections: layers added, removed, renamed, reordered or restyled (each changed style, label, filter and source setting with its before and after value, plus counts of embedded features added, removed and modified), the camera, basemap and projection, plugins, and the project title, details and preferences. When comparing against the current project, **Restore this layer** brings back the snapshot's version of a single layer — or re-adds one you deleted — without touching the rest of the project; **Undo** reverts it. Re-adding a deleted layer also brings back what deleting it removed: its dashboard widgets, comments on its features, its legend order and overrides, its story-map chapter opacity steps, its

visibility in other map panes, and Print Layout table, chart or atlas blocks that used it. Anything you changed since the snapshot keeps your change, for example a Print Layout block you pointed at another layer.

The store is capped, so history stays bounded: at most 20 snapshots per project, 10 MB per snapshot, and 50 MB in total. The oldest snapshots are dropped once a cap is hit, and a project too large to fit in a single snapshot is not autosaved.

Crash recovery is a standalone-browser feature

In the browser build, and outside an embedded (iframe) session, GeoLibre marks the session open while you work. If the tab or browser goes away without closing cleanly and a newer autosave exists than your last explicit save, the next launch offers **Recover unsaved work?** with the option to restore or discard it. The desktop app and embedded deployments keep the history list but do not show this prompt.

Snapshots are stored per project — keyed by file path, or by name for a project you have not saved yet — and live only on the device that made them. They are not uploaded, not shared, and not part of the `.geolibre.json` file.

Importing a QGIS project

Project → Import → Import QGIS Project... reads a QGIS `.qgs` or `.qgz` project and rebuilds it as a GeoLibre project: its layers, layer groups (including nested ones), group visibility, layer order, styling, and the saved map view.

The importer targets file-based vector layers plus rasters the app can open, and it reports what it could not bring across rather than failing the whole import — you get the project plus a list of skipped layers and the reason (an unsupported data provider, an unsupported file format, a missing source, a network share path, or a remote source). Layers skipped for the same reason are grouped into one line with a count, so a project where hundreds of layers share one root cause reads at a glance; expand a group to see the layer names. In the browser build, layers that reference a local path on disk are listed as skipped because a browser cannot reopen those paths; open the same project in GeoLibre Desktop to load them.

Importing an ArcGIS Pro project

Project → Import → Import ArcGIS Pro Project... reads an ArcGIS Pro `.aprx` project or standalone `.mapx` map. GeoLibre reads the CIM JSON stored in the file directly, so ArcGIS Pro and ArcPy do not need to be installed.

An ArcGIS Pro project can contain several maps; GeoLibre imports its first 2D map. The importer preserves the saved extent, file-based feature layers and GeoTIFF rasters, nested groups, visibility, simple symbols, field-based labels, ArcGIS vector-tile portal items, and cached map services.

Unsupported sources such as file geodatabases, scenes, and network-share paths are listed, grouped by reason, after the rest of the project is imported. A File Geodatabase layer is named as such rather than reported as a generic unsupported format, and feature layers and rasters are reported separately, because their workarounds differ: in the desktop build a geodatabase's feature classes can be added with **Add Data → File Geodatabase (GDB)**, while a raster stored in a `.gdb` has to be exported to GeoTIFF first. Local data paths cannot be reopened by the browser build.

Templates

Project → Save as template... stores the current project as a reusable template in your personal library, with a name and an optional description. Enable **Strip data layers** to keep the basemap, layer groups, styles, legend, widgets, and layout while dropping the data layer content — useful for a house-style starting point that a team applies to new maps.

Share

Project → Share... uploads the current project to `share.geolibre.app` and returns a public URL you can send to anyone or open in the live viewer. The shared file is the same `.geolibre.json` the app saves locally, so anyone who opens the link sees the same layers, styles, and map view. See the [Sharing & Embedding tutorial](#).

Connecting your account depends on the build:

- **Web app:** click **Sign in** in the Share dialog (or under **Share.GeoLibre account** in **Settings → Environment Variables**). A popup opens `share.geolibre.app`'s consent page; approving it connects the app, and the sign-in is kept for the browser session. Session expired prompts offer a one-click re-sign-in.
- **GeoLibre Desktop:** sign in the same way; the consent page opens in your system browser and returns to the app. The sign-in is kept in your system keychain, so it survives restarts. Once signed in, **Settings → Environment Variables** also lets you review and revoke your sessions.
- **Mobile apps, notebook embeds, and as a fallback everywhere:** paste a personal API token — created under **Settings → API tokens** at share.geolibre.app/settings — into the **Share.GeoLibre API token** field in **Settings → Environment Variables**.

Share-readiness check

A project file is mostly references, so a project can upload cleanly and still draw nothing for the person you sent it to. When the Share dialog opens it checks the data sources the project points at and lists the ones a recipient will not be able to load, with the reason and what to do about it:

- **Uses a credential that is removed when sharing.** Tokens and API keys are stripped from the upload, so the recipient gets the URL without the secret. Make the service public, or tell them to supply their own key.

- **A browser cannot fetch this host.** The host sends no cross-origin (CORS) headers, or it did not answer. Layers like this keep working in the desktop app, which is not subject to browser CORS, but stay empty in the browser viewer.
- **The service answered not found.** A signed URL that has expired, or a file that moved.
- **Points at a private or local network address.** An intranet service only resolves for people on that network, which may be exactly who you are sharing with. A file on your machine, or a layer with no source at all, is not listed here but in a separate warning the moment the dialog opens; see [Sharing local data](#) below.

The check runs in the browser, without your credentials attached, so it sees what a recipient sees. It never blocks the upload: sharing an intranet map with intranet colleagues is a normal thing to do, and the list is there to inform you, not to stop you.

Sharing local data

A project file holds references to data, not the data itself, and `share.geolibre.app` stores only that file. It never uploads files from your computer. So a layer you added from a local GeoTIFF, GeoPackage, Shapefile, or other file on disk opens fine for you and draws nothing for anyone else: recipients open the project in a browser, which cannot read your disk. The same goes for a query-backed layer (PostGIS, a DuckDB SQL layer, a sidecar result) that names no URL. A layer on a private network address (`localhost` , an intranet server) is different: it may load for colleagues on the same network, so it stays in the softer readiness list above rather than in this warning.

When the Share dialog finds such layers it lists them under **N layers will be missing from the shared map**, and the Share button reads **Share anyway**. You can still share; the map will simply not include those layers. To include them:

- **Host the data online.** Convert rasters to Cloud Optimized GeoTIFF and vectors to PMTiles or GeoParquet, put the files on a public HTTPS server such as GitHub or Hugging Face, and add them to the map by URL (**Add Data → Raster Layer** or **Vector Layer** with the URL). Tile services, WMS, ArcGIS services, and other hosted sources work as they are, as long as the host allows cross-origin requests and needs no login.
- **Let small vectors ride along.** Local vector layers added with **Add Data → Vector Layer** are embedded in the upload automatically, so they are never listed. Large vector datasets are better hosted, since every feature of an embedded layer has to be parsed when the project opens.

The dialog warns even before a share token is configured, because uploading a saved `.geolibre.json` by hand on `share.geolibre.app` drops the same layers, silently.

Export as HTML

Project → Export → Export as HTML... writes the whole project to a single standalone HTML file that runs offline with no server. Host it anywhere, or open it straight from disk.

Layer styles

Project → Export → Export Layer Styles... writes every layer's style to one JSON file keyed by layer name. **Project → Import → Import Layer Styles...** applies such a file to the open project, restyling each layer whose name matches in one undoable step, and reports any styles that matched no layer. To apply a styles file automatically, choose it under **Default layer styles** in [Settings → Startup](#): each layer you add whose name matches is styled as it arrives, while layers in opened projects keep their saved styles.

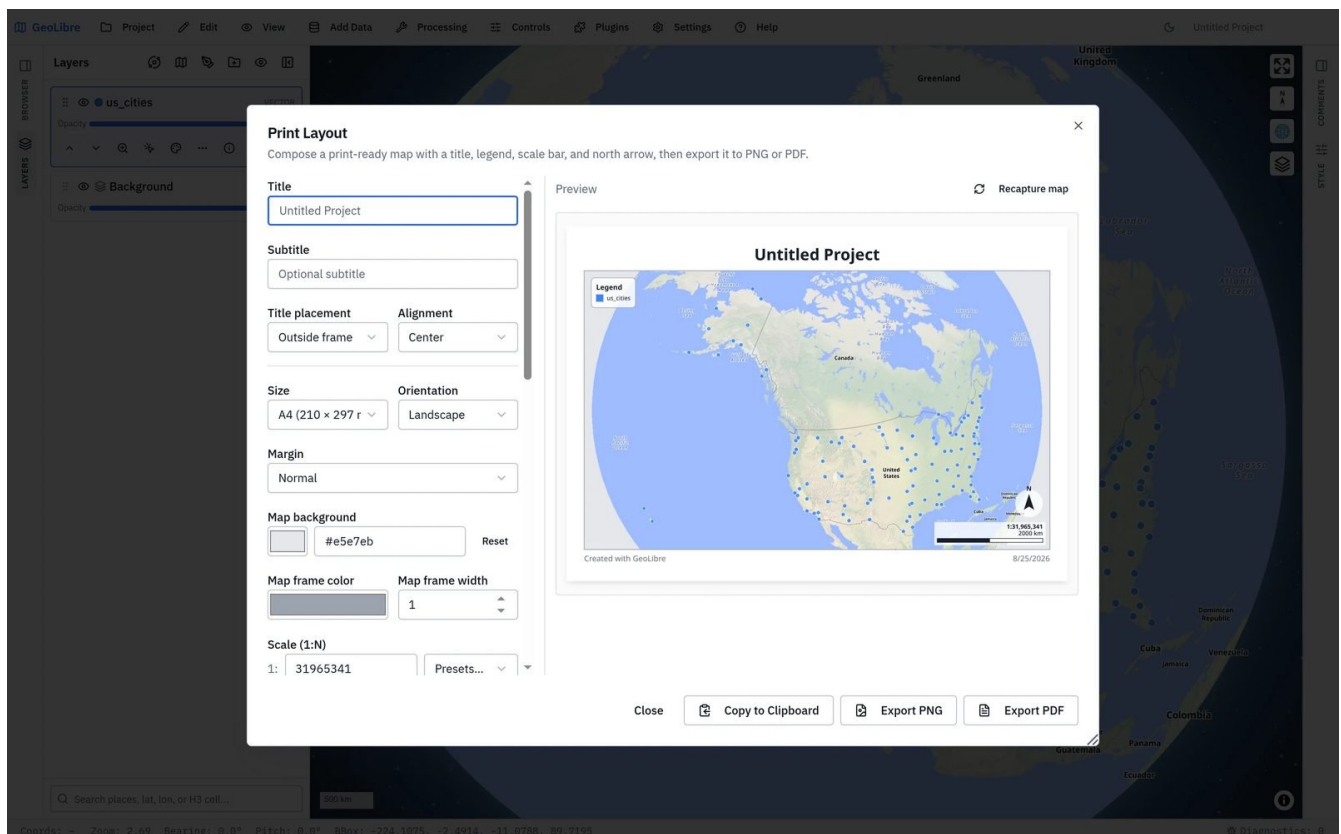
Collaborate

Project → Collaborate... starts or joins a live session in which several people edit the same project at once, with presence cursors, chat, and per-participant permissions. The feature is off unless the build configures a relay URL — see [Collaboration](#).

Offline basemap

Project → Offline Basemap... pre-caches the current map view's basemap tiles so the map still draws when the device is offline. See [Troubleshooting](#) if tiles are missing after a download.

Print



Project → Print Layout... opens the layout composer, which exports the current map to PNG, PDF, or SVG. It carries an editable title and footer, an info (title) block with project metadata, a user-editable legend, an explicit map-scale input, page-size controls, a custom print extent, attribute-table and chart blocks, Atlas / map series generation (one page per feature, or a uniform series along a line), SVG export for editing in a vector editor, and Copy to Clipboard. See [Print Layout](#) for the full guide.

Story maps

Project → Story Map... opens the scroll-driven story builder. See [Story Maps](#).

The project format

For the full schema of `.geolibre.json`, including how layers, styles, and plugin state are serialized, see [Reference → Project Format](#).

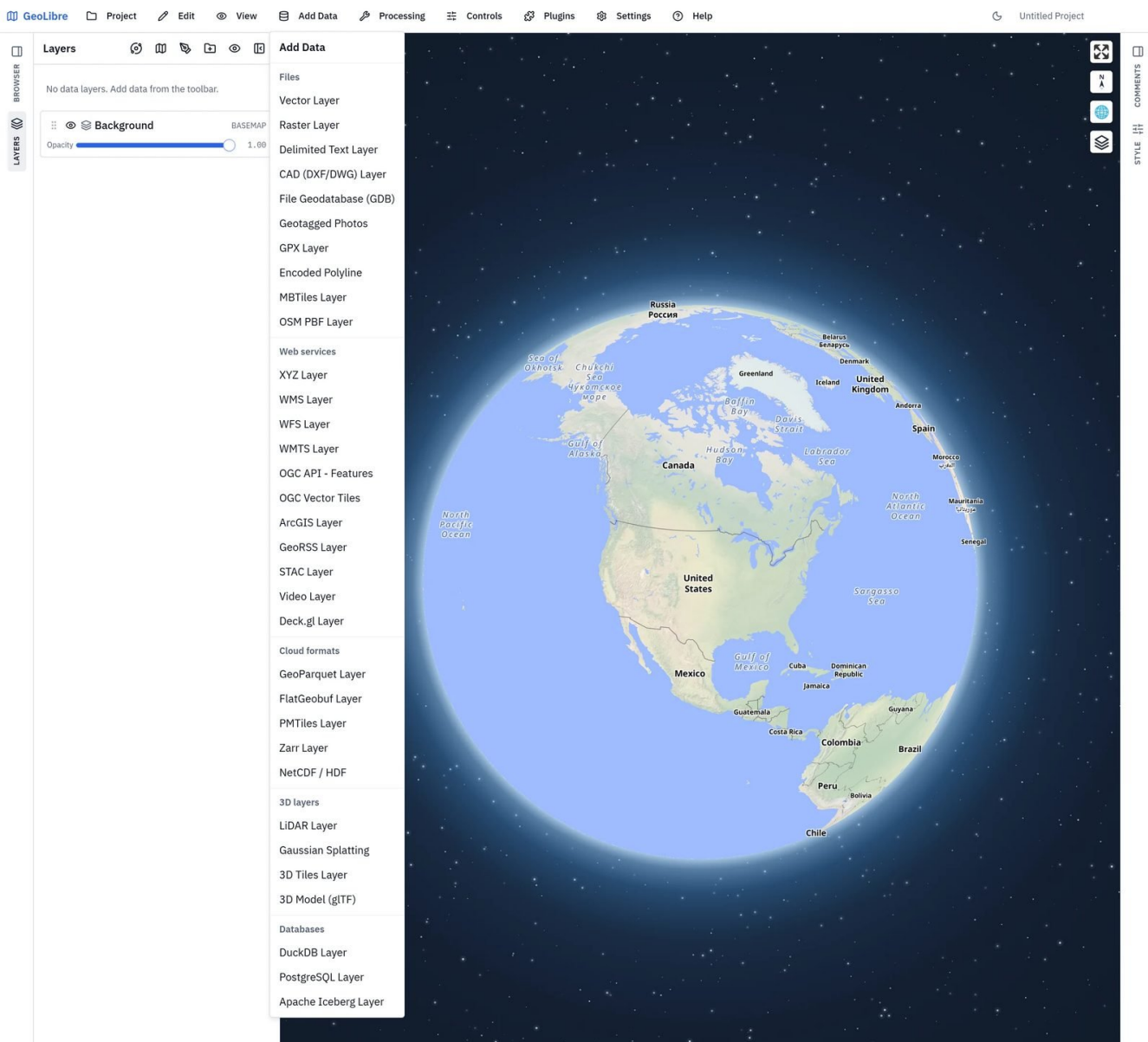
<https://geolibre.app/user-guide/adding-data/>

Adding Data

The **Add Data** menu is the main way to bring layers into GeoLibre. It groups sources into Files, Web services, Cloud formats, 3D layers, and Databases. You can also drag files straight onto the map.

For a consolidated list of file formats, service protocols, and platform limitations, see [Supported Data Formats](#).

To collect supported dataset links from a catalog or other webpage and open several at once, use the [GeoLibre Chrome extension](#), available from the [Chrome Web Store](#).



Files

Item	Notes
Vector Layer	Opens the Add Vector panel (backed by <code>maplibre-gl-vector</code>). Loads GeoJSON, GeoParquet, FlatGeobuf, zipped Shapefile, GeoPackage, KML/KMZ, GML, and other vector formats from a file or URL.
Raster Layer	Opens the Add Raster panel (backed by <code>maplibre-gl-raster</code>). Loads GeoTIFF and Cloud-Optimized GeoTIFF (COG) from a file or URL.
Delimited Text Layer	Loads CSV/TSV from a file or URL, using longitude and latitude columns to build point features, or by geocoding one or more address columns (see Geocoding). A file with no coordinates can be added as a non-spatial attribute table.
CAD (DXF/DWG) Layer	Loads AutoCAD drawings, converting their entities to vector features. Coordinate Z values (contours, 3D polylines, surveyed points) are kept and rendered in 3D unless Render Z values in 3D is unchecked.
File Geodatabase (GDB)	Opens an Esri file geodatabase (<code>.gdb</code> folder) and adds one of its feature classes as a layer. It is read by the local GeoLibre server, so it needs the desktop app.
Geotagged Photos	Reads the EXIF GPS tags from a set of photos and places each one on the map as a point with a thumbnail.
GPX Layer	Loads a GPX file or URL and splits it into separate waypoint, track, and route layers.
LandXML Layer	Loads a LandXML file or URL and imports selected TIN surfaces, horizontal alignments, vertical profile metadata, and survey points. Projected coordinates are reprojected from the selected or embedded source CRS.
Encoded Polyline	Loads Google (precision 5) or Valhalla/Mapbox (precision 6) encoded polyline strings from pasted text or uploaded text files.
MBTiles Layer	Loads a local MBTiles tile archive (desktop app).
OSM PBF Layer	Reads an OpenStreetMap <code>.osm.pbf</code> extract and adds the features you select from it.

A layer the Add Vector panel loads in GeoJSON mode, with up to 100,000 features, is handed over to GeoLibre once it has loaded, so it gets the same Style panel as a file dropped on the map: joins, virtual fields, the attribute form, popup design, editor tracking, diagrams, and geometry generators included. It then leaves the panel's own layer list and is managed from the Layers panel. The panel keeps the layers it draws as tiles, streamed GeoParquet, larger layers, and KML/KMZ layers with placemark icons; for those, the Style panel shows only the settings the panel can draw. A saved project still reopens a handed-over layer from its URL or file path, and a URL-backed one refreshes the same way.

Vector files are reprojected to EPSG:4326 on load. In the browser, vector import relies on DuckDB-WASM Spatial, with direct handling for GeoJSON, zipped Shapefiles, and KMZ archives. The source CRS is read from the file itself — the layer metadata for the GDAL-read formats, a Shapefile's `.prj` sidecar, or a GeoParquet's `geo` metadata — so a national grid such as EPSG:2100 (GGRS87 / Greek Grid) lands in the right place with nothing to configure.

GeoParquet opens across its variants: 1.0 and 1.1 files (including one carrying a declared `covering` bbox column), 2.0 files using the native Parquet `GEOMETRY` / `GEOGRAPHY` types, and plain Parquet whose geometry is only a WKB blob under a conventional name such as `geom`. The `geo` metadata block decides which column holds the geometry and what CRS it is in, so a file with several geometry columns loads the one it declares as primary; a file declaring `"crs": null` is stating that its coordinates are in no known reference system, and is drawn as it stands with no transform applied. A Parquet table with no geometry at all but a pair of `lon / lat` (or `longitude / latitude`, `lng`, `x / y`) floating-point columns loads as points, assumed to be WGS84.

Large vector files

There is no fixed size limit. Files **under 100 MB** are read by the in-memory JavaScript readers, which are fastest for everyday data. At **100 MB or larger**, GeoLibre streams the file through DuckDB instead — off the main thread, so the interface keeps responding. For a zipped Shapefile the threshold applies to the *uncompressed* `.shp`, since shapefiles compress heavily and the archive's size says little about the parse cost. This happens automatically; nothing is asked of you.

A separate check counts features once the source is open. Past **100,000 features**, GeoLibre asks before converting every one to GeoJSON in memory, because that is where memory rather than file size becomes the limit — a small GeoParquet can hold millions of rows.

For very large data, converting first still pays: **Processing → GeoLibre Toolbox → Conversion → Vector to PMTiles** writes a tiled format the map loads one tile at a time instead of reading the whole file. Converting to **GeoParquet** instead gives a compact columnar format that reads far faster than text, though it is not tiled. GeoJSON is the most expensive option at any size — it expands several-fold in memory — so prefer a Shapefile, GeoParquet, or FlatGeobuf source when you have the choice.

KML and KMZ

KML is read by an in-house parser that keeps the file's own symbology, so styled KML renders the way it does in Google Earth. A file that parser cannot handle falls back to the DuckDB Spatial reader, which loads the geometry without the styling.

KML imports also render `GroundOverlay` images as map overlays (which animate through the Time Slider when they are time-tagged) and show embedded Collada `.dae` models. **Super-Overlays** — the tiled, `NetworkLink`-driven KML that large imagery exports use — are supported too: GeoLibre serves the archive's tiles to the map instead of trying to load the whole pyramid at once.

Web services

Item	Notes
XYZ Layer	A raster or vector tile service using a <code>{z}/{x}/{y}</code> URL template.
WCS Layer	Downloads numerical GeoTIFF subsets from WCS 1.0.0 services.
WMS Layer	A Web Map Service layer, with click-to-identify through GetFeatureInfo where supported. In the desktop app, once the layers are retrieved, a layer can be requested in any coordinate reference system it offers; the tiles are reprojected to Web Mercator.
CSW Catalog	Searches an OGC Catalogue Service (CSW) for datasets and adds their supported linked resources (GeoJSON, WMS, WFS, ArcGIS) to the map.
WFS Layer	A Web Feature Service layer, with optional automatic refresh. GeoLibre asks for GeoJSON first and falls back to GML when the server offers no GeoJSON output, so GML-only services (MapServer, most INSPIRE services) load too. GML in WGS84, ETRS89, NAD83 or Web Mercator is read directly, and GML in any other EPSG coordinate system (a national grid, a UTM zone) is reprojected; a system GeoLibre does not know is reported as an error rather than drawn in the wrong place. In the desktop app, WFS requests go through the native HTTP client, so services that send no CORS headers load too.
WMTS Layer	A Web Map Tile Service layer.
OGC API - Features	An OGC API - Features endpoint; pick a collection and add it as a vector layer.
OGC Vector Tiles	An OGC API - Tiles vector tile service.
ArcGIS Layer	An ArcGIS FeatureServer, VectorTileServer, MapServer, or ImageServer layer. See ArcGIS services .
GeoRSS Layer	A GeoRSS feed, added as points and lines with the feed's titles and descriptions as attributes.
Administrative Boundaries	Pick a country and an admin level (ADM0 for the national border down to ADM3 to ADM5 where a country has them) to add its geoBoundaries polygons. Simplified geometry is the default; clear it for full-resolution boundaries, which can be several MB.
STAC Layer	Searches a STAC catalog and adds the matching raster items.

Item	Notes
Video Layer	Drapes a video over four map corner coordinates, the way MapLibre's video source does.
Deck.gl Layer	Renders a deck.gl layer specification over the map, for visualizations MapLibre's own layer types do not cover.

ArcGIS services

Pick the **Layer type** that matches the service, then give it a service URL or a portal item ID. For a secured service, set **Authentication** to **Sign in with ArcGIS** or paste an access token; see [Signing in with ArcGIS](#).

Layer type	Service	How it loads
Feature layer	FeatureServer	Downloaded page by page as a GeoJSON layer, so labels, the attribute table, identify, symbology, and export all work on it.
Vector tile layer	VectorTileServer	Rendered from the service's own style.
Map service	MapServer	A raster layer. Cached services are read as tiles; dynamic ones are drawn per tile through <code>/export</code> .
Image service	ImageServer	A raster layer drawn through <code>/exportImage</code> (or the service's tile cache).

Map and image services become ordinary raster layers, so opacity, the Style panel's brightness/contrast/saturation controls, reordering, and saving to a project all work on them.

Two optional fields shape what the service draws:

- **Sublayers** (map service) takes the sublayer ids to draw, such as `0,2,5`. Leave it blank for the service's own default set. Pasting a URL that ends in a sublayer id (`.../MapServer/3`) selects that sublayer for you.
- **Raster function** (image service) can be selected from the functions the service advertises with **Browse functions**. The selected function is applied by the server before the image is sent. The **Rendering rule** field remains available for advanced Esri raster function JSON. For example, `{"rasterFunction":"Hillshade"}` selects the Hillshade function.

Either choice draws the service dynamically, because a cached service's tiles were rendered before the choice existed and cannot honor it.

Cloud formats

Item	Notes
GeoParquet Layer	Cloud-native columnar vector format. Opens the same Add Vector panel as Vector Layer . Can be streamed in place with HTTP range requests for large remote files.

Item	Notes
FlatGeobuf Layer	Cloud-optimized vector format with spatial indexing.
PMTiles Layer	A single-file vector or raster tile archive.
Zarr Layer	Chunked, cloud-native multidimensional arrays.
NetCDF / HDF	Multidimensional scientific data cubes, with a setup step for picking the variable and the time and vertical dimensions.

3D layers

Item	Notes
LiDAR Layer	Point-cloud visualization, rendered with deck.gl.
Gaussian Splatting	Gaussian splat scenes.
3D Tiles Layer	OGC 3D Tiles, restored when reopening a project. Includes a Google Photorealistic 3D Tiles sample that reads <code>VITE_GOOGLE_MAPS_API_KEY</code> or <code>GOOGLE_MAPS_API_KEY</code> from the runtime environment.
Cesium Ion Asset	Adds a 3D Tiles tileset or imagery from Cesium ion by asset ID, using your Cesium ion token. Cesium engine only; the entry is disabled on the 2D engines.
CZML Dynamic 3D Scene	Loads a CZML document of orbits, trajectories, vehicle paths, and other time-varying 3D scenes. Cesium engine only.
KML / KMZ	Loads KML or KMZ with native globe styling, overlays, and network links on the Cesium globe, and through the regular KML importer on the 2D engines.
3D Model (glTF)	Places a glTF/GLB model at a coordinate, with scale, rotation, and altitude controls.

Databases

Item	Notes
DuckDB Layer	Query a DuckDB or DuckDB Spatial source and add the result as a layer, with identify, selection, and attribute table support.
PostgreSQL Layer	Add a layer from a PostgreSQL/PostGIS connection (desktop app, served through a local tile server).
SQL Server Layer	Add a spatial table from SQL Server or Azure SQL (desktop app).
Apache Iceberg Layer	Read an Iceberg table through DuckDB and add its spatial column as a layer.

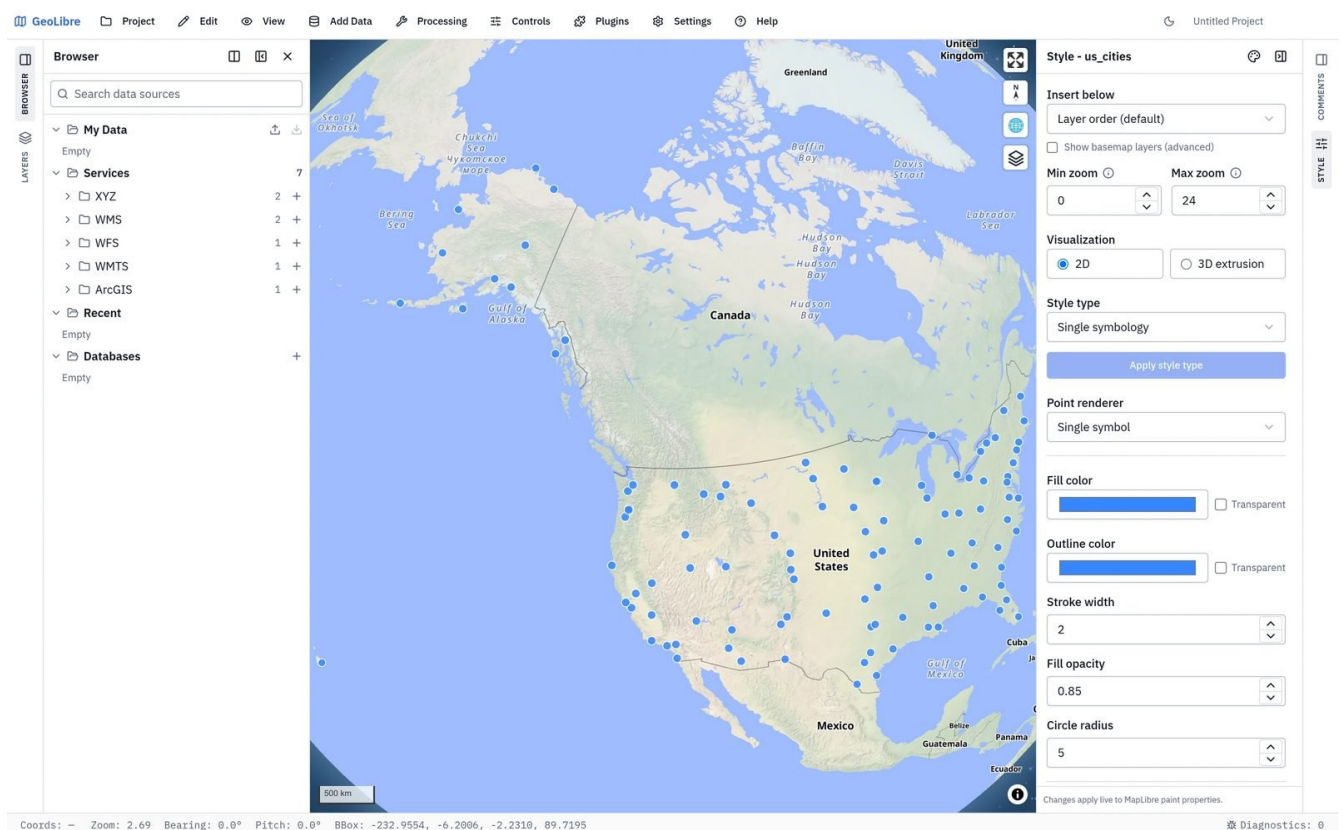
Drag and drop

Drag a vector file (GeoJSON, zipped Shapefile, KMZ, and similar) or a GeoTIFF/COG raster directly onto the map to add it as a layer. GPX files dropped on the map are split into named waypoint, track, and route layers. Other files are routed by type:

- **OSM PBF** (`.osm.pbf`) extracts are parsed into point, line, and polygon layers.
- **LAS, LAZ, and COPC** point clouds load into the LiDAR control; a LAS 1.4 file the viewer cannot read whole is converted to COPC first.
- **Geotagged photos** (JPEG, PNG, WebP, HEIC) become a point layer placed from their EXIF GPS tags.
- A single `.geolibre.json` project file opens as the current project.

The Browser panel

The **Browser** tab on the left edge of the window opens a QGIS-style Data Source Manager: a tree of everything you can add without going through a menu. Use it when you add the same sources repeatedly, or when you want to explore a service or a database before committing to a layer.



Section	What it holds
Favorites	Services, folders, and files you starred, listed first for quick access. Appears once you have favorited something.
My Data	Your personal layer library. Layer actions → Save to My Data stores a fully configured layer — source, style, labels, filters, joins, virtual fields, and attribute form — and one click here re-adds it to any later project. Import and export the library with the buttons on the section header.
Services	Saved map services, grouped by kind (WMS, WFS, WMTS, XYZ, ArcGIS, CSW). GeoLibre ships a starter set; the + on a group adds a new connection of that kind. Expand a service to browse its layers and add one. Self-hosted deployments can add read-only organization-wide services here, marked with a <i>config</i> badge (see Getting Started); they are shared with every user, cannot be edited or deleted, and are never stored in your own service library.
Recent	The projects you opened most recently, so reopening one is one click.
Databases	PostgreSQL/PostGIS and SQL Server connections, each in its own engine group with a + for a new connection. Expand a connection to browse its schemas and spatial tables; the Add Data form retains each engine's table options. The trash icon forgets a saved connection and removes its saved credential; on desktop, if the system keychain cannot delete the credential right away, GeoLibre shows a persistent warning with the password-masked connection label and retries the next time it starts. Failures for different connections are reported separately.
Files	Desktop app only. Folders you pinned with Add folder (the + on the section), each expandable to its subfolders and the loadable files inside, so you can add a file without a file dialog.

Choose the + on the PostgreSQL or SQL Server group to open that engine's connection form; the Databases section has no shared connection action.

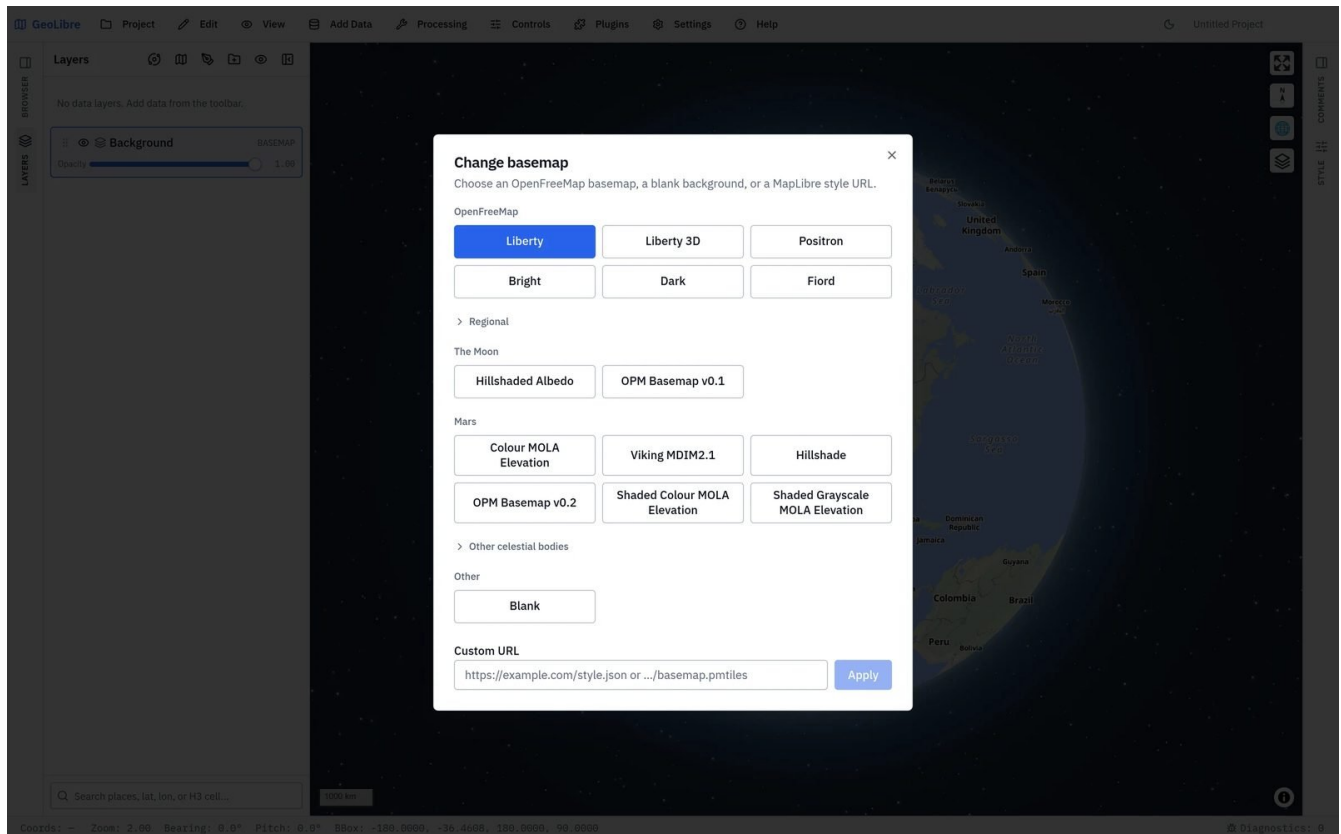
Forgetting a PostgreSQL connection invalidates table requests already in flight, so re-saving it cannot show an older request's table list. On desktop, an unreadable or malformed credential-deletion journal prevents forgetting additional connections until the journal can be read and is valid again; GeoLibre preserves it rather than discarding pending deletions. A connection's deletion result reflects the system keychain's acknowledgement, even if the journal becomes unreadable after the saved-list update.

Type in the search box to filter the whole tree, and navigate it entirely from the keyboard with the arrow keys.

If desktop database browsing reports a processing server from a previous session, quit any stray GeoLibre processes, then collapse and re-expand the connection to retry.

Basemaps

The basemap sits at the bottom of the [Layers panel](#) as the **Background** entry. **Double-click that row** to open the **Change background** dialog, or activate the **Basemaps** plugin from the [Plugins menu](#) to switch between OpenFreeMap styles (Liberty, Liberty 3D, Positron, Bright, Dark, Fiord), Protomaps styles (when the build offers them), a collapsible **Regional** group, a blank background, or a custom style URL. You can toggle basemap visibility and adjust its opacity from the Layers panel.



Other celestial bodies

GeoLibre can map worlds beyond Earth. The **Change background** dialog and the **New project** dialog group planetary basemaps into sections for **The Moon**, **Mars**, and a collapsible **Other celestial bodies** section covering **Mercury**, **Venus**, the **Galilean moons** (Io, Europa, Ganymede, Callisto), **Titan**, **Pluto**, and **Charon**. The Moon and Mars mosaics come from [OpenPlanetaryMap](#); the other bodies come from [USGS Astrogeology](#) and are reprojected to Web Mercator on the fly so MapLibre can render them.

For quick switching, use the **planet switcher** (the orbit icon) in the Layers panel header. Selecting a body sets the project's **ellipsoid**, so distance, area, and scale-bar measurements use that body's radius instead of Earth's.

More data sources

Additional catalogs and providers are available as panels and plugins rather than Add Data items, including Planetary Computer, Earth Engine, Overture Maps, and several federal Web Services. See [Data Integrations](#).

Three of them, grouped under **Plugins → Web Services**, search public dataset catalogs by keyword and add a result to the map in one click:

Panel	What it searches
ArcGIS Hub	Public datasets published to ArcGIS Hub. Search by keyword or restrict the search to the current map area, page through results, add a supported layer to the map, zoom to it, or download the data. Datasets with several layers download only the first, and the panel says so.
Socrata	Public Socrata open-data catalogs, adding their GeoJSON datasets.
CKAN	The Humanitarian Data Exchange CKAN catalog, adding its available GeoJSON resources.

Each panel shows how many of the total results you are looking at and offers **Load more** to page further. See [Web Services](#) for the other browsers in that submenu.

Browser vs desktop

URL-based sources work in both the browser and the desktop app. Local file dialogs, local MBTiles, local raster reads, File Geodatabases, PostgreSQL, and SQL Server require the desktop app. See [Getting Started](#).

<https://geolibre.app/data-formats/>

Supported Data Formats

This page lists GeoLibre's supported data formats, services, and database sources, including how to open them and the main limitations. The tables reflect the default MapLibre renderer; some specialized layers are unavailable with other renderers. See [Adding Data](#) for the menu workflow and [Projects](#) for saving and reopening data.

Vector and tabular files

Format	Extensions or source	How to add it	Notes
GeoJSON	<code>.geojson</code> , <code>.json</code> , GeoJSON URL	Vector Layer	Geometry and feature attributes; also supports drag and drop.
GeoParquet / Parquet	<code>.geoparquet</code> , <code>.parquet</code>	GeoParquet Layer or Vector Layer	Reads geometry and CRS metadata; supported coordinate-column tables can become points.
FlatGeobuf	<code>.fgb</code> , <code>.flatgeobuf</code>	FlatGeobuf Layer or Vector Layer	Spatially indexed vector files, including remote URLs.
GeoPackage	<code>.gpkg</code>	Vector Layer	Select feature tables from a multi-layer file.
Shapefile	<code>.shp</code> with companion files, or <code>.zip</code>	Vector Layer	Keep <code>.shx</code> , <code>.dbf</code> , and <code>.prj</code> companions together; use a ZIP for browser imports.
GML	<code>.gml</code>	Vector Layer	Imported through the vector reader.
MapInfo TAB	<code>.tab</code> and companion files	Vector Layer	Requires access to the dataset's companion files.
KML / KMZ	<code>.kml</code> , <code>.kmz</code>	KML / KMZ or Vector Layer	Supports folders, styles, ground overlays, embedded models, and Super-Overlays; behavior depends on the renderer.
GPX	<code>.gpx</code>	GPX Layer	Waypoints, tracks, and routes can become separate layers.
LandXML	<code>.xml</code> , <code>.landxml</code> , LandXML URL	LandXML Layer	Imports TIN surfaces, horizontal alignments, vertical profile metadata, and survey points; projected data requires a source CRS.
Delimited text	CSV, TSV, and custom-delimited text	Delimited Text Layer	Map coordinate columns, specify their CRS, or geocode address columns. CSV without coordinates can also be loaded as a table.
Excel	Excel workbooks	Delimited Text Layer	Select a worksheet and coordinate columns to create point features.

Format	Extensions or source	How to add it	Notes
AutoCAD	.dxf , .dwg	CAD (DXF/DWG) Layer	Select drawing layers and the source CRS. Coordinate Z values are kept and rendered in 3D unless you turn that off.
Esri File Geodatabase	.gdb folder	File Geodatabase (GDB)	Desktop folder access; select a feature class.
OpenStreetMap PBF	.osm.pbf	OSM PBF Layer	Reads an extract in the browser and adds the selected features.
Encoded polyline	Encoded strings or text files	Encoded Polyline	Precision 5 and 6, including Google, OSRM, Valhalla, and Mapbox conventions.

Vector import generally reprojects a known source CRS to EPSG:4326. Multi-file formats need their companion files, and large feature collections may require substantial memory. Additional formats depend on the bundled vector reader; see [vector import details](#).

Raster, tile archives, and multidimensional arrays

Format	Extensions or source	How to add it	Notes
GeoTIFF / Cloud-Optimized GeoTIFF (COG)	.tif , .tiff , remote URL	Raster Layer	Numerical pixel inspection, styling, export, and raster analysis. COGs support partial remote reads.
PMTiles	.pmtiles , remote URL	PMTiles Layer	Vector and raster tile archives. Multiple vector source layers can be added separately.
MBTiles	.mbtiles	MBTiles Layer	Local raster or vector tile archives; requires desktop filesystem access.
Zarr	Remote store or local folder	Zarr Layer	Multidimensional arrays with variable and dimension selection; also supports Icechunk repositories.
NetCDF-4 / HDF5	Local scientific data files	NetCDF / HDF	Select variables and their spatial, time, and vertical dimensions.
Cloud-Optimized NetCDF / HDF	Kerchunk references	NetCDF / HDF	Reads referenced chunks from remote data.

Web services and catalogs

Service	How to add it	Data and limitations
XYZ tiles	XYZ Layer	Raster or vector tiles from a {z}/{x}/{y} URL template.
WMS	WMS Layer	Rendered map images; discover layers and query GetFeatureInfo where supported.

Service	How to add it	Data and limitations
WCS 1.0.0	WCS Layer	Numerical GeoTIFF subsets with chosen extent, pixel dimensions, and an EPSG:4326 or EPSG:3857 request CRS.
WFS	WFS Layer	Vector features with feature-type discovery and optional refresh. Loads GeoJSON, or GML (2, 3.1, 3.2) from servers that offer no GeoJSON output; GML in other EPSG coordinate systems is reprojected.
WMTS	WMTS Layer	Map tiles from a Web Map Tile Service.
OGC API - Features	OGC API - Features	Browse collections and load their features.
OGC API - Tiles	OGC Vector Tiles	Vector tile services.
ArcGIS FeatureServer	ArcGIS Layer: Feature layer	Features and attributes from a service URL or portal item.
ArcGIS VectorTile Server	ArcGIS Layer: Vector tile layer	Vector tiles and their styling.
ArcGIS MapServer	ArcGIS Layer: Map service	Cached tiles or rendered exports, with sublayer selection.
ArcGIS ImageServer	ArcGIS Layer: Image service	Rendered imagery with advertised raster functions or a custom rendering rule. Use WCS when numerical raster subsets are needed and the service exposes WCS.
GeoRSS	GeoRSS Layer	Geographic features from a feed URL or file.
geoBoundaries	Administrative Boundaries	Country, state, and district boundaries (ADM0 to ADM5, depending on the country) under open licenses, in simplified or full geometry.
STAC	STAC Layer	Search catalogs and load raster assets.
OGC CSW 2.0.2	CSW Catalog	Search metadata records; open supported linked GeoJSON, WMS, WFS, and ArcGIS resources.

Remote services must allow browser cross-origin requests (CORS) for the hosted web app. Some services need credentials or impose their own request limits. Provider-specific catalogs, including NASA, USGS, ArcGIS Hub, Socrata, and CKAN, are described in [Web Services](#) and [Data Integrations](#).

3D, time-dependent data, and media

Source	How to add it	Notes
LiDAR point clouds	LiDAR Layer	LAS/LAZ and cloud-optimized point-cloud data. The layer menu exports a cloud to LAS, LAZ or COPC, and Whitebox LiDAR tool outputs load as new point cloud layers. LAS, LAZ and COPC files can also be dropped on the map.
Gaussian splats	Gaussian Splatting	Point-based scene rendering.
OGC 3D Tiles	3D Tiles Layer	Streamed 3D tilesets; authenticated tilesets can use request headers.
ArcGIS I3S	3D Tiles Layer (SceneServer URL)	Integrated Mesh and 3D Object scene layers.
Cesium ion assets	Cesium Ion Asset	3D Tiles tilesets or imagery hosted on Cesium ion; requires the Cesium renderer.
CZML	CZML Dynamic 3D Scene	Time-dependent scene descriptions; requires the Cesium renderer.
glTF / GLB	3D Model (glTF)	Place a local or remote model at coordinates with scale, rotation, and altitude.
Georeferenced video	Video Layer	Video draped over four map corner coordinates.
Geotagged photos	Geotagged Photos	EXIF GPS locations with thumbnails; photos without coordinates can be placed manually.
Deck.gl visualization data	Deck.gl Layer	Supported visualization types from local files or URLs.

Databases and spatial query sources

Source	Entry point	Notes
DuckDB / DuckDB Spatial	DuckDB Layer or SQL Workspace	Query local files, remote data, or a database and add spatial results to the map.
PostgreSQL / PostGIS	PostgreSQL Layer or Browser panel	Desktop database connections use a local tile server.
SQL Server / Azure SQL	SQL Server Layer or Browser panel	Desktop only; spatial tables load as editable layers.
Apache Iceberg	Apache Iceberg Layer	Read spatial tables through DuckDB.
PostGIS via PGLite	SQL Workspace	In-browser spatial SQL engine.

Source	Entry point	Notes
Apache Sedona	SQL Workspace	An additional spatial query engine.

Project formats

Format	Entry point	Notes
GeoLibre project (<code>.geolibre.json</code>)	Project > Open From	Layers, styles, groups, and map state; remote sources and local file references still need to be accessible.
QGIS project (<code>.qgs</code> , <code>.qgz</code>)	Project > Import	Imports supported layers and settings; reports skipped sources. Local paths require desktop access.
ArcGIS Pro project or map (<code>.aprx</code> , <code>.mapx</code>)	Project > Import	Imports supported layers and settings; see the project import guide for requirements.

WCS raster subsets

Use **Add Data > WCS Layer** to download numerical raster data from an OGC Web Coverage Service. Unlike a rendered WMS image, the returned GeoTIFF retains band values for pixel inspection, raster styling, export, and raster analysis.

1. Enter the service's WCS endpoint and click **Retrieve coverages**.
2. Select a coverage. Choose **Use view**, **Use coverage extent**, or enter west, south, east, and north in longitude/latitude degrees.
3. Pick a **Request CRS** (see below), set the output width and height in pixels, and click **Add layer**.
4. Use the layer's pixel inspector or raster tools. Export the raster as a GeoTIFF to keep a copy of the downloaded data.

The sample selector includes USGS 3DEP elevation and the Illinois statewide LiDAR DEM. Each sample selects a small area near Elkhart, Illinois. A bare ArcGIS ImageServer REST URL is also accepted and converted to its WCS endpoint. This is a convenience; GeoServer and other WCS endpoints use the same protocol.

Supported requests

The initial implementation supports **WCS 1.0.0** KVP `GetCapabilities` , `DescribeCoverage` , and `GetCoverage` . Services must advertise GeoTIFF output; unsupported versions and formats produce an error before the coverage download. WCS 1.1 and 2.x requests are not yet implemented.

The coverage is requested in **EPSG:4326** or **EPSG:3857** (Web Mercator). The box is always entered in longitude/latitude and projected to the chosen CRS. **Automatic** picks EPSG:4326 when the coverage advertises it, then EPSG:3857, and otherwise still tries EPSG:4326: many services (for example PDOK's AHN elevation service, which lists only EPSG:28992) serve CRSes they do not advertise. If the server

rejects a CRS it did not advertise, the error names the advertised list so you can try the other option. Striped GeoTIFF responses, which MapServer-based services return, are converted to a Cloud-Optimized GeoTIFF in the browser before loading.

The selected extent and pixel dimensions determine output resolution. The server may resample values; this is not a guarantee of native-resolution data. Each dimension must be 1–4096 pixels, with a 128 MB download limit; coverage listings and descriptions are capped at 32 MB. The server may impose smaller limits. Areas crossing the antimeridian must be split into two requests. Each load is a fixed subset; panning does not fetch more coverage.

Downloads become file-backed raster layers and follow the same persistence rules as other imported GeoTIFFs. Export the raster before ending the session; saving only the project does not retain the downloaded raster bytes.

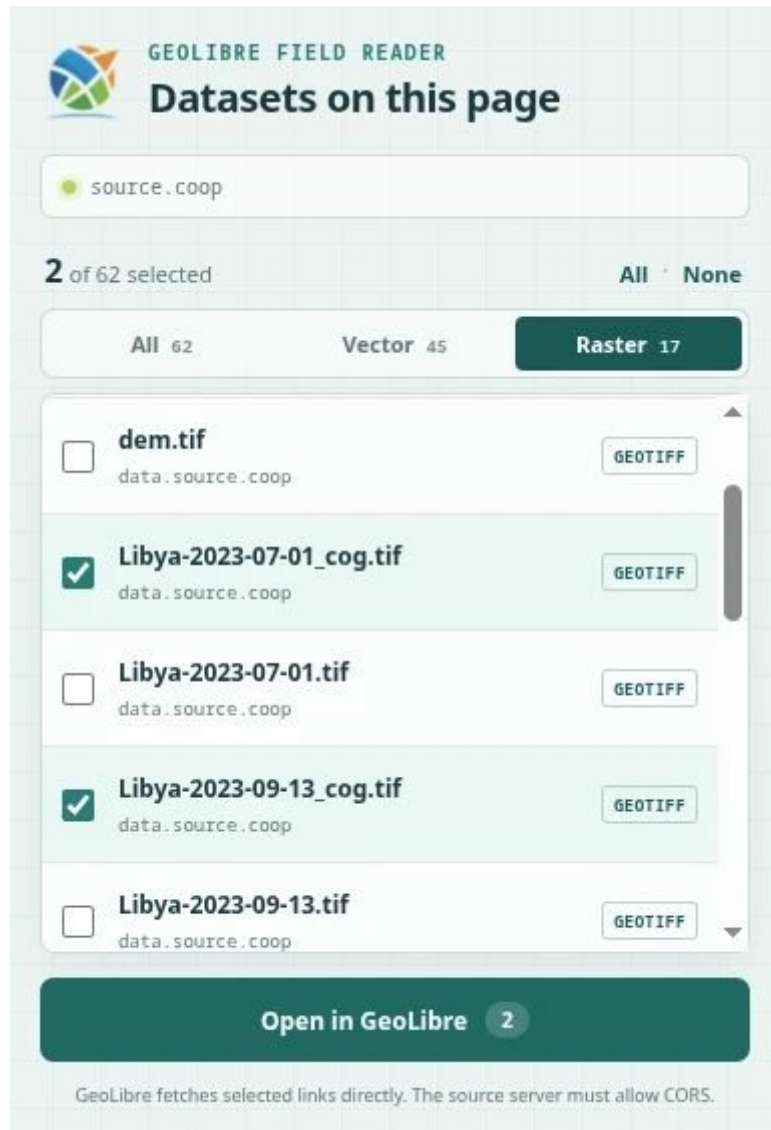
Desktop requests use native HTTP. The hosted web app requires the service to allow browser cross-origin requests (CORS). Development uses the existing service proxy, so a successful development test alone does not establish CORS support on the hosted site.

References: [OGC WCS 1.0.0 conformance tests](#) and [GeoServer WCS documentation](#).

<https://geolibre.app/user-guide/chrome-extension/>

Chrome Extension

The **Open data in GeoLibre** Chrome extension finds supported geospatial data links on the current webpage and opens the datasets you select together on one GeoLibre map.



Install the extension

The extension is published on the Chrome Web Store, so it installs in one click and updates automatically:

[Get Open data in GeoLibre from the Chrome Web Store](#)

1. Open the [Chrome Web Store listing](#).
2. Select **Add to Chrome**, then confirm.
3. Pin **Open data in GeoLibre** to the Chrome toolbar for convenient access.

It works in any Chromium-based browser that can install from the Chrome Web Store, including Chrome, Edge, Brave, Vivaldi, Opera, and Arc.

Install the packaged release manually

Prefer not to install from the Store, or want to run a build that is newer than the reviewed listing? Load the packaged release as an unpacked extension:

1. Open the [latest GeoLibre release](#) and download the ZIP asset whose name starts with `geolibre-chrome-`.
2. Extract the ZIP archive to a folder you intend to keep. Chrome loads the extension from this folder, so do not delete it after installation.
3. Open `chrome://extensions` in Chrome.
4. Turn on **Developer mode**.
5. Select **Load unpacked** and choose the extracted `geolibre-chrome-<version>` folder.
6. Pin **Open data in GeoLibre** to the Chrome toolbar for convenient access.

A manually installed copy does not update itself. To update it, download and extract the newer release asset, then select the extension's **Reload** button on `chrome://extensions`. If you extract it to a different folder, remove the old copy and load the new folder instead.

Open datasets from a webpage

1. Visit a webpage or data catalog containing geospatial file links.
2. Select the GeoLibre icon in the Chrome toolbar.
3. Use **All**, **Vector**, **Raster**, or **LiDAR** to filter the discovered datasets.
4. Check one or more datasets. Nothing is selected by default.
5. Select **Open in GeoLibre** to load the selected datasets on the same map.

The extension recognizes GeoJSON and spatial JSON, GeoParquet and Parquet, PMTiles, GeoTIFF and Cloud-Optimized GeoTIFF, LiDAR point clouds (LAS, LAZ, COPC, and EPT `ept.json`), ZIP archives containing GeoJSON, JSON-LD download metadata, and existing GeoLibre data links. A neighboring `name.style.json` or `name.geolibre.style.json` link is paired with the matching dataset. It also offers the map services the page's own maps requested: WMS, WMTS, WFS, OGC API - Features, ArcGIS feature services, XYZ/TMS image tiles, and vector tiles. On Source Cooperative repository pages, it also reads the embedded inventory so datasets outside the currently visible list can be selected.

Use a different GeoLibre instance

By default, selected datasets open on the hosted `web.geolibre.app`. To open them on a self-hosted GeoLibre instance instead, right-click the toolbar icon, choose **Options**, and enter its URL. Choose **Reset to default** to go back to the hosted instance.

Access and privacy

GeoLibre fetches selected links directly, so the source server must allow cross-origin requests (CORS). Complete HTTP(S) URLs, including signed query parameters, are forwarded to GeoLibre. Cookies and other browser-session credentials are not forwarded, so cookie-bound or session-authenticated links might fail. Temporary `blob:` URLs cannot be transferred.

The extension requests access only to the active tab, and only from the moment you click its icon. It holds no standing permission to any website, sends no analytics, and runs nothing in the background. The only thing it stores is the custom base URL described above, if you set one. Map services are recognized by reading back the addresses of the requests the page has already made, which the page records for itself, rather than by watching your browsing.

<https://geolibre.app/user-guide/layers/>

Managing Layers

The **Layers panel** on the left lists every layer in the project, from the topmost drawing layer down to the basemap. Selecting a layer here drives the [Style panel](#) and the [Attribute table](#).

The on-map **Layer Control** also lists project layers, including ones currently hidden or still loading, nested under the same [layer groups](#) as the left panel. Its checkboxes, opacity sliders, and group folders mirror the same settings as the left panel, and dragging a layer or using its right-click **Move** actions reorders the project's layers.

The Layers panel: a group, two vector layers with the selected one's action buttons expanded, and the basemap at the bottom

Layer order and visibility

- **Visibility:** click the eye button to show or hide a layer. The **Hide all layers** button at the top of the panel hides every layer at once.
- **Order:** drag the grip handle beside a layer's name onto another layer to reorder it, or use the move up and move down actions. Layers higher in the list draw on top. The basemap (**Background**) always stays at the bottom. Layer-handle drags stay inside the panel; dropping files from Finder into the desktop app remains a separate data-import action.
- **Opacity:** each layer has an opacity slider from 0 to 100 percent.

Hover tooltips

When at least one layer shows a [hover tooltip](#), the Layers panel (and the read-only viewer) shows a **Hover tooltips** checkbox. Untick it to pause every layer's hover tip while you explore the map or inspect attributes; tick it again to bring them back with the same per-layer choices and fields. This is a viewing switch only: it does not change or dirty the saved project, and it turns back on when a project opens.

Blend modes

The [Style panel](#) carries a **Blend** menu, above the symbology controls, that sets how a layer's colours combine with the map below it. Opacity dilutes a layer; a blend mode mixes it, so the layers underneath still read through at full saturation.

The classic use is shading: put a hillshade, a terrain raster, or a dark basemap under a thematic fill or an aerial image, set the top layer to **Multiply**, and the relief shows through the colour instead of being hidden by it. Lowering opacity instead would wash out both.

Mode	What it does	Typical use
Normal	Ordinary transparency — the default	Everything else
Multiply	Darkens: the two colours are multiplied	Colour or imagery over a hillshade; adding shadow
Screen	Lightens: the inverse of Multiply	Lifting a dark layer out of a dark basemap
Lighten	Keeps whichever colour is brighter	Overlaying bright features without darkening the map
Add	Sums the colours, clipping toward white	Glow effects, heat and density overlays

Blending applies to a layer's fills, outlines, points, markers, and raster tiles. **Labels are deliberately excluded** and always draw normally, so place names stay legible over a multiplied hillshade.

Overlapping points and 3D buildings blend twice

Polygons, lines, and raster tiles blend as a whole layer: two overlapping polygons in the same layer combine with the map beneath them, not with each other. Points and 3D extrusions cannot, because MapLibre provides no layer-level compositing step for them, so each symbol blends separately. Where two point symbols overlap on screen the overlap is blended twice and reads noticeably darker under Multiply than the rest of the symbol.

If that shows in your map, reduce the overlap (a smaller radius, clustering, or a scale-dependent minimum zoom), or convert the points to polygons.

A blend mode is saved with the project and travels with a copied style, but it is a GeoLibre rendering setting with no equivalent in the MapLibre or Mapbox style specification, so it is dropped when you export a layer's style to a style file, SLD, or QML.

Why these five modes

MapLibre draws every layer into one WebGL canvas and offers no per-layer blending API, so GeoLibre applies these modes inside the renderer using the GPU's fixed-function blend stage. That stage can express these five and no more. Photoshop-style modes such as Overlay, Colour Dodge, and Soft Light need to read the colours already on the canvas from inside a shader, which is not possible here without an extra full-frame copy on every draw. Darken and Subtract are not offered either: their GPU equations also act on the transparency channel, which would erase the map wherever the layer does not cover it. See [maplibre-gl-js#8073](#) for the upstream work that would widen this list.

Blending is applied while MapLibre draws a layer, so it reaches only the layers GeoLibre itself styles. The following are drawn or styled elsewhere, and so have no Blend menu:

- **3D Tiles, Gaussian splats, LiDAR point clouds, and deck.gl overlays**, which draw with their own WebGL renderer instead of MapLibre's.
- **Cloud-Optimized GeoTIFFs** added through **Add Data > Raster Layer**, for the same reason on their default rendering engine.

- **Vector layers added through Add Data > Vector Layer**, which do draw through MapLibre but are styled by their own control rather than by GeoLibre, so there is no point at which the blend mode could be applied.

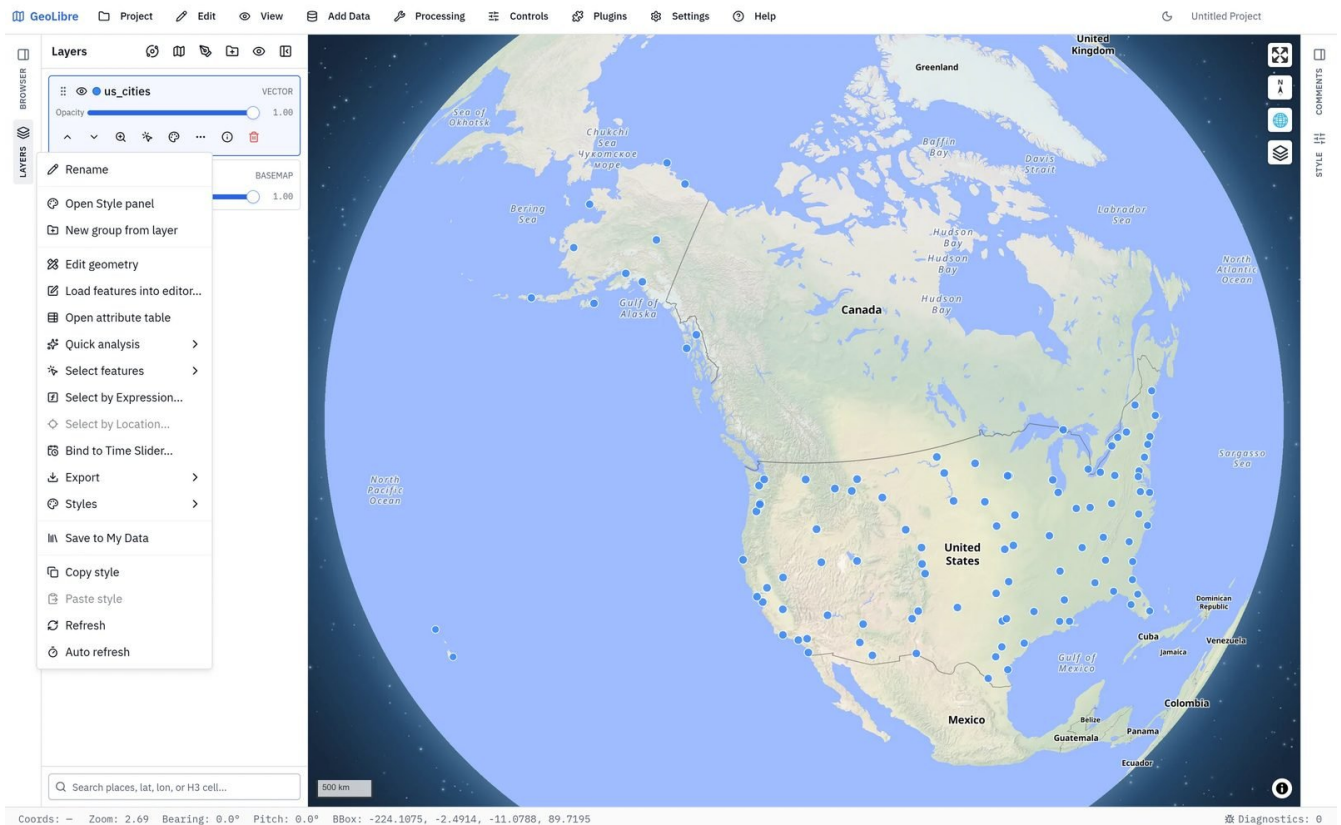
Everything else blends: local and remote GeoJSON, ordinary raster layers (XYZ, WMS, WMTS), PMTiles, MBTiles, and vector tiles.

Per-layer actions

Selecting a layer expands a row of icon buttons on its card:

Button	What it does
Move up / Move down	Shift the layer one position in the stack.
Zoom to layer	Fit the map to the layer's extent (for layers whose bounds are known).
Identify features	Click features on the map to see their attributes in a popup. On a raster layer this reads the pixel value instead, and on a multiband raster it also builds a spectral profile . On an editable vector layer, each result also offers Edit geometry (opens the GeoEditor with that feature selected) and Edit attributes (opens the attribute table in edit mode on that feature's row).
Open Style panel	Select the layer and open its styling controls .
Layer actions	The full menu, below.
Metadata	Inspect the layer's source and configuration, and edit its descriptive metadata (abstract, keywords, license, contact, lineage, temporal extent, and links). Export as STAC Item writes that metadata out as a STAC Item.
Remove layer	Delete the layer from the project.

The **Layer actions** menu (the  button) holds everything else:



Item	What it does
Rename	Change the layer's display name.
Open Style panel	Same as the palette button on the card.
New group from layer	Wrap this layer in a new group .
Edit geometry	Hand the layer to the GeoEditor for vertex-level editing.
Load features into editor...	Copy features from this layer into the GeoEditor's sketch layer.
Open attribute table	Show this layer's records in the Attribute table .
Quick analysis	Run a buffer, centroids, convex hull, or bounding box over the whole layer with no dialog. See Right-click quick actions .
Select features	The interactive selection modes: by click, rectangle, polygon, freehand, or radius, plus Clear Selection . Hold Shift to add, Alt to remove, Shift + Alt to intersect, and Esc to cancel.
Select by Expression... / Select by Location...	Build a selection from an attribute expression or a spatial relationship. Select by Expression can also apply the expression as a persistent layer filter, hiding non-matching features without creating a new layer. Both are also on the Edit menu .

Item	What it does
Bind to Time Slider...	Drive the Time Slider from one of this layer's date or number fields.
Export	Write the layer out as GeoJSON, GeoParquet, GeoPackage, KML, KMZ, zipped Shapefile, or CSV (attributes only). On a raster layer it offers GeoTIFF (COG) and Extract subset... , and on a LiDAR layer LAS, LAZ, or COPC. A polygon layer drawn as a 3D extrusion can also be exported as a 3D model (glTF <code>.glb</code> , OBJ, or STL) for Blender and other 3D tools. See 3D model export .
Styles	Import and export symbology — see below .
Save to My Data	Store the fully configured layer in your personal library, ready to re-add from the Browser panel in any later project.
Copy style / Paste style	Carry symbology from one layer to another.
Refresh / Auto refresh	Reload the source now, or on an interval — see Refreshing live layers .
Reload from disk / Watch file for changes	Desktop app, layers read from a local file: re-read the file now, or reload it whenever it changes on disk. These replace Refresh / Auto refresh on such layers.
Save edits to source file	Desktop app: write attribute and geometry edits back to the layer's GeoJSON or GeoPackage file. On an ArcGIS feature layer, a PostGIS table, or a SQL Server table it reads Save edits to ArcGIS service , Save edits to PostGIS table , or Save edits to SQL Server table .
Materialize to editable layer	DuckDB layers: copy the query result into an editable GeoJSON layer.
Edit query in SQL Workspace	SQL query layers: reopen the layer's query in the SQL Workspace .

Some entries are unavailable on layers they do not apply to: **Select by Location** needs a second layer to compare against, and **Paste style** needs a style on the clipboard.

Importing and exporting styles

Vector layers have a **Layer actions** → **Styles** submenu for symbology interchange:

- **Export GeoLibre URL style** writes a compact `.geolibre.style.json` file designed for the [data and style URL parameters](#). It contains no feature data. Its MapLibre render layers use the original GeoJSON filename stem as `source`, allowing one style document to distinguish GeoJSON members in a ZIP.
- **Export as Mapbox GL style** writes a self-contained Mapbox/MapLibre style with the layer's GeoJSON embedded.
- **Export as OGC SLD** and **Export as QGIS QML** produce styles for other desktop and server GIS software.

- **Import style from file (GeoLibre URL / Mapbox GL / SLD / QML)...** applies a supported style file to the selected layer. When importing a GeoLibre URL style interactively, the filename-based `source` association is ignored because the selected layer is the target.
- **Import style from text...** opens a box to paste a style into, for one copied out of QGIS, a gist, or a catalog page rather than saved to disk. It reads the same four formats, decided from the text rather than a file extension, and reports the same warnings. The Style panel header carries the same box, so you do not have to leave the panel to reach it.
- **Saved styles (Style Manager)...** applies a preset from your style library, or saves the current symbology into it. The same library is reachable from **Settings → Style Manager**.

To use a GeoLibre URL style, upload it to a CORS-enabled web host and open GeoLibre with both URLs:

```
https://web.geolibre.app/?data=https://assets.geolibre.app/data/places.geojson&style=https://assets.geolibre.app/data/sample.style.json
```

See [Embedding & Sharing](#) for GeoParquet and PMTiles deep links, ZIP source matching, REST API responses, raster-style JSON, and encoding nested URLs.

3D model export

When a polygon layer's **Visualization** is set to **3D extrusion** in the Style panel, **Layer actions → Export** also offers **3D model: glTF (.glb)**, **3D model: OBJ**, and **3D model: STL**. Each feature becomes a closed solid (floor, walls, and roof) with the height, base, and color the map draws. Categorized, graduated, and expression-based colors and heights come through as drawn.

- **glTF (.glb)** keeps one named object per feature with its attributes as custom properties, and one material per color. Blender opens it with **File → Import → glTF 2.0**.
- **OBJ** keeps one object per feature and stores colors on the vertices.
- **STL** writes one uncolored mesh, for 3D printing and CAD tools.

Coordinates are in meters around the center of the layer. glTF and OBJ use Y as the up axis, and STL uses Z. The glTF file stores the longitude and latitude of that center on its root object. Features whose height is not above the extrusion base, or that have no area, are skipped, and a notice says how many.

Layer groups

Groups are folders in the layer stack. They can nest, so a project can carry a real hierarchy rather than one flat list.

- **Create: New group** adds an empty folder. **New group from layer** wraps the layer you are on, and **New group from selected layers** wraps a multi-selection.
- **Fill: Move to group** moves one layer, **Move selected layers to group** moves a whole selection in one step (keeping their relative order), and **Add data to group** opens Add Data with the new layer targeted at that group.

- **Organize:** rename a group, collapse or expand it, move it up or down, and set a group-level opacity that applies to everything inside.
- **Sort:** **Sort A to Z** and **Sort Z to A** order a group's contents by name, top of the list first. Numbers sort naturally (Parcel 2 before Parcel 10) and case is ignored. Subgroups are sorted among themselves and move with everything inside them, while the group's own layers stay together as one block. Undo restores the previous order.
- **Visibility:** hiding a group hides its layers. A layer inside a hidden group is marked *Hidden because its group is not visible*, so you can tell it apart from a layer you turned off yourself.
- **Remove:** **Ungroup (keep layers)** dissolves the folder and leaves its layers in place; **Delete group and layers** removes both.
- **On-map Layer Control:** groups appear there as collapsible folders with their own checkbox and opacity slider, and toggling, fading, or collapsing a group in either place updates the other. A layer dragged or moved there stays inside its own group. A group's own layers always sit together, with its subgroups above or below them, so a move that would split them (one layer above a subgroup while its siblings stay below) is not applied and the layer returns to its place.

Groups and their nesting are saved with the project, and [importing a QGIS project](#) brings that project's group tree across.

Refreshing live layers

WFS and GeoJSON URL layers can refresh automatically so the map stays current with a changing source. Open the layer's refresh configuration and choose an interval (for example off, 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, or a custom value), or trigger a manual refresh.

Each reloadable layer persists a **connection** record with the project, so the refresh cadence survives a save and reopen. The record also carries the layer's synchronization status — when it last succeeded and the most recent error — which the Layers panel shows, and an on-failure policy that decides whether a failed refresh keeps the last good data or clears it. See [Project Format](#) for the schema.

DuckDB layers

Layers added from a [DuckDB source](#) or produced by the [SQL Workspace](#) support identify, selection, and the attribute table like any vector layer. You can also materialize a DuckDB query result into an editable GeoJSON layer when you want to edit its geometry or attributes.

The basemap

The **Background** entry at the bottom of the panel is the basemap. Toggle its visibility and adjust its opacity here. To change which basemap is shown, double-click the row to open the **Change background** dialog, or use the **Basemaps** plugin from the [Plugins menu](#). See [Adding Data](#).

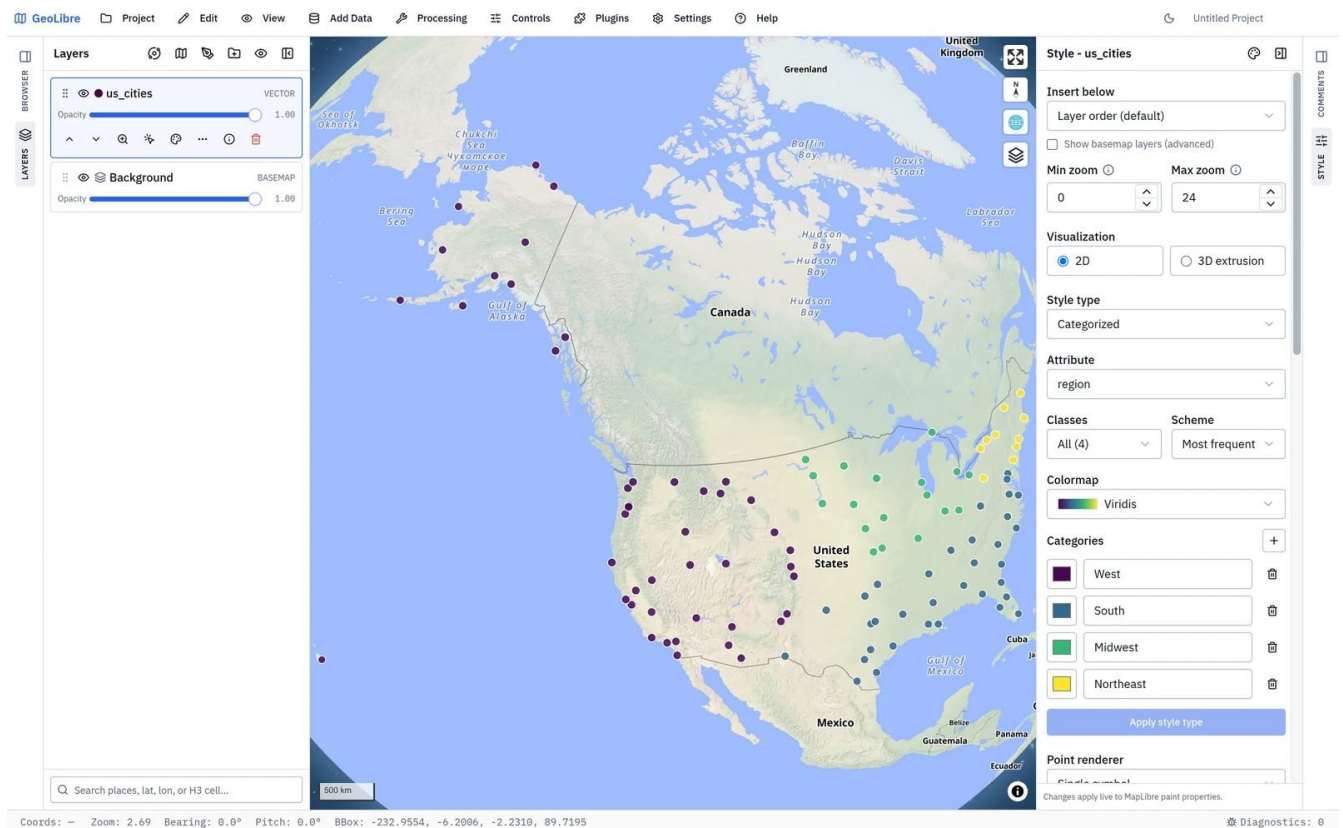
Editing geometry

To draw or edit features directly on the map, activate the **GeoEditor** plugin from the [Plugins menu](#). It adds drawing, vertex editing, and deletion tools for GeoJSON layers.

<https://geolibre.app/user-guide/styling/>

Styling Layers

The **Style panel** on the right edits the appearance of the layer selected in the **Layers panel**. Vector and raster layers each get their own set of controls.



Vector styling

For vector layers the Style panel covers fill, stroke, points, labels, and 3D extrusion:

- **Fill:** fill color and fill opacity for polygons.
- **Stroke:** line color and width for lines and polygon outlines.
- **Points:** circle radius for point layers, or **Use marker icon** to draw a marker instead: a built-in shape (circle, square, triangle, diamond, star, cross, or pin) with a color and size, or a **Custom image**. **Upload image...** accepts SVG, PNG, JPEG, or GIF; raster images draw as they are, so the marker color and per-class colors apply only to SVG markers.
- **Labels:** text color, size, halo color, and halo width.
- **3D extrusion:** turn polygons into extruded blocks, with a height field, height scale, base height, and color. Advanced expressions are available for both height and color.

You can also set a per-style minimum and maximum zoom so a style only applies within a zoom range.

Point renderer (heatmap and clustering)

For point-only GeoJSON layers — whether dropped on the map, produced by a tool, or loaded through **Add Vector Layer** in the geojson render mode — the Style panel adds a **Point renderer** control:

Renderer	Description
Single symbol	One circle per point (the default).
Heatmap	A density surface colored from cold to hot. Adjust Heatmap radius (the kernel size in pixels) and Heatmap intensity .
Clustered	Group nearby points into bubbles labeled with the count; zooming in splits them apart. Adjust the Cluster radius (in pixels) and the Cluster max zoom above which points stop clustering. Individual (unclustered) points keep the layer's circle style.

The renderer choice is saved with the project.

Style type (data-driven styling)

The **Style type** control chooses how feature values map to color:

Style type	Description
Single symbology	One uniform style for every feature.
Graduated	Classify a numeric attribute into classes and colour them from a ramp.
Categorized	Give each of an attribute's distinct values its own colour.
Rule-based	Build a list of rules, each with its own filter, symbol properties, and optional scale-dependent visibility.
Advanced expression	Drive styling with a custom MapLibre expression for full control.

Graduated and categorized styles share four more controls, and then list the generated stops so you can fine-tune an individual colour before applying:

- **Attribute** — the field the style reads.
- **Classes** — 2 to 12 classes for a graduated style; 1 to 12 for a categorized one, plus an **All (n)** option that gives every distinct value its own colour.
- **Scheme** — how the stops are chosen. Graduated offers **Equal interval**, **Quantile**, **Natural breaks**, **Standard deviation**, **Geometric interval**, and **Manual**; categorized offers **Most frequent**, **Alphabetical**, and **First values**.
- **Colormap** — the named colour ramp the classes are drawn from.

Nothing reaches the map until you click **Apply style type**, so you can adjust the classification and watch the stop list update first.

The graduated schemes differ in where they put the class breaks:

- **Standard deviation** centres classes one standard deviation wide on the mean, so it shows how far each feature is from average. An even class count puts a break on the mean; an odd count gives a middle class that straddles it. Breaks that would fall outside the data's range are dropped, so you can get fewer classes than you asked for. A diverging colormap suits it.
- **Geometric interval** grows the breaks by a constant ratio from the smallest positive value to the maximum, which suits skewed data spanning orders of magnitude (population, income, concentrations). Zero and negative values fall into the first class; if no value is positive, it classifies by equal interval instead.
- **Manual** is set for you as soon as you edit a class value, add a class, or remove one, so the method shown never claims to have produced breaks you typed. Choose it yourself to keep the current breaks while you edit them. Changing the class count, colormap, or attribute regenerates the breaks by equal interval.

The map legend names the method under a graduated layer's field, for example "Quantile, 5 classes", so a reader of an exported or shared map can tell how the classes were built. A project saved before the **Manual** scheme existed keeps the method it was classified with even if its breaks were edited by hand afterwards; reapply the classification, or pick **Manual**, to correct the caption.

Diagram symbology

Below the renderer controls, a **Diagram** control draws a small chart on each feature instead of (or as well as) a plain symbol: **Pie chart**, **Donut chart**, **Bar chart**, or **Stacked bar chart**, built from the numeric fields you pick. Set it to **None** to turn diagrams off. It is the usual way to show a composition — vote share, land-cover mix, age structure — per polygon or per point.

Quick filters

Below the renderer and label controls, the **Quick filters** section narrows what the layer draws without writing an expression. Pick a field from **Add a filter...** and GeoLibre reads the data to decide which control it deserves:

- a **Values** list of checkboxes with a count beside each value (with a search box once there are more than a handful),
- a **Range** slider plus typed minimum and maximum for a numeric field,
- a **Dates** from/to pair for a timestamp field, with both days included in full,
- a **Text** match — *contains*, *starts with*, or *is exactly* — which ignores case.

The **Filter type** dropdown beside a field switches between the controls it supports, so a numeric code column that opens as a range can be answered with checkboxes instead. Several filters on one layer narrow it together, and they combine with a Time Slider window, an embed `setFilter`, and rule-based symbology rather than replacing them.

A quick filter hides features; it does not select them. **Select by Expression** can either build a live selection or apply a persistent expression filter to the layer. The latter provides definition-query behavior without creating a copy of the matching features. A saved filter keeps the expression itself live but not the builder's `@` variables: `@map_zoom`, `@map_scale`, `@feature_count`, `@project_name`, and `@layer_name` are replaced by their values when you click **Filter layer**. Write `["zoom"]` rather than `@map_zoom` when the filter should keep responding to the camera.

A layer with an active filter shows a funnel icon on its row in the Layers panel, so a filtered layer is never mistaken for missing data, and its **Layer actions** → **Clear filters** empties every control at once while keeping the controls themselves. Clearing the last checkbox in a Values list is the same as clearing that filter: it shows every value, not none. **Remove all** at the top of the section deletes the controls outright.

Filters are saved with the project, so a shared map opens with the same view of the data. Quick Filter controls also appear in the read-only viewer chrome (`?layout=viewer`), so the person you shared it with can adjust those controls even though the authoring panels are hidden. A persistent expression filter has no control to offer there, but the layer still carries the funnel icon, so a viewer can see that features are being held back rather than missing.

Tile-backed layers (vector tiles, PMTiles, MBTiles) carry no local features, so their value lists are read from the tiles currently loaded and grow as you pan and zoom; the section says so under the controls.

One limit is worth knowing, and it now cuts two ways. MapLibre builds clusters at the source, before the renderer evaluates any filter. A persistent expression filter and Quick Filters are applied to a clustered layer's source data first, so its bubbles and counts do follow them. A Time Slider window, a rule-based filter, and an embed `setFilter` stay per-feature render filters, so while clustering is on the bubbles and their counts still describe data those have not narrowed. Switch the point renderer back to **Single symbol** to see the filtered features drawn individually, one circle each, with no cluster counts to misread.

Style interchange and URL styles

The selected vector layer's **Layer actions** → **Styles** submenu imports and exports GeoLibre URL style JSON, Mapbox/MapLibre style JSON, OGC SLD, and QGIS QML. A GeoLibre URL style is a compact MapLibre style whose feature data is supplied separately through the `data` URL parameter. Export one when you want to publish the current symbology beside hosted GeoJSON or a ZIP of GeoJSON files; import it when you want to apply that symbology to a layer already open in GeoLibre.

See [Managing Layers](#) for the menu workflow and [Embedding & Sharing](#) for the JSON conventions.

To restyle many layers at once, **Project** → **Export** → **Export Layer Styles...** writes every layer's style to one file keyed by layer name, and **Project** → **Import** → **Import Layer Styles...** applies it to the layers whose names match. See [Projects](#).

Choropleth maps

To make a choropleth, select **Graduated**, pick a numeric attribute, choose a colormap, and click **Apply style type**. See the [Your First Map tutorial](#).

Popups and hover tooltips

Further down the Style panel, the **Popup** section designs what a viewer sees when they interact with a feature, rather than the raw column dump the Identify popup shows by default. It matters most for a map you share: a `layout=viewer` embed, a shared project, or a story-map chapter, where the recipient clicks a feature and would otherwise get join artifacts, editor-tracking columns, and internal ids alongside the fields you meant them to read.

- **Show popup on click (Identify)** and **Show tooltip on hover** are independent switches. The click popup is on by default; the hover tooltip is off.
- **Title field** leads the popup with the feature's own name rather than the layer's. **Title expression** does the same from an expression, and wins over the field.
- **Body expression** replaces the whole popup body — the field rows and the feature id row with them — with a sentence built from the feature's properties, for authors who want prose rather than a table. Both boxes open the same expression builder that backs [data-driven styling](#), labels, and filters.
- **Fields** chooses which attributes appear and in what order. Add a field from the picker, drag its handle (or use the arrow buttons) to reorder, and give it a **Label** to show instead of the raw column name. Leave the list empty — or remove every field you added — to keep the default: every visible field, in the data's own order.
- **Value type** formats the value: **Number** (decimals, thousands separator), **Date** (date, date and time, time, ISO 8601, or year), **Link**, **Image**, or plain **Text**. For **Text**, **Number** and **Date** fields, **Prefix** and **Suffix** wrap the result, which is where a currency symbol or a unit goes.
- **In hover tooltip** puts a field in the tooltip's short subset. Keep it to one or two fields: the tip follows the pointer, so it is a glance, not a table. Image fields are skipped there and stay in the click popup, where there is room to show the picture.

Layers loaded through **Add Vector Layer** are designed here too, once the panel has handed them over to GeoLibre, which it does for every layer it loads in GeoJSON mode with up to 100,000 features (see [Adding data](#)). A layer the panel keeps, such as a tiled or streamed one, has no features in GeoLibre for the designer to read, so the Popup section does not appear for it. The panel's own **Popup** checkbox opens a separate, unstyled attribute table on click; it is off by default so the two do not both answer the same click. Tick it only if you want the control's raw table back.

Fields you hid or excluded from the [attribute table](#) stay out of the popup: the popup design selects from the visible fields and cannot re-expose a hidden one. A layer with nothing configured behaves exactly as before, and the whole design saves with the project, so it travels to the Python package and the MCP authoring tools as well.

Raster styling

For raster layers the Style panel exposes image adjustments:

- **Brightness** (minimum and maximum)
- **Saturation**
- **Contrast**
- **Hue Rotate** (in degrees)

These let you tune the look of GeoTIFF, COG, and tile-based raster layers without changing the underlying data.

A WMS layer also gets a **Legend** section. **Get legend** shows the service's legend image, from the capabilities `LegendURL` when one is advertised or a `GetLegendGraphic` request otherwise; if the service has none, you can enter a legend image URL instead. **Add to map** places the legend on the map in an HTML control.

Raster symbology and histogram stretch

GeoTIFF and COG layers add a **Raster symbology** section with a **Render mode**:

- **Single band (pseudocolor)** draws one **Band** through a colormap, which can be reversed or replaced with your own colors, and optionally **Classify into discrete classes** (equal interval, quantile, or manual breaks).
- **RGB composite** maps three bands to red, green, and blue.
- **Index (normalized difference)** computes an index such as NDVI from two bands.

Min and **Max** set the stretch range (left empty, they show `auto`), and **Stretch** (**Linear**, **Logarithmic**, or **Square root**) and **Gamma** shape it. **No data** reads the nodata value from the file, renders every pixel, or uses a custom value.

Under **Histogram**, a single-band (unclassified) or RGB raster shows the distribution of its pixel values with the active stretch window shaded between two handles:

- **Drag a handle** to set the minimum or maximum; the **Min/Max** inputs follow, and typing a value moves the handle.
- From the keyboard, focus a handle and use the arrow keys, with `Shift` or `Page Up / Page Down` for steps ten times larger and `Home / End` for the ends.
- An **RGB composite** shows one histogram per channel, each with its own **Min** and **Max**. Editing one channel pins the others at their current automatic range so they do not jump.
- With no range set, the automatic window is the 2nd to 98th percentile of the band.

For a single-band or index raster that is not classified, **Viewport stretch** fits the range to the pixels currently on screen instead: **Viewport min / max**, **Viewport 5–95 percentile**, or **Viewport mean $\pm$ 2 standard deviations**, then **Apply to viewport**. Tick **Update automatically when the map moves** to keep refitting as you pan and zoom. The stretch is saved with the layer.

Spectral profile

For a **multiband** raster — a stacked Landsat or Sentinel scene, a NetCDF/HDF cube, or any COG with more than one band — the Style panel adds a **Spectral profile** chart of one pixel's value across every band.

Turn on **Identify** for the layer in the [Layers panel](#), then click the map. Each click adds a numbered dot on the map and a matching curve in the chart, so you can click water, vegetation, and asphalt and compare the three responses side by side. Up to six points are compared at once; older points age off as you add more.

- The chart plots against **wavelength** when the file declares one wavelength per band, and against **band number** otherwise. A wavelength list that doesn't match the band count is ignored rather than trusted, so a stale one can't mislabel the axis.
- Only the tile containing the clicked pixel is fetched, not the whole scene, so profiling a large remote COG stays fast.
- **Pop out** floats the chart in a draggable, resizable window over the map; **PNG** and **CSV** export it.

Single-band rasters produce no profile — use Identify on its own to read the pixel value. Clicks outside the raster, or on a pixel that is nodata in every band, are dropped without disturbing the points you have already collected.

A remote GeoJSON, GeoParquet, or vector PMTiles layer opened with the `data` URL parameter can receive a GeoLibre/MapLibre vector style through `style`. A remote COG can receive a raster style instead. Raster URL styles support band mode and selection, rescale ranges, colormap and reversal, nodata, opacity, gamma, stretch, and normalized-difference index presets. See the [remote data examples](#).

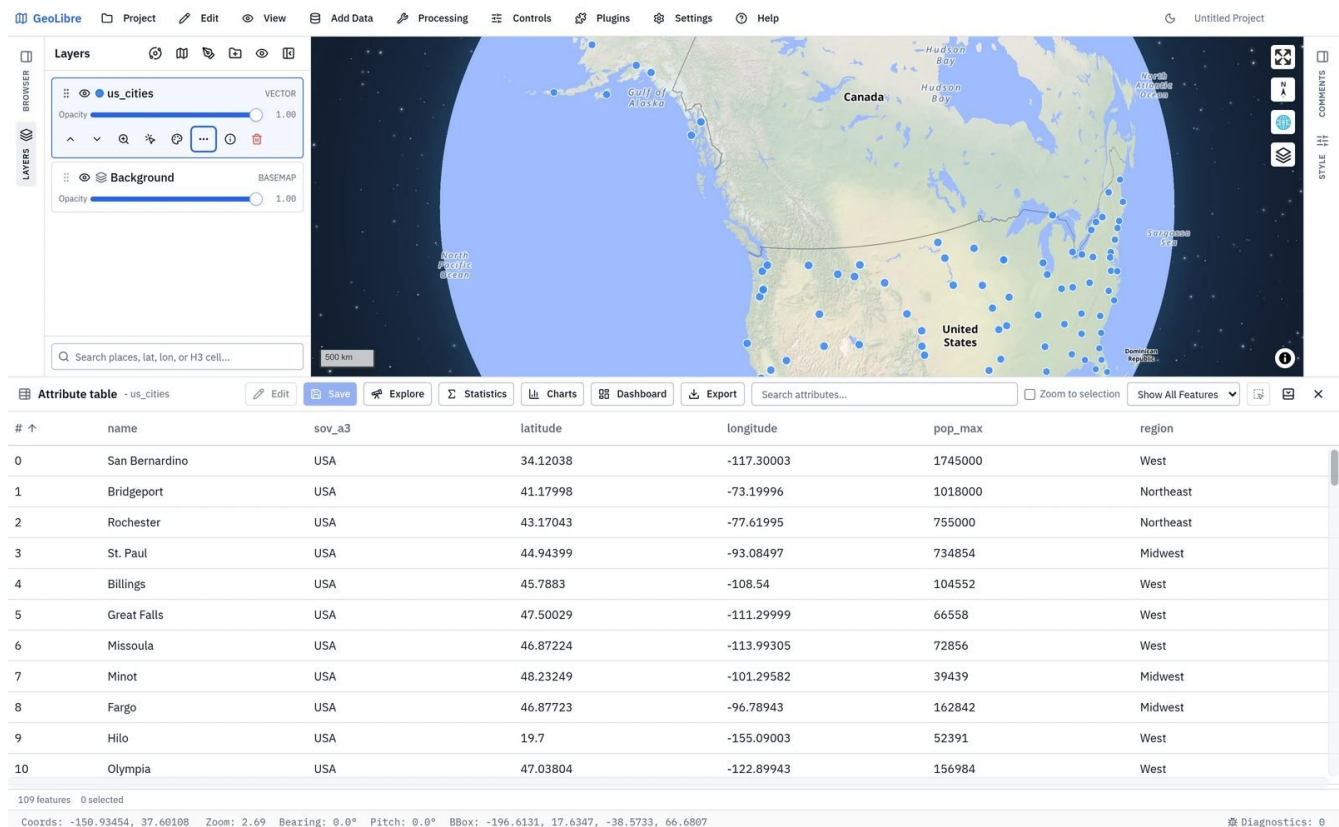
Legends and colorbars

To display a legend or a continuous colorbar on the map, open them from the [Controls menu](#). They reflect the styling you set here.

<https://geolibre.app/user-guide/attribute-table/>

Attribute Table

The **Attribute table** shows the records of a vector or DuckDB layer. Open it from the layer's actions menu in the **Layers panel** with **Open attribute table**. It is available for GeoJSON layers (including files loaded from disk and layers produced by processing tools), CZML layers, **Add Data → Vector Layer** layers, and DuckDB query layers.



The screenshot shows the GeoLibre interface with a map of North America. The 'Layers' panel on the left shows 'us_cities' (VECTOR) and 'Background' (BASEMAP). The 'Attribute table' is docked at the bottom, displaying the following data:

#	name	sov_a3	latitude	longitude	pop_max	region
0	San Bernardino	USA	34.12038	-117.30003	1745000	West
1	Bridgeport	USA	41.17998	-73.19996	1018000	Northeast
2	Rochester	USA	43.17043	-77.61995	755000	Northeast
3	St. Paul	USA	44.94399	-93.08497	734854	Midwest
4	Billings	USA	45.7883	-108.54	104552	West
5	Great Falls	USA	47.50029	-111.29999	66558	West
6	Missoula	USA	46.87224	-113.99305	72856	West
7	Minot	USA	48.23249	-101.29582	39439	Midwest
8	Fargo	USA	46.87723	-96.78943	162842	Midwest
9	Hilo	USA	19.7	-155.09003	52391	West
10	Olympia	USA	47.03804	-122.89943	156984	West

Below the table, it indicates '109 features 0 selected' and provides coordinates: 'Coords: -150.93454, 37.60108'. The toolbar at the bottom includes buttons for Edit, Save, Explore, Statistics, Charts, Dashboard, and Export.

The table docks along the bottom of the window. Drag its top edge to resize it, or use the buttons at the end of the toolbar to collapse, expand, or close it. Rows are virtualized, so large layers scroll smoothly; there is no paging and no row limit, but sorting, searching, and the dialogs work over every row in memory.

The toolbar

Button	What it does
Edit / Save	Turn on inline editing, then commit the changes. See Editing values .
Add field	Add a field to the layer. See Managing fields .
Calculate	Open the Field calculator .
Fields	Show or hide columns (Show fields , Show all fields).
Explore	Open the Column explorer .

Button	What it does
Statistics	Summary statistics for one field. See Field statistics .
Charts	Chart one or two fields. See Charts .
Dashboard	Open the Dashboard with this layer preselected.
Refresh	Re-read the features from the source to pick up changes made elsewhere, such as edits by other clients of a service. Shown for layers that can be refreshed ; on desktop, a layer read from a local file is reloaded from disk. Disabled while an ArcGIS layer has unsaved edits.
Export	Write the layer to a file. See Exporting .
Search attributes...	Show only rows whose values (or id) contain the text, ignoring case.
Zoom to selection	Zoom the map to each new selection.
Show All Features / Show Selected (n)	Show every record, or only the selected ones.
Clear selection	Deselect everything.

Add field, **Calculate**, and **Fields** appear for editable GeoJSON layers when you are not editing. **Explore**, **Statistics**, **Charts**, and **Dashboard** are hidden while editing. On a narrow screen the buttons show only their icons.

The footer reports the number of features, how many are shown when a search or **Show Selected** narrows the list, and how many are selected.

Reading and navigating

- **Sort** by clicking a column header: ascending, then descending.
- **Resize** a column by dragging its border. Column widths are not saved.
- **Open links**: a cell that holds a URL shows a link that opens in your browser.

Selecting features

The table and the map share one selection:

- Click a row to select it, **Ctrl / Cmd** + click to add or remove a row, **Shift** + click to select a range, and **Shift** + **Ctrl / Cmd** + click to add a range.
- Selected rows are highlighted on the map, and a feature picked on the map scrolls the table to its row.
- **Zoom to selection** keeps the map framed on the selection as it changes, and **Show Selected (n)** narrows the table to it.

For selections by rule rather than by hand, use the **Edit** menu (also on the layer's actions menu):

Command	What it does
Select by Expression...	Select features matching an expression, built by hand or with Expression builder... Modify current selection by creates a new selection, adds to it, removes from it, or selects within it. The same dialog can instead Filter layer to save the expression as the layer's filter.
Select by Location...	Select features that intersect, are within, contain, or are disjoint from the features of another layer.
Invert Selection / Clear Selection	Flip or empty the selection.
Zoom to Selection	Fit the map to the selected features.
Export Selected Features as Layer	Copy the selection into a new layer.

The layer's actions menu also has **Select features**, which draws a selection on the map by click, rectangle, polygon, freehand, or radius.

Managing fields

Each column header has a ... menu with:

- **Rename field:** edit the name in the header; **Enter** saves and **Esc** cancels. Styles and labels that use the field follow the new name.
- **Hide field:** hide the column (bring it back from **Fields**).
- **Exclude field on export / Include field on export:** keep the column in the table but leave it out of exported files.
- **Move left / Move right:** reorder the columns.
- **Delete field:** remove the field from every feature, after a confirmation. This cannot be undone, and it clears any style or label that used the field.

Add field asks for a **Field name**, a **Type** (Text, Number, or Boolean), and a **Default value** for the existing features. A field's type cannot be changed after it is created; calculate a new field instead.

Hidden fields, column order, and export exclusions are saved with the project. Field management is not available for DuckDB layers or Add Vector Layer layers, which are read-only here.

Calculated fields

GeoLibre has two kinds of calculated field.

Field calculator

Calculate writes values into a field once, like QGIS's field calculator. The expression itself is not kept.

1. Under **Target field**, choose **Update field** (an existing field) or **Create field** (type a new name), and an **Output type: Auto (keep computed type)**, Text, Number, or Boolean.
2. Write the **Expression**. Refer to a field by its name, or as `props["field name"]` when the name has spaces or punctuation; click a field or function chip to insert it. `$index` is the row number, and `PI` and `E` are available.
3. Check the live **Preview**; optionally tick **Only update the selected features**, and press **Calculate**.

Available functions: `abs`, `ceil`, `floor`, `sqrt`, `exp`, `ln`, `log10`, `pow`, `min`, `max`, `round(x, digits)`, `toNumber`, `toString`, `upper`, `lower`, `trim`, `length`, `concat`, `substr`, `replace`, `isNull`, `coalesce`, and `iif(condition, then, else)`. Geometry measures are `$length(unit)`, `$perimeter(unit)`, and `$area(unit)`; the **Geometry**: row inserts them with a unit for you, offering only the measures that fit the layer's geometry. Lengths take `meters` (the default), `kilometers`, `miles`, `feet`, `yards`, or `nautical-miles`; areas take `square-meters` (the default), `square-kilometers`, `square-miles`, `hectares`, `acres`, or `square-feet`.

```
round($area("square-kilometers"), 2)
iif(pop > 100000, "city", "town")
upper(concat(name, " (" , state, ")"))
```

A row whose expression fails is written as null, and a message says how many did.

Virtual fields

A virtual field is recomputed from an expression whenever the data changes, like a QGIS virtual field. Add one in the layer's **Style** panel under **Virtual fields** → **Add virtual field**, with a **Field name** and an **Expression** in MapLibre expression syntax, for example `["/", ["get", "pop"], ["get", "area_km2"]]`. **Open builder** opens the Expression Builder.

Virtual fields are saved with the project and can be switched off without being deleted (**Apply this virtual field**). In the table they are read-only and shown in italics with a function icon; you can style, label, and filter by them like any other field.

Joins

A join adds the fields of another table to a layer by matching a key, like a QGIS table join. Joins live in the layer's **Style** panel under **Joins**:

1. **Add join**, then choose the **Join layer**: any other layer with features (its geometry is ignored). To join a spreadsheet, use **Choose file** to load a CSV, TSV, TXT, or Excel file (with **Worksheet** and **Delimiter** options) as a table layer first.
2. Pick the **Join field** in the join layer and the **Target field** in this layer.

3. Choose the **Fields to join** (all but the key by default) and an optional **Field name prefix**, such as `census_`.

4. **Add**.

How a join behaves:

- It is a left join: every feature is kept, and features without a match get null in the joined fields. Each join reports how many features matched and how many join rows went unmatched.
- Keys are compared as text, so `5` matches `"5"`. Empty keys never match.
- It is one-to-one: when several join rows share a key, the first wins.
- A joined field whose name already exists on the layer is skipped; use a prefix to avoid clashes.
- The join is live: it re-runs when either layer changes, and it is saved with the project. **Apply this join** switches it off without removing it.

Joined fields are read-only in the attribute table; edit the source table instead. For a permanent, one-off join (including an inner join), use **Processing → Vector → Join → Attribute join**, which writes a new layer.

Attribute forms

An attribute form controls how each field is edited, like QGIS's attributes form. Set it up in the layer's **Style** panel under **Attributes Form**, with **Configure field** for each field:

Setting	Effect
Edit widget	Text, Number, Range, Checkbox, Date, or Value map (a dropdown).
Alias	A friendlier label for the field in forms.
Required	The field must have a value.
Value map entries	One per line, as <code>value</code> or <code>value=Label</code> .
Min, Max, Step	Bounds for Number and Range widgets (Step for Range).
Constraint expression / Constraint description	A MapLibre expression the value must satisfy, and the message shown when it does not.
Visibility expression	Show the field only when the expression is true.

When you edit the table, value-map fields become dropdowns, checkbox fields become checkboxes, and number, range, and date fields get typed inputs. A cell that breaks a rule turns red with the reason (for example "A value is required"), and **Save** is blocked until it is fixed. Only problems your own edits introduce are flagged, so existing data that breaks a rule does not stop you saving other changes.

The same form drives [Field Collection](#), so a layer set up here captures observations with the same widgets and rules. Aliases are used in forms, not as table headers. Attribute forms are not available for DuckDB layers.

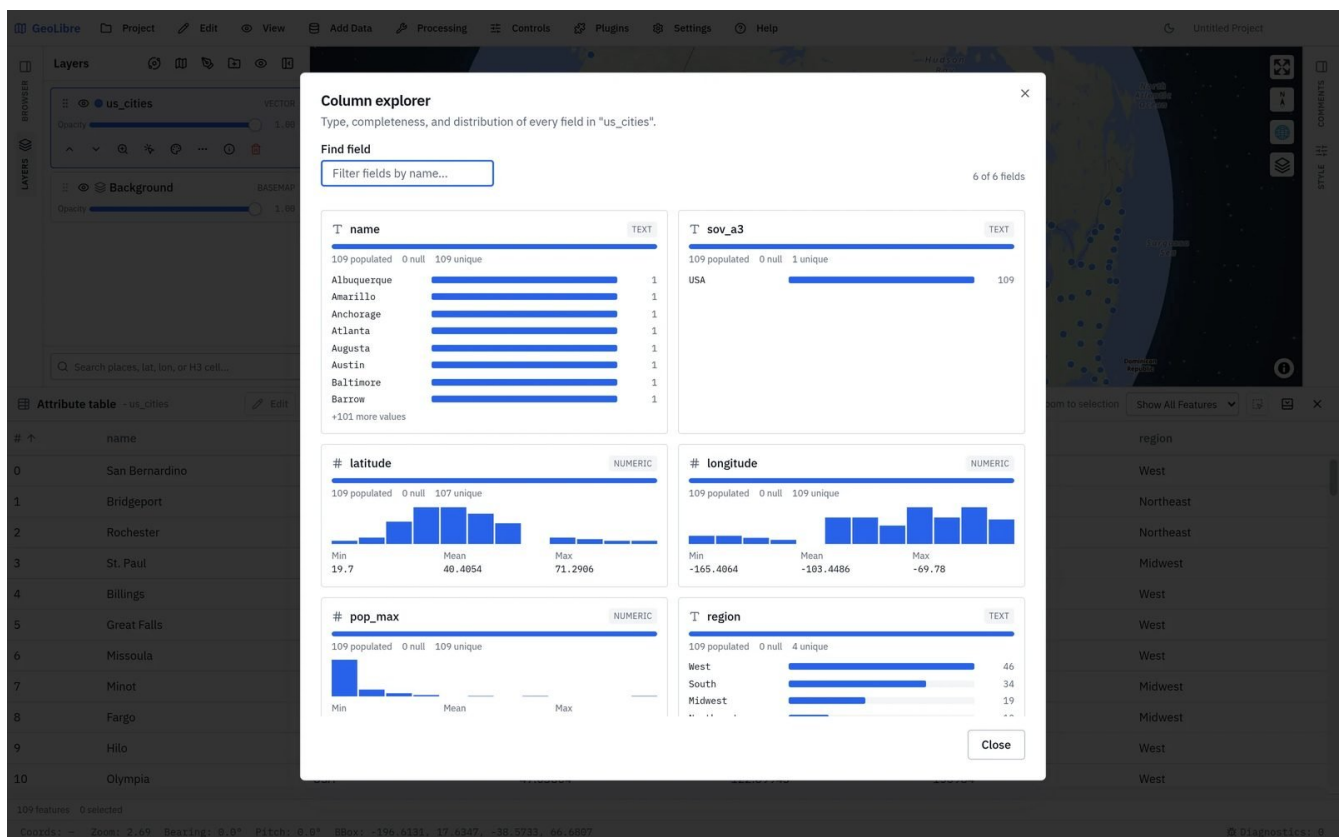
Quick filters

Quick filters are point-and-click filters on the map, saved with the layer. Add them in the layer's **Style** panel under **Quick filters** → **Add a filter...**, one per field. Depending on the field, a filter is a list of **Values** with counts, a numeric **Range**, a span of **Dates**, or a **Text** match (contains, starts with, or is exactly). See [Quick filters](#) for the details.

Quick filters hide features on the map only: they do not narrow the attribute table and do not change the selection. To filter the table, use **Search attributes...** or **Show Selected**.

Column explorer

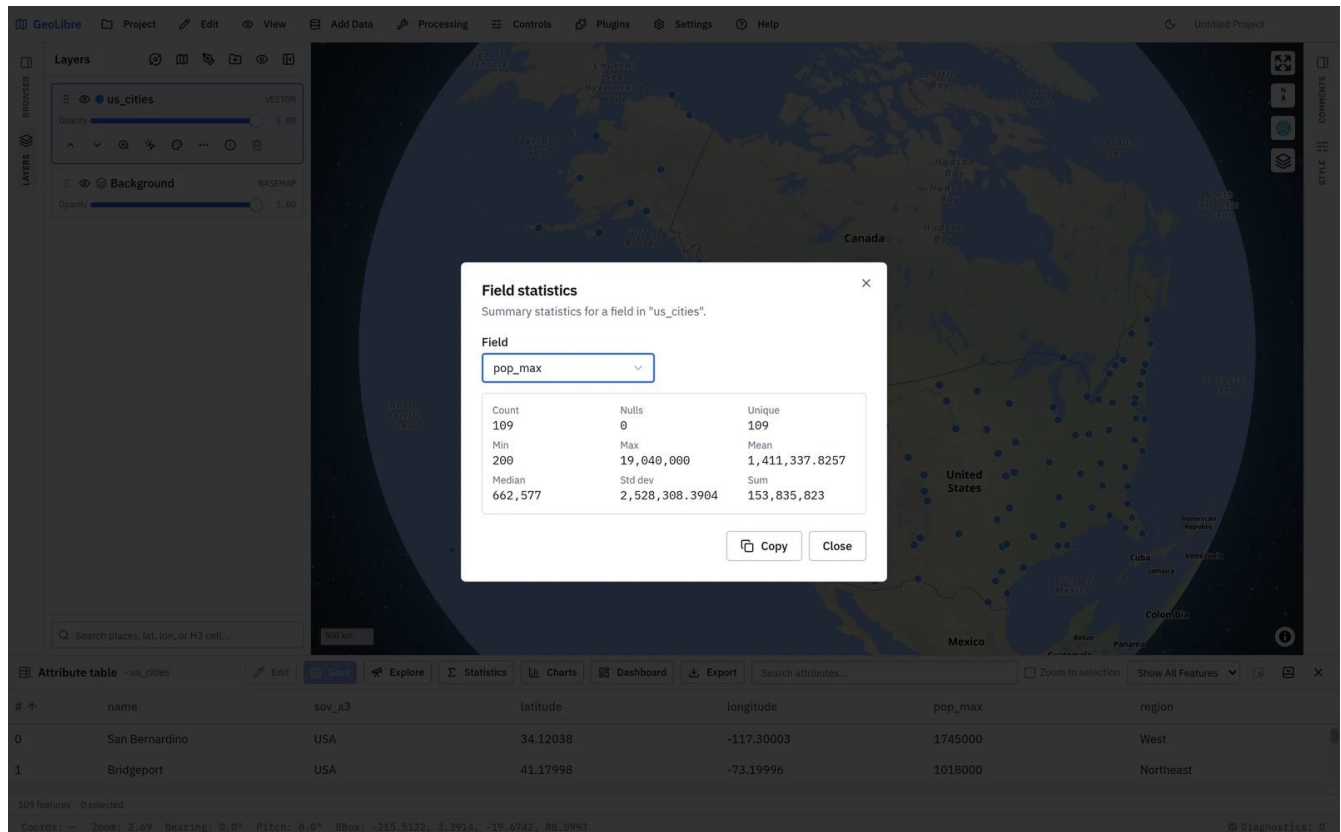
Explore profiles every field in the layer at once: its type, how many records are populated, how many are null, how many distinct values it holds, and the shape of its distribution — a ranked bar chart of the commonest values for text fields, a histogram with min, mean, and max for numeric ones. Filter the field list by name, and set the **Scope** to all features or only the filtered ones.



It is the fastest way to answer "what is actually in this data?" before styling or filtering it — a field that is 90% null, or a "numeric" field with one non-numeric outlier, shows up immediately.

Field statistics

Statistics summarizes one **Field** over a **Scope** of all, filtered, or selected features. Every field reports **Count**, **Nulls**, and **Unique**; a numeric field adds **Non-numeric**, **Min**, **Max**, **Mean**, **Median**, **Std dev**, and **Sum**, while a text field lists its five most frequent values with their counts. **Copy** puts the whole summary on the clipboard.

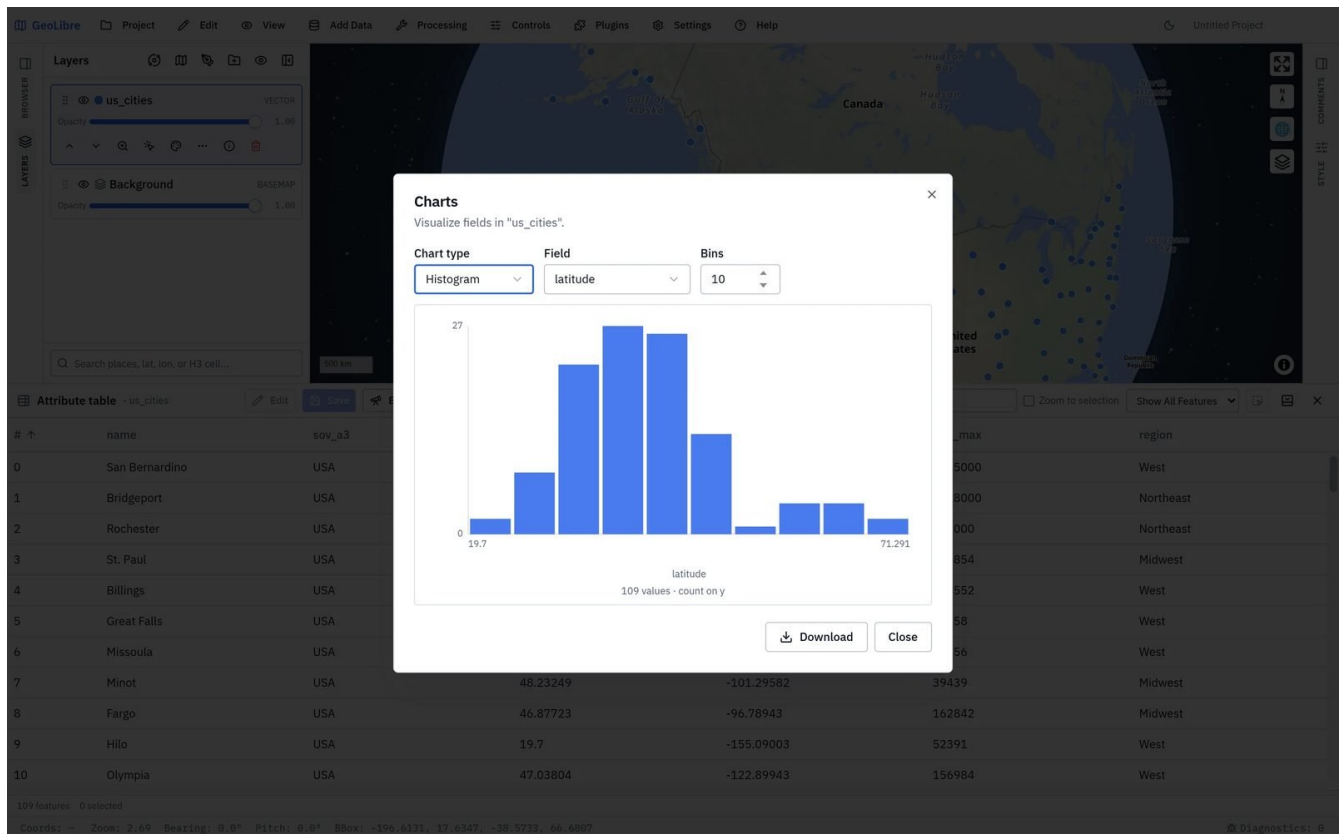


Charts

Charts plots the layer without adding anything to the project:

Chart	Options
Histogram	A number field and Bins (1 to 50).
Scatter	An X axis and Y axis field; up to 2,000 points.
Bar	A Category and an Aggregate (Count, Sum, or Average); up to 20 bars.
Line	A number field against row order.
Box plot	A number field.
Pie	A Category and an Aggregate (Count or Sum); up to 8 slices, the rest grouped as "(other)".

Charts use all features. **Download** saves the chart as a PNG image or an SVG.



Charts here are throwaway views. For charts that are saved with the project and cross-filter each other, build them in the [Dashboard](#) instead.

Editing values

Click **Edit** to turn the cells into inputs. Changed cells are highlighted; **Save** commits them and the reset button discards every unsaved edit. A cell that holds an object must contain valid JSON before you can save.

- Joined, virtual, and editor-tracking fields are read-only.
- **Edit** is unavailable while you are editing geometry, and for Add Vector Layer layers.
- **Ctrl / Cmd + Z** undoes saved layer changes (not while you are typing in a cell).
- To write edits back to the file, service, or database the layer came from, use the layer's **Save edits to source file** (or the ArcGIS, PostGIS, and SQL Server equivalents).

Combine this with the **GeoEditor** plugin to edit geometry and attributes together. See [Managing Layers](#).

DuckDB layers

Layers produced by the [SQL Workspace](#) or added from a [DuckDB source](#) behave like vector layers here, with identify, selection, sorting, search, statistics, and charts. Edits change only the rows held in memory, and **Add field**, **Calculate**, **Fields**, the column menus, attribute forms, and **Export** are not available for them. Export a DuckDB result from the SQL Workspace instead.

Exporting

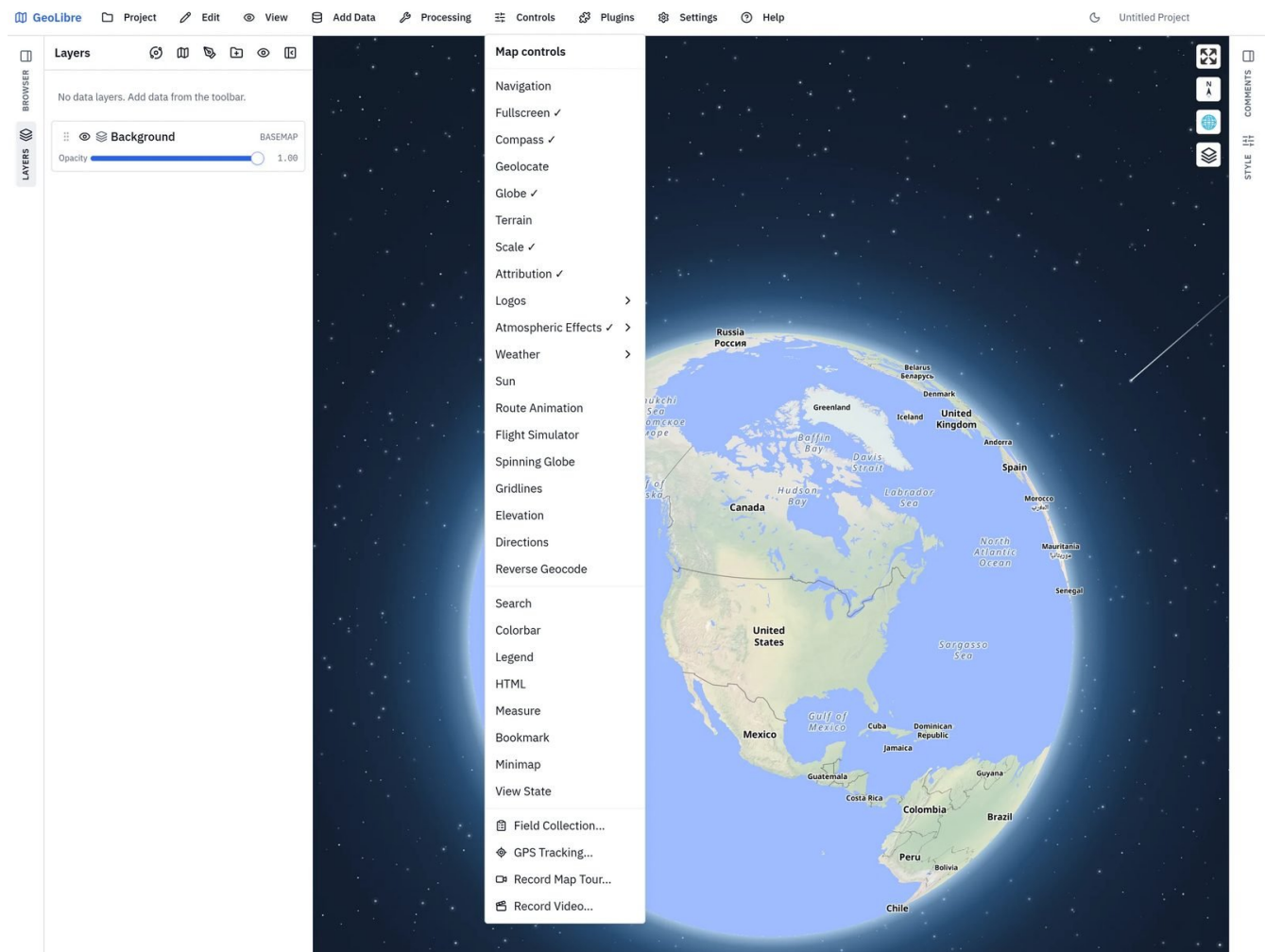
Export writes the layer, including unsaved edits, as GeoJSON, GeoParquet, GeoPackage, KML, KMZ, a zipped Shapefile, or CSV (attributes only). Line layers also export as an encoded polyline (precision 5 or 6). Fields marked **Exclude field on export** are left out, and a Shapefile export warns when field names are cut to the format's 10-character limit.

The same formats are available from **Layer actions** → **Export** in the [Layers panel](#). The [SQL Workspace](#) exports query results as CSV or GeoParquet, and the [Conversion tools](#) write cloud-native formats.

<https://geolibre.app/user-guide/map-controls/>

Map Controls & Tools

The **Controls** menu toggles two kinds of on-map helpers: the built-in MapLibre map controls, and the component panels that add tools like Measure and Bookmark. A check mark next to an item means it is currently shown.



Map controls

These are the standard MapLibre controls that sit in the map corners:

Control	Description
Navigation	The zoom in / zoom out buttons.
Compass	A north arrow that resets the bearing to north when clicked.
Fullscreen	Expand the map to fill the screen.
Geolocate	Center the map on your current location.

Control	Description
Globe	Switch between the flat map and a 3D globe projection.
Terrain	Toggle terrain (3D elevation) rendering. Double-click the control to set the vertical exaggeration.
Scale	Show a scale bar, in the units set by Scale bar units .
Elevation	Report the ground elevation under the pointer in the status bar. Off by default — see Elevation readout .
Attribution	Show data attributions.
Logos	Show or hide the MapLibre and provider logos.

Component tools

These are interactive panels provided by the MapLibre components plugin:

Tool	Description
Search	Search for places by name and fly to the result.
Colorbar	Display a continuous color scale for raster values.
Legend	Show a legend describing the layers on the map.
HTML	Display custom HTML content in an on-map panel.
Image	Place images from <code>https://</code> URLs on the map, such as a logo, a north arrow, or a legend, in any corner. Each image has a title bar that folds it away, and a map can hold up to 20.
Measure	Measure distances and areas interactively, with heading and terrain-aware 3D readouts. See Measuring distance, area, and heading .
Bookmark	Save named map views and jump back to them. Bookmarks and their folders are saved in the project, so they travel with the file (bookmarks saved by older versions in this browser are copied into the first project you open that has none and saved there; the old browser copy is left in place but no longer updated).
Mini map	Show an overview map of the current extent.
ViewState	Read and edit the exact center, zoom, bearing, and pitch.

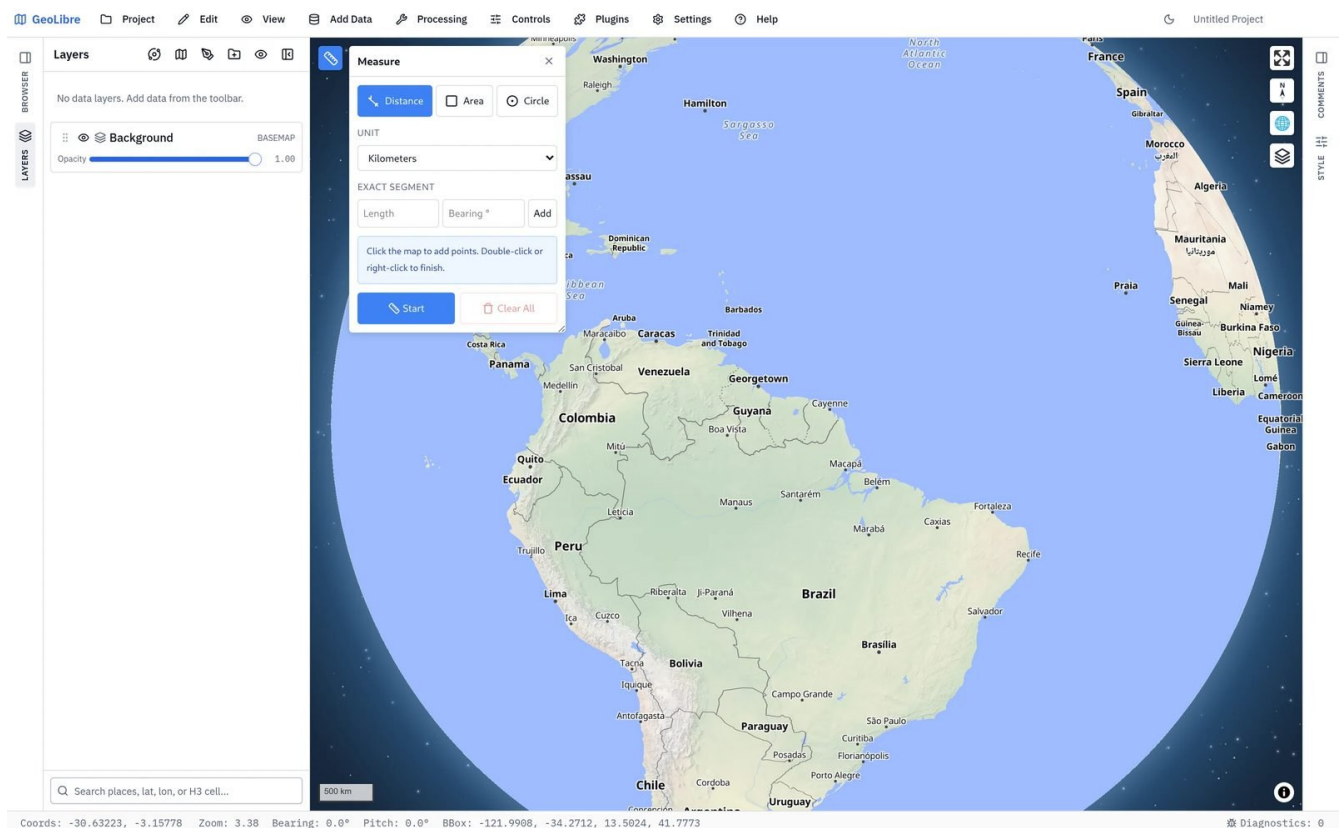
The **Print Layout** composer lives under the Project menu; see [Print Layout](#).

Control position

Plugin-backed controls can be positioned in any map corner. For plugins that support it, set the corner from the [Plugins menu](#) (top left, top right, bottom left, or bottom right).

Measuring distance, area, and heading

Controls → **Measure** opens a panel with three modes — **Distance** (a multi-segment line), **Area** (a polygon), and **Circle** (a radius from a center point) — plus a **Unit** picker. Click **Start**, then click the map to add points and double-click or right-click to finish; **Clear All** removes the measurements.



Exact segment is the keyboard route to the same thing: type a length and a bearing and click **Add** to extend the measurement by a segment of exactly those dimensions, instead of clicking for it. Useful for retracing a deed description or a survey traverse.

Beyond distance and area, the panel adds further readouts:

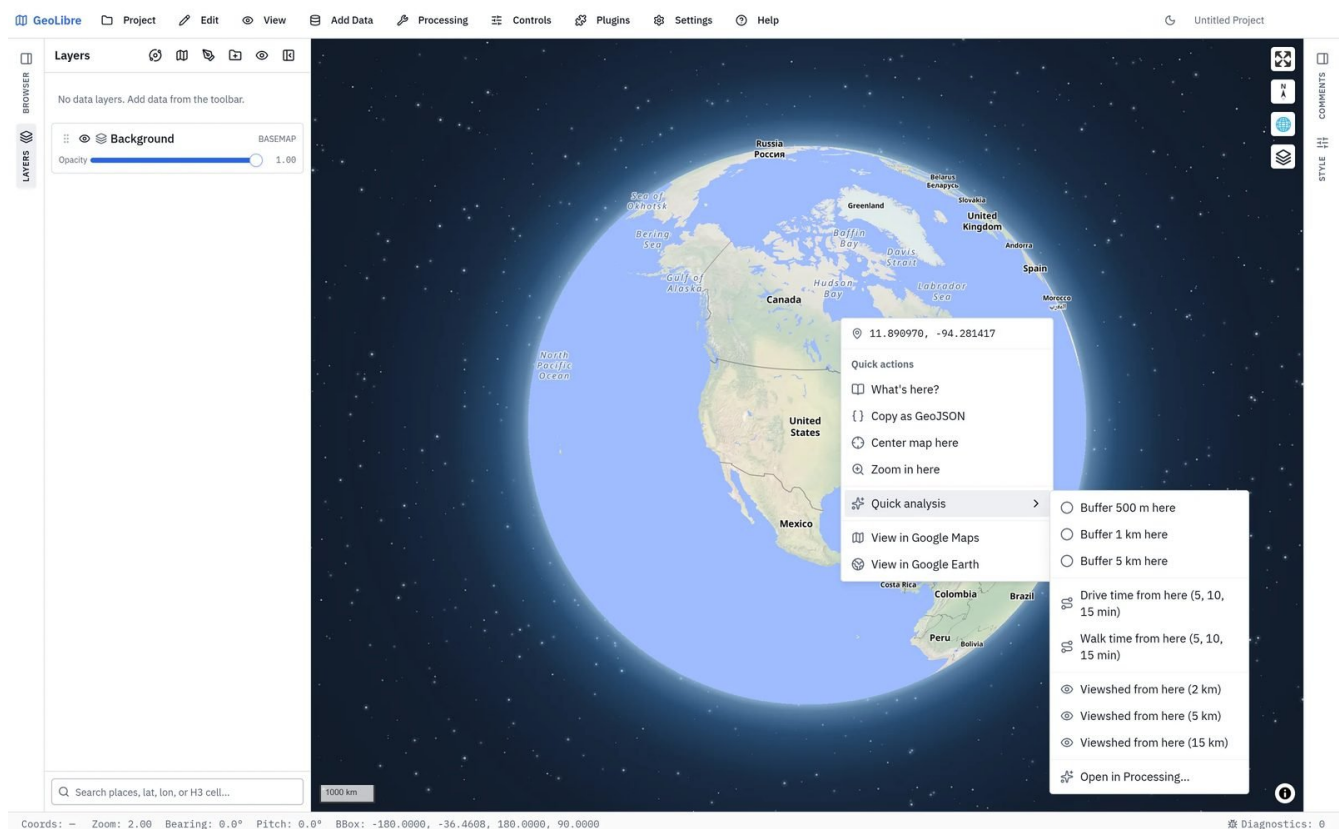
- **Heading** — the direction of the measured line, as degrees plus a 16-point compass label (for example **310° NW**). This is a true great-circle initial bearing, not the angle the line makes on screen: on a Mercator map Washington to London *looks* due east, while the real initial bearing is about 50°. On a path with more than two points, the heading is the overall first-to-last direction rather than one row per segment.

- **Final heading** — the bearing arriving at the far end. A great circle changes direction along its length, so this row appears only on lines long enough for the convergence to reach a degree; short measurements stay a single row.
- **Terrain (3D)** — slope-following distances when 3D terrain is enabled. This section hides itself when no elevation data is available; the heading rows do not, because a bearing is pure geometry on the measured coordinates and needs no DEM.

Right-click quick actions

Right-clicking the map opens a context menu built around the coordinate you clicked:

Item	What it does
The coordinate	Shown at the top in a monospace font; click it to copy to the clipboard.
What's here?	Pull up the Wikipedia knowledge card for that place.
Copy as GeoJSON	Copy the clicked point as a GeoJSON <code>Feature</code> .
Center map here / Zoom in here	Fly the camera to the point, optionally one zoom level closer.
Quick analysis	A submenu of one-click analyses — see below.
View in Google Maps / View in Google Earth	Open the same location in an external map.



Quick analysis from a clicked point

The **Quick analysis** submenu runs a tool on the coordinate you right-clicked, with no dialog and no need to first create a point layer:

Action	Description
Buffer ... here	Three distance presets around the point. The presets follow your Scale bar units preference — 500 m / 1 km / 5 km for metric, 0.25 mi / 1 mi / 5 mi for imperial.
Drive time from here	5-, 10-, and 15-minute drive-time isochrones.
Walk time from here	The same contours on foot.
Viewshed from here	What is visible from the point, within 2 km, 5 km, or 15 km. See Viewshed .
Open in Processing...	Open the full Processing vector dialog instead, for the parameters the presets don't expose.

Results are added as new layers, and a banner reports progress and any failure with a link to the run's entry in Processing History.

The same **Quick analysis** submenu also appears in each vector layer's actions menu in the [Layers panel](#), where it operates on the whole layer instead of a point: **Buffer all features by ...**, **Centroids**, **Convex hull**, and **Bounding box**.

Drive and walk time call an external service

Isochrones are computed by a Valhalla routing server — by default the public FOSSGIS instance at valhalla1.openstreetmap.de, so the clicked coordinate leaves your device. GeoLibre asks for consent the first time. Self-hosted deployments can point at their own server; see [Self-Hosting](#).

Viewshed from a clicked point

Quick analysis → **Viewshed from here** computes the terrain visible from the clicked point and drapes it over the map as a translucent overlay.

- **No DEM to prepare.** The elevation comes from the same public terrain tiles the map's 3D terrain control uses, fetched on demand for the square around your point. You do not have to find, download, or load a DEM first, and 3D terrain does not have to be switched on.
- **Observer height** is 1.8 m — a standing person — above the ground elevation at the clicked point.
- **Radius** is one of the three presets, from 2 km to 15 km. The underlying tool accepts anything from 100 m up to a 50 km ceiling.
- **The result is an image overlay layer** named `Viewshed (5 km)` or similar, so it gets opacity, reordering, zoom-to, and removal like any other layer, and it is saved with the project.

What the quick viewshed does not model

Earth curvature and atmospheric refraction are ignored. Over a few kilometres that is immaterial, but at the 50 km ceiling the curvature drop alone reaches roughly 180 m — enough to matter for a radio line-of-sight study, if not for "what can I see from this overlook". The public terrain tiles are also a global, generalized elevation model rather than a survey-grade DEM. For a rigorous analysis against your own DEM and your own station points, use the **Viewshed** tool under [Processing → Whitebox Toolbox](#), which this quick action deliberately does not replace.

Camera, overlay, and recording tools

The Controls menu also carries tools that move the camera, drape live data over the map, or capture what you see:

Tool	Description
Sun	Simulate the sun's position and the day/night terminator for any date and time.
Weather	Overlay near-realtime Clouds (NASA satellite imagery, animated day by day) and Precipitation (RainViewer radar, animated over roughly the last two hours).
Gridlines	Draw a coordinate grid with edge labels, including a UTM easting/northing mode and an MGRS/USNG mode (grid zones with the Norway and Svalbard exceptions, lettered 100 km squares, then 10 km and 1 km lines as you zoom in).
Spinning Globe	Slowly rotate the globe, optionally bounded to a region.
Route Animation	Animate a marker along a line layer with play/pause, speed, a trail, and camera follow.
Flight Simulator	Fly over terrain and 3D layers with continuous keyboard controls.
Atmospheric Effects	Render a deep-space backdrop, starfield, comets, and an atmospheric halo at low zoom.
Directions	Click the map to add waypoints and get a route. Waypoints are sent to the public OSRM demo server.
Reverse Geocode	Click the map to look up an address at those coordinates through the configured geocoding provider .
Record Map Tour...	Capture an animated keyframe tour to video, with per-keyframe recapture, hold and transition durations, and a saveable tour setup.
Record Video...	Record the map canvas, or a drawn bounding box, to a video file, optionally burning in on-map HTML, legend, and colorbar panels.

Some tools call external services

Directions and Reverse Geocode send your coordinates to a third-party service — the OSRM demo server and your configured geocoding provider. Weather fetches overlay imagery from NASA and RainViewer. The rest of the table runs locally against tiles the map already fetches.

Annotations and the Elements panel

The annotation tools draw on top of the map: **Text**, **Arrow**, **Rectangle highlight**, **Ellipse highlight**, **Freehand highlight**, **Pin marker**, **Sticky note**, and **Placed image**. Each has a color, and the stroked shapes (Arrow, Rectangle, Ellipse, and Freehand) also honor the line width. You can delete the last annotation or clear them all. Use the chevron at the top of the toolbar to collapse the tools when you need more room for controls beneath them.

Annotations are saved with the project, and the **Elements** panel lists them so you can find and manage each one instead of hunting for it on the canvas. Most elements are anchored **At Point** — a geographic coordinate they move with. **Placed image** additionally offers **Pinned to Extent**, which stretches it across a bounding box so it scales with the view.

Review comments

Comments are anchored notes for review and feedback, kept separate from annotations because they are a conversation rather than map decoration.

- Activate the comment tool from the **Comments** panel on the right sidebar, or by pressing **C**, then click the map to drop a pin and write the note. Pressing **C** again turns the tool back off. GeoLibre asks for your name once and remembers it.
- Each pin opens a thread you can reply to, resolve, reopen, or delete.
- Filter the panel by **Open**, **Resolved**, or **All**. Resolved pins are hidden from the map while the Open filter is active.
- In a live [collaboration](#) session, adding, replying, resolving, and deleting all sync to the other participants.

Comments are stored in the `.geolibre.json` file, so they travel with a shared or saved project even when no session is running.

GPS tracking

Controls → GPS Tracking... follows a live position on the map, records a track log, and can digitize new features straight from the feed. Pick one of two entries under **Position source**:

- **This device** — the browser or OS geolocation API. This is the default and needs no extra hardware.
- **NMEA receiver (serial or Bluetooth)** — an external GPS/GNSS receiver, for survey-grade or high-rate positions. Choose a baud rate and use **Connect serial** or **Connect Bluetooth**; the dialog then reports the device name and a running count of parsed sentences and fixes.

NMEA needs a Chromium browser

Reading a receiver uses the Web Serial and Web Bluetooth APIs, which Chromium browsers such as Chrome and Edge provide but Firefox and Safari do not. Most Bluetooth GPS receivers speak *classic* Bluetooth rather than Bluetooth Low Energy: pair those in your operating system's settings and they appear here as a serial port. Use **Connect Bluetooth** only for Bluetooth Low Energy receivers.

Start GPS to follow your position, record a track log and save it as a layer or export it as GPX or GeoJSON, and capture points where you stand. **Controls → Field Collection...** is the related tool for capturing point, line, and polygon observations against a custom form. See [GPS Tracking & Field Collection](#) for both.

Map navigation basics

- **Pan**: drag the map, or use the arrow keys while the map has focus.
- **Zoom**: scroll wheel, pinch, the navigation control, or the `+` / `-` keys.
- **Rotate**: hold the right mouse button and drag, use the compass, or `Shift` + `←` / `→`.
- **Tilt**: hold `Ctrl` / `Cmd` and drag to tilt the map into a perspective view, or `Shift` + `↑` / `↓`.
- **Reset the view**: press `N` for north up, `U` for a top-down view, or `R` to reset both pitch and bearing. See [the interface guide](#) for the full shortcut list.

<https://geolibre.app/user-guide/field-collection/>

GPS Tracking & Field Collection

GeoLibre has two tools for work in the field, both on the **Controls** menu:

- **GPS Tracking...** streams a live position onto the map, records a track log, and captures points where you stand.
- **Field Collection...** captures point, line, or polygon observations against a form you define, with optional photos.

Both write ordinary GeoJSON layers, so everything they capture is saved with the project, shows up in the [attribute table](#), and exports through the usual **Layer actions** → **Export** formats. Neither has a server sync or upload step: captures live in the project until you save or export it.

GPS Tracking

Controls → **GPS Tracking...** opens the **GPS Tracking** dialog.

Position source

Pick one entry under **Position source**:

- **This device** uses the browser or operating system's location service with high accuracy requested. In the Android and iOS apps the OS asks for location permission the first time. If permission is refused the dialog says so; allow location for the site (browser) or app (system settings) and press **Start GPS** again.
- **NMEA receiver (serial or Bluetooth)** reads an external GPS/GNSS receiver. Choose a **Baud rate** (4800, 9600, 19200, 38400, 57600, or 115200; 9600 by default, and remembered), then **Connect serial** or **Connect Bluetooth**. Once connected the dialog shows the device and a running count such as `1,204 sentences · 300 fixes` with the current fix quality (for example **RTK fixed**, **RTK float**, **Differential GPS**). **Disconnect** releases the receiver.

GeoLibre parses the GGA, RMC, VTG, GSA, and GST sentences from any talker (GP, GN, GL, ...). Accuracy comes from GST when the receiver sends it; otherwise it is estimated from HDOP.

NMEA needs a Chromium browser

The receiver is read through the Web Serial and Web Bluetooth APIs, which Chrome and Edge provide and Firefox and Safari do not. Most Bluetooth GPS receivers use *classic* Bluetooth: pair those in your operating system's settings and connect them as a serial port. **Connect Bluetooth** is for Bluetooth Low Energy receivers only.

Following your position

Start GPS begins streaming. The map shows a blue dot, a heading arrow when a heading is reported, and a translucent circle for the reported accuracy. The readout lists the accuracy (± 4 m), **Satellites**, altitude, speed in km/h, and heading in degrees.

Keep map centered on my position (on by default) recenters the map on each fix. Dragging the map turns it off.

When you close the dialog while GPS is running, a small floating panel stays in the bottom corner of the map with the coordinates, accuracy, satellites, speed, the track statistics, **Capture point**, a follow toggle, **Pause/Resume**, and a button that reopens the dialog.

Recording a track

Under **Track log**:

1. **Record track** clears any previous unsaved track and starts recording, turning GPS on if needed.
2. **Pause** stops logging; **Resume** continues in a new segment, so a gap is not drawn as a straight line.
3. While a track exists the dialog shows its point count, distance, and duration, with four actions:
 - **Save as layer** adds the track to the project as a line layer named like `GPS Track 2026-10-04 0930` (a `MultiLineString` when it has several segments), keeping each vertex's time, accuracy, and satellite count.
 - **Export GPX** writes GPX 1.1, one `<trkseg>` per segment.
 - **Export GeoJSON** writes the same track as GeoJSON.
 - **Discard** throws the track away.

Saving and exporting need at least two points.

Unsaved tracks are not kept

A track lives only in memory until you press **Save as layer** or export it. Closing the app or reloading the page loses an unsaved track.

Capturing points

Capture point under **Digitize** adds a point at the current fix to a layer named **GPS Points**, creating it on first use. Each point records `time`, `accuracy_m`, `satellites_used`, `ele`, `speed_mps`, and `heading_deg`. To capture into a layer of your choice, with a form, use [Field Collection](#) instead.

Logging filters

Filter	Effect
Min distance (m)	Skip track fixes closer than this to the previous one.
Min time (s)	Skip track fixes sooner than this after the previous one.
Max accuracy (m)	Do not log fixes less accurate than this, and block Capture point while the fix is worse.

Each is off at 0. The filters are remembered between sessions.

Field Collection

Controls → **Field Collection...** captures observations into a collection layer against a custom form.

Set up a collection layer

1. Under **Collection layer**, pick an existing collection layer or **New collection layer....**
2. For a new layer, enter a **Layer name** and pick a **Geometry: Point, Line, or Polygon**. A layer holds one geometry type.
3. **Add field** for each attribute. Give it a **Label**, a **Type (Text, Number, Date, or Choice)**, tick **Required** if it must be filled, and for a Choice field list the **Options (comma-separated)**.
4. **Create layer**.

A layer with no fields captures location-only observations. Property names are derived from the labels. Fields have no default values.

If the layer has an [attribute form](#) configured, Field Collection uses it instead of the plain field list: its aliases, dropdowns and checkboxes, number, range and date widgets, conditional visibility, and constraints all apply.

Capture an observation

For a **point**:

- **Use GPS** takes a single high-accuracy fix from this device (up to 15 seconds). It does not read the NMEA receiver connected in GPS Tracking.
- **Pick on map** places the point where you click; **Esc** cancels.
- **Reposition point** moves a point you have already placed.

For a **line** or **polygon**, **Draw on map** opens a toolbar along the bottom of the map. Click to add vertices, or **Add GPS vertex** to add one at your current position; **Undo** removes the last vertex, and **Finish** (or a double-click) closes the shape once it has enough vertices.

Fill in the form, attach photos if you like, and press **Save point**, **Save line**, or **Save polygon**. Required fields are marked *, and the form refuses an empty required field, a non-numeric number, or a choice outside the listed options.

Photo (optional) → **Choose photo...** attaches one or more images, up to 2 MB each. Photos are stored on the feature, so they travel with the project.

Session behavior

Done ends the collection session. Closing the dialog with the X button or **Esc** only hides it: an **Open Field Collection** pill stays on the map so you can capture the next observation without setting up again.

The GPS accuracy shown while collecting is for your information and is not stored on the feature. If you need per-point accuracy and satellite counts, use **Capture point** in GPS Tracking.

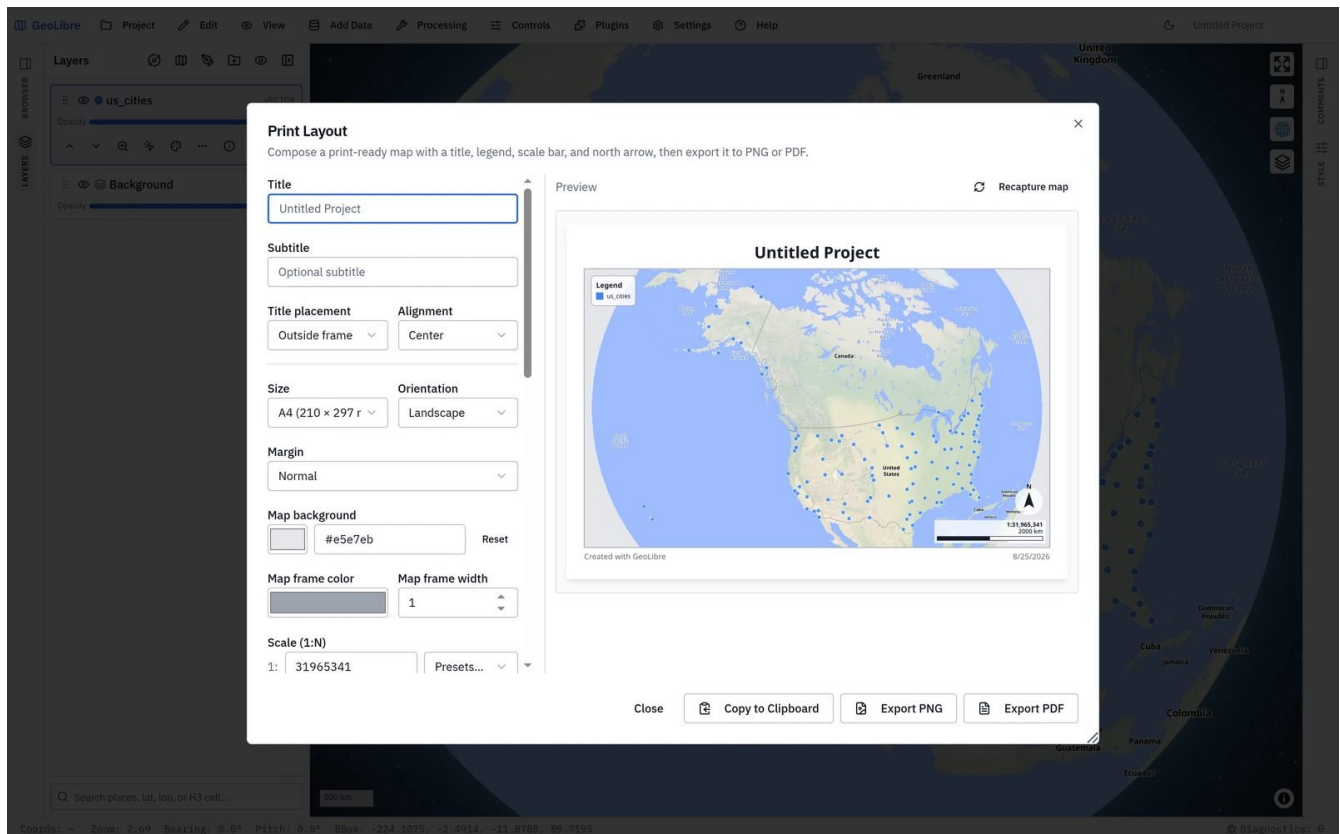
Working offline

Captures are kept in the project, so collecting needs no connection once the app is loaded. Cache the basemap for the area before you go with [Project → Offline Basemap...](#), and save the project before closing the app.

<https://geolibre.app/user-guide/print-layout/>

Print Layout

Project → Print Layout... composes a print-ready map with a title, legend, scale bar, north arrow, and optional data blocks, then exports it to PNG, PDF, or SVG. It can also generate an atlas: a map series with one page per feature or a run of pages along a line.



The controls are on the left and the live **Preview** on the right. The preview is built from a capture of the live map; after you pan, zoom, or restyle the map, press **Recapture map** to refresh it. Drag the splitter between the two halves, or the dialog's corner grip, to make more room.

The layout is saved with the project, so reopening the composer brings back the same page, elements, and atlas settings. A project has one layout.

Title

Title (defaults to the project name) and **Subtitle** head the page. **Title placement** puts them **Outside frame** (above the map) or **Inside frame** (over it), and **Alignment** sets **Left**, **Center**, or **Right**.

Page

Size offers paper and screen presets:

Group	Sizes
Paper	A4 (210 × 297 mm), A3 (297 × 420 mm), Letter (8.5 × 11 in), Legal (8.5 × 14 in), Tabloid (11 × 17 in)
Screen / digital	Full HD (1920 × 1080 px), HD (1280 × 720 px), 4K UHD (3840 × 2160 px), Square (1080 × 1080 px)

Custom... takes a **Width** and **Height** in px or mm. **Orientation** switches between **Portrait** and **Landscape** (not for custom sizes), and **Margin** is **Normal**, **Narrow**, or **None (borderless)**.

Map frame, scale, and extent

- **Map frame:** the **Map background** color shows where the map has no data; **Map frame color** and **Map frame width** draw the border around the map (width **0** hides it).
- **Scale (1:N):** type a scale or pick one from **Presets...** (1:500 up to 1:100,000). Applying it zooms the live map to that scale and recaptures. It is available on paper (mm) pages once the map has been captured. If the map's zoom limits cannot reach the scale, the composer uses the closest one it can and says so.
- **Print extent:** **Draw extent on map** lets you drag a box on the map to set the exact print area; hold **Shift** to match the page proportions. Then choose **Use active viewport** or **Clip to custom extent**, or **Clear extent**. While clipping to a custom extent, the scale input is disabled because the extent fixes the scale.

Map elements

Tick the elements to place on the page under **Map elements**:

Element	Notes
Title, Subtitle	See Title .
Legend	Built from the visible layers. See Legend editor .
Scale bar, North arrow	Group north arrow & scale bar places them together.
Date	The export date.
Include GeoLibre attribution	Adds "Created with GeoLibre".
Footer text	Free text along the bottom of the page.
Page border	With Border color and Border width .
Info block (title block)	An engineering-style block with Author , Project No. , CRS , Revision , and the scale.
Colorbar	A continuous color scale with Colormap , Min value , Max value , Label , Orientation , Position , and Length .
Custom legend	Hand-made legend entries. See Custom legend .
Attribute table	A table of a layer's records. See Data blocks .
Chart	A bar, pie, or line chart. See Data blocks .

Blocks snap to the page corners through their **Position** setting; there is no free placement by dragging. The composer has no free text boxes, image or logo elements, inset maps, or layout grid. A coordinate grid on the map itself (**Controls** → **Gridlines**) is captured with the map, and the map is fitted so its edge labels are not cropped.

Legend editor

When **Legend** is ticked, the legend editor lists one entry per visible layer (and its classes). For each entry you can:

- edit the label to rename it on the page,
- hide or show it with the eye button,
- move a layer entry up or down.

Legend title sets the heading, **Group classes by layer** nests classes under their layer, and the reset button returns to the automatic legend. The editor changes the project's legend, the same one **Controls** → **Legend** shows, so edits appear in both places. Entries come from layers; to add items that are not layers, use a custom legend.

Custom legend

Custom legend draws a legend you build by hand: a **Legend title**, then a color and **Label** for each row, with **Add item** and remove buttons, and a **Position**. **Import from dictionary** replaces the rows from JSON such as `{"Forest": "#228b22", "Water": "#1e90ff"}`.

Data blocks

Attribute table and **Chart** read a vector layer whose features are loaded in the app (a GeoJSON or other local vector layer).

- **Attribute table**: choose the **Layer**, a **Heading**, the **Columns**, **Sort by** and **Order**, and **Max rows** (up to 50), or **Adjust number of rows to fit page**.
- **Chart**: choose the **Layer**, a **Chart type** (**Bar**, **Pie**, or **Line**), the **Category field**, and an **Aggregate** (**Count**, **Sum**, or **Mean**, with a **Value field** for the last two).

Both take a **Page extent filter**: **All features**, only features completely within the page extent, or only features intersecting it. In an atlas they can also show only the current atlas feature.

Atlas (map series)

Tick **Generate a multi-page map series** to turn the layout into a template that is repeated for a series of extents. The atlas needs a flat map; it is not available on the Cesium globe or the ArcGIS 3D scene.

1. Pick a **Coverage layer**, a vector layer whose features drive the pages.

2. Choose the **Coverage**:

- **One page per feature** frames each feature in turn.
- **Pages along a line** walks each line feature in order, one page per **Segment length (km)** (20 km by default). A series is capped at 5,000 pages; increase the segment length if you hit the cap.

3. Choose a **Page name field** (by default the feature number).4. Set the **Page extent: Margin around feature** (a percentage) or, on paper pages, a **Fixed scale**.

5. Optionally:

- **Mask area outside current feature** dims everything outside the current polygon (one page per feature, polygon layers only).
- **Sort by** and **Order** set the page order (one page per feature).
- **Filter** limits the pages to matching features, for example `POP2000 > 100000` and `ST = "CA"`. Comparisons are `=`, `!=`, `>`, `>=`, `<`, `<=`, and `contains`, joined with `and`.

The preview shows **Page X of Y** with previous and next buttons, and moves the live map to each page's feature.

Atlas tokens

Use these tokens in the title, subtitle, footer, and the **Page file name (ZIP export)**:

Token	Value
<code>{atlas.name}</code>	The page name.
<code>{atlas.pagenumber}</code>	The page number.
<code>{atlas.total}</code>	The number of pages.
<code>{atlas.attr:FIELD}</code>	The current feature's <code>FIELD</code> value. Along a line, <code>km_start</code> , <code>km_end</code> , <code>segment</code> , and <code>segments</code> are also available.

The default page file name is `{atlas.pagenumber}-{atlas.name}`.

Exporting

Button	Output
Export PNG	A raster image of the page. In an atlas it becomes Export PNG ZIP : one PNG per page in a ZIP file.
Export PDF	A PDF page of the chosen size. In an atlas it becomes Export PDF (N pages) : one multi-page PDF.
Export SVG	An SVG of the page. Not available in an atlas.
Copy to Clipboard	The page as a PNG on the clipboard.

The file is named after the title. Paper pages render at 150 DPI; screen presets and pixel custom sizes render at their exact pixel size. An atlas export reports its progress page by page, and the dialog stays open until it finishes.

What is vector and what is raster

SVG is the format to choose when the layout is going on to a vector editor such as Inkscape or Illustrator: the title, legend, scale bar, north arrow, tables, charts, and borders are written as editable text and paths. The map itself, and any image-based marker icons, stay embedded as images. The **PDF** is a 150 DPI image of the whole page placed on a PDF page, not vector content.

Getting a sharper map

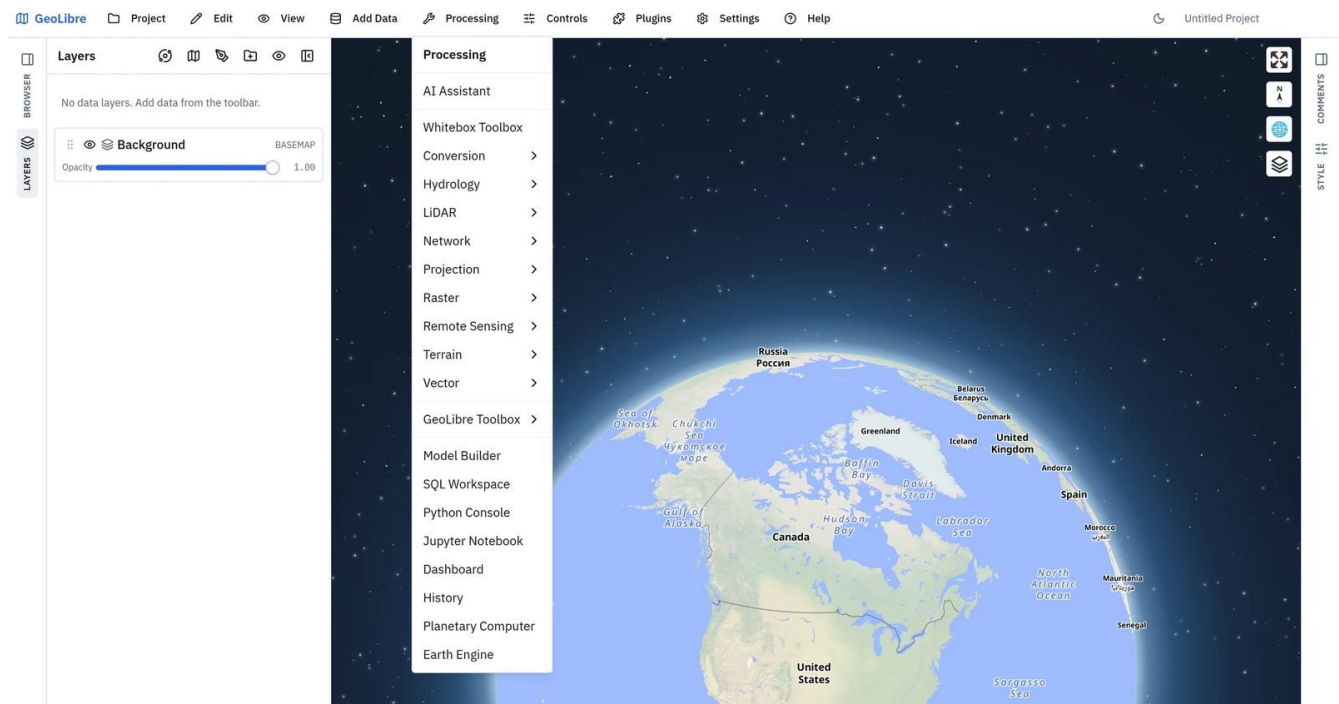
The map is captured from the on-screen map, so its detail is limited by the size of the map on your screen, whatever the page size. Maximize the window (or use a high-DPI display) before **Recapture map** for a large print. Text and other layout elements are drawn at full output resolution.

A **View → Color vision preview** does not affect exports: the layout is always exported in the map's real colors.

<https://geolibre.app/user-guide/processing/>

Processing Tools

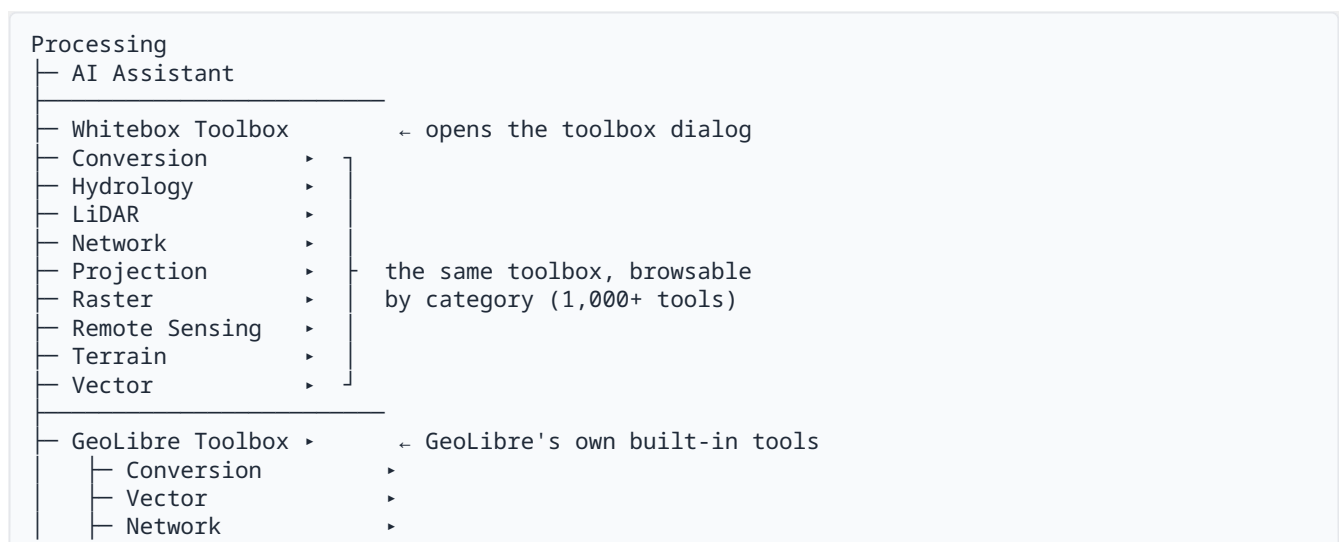
The **Processing** menu collects GeoLibre's analysis and conversion tools. It holds **two separate toolboxes** (why), plus the [SQL Workspace](#), [Python Console](#), [AI Assistant](#), and [AI Segmentation](#), which have their own pages.



Two toolboxes in one menu

This is the single most common source of confusion in the Processing menu, so it is worth reading before anything else.

The menu is laid out like this:





The block between the first and second separators belongs to the **Whitebox Toolbox**; everything under **GeoLibre Toolbox** is GeoLibre's own. Because both toolboxes cover vector, raster, conversion, and network work, several category names appear twice. They are not duplicates, and they do not open the same thing:

	Whitebox Toolbox	GeoLibre Toolbox
Where in the menu	The Whitebox Toolbox item and the nine category submenus directly below it	The GeoLibre Toolbox submenu below the separator
What it contains	1,000+ tools: the Whitebox Next Gen suite plus GeoLibre's own Rust tools from geolibre-rust	GeoLibre's built-in tools, written for this app
Categories come from	The tool manifests, generated automatically	Hand-curated for this app
What clicking a tool opens	One shared dialog, with the tool preselected and its form built from the tool's parameter manifest	A purpose-built dialog per tool family (Vector tools, Raster tools, Conversion, ...)
Where it runs	WebAssembly in the browser by default, optionally the Python sidecar	Turf.js or Pyodide in the browser, or the Python sidecar, depending on the tool
Input	A map layer or a file	Usually a layer already on the map

The two paths that look alike

The pair that trips people up most often:

- `Processing → Vector → GeoLibre (WASM)` lists the several hundred **vector tools GeoLibre contributes to the Whitebox toolbox**. They are compiled from [geolibre-rust](#) to WebAssembly and sit in the same catalog as the Whitebox Next Gen tools, so the menu groups them under a `GeoLibre (WASM)` subheading to show where they came from. The same subheading appears under Conversion, Raster, Hydrology, and LiDAR. Every one of these opens the **Whitebox Toolbox dialog**.
- `Processing → GeoLibre Toolbox → Vector` opens GeoLibre's own [Vector tools dialog](#): buffer, clip, spatial join, and the rest of the [list below](#), with an engine picker (Turf.js, GeoPandas sidecar, or Pyodide).

So the first is *"the vector part of the Whitebox toolbox, filtered to the tools GeoLibre wrote"*, and the second is *"GeoLibre's own vector toolbox"*. Both exist because they are different implementations with different strengths, not because one is a copy of the other.

A few individual **tool** names collide the same way. `Processing → Conversion → GeoLibre (WASM) → Vector to PMTiles` is a WASM catalog tool that opens the Whitebox Toolbox dialog; `Processing → GeoLibre Toolbox → Conversion → Vector to PMTiles` is the Conversion dialog documented [below](#), with its own file pickers and zoom, compression, and layer-name options. `Raster to PMTiles` appears in both places too. When a step in these docs or a tutorial names a Conversion tool, it means the GeoLibre Toolbox one.

Which one should I use?

Start with **GeoLibre Toolbox** for everyday work on layers already on the map: fewer tools, guided dialogs, and a choice of engine. The analysis tools add their result to the map as a new layer; the Conversion tools write a file instead. Reach for the **Whitebox Toolbox** when you need something the GeoLibre Toolbox does not have, which is most terrain, hydrology, remote sensing, and LiDAR analysis, or a specific named tool from the full catalog.

The dialog's Source filter is a third, narrower thing

Inside the Whitebox Toolbox dialog, the **source** dropdown offers *All sources / GeoLibre tools / Whitebox tools*. That filter splits the **Whitebox toolbox catalog only** by who wrote each tool. "GeoLibre tools" there means the same WASM tools that the menu labels `GeoLibre (WASM)`. It never shows the GeoLibre Toolbox dialogs, which are not in that catalog at all.

GeoLibre Toolbox

GeoLibre's own tools, under **Processing → GeoLibre Toolbox**.

Vector

Processing → GeoLibre Toolbox → Vector opens the Vector tools dialog. Pick a tool from the list, choose the input layer and parameters, select an [engine](#), then **Run**. Output appears as a new layer.

Skipping the dialog

Several of these are also one click away from the **Quick analysis** submenu. Right-clicking the map runs a buffer or a drive- or walk-time isochrone on the clicked point; the submenu on a layer row runs a buffer, centroids, convex hull, or bounding box over the whole layer. See [Right-click quick actions](#).

Geometry

Tool	Description
Buffer	Create a buffer polygon around each feature by a fixed distance. Pick the side (outside, inside, or both), and optionally dissolve the buffers into a single attribute-less feature.
Centroids	Compute the centroid point of each feature.
Convex hull	Compute the convex hull enclosing all features.
Dissolve	Merge polygon features into a single geometry, optionally grouped by a field.
Bounding box	Compute the rectangular envelope of all features.
Simplify	Reduce the number of vertices using Douglas-Peucker.
Reproject	Reinterpret a layer's coordinates as a source CRS and transform them to WGS84 so they display in the right place. Needs the Sidecar or Python engine.
Explode	Split multipart geometries into single-part features, one per part, keeping the parent's attributes.
Aggregate by attribute	Dissolve features sharing an attribute value into one geometry per group, with a summary statistic.
Smooth	Round the corners of lines and polygons with Chaikin's algorithm (this <i>adds</i> vertices, unlike Simplify). Z values are preserved.
Extract vertices	Turn every vertex of a line or polygon layer into a point feature, keeping the parent's attributes.
Points along geometry	Place points at a fixed spacing (or a fixed count) along each line or polygon boundary.
Regular grid	Generate a rectangular grid (fishnet) over the map view, a layer's extent, or a manual bounding box.
Voronoi / Delaunay	Build a Voronoi diagram (one polygon per point, clipped to the points' extent) or a Delaunay triangulation from a point layer.
Cell-site coverage	Build antenna sector polygons from point sites using azimuth, radius, and beamwidth read from fields or fixed values.
Decode polyline	Decode encoded polyline strings (precision 5 or 6) from an attribute field into a LineString vector layer.
Encode line to polyline	Encode LineString and MultiLineString geometries into an encoded polyline string stored in a new attribute field.

Overlay

Tool	Description
Clip	Clip the input layer to the area covered by an overlay layer (keeps input attributes).
Intersection	Keep only the areas where both polygon layers overlap (merges attributes from both).
Difference	Remove the overlay layer's area from the input layer (keeps input attributes).
Union	Merge two polygon layers into a single combined geometry (attributes are not preserved on either engine).

Join

Tool	Description
Spatial join	Attach attributes from a join layer to each input feature based on a spatial relationship (intersects, within, or contains). Choose an <i>inner</i> join to keep only matched features or a <i>left</i> join to keep all input features. Works with any geometry type.
Attribute join	Attach attributes from a join layer (a table) onto each input feature where a key field matches — no geometry involved (e.g. join census stats to boundary polygons by FIPS code). One-to-one: the first matching join row wins. Choose which fields to bring over, and an <i>inner</i> join (keep only matched) or <i>left</i> join (keep all input).

Select

Tool	Description
Select by value	Extract features whose attribute matches a condition into a new layer. Pick a field, an operator ($=$, $\neq$, $>$, $\geq$, $<$, $\leq$, contains, starts with, is empty, is not empty) and a value. Comparisons are numeric when both sides are numbers, otherwise text.
Select by location	Extract features by their spatial relationship to a second layer (intersects, within, contains, or disjoint) into a new layer. Works with any geometry type.
Random extract	Extract a random subset of features into a new layer, sized by a feature count or a percentage of the input.

Movement & time

Tool	Description
Trajectory speed	Order points by time per target and connect consecutive fixes into segments carrying distance, duration, and speed.
Detect stops	Find where a target dwells: runs of consecutive fixes staying within a distance for at least a minimum duration. One point per stop.
Space-time proximity	Find pairs of points close in both space and time (two targets meeting, say). Outputs a line per qualifying pair with the distance and time gap.

Data management

Tool	Description
Merge layers	Combine several layers into one, through a multi-layer picker that unites their attribute schemas and can record each feature's source layer in a new field.

Data quality

Tool	Description
Check validity	Find features with invalid geometry (self-intersecting rings, holes outside shells, ...) and mark them on the map.
Fix geometries	Repair invalid geometries with <code>ST_MakeValid</code> ; valid features pass through untouched.

Tool	Description
Check topology rules	Validate a layer against topology rules (overlaps, gaps, self-intersections, dangles) and mark each violation.
Fix topology	Snap nearly-connected line endpoints, fix dangles, and project points onto lines. Free ends with nothing nearby are left alone.

Engines

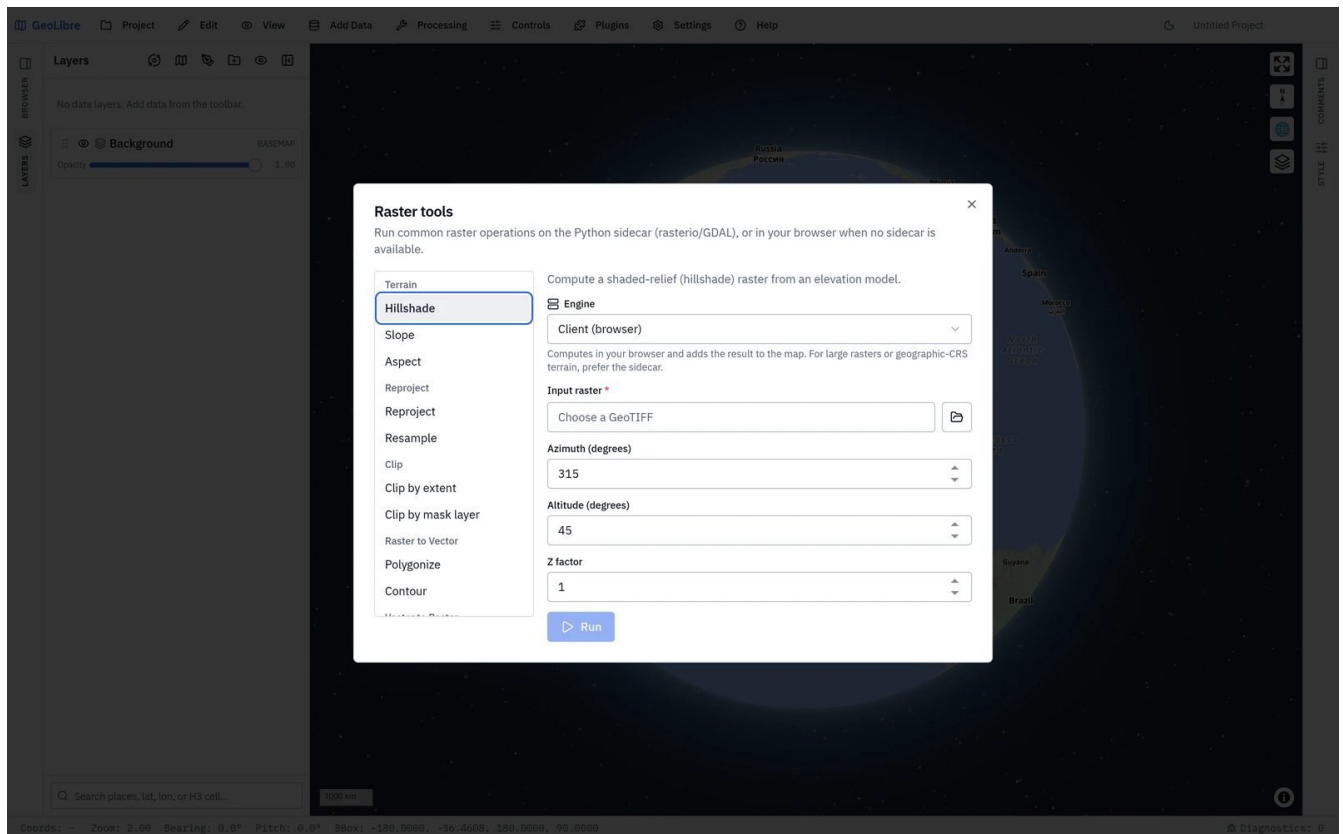
The vector tools run on one of three engines, selectable in the dialog (a few, such as Reproject, need the Sidecar or Python engine):

- **Client (Turf.js)**: runs entirely in the browser. No setup, works offline, and operates on the layer's GeoJSON.
- **Sidecar (GeoPandas)**: runs on the optional Python sidecar for projection-aware results, backed by GeoPandas and Shapely. The dialog falls back to the client engine when the sidecar's optional `vector` extra is not installed.
- **Python (Pyodide)**: runs the same GeoPandas/Shapely code as the sidecar, but **entirely in your browser** via [Pyodide](#) — no server, so it works on the web build and the public demo too. The first run downloads the Python runtime once (a few tens of MB, fetched lazily from a CDN, so an internet connection is needed the first time); later runs reuse the warmed-up runtime. Because it shares the sidecar's Python, results match the Sidecar engine. By default the runtime loads from the public jsDelivr CDN, which is a trust assumption: a tampered CDN response would run unverified (Pyodide loads its own `pyodide.asm.js` /WASM internally, so a subresource-integrity check on the entry script alone is not sufficient). For production or offline use, **self-host** the runtime by pointing `VITE_PYODIDE_INDEX_URL` at a mirrored copy of the Pyodide distribution, which removes the CDN dependency entirely.

See the [Vector Analysis tutorial](#).

Raster

Processing → GeoLibre Toolbox → Raster opens the Raster tools dialog. Most raster tools run on the rasterio Python sidecar: they take a file path in and write a file path out, then add the result to the map.



Eight of them also offer a **Client (browser)** engine that computes on the loaded raster with no sidecar at all, so they work on the web build and on mobile: **Hillshade**, **Slope**, **Aspect**, **Clip by extent**, **Raster calculator**, **Spectral index**, **Reclassify**, and **Focal statistics**. The rest — Reproject, Resample, Clip by mask layer, Polygonize, Contour, Interpolation, Zonal statistics, and Mosaic / merge — need the sidecar. The engine picker in the dialog shows which engines a tool accepts, and the line beneath it says what the selected one will do.

Terrain

Tool	Description
Hillshade	Compute a shaded-relief raster from an elevation model.
Slope	Compute slope (steepness) from an elevation model.
Aspect	Compute aspect (compass direction of the steepest slope) from an elevation model.

Reproject

Tool	Description
Reproject	Warp a raster to a different coordinate reference system.
Resample	Resample a raster to a different pixel size (resolution).

Clip

Tool	Description
Clip by extent	Crop a raster to a bounding box (in the raster's CRS).
Clip by mask layer	Clip a raster to the geometries of a vector mask file.

Raster to Vector

Tool	Description
Polygonize	Convert a raster band into vector polygons grouped by pixel value.
Contour	Generate contour lines from an elevation model.

Vector to Raster

Tool	Description
Interpolation (IDW / Kriging)	Interpolate a point layer's numeric attribute into a continuous raster surface using inverse distance weighting or ordinary kriging. The output grid spans the points' extent at the chosen pixel size, in the layer's CRS.

Analysis

Tool	Description
Zonal statistics	Summarize raster values within each polygon of a zone layer.
Raster calculator	Evaluate an expression across one or more aligned rasters.
Spectral index	Compute NDVI, NDWI, EVI, and other indices from band presets.
Reclassify	Map pixel value ranges onto new values.
Mosaic / merge	Combine several rasters into one.
Focal statistics	Compute a moving-window statistic (mean, min, max, ...) over a raster.

Download Global DEM sits at the top of the same submenu: it downloads a DEM for the current map view or a box drawn on the map from the public AWS Terrain Tiles dataset (no API key needed), and opens in the Whitebox Toolbox dialog rather than the Raster tools dialog. **Georeferencing** sits at the bottom: it pins a non-georeferenced image to the map with ground control points using a least-squares affine fit, reporting per-GCP and RMS residuals.

See the [Terrain Analysis tutorial](#).

Conversion

Processing → GeoLibre Toolbox → Conversion writes data to cloud-native formats:

Tool	Description
Vector to Vector	Convert between any formats DuckDB's spatial extension supports; input and output formats are detected from the file extensions. The desktop app (sidecar) writes any GDAL format (FlatGeobuf, GeoPackage, Shapefile, KML, GML, SQLite, ...); the browser writes GeoJSON, CSV, GeoParquet, GeoPackage, FlatGeobuf, and Shapefile.
Vector to GeoParquet	Hilbert-sorted, compressed GeoParquet.
Vector to FlatGeobuf	Hilbert-sorted, cloud-optimized, spatially indexed vector.
Vector to Shapefile	Hilbert-sorted, zipped ESRI Shapefile (field names truncated to 10 characters).
Vector to GeoPackage	Hilbert-sorted GeoPackage for sharing with QGIS/ArcGIS.
CSV to GeoParquet	Convert a CSV with coordinates to GeoParquet.
Vector to PMTiles	Build a vector tile archive.
Raster to PMTiles	Render one raster band through a colormap into a PMTiles archive of Web Mercator PNG tiles.
Raster to COG	Write a Cloud-Optimized GeoTIFF.

Every one of these has a client-side engine, so all of them work in the browser and on the Mac App Store build. On desktop they run on the Python sidecar instead, which reads and writes native file paths and, for Vector to Vector, can write any format GDAL supports rather than the browser's shorter list. Raster to PMTiles is the exception: it has no sidecar endpoint at all and always runs in WebAssembly. The dialog names the engine it is about to use in its status line.

The sidecar can also confine conversion inputs and outputs to an allowlist of directories, set through `GEOLIBRE_CONVERSION_ROOTS`. It is unset on the desktop app, where the paths are your own filesystem and there is nothing to confine. The Docker image sets it, because there the sidecar is reachable same-origin through the nginx proxy and must not be able to read or overwrite arbitrary container paths. See [Self-Hosting](#).

Not the same as Processing → Conversion

The bare **Conversion** submenu higher up the menu is the Whitebox toolbox's conversion *category* (format and raster/vector conversion tools from the WASM catalog). The dialogs in this table live under **GeoLibre Toolbox → Conversion**. See [Two toolboxes in one menu](#).

Network

Processing → GeoLibre Toolbox → Network runs routing analysis against a [Valhalla](#) server (the public FOSSGIS instance by default, so the coordinates you submit leave your device; self-hosted deployments can point at their own, see [Self-Hosting](#)).

Tool	Description
Isochrone / service area	Polygons reachable within given travel times from one or more origins.
OD cost matrix	Travel time and distance for every origin-destination pair between two point layers.
Sequential route (directions)	Route through a set of stops in order, with turn-by-turn directions.

Spatial Statistics

Processing → GeoLibre Toolbox → Spatial Statistics runs pattern analysis on a point or polygon layer and adds the result to the map.

Tool	Description
Global Moran's I	A single measure of whether a numeric attribute is clustered, dispersed, or random across the layer.
Local Moran's I (LISA)	Per-feature clustering: high-high, low-low, and spatial outliers.
Getis-Ord Gi* hotspots	Per-feature hot and cold spots with significance levels.
Average nearest neighbor	Whether a point pattern is more clustered or dispersed than random.
Kernel density (heatmap)	A continuous density surface from a point layer.
Emerging Hot Spot (space-time cube)	Hot-spot trends over time from a space-time cube.
Composite score (suitability index)	Normalize, weight, and combine several numeric fields into one index (0-100 by default, or 0-1), styled on the map by the score.

The **Composite score** builder is the suitability and index workflow: pick two or more numeric fields, give each a normalization (min-max, z-score, rank, or quantile), a relative weight, and the end of its range that scores well, then combine them with a weighted arithmetic mean, or a geometric mean when a strong field should not be able to compensate for a weak one. The weights are shown as the share of the total each field carries, so their effect is legible while editing.

Two choices decide what the map means, so the dialog asks rather than assumes:

- **Features missing a value** either drop out of the result or have their weights renormalized over the fields they do have. There is no default; both produce plausible maps, and only one of them is the index you meant.
- **Keep normalized components** writes each field's normalized value as its own column, so a score can be audited instead of taken on trust.

Score range writes the index on 0 to 100 by default; switch it to 0 to 1 when the score feeds another calculation rather than a legend. **Score field name** decides the column it lands in, and the normalized components, when kept, are written on the same scale beside it.

Rank and quantile normalization share ranks across ties (the mid-rank convention), so equal inputs always score equally. The result lands as a new layer already styled by the graduated renderer on the score.

DGGS

Processing → GeoLibre Toolbox → DGGS works with discrete global grid systems (H3, S2, A5, DGGRID, DGGAL).

Tool	Description
DGGS Generator	Fill an area with DGGS cells, from a layer's geometry or extent, the map view, or a manual bounding box.
DGGS Binning	Aggregate a point layer into DGGS cells (count, or sum/mean/min/max of a numeric field).
DGGS Compact	Compact DGGS polygon cells, or expand them to a uniform resolution.

Geocode Addresses and Batch tools

Two more entries sit below the separator in the GeoLibre Toolbox submenu:

- **Geocode Addresses** turns a table of addresses into a point layer.
- **Batch tools** runs one tool across many input layers (*Batch*), or chains tools so each step's output feeds the next and saves the chain with the project (*Models*). Both modes draw on the Vector tools above and run on the client engine. For a graphical way to build the same kind of chain, see [Model Builder](#).

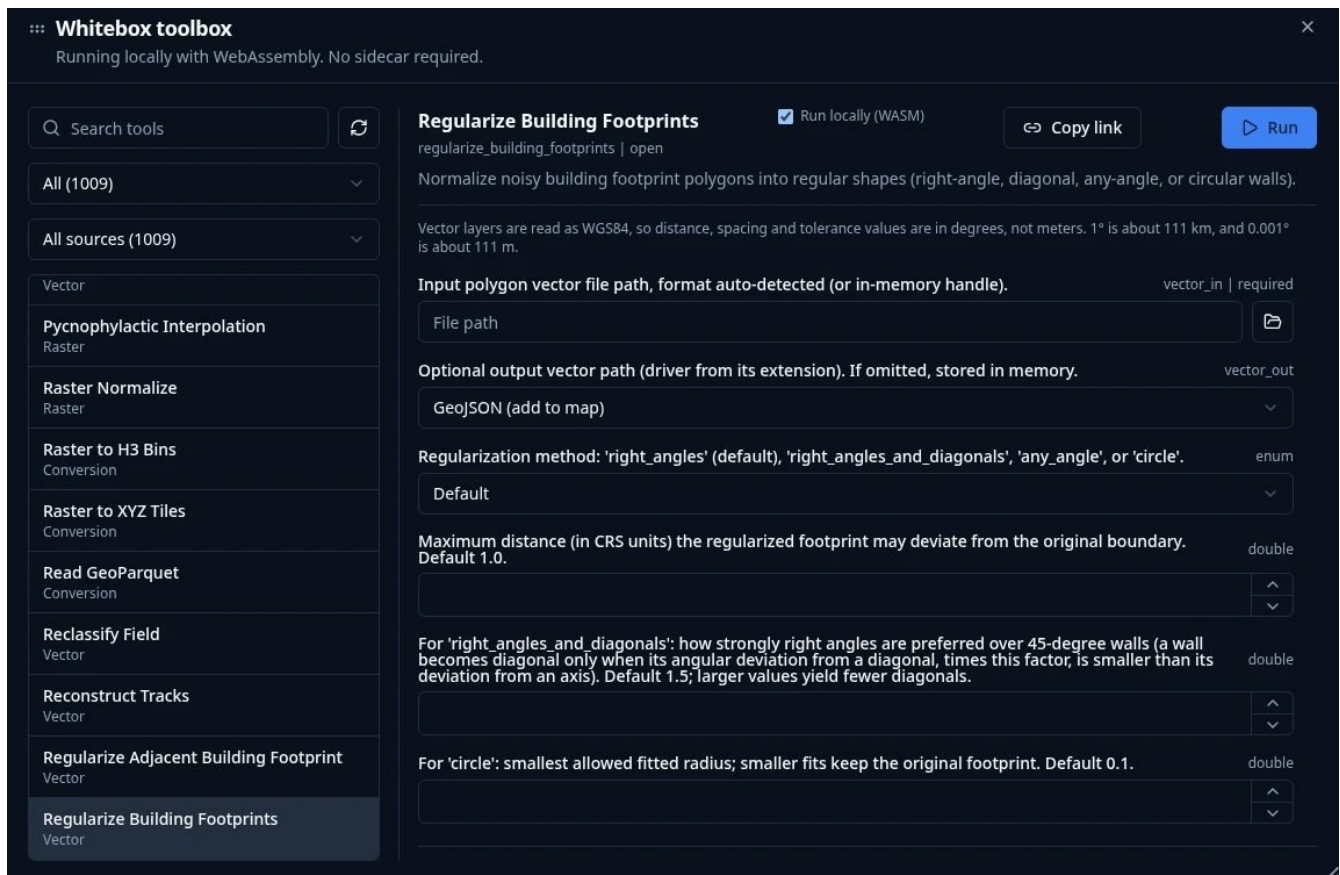
AI Segmentation, Object Detection, Segment Everything

- **AI Segmentation** turns imagery into vector features with [segment-geospatial](#) (SamGeo) and Meta's SAM 3 model: choose a GeoTIFF, type a text prompt ("*trees*", "*buildings*") or run automatic segmentation, and the resulting polygons are added as a new layer. It runs the model in a separate `samgeo-api` server (a GPU is recommended) that the sidecar proxies. See the dedicated [AI Segmentation](#) page for setup and usage.

- **Object Detection** and **Segment Everything** run client-side in the browser on `onnxruntime-web`, so they need no sidecar and stay available on the web build and on mobile.

Whitebox Toolbox

Processing → **Whitebox Toolbox** opens the geoprocessing toolbox: **1,000+ tools** covering vector, raster, remote sensing, hydrology, terrain, LiDAR, conversion, network, and projection analysis.



The tools come from the [Whitebox Next Gen](#) suite together with GeoLibre's own WASM tools from [geolibre-rust](#), which the dialog mixes into the same catalog (use the **source** dropdown to filter to one or the other). The category and subcategory names, and the tool names themselves, come straight from the tool manifests and are generated automatically, so they follow upstream naming rather than GeoLibre's.

Where the tools run

They run **in the browser**, through a WebAssembly runtime with raster and vector I/O — no Python sidecar, no server, and no data leaving your machine. That means the full toolbox is available on GeoLibre Web and on Android and iOS, not just the desktop app.

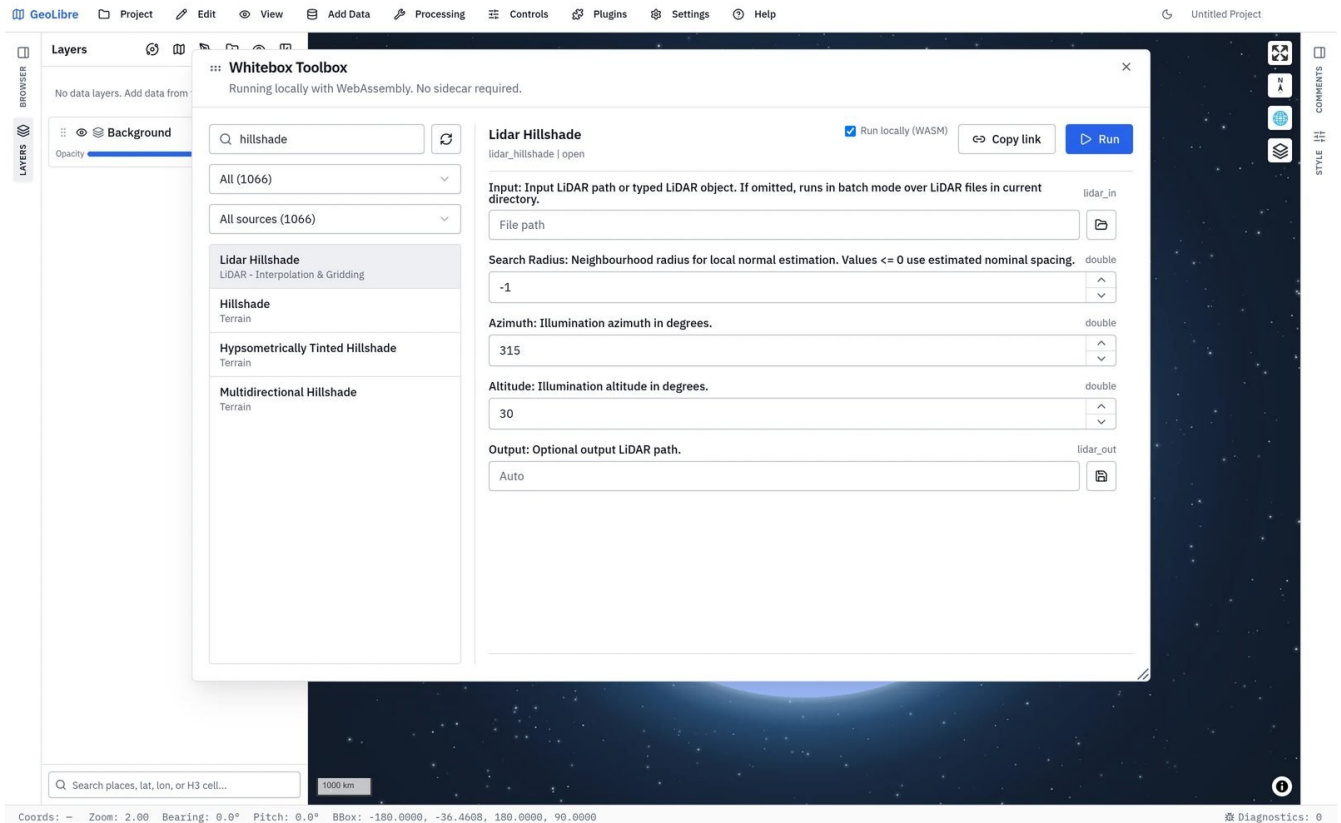
The **Run locally (WASM)** checkbox controls the engine:

- **Checked** runs the tool in WebAssembly on the layer or file you pick. This is the default in the browser build, where no sidecar can be started. The Mac App Store build has no sidecar to switch to at all, so the checkbox does not appear there — WebAssembly is the only runtime.
- **Unchecked** sends the job to the Python sidecar instead. This is the default on desktop, where the sidecar is available. The sidecar can read native file paths that the in-browser runner cannot fetch, and is the better choice for batch runs across a directory of files on disk.

Either way the tool list is the same; only the executing engine changes.

Finding and running a tool

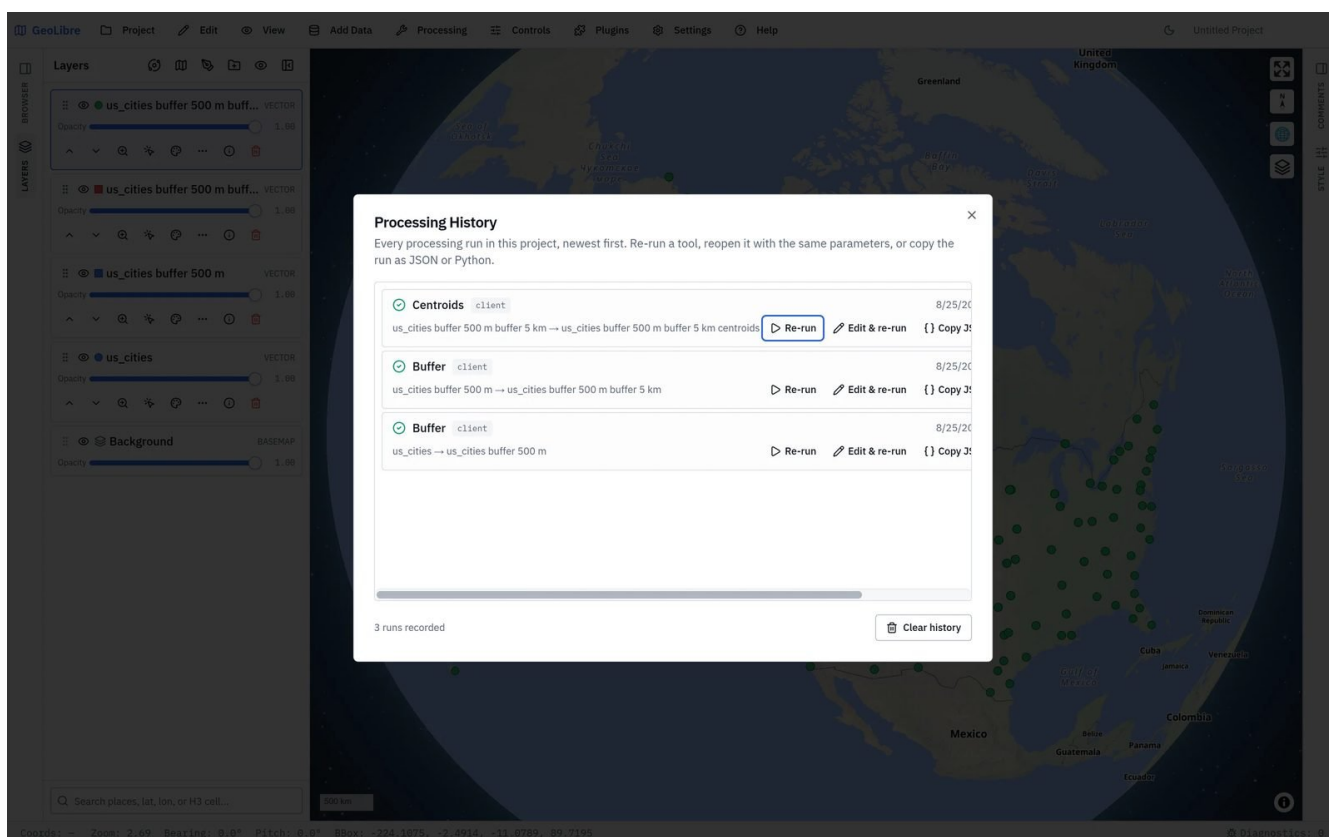
- **Search** by name at the top of the tool list, or narrow with the **category** and **source** dropdowns.
- **Browse by category** without opening the dialog at all: the Processing menu has a submenu per category (Conversion, Hydrology, LiDAR, Network, Projection, Raster, Remote Sensing, Terrain, Vector) with nested subcategory submenus, including the **GeoLibre (WASM)** subheading for GeoLibre's own tools. Picking a tool opens the dialog with it preselected. The catalog is bundled offline, so the menu works with no network.
- **Fill in the form** — the dialog builds it from the tool's own parameter manifest, with a file picker for path inputs and an output-format dropdown for vector outputs. Parameters that are ground distances get a metric unit picker.
- **Run**, and the output is added to the map. Raster outputs are Cloud Optimized GeoTIFFs. A LiDAR output from the in-browser (WASM) engine, such as a classified point cloud, is added to the map as a point cloud layer (converted to COPC first when needed), and is downloaded as a file only if that fails. A LiDAR layer's **Export** menu saves it as LAS, LAZ, or COPC.



Share a link to a tool

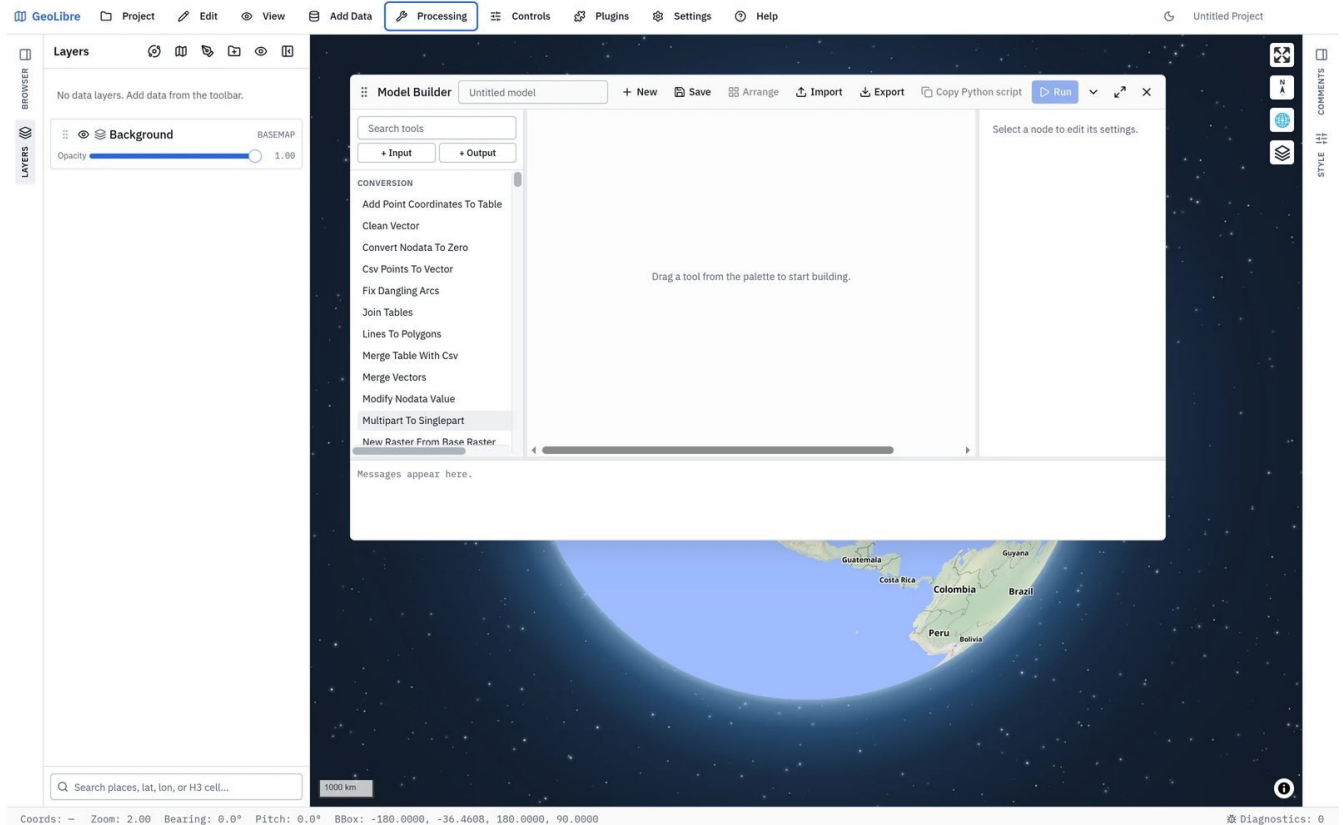
Copy link builds a URL with a `?tool=` parameter that reopens the app with that tool preselected and its form pre-filled — handy for documentation, teaching, and bug reports.

Every run is recorded in **Processing → History**, newest first, with the tool, the engine it ran on, how long it took, and the layers in and out. Each entry offers **Re-run**, **Edit & re-run** (reopens the tool with the same parameters), **Copy JSON**, and **Copy Python**. Both toolboxes write to the same history, and so do the [quick analyses](#).



Model Builder

Processing → Model Builder opens an ArcGIS-style canvas for building a processing workflow as a graph instead of running one tool at a time.

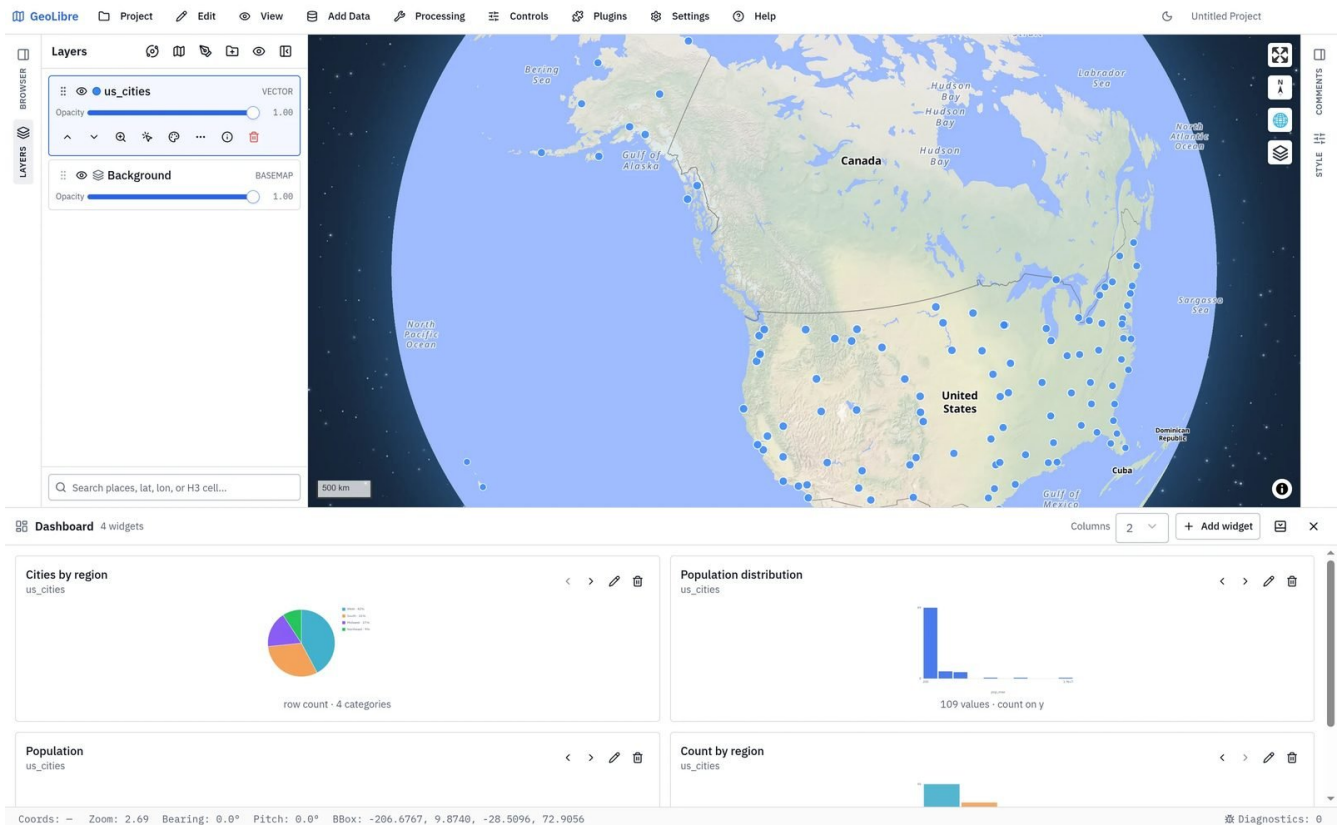


- **Drag a tool from the palette** onto the canvas to add it as a node, then wire one node's output into the next node's input. **+ Input** and **+ Output** add the model's own entry and exit points, so the same graph can be re-run against different layers.
- The palette draws on the client-side [GeoLibre Toolbox](#) vector tools plus the open [Whitebox](#) catalog, so a raster chain such as fill depressions → flow accumulation → extract streams is a valid model.
- The canvas checks the graph as you build it (missing inputs, unconnected ports, mismatched data types, loops), and **Run** executes the chain as one job, with progress in the message log at the bottom.
- **Save** stores the model with the project, **Import** / **Export** move it between projects as a file, and **Copy Python script** puts the equivalent [geolibre](#) [Python](#) code on the clipboard.

See [Model Builder](#) for the full guide, including node types, validation messages, how runs execute, the model file format, and Python export. The [AI Assistant](#) can also author a model from a plain-language description and open it here for review before you run it.

Dashboard

Processing → Dashboard opens a panel of chart widgets below the map that summarize the layers in the project. Unlike the [attribute table's Charts dialog](#), dashboard widgets are saved with the project and cross-filter each other.



Add widget asks for a layer, a widget type, and the fields to plot:

Widget	What it shows
Histogram	The distribution of one numeric field, with a configurable bin count.
Scatter	Two numeric fields against each other.
Bar / Line	An aggregate per category or per ordered value.
Box plot	The spread of a numeric field, optionally grouped.
Pie	The share of each category.
Indicator	A single big number — count, sum, mean, min, max, or median — with an optional prefix and suffix.
Selector	A categorical field rendered as chips that cross-filter every other widget bound to the same layer, in single- or multi-select mode.
List	A field's values as a scrollable list, for reading records rather than aggregating them.

Each widget carries a title and a color, and the **Columns** control sets how many sit side by side. A selector never filters itself, so a choice can always be changed or cleared, and selections start empty each time the dashboard opens — they are a way of looking at the data rather than a property of it.

Any vector or DuckDB query layer can be charted; the panel says so when the project holds none yet.

Jupyter Notebook

Processing → Jupyter Notebook docks a notebook beside the map: [JupyterLite](#) on a Pyodide kernel in the web build, and a real JupyterLab server on the desktop app, which gives you full CPython with geopandas, rasterio, and GDAL. Either way a `geolibre` client is preloaded, so a cell can drive the live map next to it. See [Notebook Panel](#).

Planetary Computer and Earth Engine

The Processing menu also opens the **Planetary Computer** and **Earth Engine** panels for browsing and loading cloud datasets. See [Data Integrations](#).

The Python sidecar

The raster tools, the sidecar conversion tools, and the optional GeoPandas vector engine use a local FastAPI sidecar that the desktop app starts on demand. The Whitebox Toolbox can *optionally* use it too, but does not need it — it runs in WebAssembly by default. The vector tools' client engine and the browser-based conversions need no sidecar either. See [Getting Started](#) for setup and [Reference → Architecture](#) for how it works.

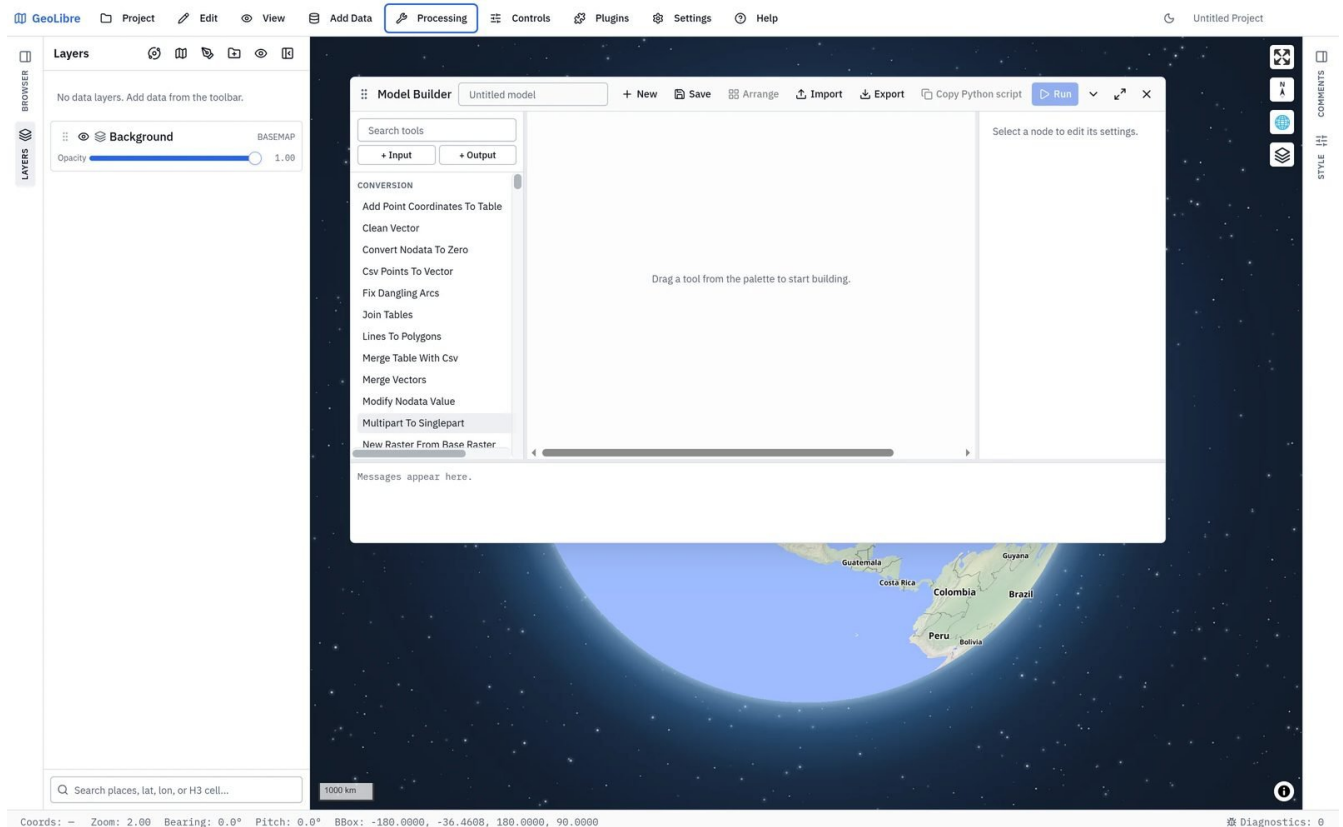
Browser vs desktop

The [Whitebox Toolbox](#), the client-side vector tools, every [Conversion](#) tool, and the eight [raster tools with a client engine](#) run in the browser. Vector to Vector's full any-format output and the remaining raster tools require the desktop app and the Python sidecar.

<https://geolibre.app/user-guide/model-builder/>

Model Builder

Processing → Model Builder builds a processing workflow as a graph: wire the output of one tool into the input of the next, then run the whole chain as one job. A saved model can be re-run on other layers, exported to a file, or turned into a Python script.



Model Builder opens as a floating panel over the map that you can move, resize, maximize, and minimize. The tool palette is on the left, the canvas in the middle, the selected node's settings on the right, and the message log along the bottom. Drag the splitters between them to resize the columns. On a narrow screen the palette and settings become toggle buttons.

The title bar

Control	What it does
Model name	The model's name (Untitled model until you name it).
New	Start an empty canvas, asking first if the current one has unsaved changes.
Save	Store the model in the project. See Saving and sharing models .
Arrange	Lay the nodes out left to right along the flow.
Import / Export	Read or write a <code>.model.json</code> file.
Copy Python script	Copy the equivalent <code>geolibre</code> Python code. See Python export .

Control	What it does
Run / Cancel	Run the model, or stop a run in progress.

Building a model

Nodes

A model has three kinds of nodes:

- **Input:** a layer from the project that feeds the model. Pick it under **Source layer** in the settings panel. Add one with **+ Input**.
- **Tool:** one processing tool. Its parameters are set in the settings panel.
- **Output:** a result that is added to the map when the model runs, named by its **Result name**. Add one with **+ Output**.

The palette offers the client-side [GeoLibre Toolbox](#) vector tools and the open [Whitebox](#) tool catalog. Type in **Search tools** to filter it, then drag a tool onto the canvas (or click it, or press **Enter**, to add it at a default spot).

There are no value or parameter nodes, loops, iterators, or conditions: every model is a straight data flow from inputs to outputs, and a model that contains a loop is rejected.

Connecting nodes

Drag from an output port (on the right of a card) to an input port (on the left). From the keyboard, activate an output port to arm it, then an input port to connect them. Each port carries vector data, raster data, or either, and ports that carry different kinds of data cannot be connected.

- Connecting to an input that is already wired replaces the old connection.
- Click a connection's curve to remove it.
- Remove a node with the trash button in its settings panel.

A tool input that is not connected can take a layer chosen directly in the settings panel instead, which is how a one-tool model gets its data.

Drag node cards to arrange them by hand, or press **Arrange** to lay the graph out automatically (this replaces your own positions). The canvas scrolls to pan; it does not zoom. There is no undo or copy and paste inside Model Builder, so export a copy before a large rework.

Keeping intermediate results

Only results connected to an **Output** node are kept; everything in between is discarded after the run. To keep an intermediate result, select its tool and press **Keep this result** (or **Keep "port"** for a tool with several outputs), which adds and connects an Output node for it.

Checking the model

The canvas checks the model as you build it and lists each node's problems in red in its settings panel:

- **Choose an input layer.**
- **"port" needs a connection or a value.**
- **"port" already has an incoming connection.**
- **Those ports carry different kinds of data.**
- **The model contains a loop.**
- **Add an output node to keep a result.**
- **Unknown tool "tool".**, or a connection to a node or port that no longer exists.

Run stays disabled until the model has no problems.

Running a model

Run executes the nodes one at a time, each after everything that feeds it. Node cards show whether they are running, done, or failed, and the log at the bottom shows each tool's own messages.

- Vector tools run in the browser.
- Whitebox tools run through the in-browser WebAssembly runner, in the desktop app as well; Model Builder does not use the Python sidecar.
- Each Output node adds a new layer to the map: a GeoJSON layer for vector results, a GeoTIFF for raster results.
- The run stops at the first node that fails, and reports **Run failed:** Outputs added before the failure stay on the map.
- **Cancel** stops the run before the next node starts. A Whitebox tool that is already running finishes first.

Saving and sharing models

Save stores the model in the project's `.geolibre.json`, so it travels with the project. Saved models are listed under **Saved models** below the canvas; pick one from **Load a saved model...** to open it, or **Delete** to remove the saved copy (the canvas is left as it is).

Export downloads the model as `<name>.model.json`, and **Import** opens one. The file holds the graph: its nodes (inputs, tools with their parameters, and outputs, with their positions) and the connections between them:

```
{
  "$schema": "https://geolibre.app/schemas/model-graph-v1.json",
  "version": "1.0.0",
  "name": "Roads buffer",
  "graph": { "nodes": [ ... ], "edges": [ ... ] }
}
```

An imported file must be a GeoLibre model, no larger than 16 MB, 2,000 nodes, or 4,000 connections. An Input node stores the id of its layer, so after importing a model into another project, choose each input's **Source layer** again.

Python export

Copy Python script becomes available after the current model has run successfully. It copies a script that hands the whole graph to the [geolibre Python package](#), which asks the app to check and run it and returns the ids of the output layers:

```
# Generated by GeoLibre Model Builder.  
# Run with the model's input layers loaded in the displayed map.  
  
m.run_model_builder({"nodes": [...], "edges": [...]})
```

`m` is a displayed `geolibre.Map`, in a notebook or the [Python Console](#). The model's input layers must be loaded in that map. `run_model_builder` takes an optional `timeout` in seconds (600 by default).

Building a model with the AI Assistant

The [AI Assistant](#) can author a model from a plain-language description, such as "buffer the roads by 100 m and clip the result to the county". It looks up the tools, checks every parameter against the tool's definition, saves the model to the project, and opens it in Model Builder for you to review before you press **Run**. It asks before replacing unsaved work on the canvas.

<https://geolibre.app/user-guide/segmentation/>

AI Segmentation

Processing → GeoLibre Toolbox → AI Segmentation turns imagery into vector features using [segment-geospatial](#) (SamGeo) and Meta's **SAM 3** model. Describe what you want — *"trees"*, *"buildings"*, *"water"* — or run automatic segmentation, and GeoLibre adds the resulting polygons as a new vector layer.

Requires the Python sidecar and a model backend

Segmentation runs deep-learning models, so it needs the GeoLibre desktop app with the Python sidecar **plus** a running `samgeo-api` model server. A CUDA GPU is strongly recommended; CPU inference works but is slow. See [Setup](#) below.

How it works

The heavy model stack (PyTorch + SAM 3) does **not** run inside the GeoLibre sidecar. Instead the sidecar exposes a thin `/ml` reverse-proxy in front of a separate `samgeo-api` server (the REST server shipped with `segment-geospatial`). GeoLibre uploads the image to the sidecar, which forwards it to `samgeo-api`, runs SAM 3, and returns GeoJSON polygons.

```
SegmentationDialog —image+prompt—► sidecar /ml/segment/* —► samgeo-api (SAM 3) —► GeoJSON
```

This keeps the GeoLibre sidecar small and lets the model server run wherever you have a GPU.

Setup

Install `segment-geospatial` with the API and SAM 3 extras in a Python environment that has a working PyTorch build (ideally CUDA):

```
pip install "segment-geospatial[api,samgeo3]"
```

Then either let GeoLibre launch the model server for you, or run it yourself:

- **Auto-launch (default).** If `samgeo-api` is on the `PATH` of the environment the sidecar runs in, GeoLibre starts it automatically on first use.
- **Run it yourself / on another machine.** Start the server and point GeoLibre at it:

```
samgeo-api --port 8000  
# then run the sidecar with:  
GEOLIBRE_ML_SAMGEO_URL=http://127.0.0.1:8000
```

Desktop app: point it at an external `samgeo-api`

The desktop app runs its sidecar in a managed (uv) environment that ships the `m1` extra but **not** the heavy `segment-geospatial` model stack, so `samgeo-api` is not on its `PATH` and auto-launch does not apply. Install `segment-geospatial[api,samgeo3]` in a PyTorch-capable environment, start `samgeo-api` there, and launch the desktop app from a shell that exports `GEOLIBRE_ML_SAMGEO_URL` so the sidecar proxies to it:

```
# in your PyTorch env
samgeo-api --port 8000

# launch the desktop app with the proxy target set
GEOLIBRE_ML_SAMGEO_URL=http://127.0.0.1:8000 npm run tauri:dev
```

The Tauri process passes its environment to the sidecar it spawns. Without `GEOLIBRE_ML_SAMGEO_URL`, the sidecar has no model backend and `/m1/status` reports the segmentation backend as unavailable.

Install the sidecar's optional `m1` extra (just an HTTP client — the models live in `samgeo-api`):

```
pip install -e "backend/geolibre_server[m1]"
```

Configuration

Environment variable	Purpose
<code>GEOLIBRE_ML_SAMGEO_URL</code>	Base URL of an already-running <code>samgeo-api</code> . When set, the sidecar proxies here and does not launch a child process.
<code>GEOLIBRE_ML_SAMGEO_CMD</code>	Command used to launch <code>samgeo-api</code> on demand (default <code>samgeo-api</code>). <code>--host</code> / <code>--port</code> are appended automatically.
<code>GEOLIBRE_ML_DEFAULT_MODEL</code>	Model the UI defaults to (default <code>sam3</code>).

Using it

1. Open **Processing** → **GeoLibre Toolbox** → **AI Segmentation**.
2. If the backend isn't reachable, the dialog shows a message and a **Start server** button (desktop) — click it to launch the sidecar and model server.
3. **Choose a GeoTIFF**. Pick a georeferenced raster (`.tif` / `.tiff`; PNG/JPG also work but produce pixel-space results). The polygons come back in the raster's coordinate system, so they land in the right place on the map.
4. **Pick a mode:**
 - **Text prompt** — type what to segment (e.g. `trees`, `buildings`, `cars`) and set a **Confidence threshold** (0–1; lower finds more, higher is stricter). SAM 3 finds every matching object in the image.

- **Automatic (everything)** — segments all distinct objects without a prompt.
5. Click **Segment**. The result is added as a new GeoJSON polygon layer named after the prompt, and the map zooms to it.

Notes & limitations

- **SAM 3 only**. GeoLibre uses the SAM 3 backend, which covers text, box/point, and automatic prompts well. (SAM 2 is intentionally not wired up.)
- **Desktop only**. Like the raster tools, segmentation needs the desktop app and the Python sidecar; it is not available in the browser-only build.
- **No objects found** is a normal outcome — try a different prompt or lower the confidence threshold.
- **Box / point prompts** drawn on the map are available in the interactive [SamGeo plugin](#) below; this dialog covers text and automatic modes only.

Interactive SamGeo plugin

Plugins → **SamGeo** opens a dockable panel that talks to `samgeo-api` directly (default `http://127.0.0.1:8000`, editable in the panel) rather than through the sidecar, so it works in the browser build as well as on desktop. It mirrors `segment-geospatial`'s own interactive map: one uploaded image is shared by every prompt type, and results land as GeoJSON polygon layers.

1. Start the model server (`samgeo-api --port 8000` ; the panel links to the [setup guide](#)) and click **Check connection**.
2. **Choose the image**. *Image source* lists every COG or GeoTIFF layer already on the map — pick one to segment it in place — or leave it on **Upload a file** and choose a local `.tif / .png / .jpg`. The **Aerial imagery (UC Berkeley)** sample in *Add Data* → *Raster Layer* is a good first image.
3. **Pick a mode**:
 - **Text prompt** — a concept such as `building` or `tree`, with a confidence threshold and mask-size limits.
 - **Point prompts** — click **+ Foreground point** / **- Background point**, then click the map; green and red markers show the prompts. At least one foreground point is required.
 - **Bounding box (find similar)** — click **Draw box on map** and drag a rectangle; SAM 3 returns every object that resembles the boxed one.
 - **Automatic (everything)** — segments all distinct objects. SAM 3 is a prompt-driven model, so this mode runs SAM 2's automatic mask generator instead (choose the checkpoint under *SAM2 model*; `points_per_side`, `IoU` and stability thresholds apply here).
4. Click **Segment**. Polygons are reprojected from the raster's CRS to WGS84 and added as a layer named after the prompt or mode; the map zooms to it. With `segment-geospatial` 1.4.2 or newer each feature carries a `score` attribute with the model's confidence, visible in the attribute table.

Use **Clear prompts** to remove the points or box. Switching modes clears the other mode's prompts, and closing the panel removes the on-map prompt overlay. The panel's settings (API URL, mode, model, thresholds) persist with the project.

Image uploads go to the API URL you configure

The image is POSTed to the configured server. A project file can carry a saved API URL, so check the field before segmenting a local image with a project you did not author.

API (advanced)

The sidecar exposes these endpoints (proxied to `samgeo-api`):

Method	Endpoint	Description
GET	<code>/ml/status</code>	Backend availability, default model, version, and model list.
GET	<code>/ml/models</code>	The backend's model catalog (available and loaded models).
POST	<code>/ml/segment/text</code>	Text-prompt segmentation (<code>prompt</code> , <code>confidence_threshold</code>).
POST	<code>/ml/segment/automatic</code>	Automatic mask generation.
POST	<code>/ml/segment/predict</code>	Box/point prompt segmentation (<code>boxes</code> , <code>point_coords</code> , <code>point_labels</code> , <code>point_crs</code>).

Each `segment/*` endpoint takes a multipart `file` (the image) plus `model_version` (default `sam3`) and `output_format` (default `geojson`) and returns a GeoJSON `FeatureCollection`.

See also [Processing Tools](#) and [Reference → Architecture](#).

<https://geolibre.app/user-guide/point-cloud-annotation/>

Point cloud annotation

The **Point Cloud Annotation** plugin labels LiDAR points with ASPRS classes directly on the map. Use it to fix misclassified points (for example, stadium seating left as "never classified") or to build training labels for point cloud segmentation models. It is the first slice of the plan tracked in opengeos/GeoLibre#2749.

Start a session

1. Load a point cloud with **Add Data → LiDAR Layer** (LAS, LAZ, COPC, or EPT).
2. Zoom to the area you want to label. A COPC or EPT cloud streams by level of detail, so the session holds the points loaded at the current view. For a streamed COPC, **Full detail in view** loads every point inside the view at the file's full resolution (up to 4 million points) and keeps it loaded however you zoom, so labels go on every point; **Release full detail** lets the area stream normally again. It works before or during a session; during one, the area must fit in the space streaming left free. A large area can take a minute or two to download.
3. Open **Plugins → Point Cloud Annotation**, pick the cloud, and choose **Start annotating**.

While a session runs, the plugin colours the cloud by classification and pauses streaming, so no edited point is dropped when you move the map. Scroll to zoom and right-drag to tilt or rotate as usual.

Select and assign

- **Box (B)**, **Lasso (L)** and **Brush (P)**: drag on the map to select points. The brush selects everything within its radius of the dragged path; set its size in the panel or with [and]. Hold **Shift** to add to the selection or **Alt** to remove from it, or pick a mode in the panel. **Pan** returns left-drag to moving the map.
- **Polygon (G)**: click vertices, then double-click, press **Enter** or click the first vertex to close the ring and select what it encloses. **Esc** cancels. Hold **Shift** or **Alt** on the closing click to add or subtract.
- Selection goes through all depths, like a camera frustum. Narrow it with the **Min Z / Max Z** filter (metres) or **Only points in class**, which relabels just the points currently in one class.
- **Lock** a class in **Classes in session** to protect its points: locked classes, hidden classes (toggled in the LiDAR panel's legend) and points outside the LiDAR panel's elevation filter are never selected.
- Selected points are drawn in yellow. Choose the class to assign (or press **0-9** for ASPRS classes 0-9) and press **Apply (Enter)**; **Clear (Esc)** drops the selection. Clicking a row in **Classes in session** makes it the class to assign.
- **Undo** and **Redo** step through the class assignments of the session.

Custom classes

Open **Custom classes** under the assign controls to add classes of your own (for example *Car* or *Solar panel*). Give each a code from 19 to 255 (ASPRS reserves 19-63 for future standard classes and leaves 64-255 to users, so the panel suggests the first free code from 64), a name and a colour, then press **Add**. A custom class can be assigned, locked and boxed like a standard one; the LiDAR panel's legend and tooltips show its name and colour. Custom classes are saved with the project, and the exports write their codes (the LAS classification byte holds 0-255). A notebook or agent can define them ahead of time with `Map.set_point_cloud_classes` or the `set_point_cloud_classes` MCP tool.

Instances

Instances group points that belong to one object, such as one car or one tree, for instance segmentation. Select its points and press **New instance (N)**: they get the class chosen under **Assign class** and a new instance id, as one undoable edit. **Assign class to points** on a 3D box does the same for the points inside the box. **Instances** lists each one with its most common class and point count; **Select** selects its points again and **Dissolve** removes the instance while its points keep their class. Instance ids are saved with the project like classes, and exported with the points (see below).

Polygons to points

Polygons to points lifts a polygon layer straight up through the cloud: masks from **Plugins** → **SamGeo** segmenting imagery, building footprints, or any other polygons in the project. Pick the layer, then:

- **Select inside** selects the points whose position falls inside any polygon (holes excluded), combined with the current selection like a drawn shape.
- **Instance per polygon** gives the points inside each polygon the class to assign and their own [instance](#), as one undoable edit. A point in overlapping polygons belongs to the first one.

A footprint takes every point above and below it, ground included, so narrow it with the **Min Z / Max Z** and **Only points in class** filters first (for example, only unclassified points above the ground). Locked, hidden and elevation-filtered points are skipped, as in a drawn selection.

Pre-labelling with Whitebox

Pre-label (Whitebox) runs a Whitebox LiDAR classifier on the session's points in the browser (the same WebAssembly build as the **Processing** → **Whitebox Toolbox**) when you press **Run pre-label**, and applies its classes as one undoable edit:

- **Ground (improved ground point filter)** separates ground (2) from everything else (1).
- **Ground, vegetation and unclassified (classify LiDAR)** also marks vegetation.

Only relabel unclassified points (0 and 1), on by default, keeps every point you or the survey already classified. Locked and hidden classes are never changed. The tool runs in tiles of about 750,000 points with a 20 m overlap, since the WebAssembly build runs out of memory on a few million points at once; a 4.6 million point session takes a couple of minutes. The result is a starting point: review it, then correct it with the selection tools.

Notebooks and agents can pre-label without the app: `Map.prelabel_point_cloud` in the [Python package](#) and the `prelabel_point_cloud` [MCP tool](#) run the same classifiers on a local copy of the layer's file (with the `geolibre[pointcloud]` extra) and save the result as the layer's labels, which the annotator shows when the project opens.

3D boxes (cuboids)

Label objects such as buildings, trees or vehicles with oriented 3D boxes:

- **Auto box (A)**: click a point on an object. The annotator grows a cluster from it through points within 0.75 m of each other, skipping ground, water, noise and locked or hidden classes, and fits the tightest box around it: the minimum-area rectangle of the cluster's footprint, rotated to the object, and its full height.
- **Box from selection**: fits a box to the points you selected with the box, lasso or brush tools.

New boxes take the class chosen under **Assign class**. Each box in **Objects (3D boxes)** has its own class and three actions: **Select points** (the points inside it), **Assign class to points** (undoable like **Apply**; the points also become a new [instance](#)) and **Delete**. Click a box to open **Box views**, three orthographic views of the points around it: top, side (along its length) and front (across it). Drag inside the box to move it, drag an edge to resize it (the opposite face stays put), drag the knob beyond the front edge in the top view to rotate it, and scroll to zoom about the pointer. While a box is open in the views, the keyboard nudges it: arrow keys move it 10 cm north/south/east/west (1 m with **Shift**), **Q** and **E** rotate it by 1° (5° with **Shift**), and **+** / **-** raise or lower its top by 10 cm.

Each box also has a review **status** (New, Reviewed or Flagged) and free-form **Attributes**: name/value pairs such as `make: Ford` or `occluded: yes`, up to 32 per box. Expand **Attributes**, type a name and value and press **Add**; edit a value in place, or **Remove** it.

Boxes on a cloud loaded from a URL are saved with the project, with their status and attributes. Export them as **Boxes as GeoJSON** (one footprint polygon per box, with its class, `z_min`, `z_max`, size, heading, `status` and attributes as properties) or **Boxes as Segments.ai JSON** (a `pointcloud-cuboid` label in the same CRS and units as the LAS export, with the heading measured from grid east, and the status and attributes in each annotation's `attributes`).

3D vectors

Trace linear and point features in 3D: kerbs, lane markings and power lines as **Polylines (V)**, roof outlines and footprints as **3D polygons**, and poles or tree tops as **Keypoints (K)**. Click points on the cloud; each vertex snaps to the nearest drawn point within 12 pixels and takes its elevation, so the vector follows the surface. Double-click or press **Enter** to finish a polyline or polygon, **Backspace** removes the last vertex and **Esc** cancels. The map's zoom and rotation are held while a vector is being drawn. Vectors take the class chosen under **Assign class**; change it, or delete a vector, in the **3D vectors** list.

Vectors on a cloud loaded from a URL are saved with the project. Export them as **Vectors as GeoJSON** (3D `Point`, `LineString` and `Polygon` features in WGS 84 with the class and 3D length) or **Vectors as Segments.ai JSON** (a `pointcloud-vector` label with `polyline`, `polygon` and `point` annotations in the same CRS and units as the LAS export).

Saving labels with the project

Labels on a cloud loaded from a URL are saved with the project. Each edit is stored against the point's source node and its index in that node, not its position in memory, so the labels are re-applied when the project reopens and as streamed nodes load again, whatever order they arrive in. Labels on a local file are not saved (the file cannot be reopened from the project), so export them.

Export

- **LAS 1.4** writes every point loaded in the session (point format 7 with RGB, or 6 without) with its edited class, intensity, returns, GPS time and scan angle. Coordinates are written back in the source file's CRS, feet included, when its WKT is known, and in WGS 84 otherwise. When the session has instances, each point also carries an `instance` extra-bytes dimension (uint32, 0 for none), which laspy, PDAL and LAsTools read by name.
- **LAZ (compressed)** writes the same records as the LAS export, compressed with LASzip in the browser (a laz-rs WebAssembly build). It is typically about a quarter of the LAS size.
- **NumPy (.npy)** writes a structured array with `x`, `y`, `z` (float64, in the same CRS as the LAS export), `intensity`, `classification`, `red`, `green`, `blue` when the cloud has colour, and `instance` when the session has instances, ready for `numpy.load`.
- **Segments.ai JSON** writes a `pointcloud-segmentation` label. Its `point_annotations` line up point for point with the LAS file, with one annotation per instance and one per class for points in no instance, each with `category_id` set to the class code.

The exports above cover the points loaded in the session. **Full file with labels** writes a whole local copy of the point cloud with every saved label (and instance id) applied instead: enter the path of the local copy (the same LAS, LAZ or COPC file the layer streams) and an output `.las` or `.laz` path, then **Write file**. It runs in the GeoLibre server, which reads the file chunk by chunk, so files larger than

memory work; a COPC is written as plain LAS/LAZ. The section shows when the server is running and the cloud was loaded from a URL (only those have saved labels). In the web build the paths must be inside the server's allowed folders.

Finish session resumes streaming.

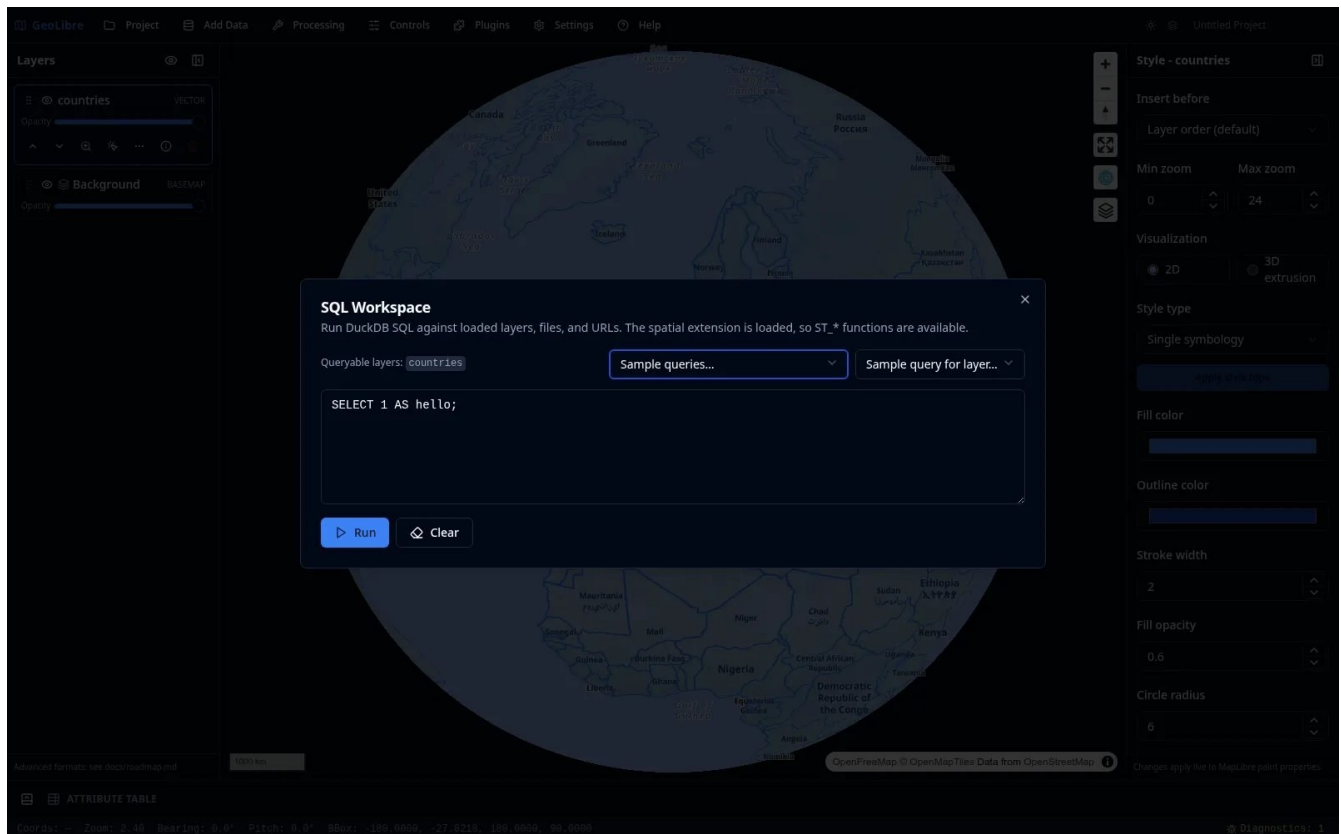
Limitations

- MapLibre renderer only.
- COPC output is not available; export LAZ and convert it with PDAL (`writers.copc`) if you need a COPC file.
- Boxes rotate about the vertical only (no pitch or roll).
- Instances are not colored by id on the map; select one from **Instances** to see its points highlighted.

<https://geolibre.app/user-guide/sql-workspace/>

SQL Workspace

The **SQL Workspace** runs DuckDB Spatial SQL right in the app, against your loaded layers, local files, and remote URLs. Open it from **Processing** → **SQL Workspace**. The spatial extension is loaded for you, so `ST_*` functions are available.



Querying loaded layers

Every loaded vector layer is exposed as a queryable table; the workspace lists the available table names under **Queryable layers**. Write a query in the editor and click **Run** (or press `Ctrl/Cmd+Enter`) to see the results. `Tab` or `Ctrl+Space` autocompletes.

```
SELECT NAME, CONTINENT, POP_EST
FROM countries
ORDER BY POP_EST DESC
LIMIT 10;
```

Reading files and URLs

You can query files and remote URLs directly. The workspace auto-wraps a bare URL into the matching reader (for example Parquet, CSV, JSON, or GeoJSON) and streams remote files over HTTP range requests, so you do not have to download them first.

```
SELECT NAME, CONTINENT, POP_EST, geom
FROM https://data.source.coop/giswqs/opengeos/countries.parquet
LIMIT 100;
```

Cloud URLs (s3://, gs://, az://)

Cloud object-store URLs are transparently rewritten to their public HTTPS equivalents, so you can use them directly in queries:

```
-- Amazon S3
SELECT * FROM read_parquet('s3://bucket/path/to/data.parquet') LIMIT 10;

-- Google Cloud Storage
SELECT * FROM read_parquet('gs://bucket/path/to/data.parquet') LIMIT 10;

-- Azure Blob Storage
SELECT * FROM read_parquet('az://account/container/data.parquet') LIMIT 10;
```

The bare `FROM s3://...` form works too — the workspace wraps it in the matching reader automatically.

Private S3 buckets

An `s3://` URL in a bucket covered by an S3 connection (**Settings** → **Cloud Storage**) is read through a presigned URL signed with that connection's credentials; every other cloud URL is read anonymously. Name individual files: globs are not signed. See [Cloud Storage](#). `gs://` and `az://` are public-only.

CORS

Browser-side reads require the bucket's CORS policy to allow cross-origin requests. Most public dataset buckets (e.g. AWS Open Data, Source Cooperative) already allow this. If you hit a CORS error, check the bucket's CORS configuration.

Choosing a SQL engine

The **SQL engine** menu (top right) selects which SQL engine runs your query. All three load a spatial extension, so `ST_*` functions are available in each.

- **DuckDB** (default) — DuckDB Spatial, in-browser. Queries loaded layers, local files, and remote URLs (including `s3://` / `gs://` / `az://` public data). Works offline after first use.
- **PostGIS** — PGLite + PostGIS, in-browser. Queries loaded layers using full PostGIS SQL. The first run loads a ~19 MB engine.

- **Apache Sedona** — Sedona spatial SQL over your loaded layers. It runs in-browser on [CereusDB](#), a WebAssembly build of Apache [SedonaDB](#). On the desktop app it uses the **SedonaDB sidecar** instead when the optional `sedona` extra is installed (better for larger layers); otherwise it falls back to the in-browser engine automatically. The CereusDB engine is downloaded lazily on first use.

Sedona reads loaded layers

Like PostGIS, the Apache Sedona engine queries layers you have loaded into the map (listed as **Queryable layers**). The geometry column is named `geometry`, so spatial functions look like `ST_Centroid(geometry)`, `ST_Area(geometry)`, or `ST_Buffer(geometry, 0.1)`.

Attribute columns in the browser engine

With the in-browser CereusDB engine, `SELECT *` returns each layer's attributes as columns (and "Add as layer" preserves them), but you cannot yet reference an attribute by name in SQL (e.g. `WHERE name = 'Texas'`) — that build exposes attributes only through the `geometry` column and spatial functions. Run the **SedonaDB sidecar** (install the `sedona` extra) for full attribute SQL.

Sample queries and history

- The **Sample queries...** and **Sample query for layer...** menus drop ready-made queries into the editor to get you started.
- Your previous queries are kept under **History...** so you can rerun them.

Using the results

When a query returns geometry, **Add as layer** adds the result to the map as a new layer (with an optional layer name). The result layer behaves like any vector layer, with [identify](#), [selection](#), and [the attribute table](#). With the DuckDB engine, **Add as query layer** instead adds a live layer that re-runs the SQL when you refresh it from the Layers panel. **Export CSV** and **Export GeoParquet** save the results to a file (GeoParquet needs a geometry column).

Multiple result layers

You can add several DuckDB query-result layers to the same project and keep them all open at once.

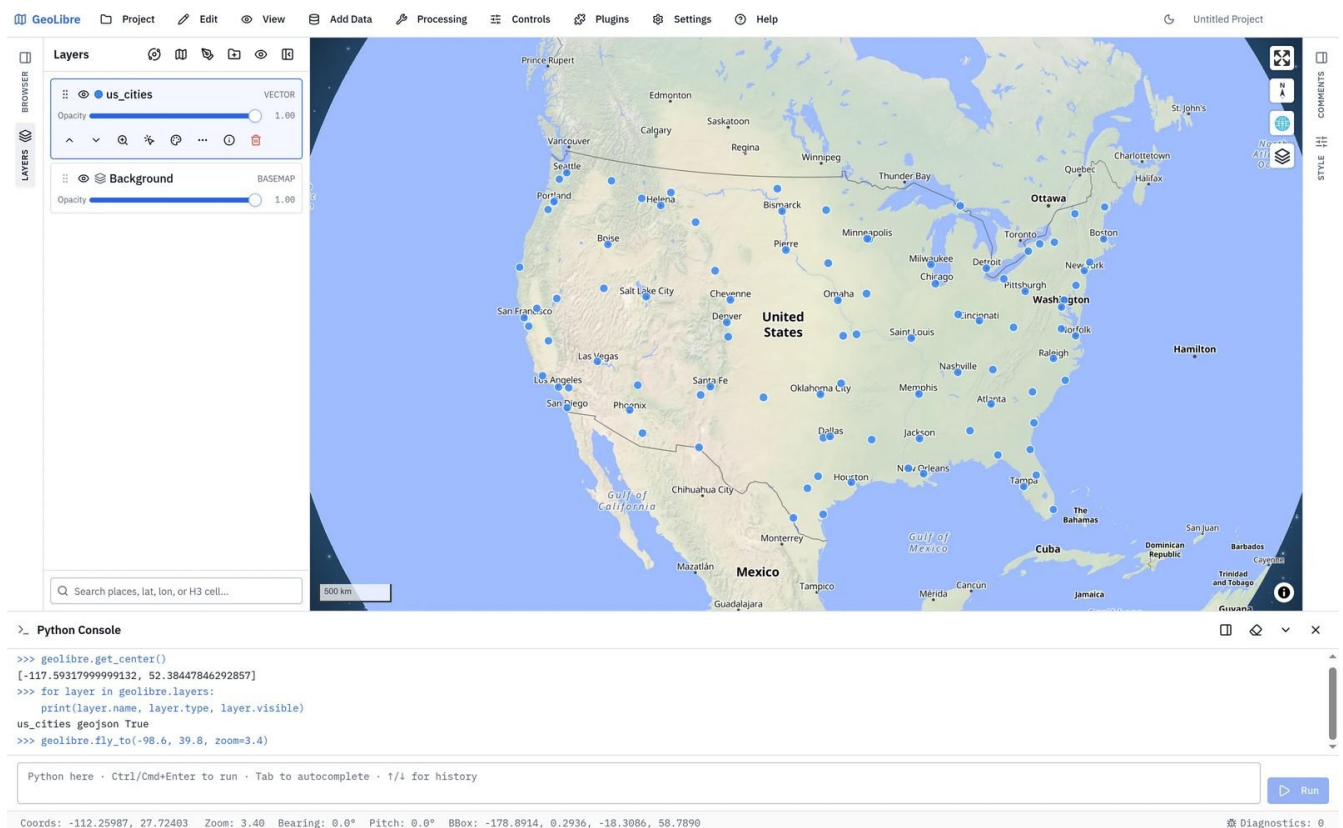
See the [Spatial SQL tutorial](#) for an end-to-end walkthrough. The SQL Workspace works in both the browser and the desktop app because it runs on DuckDB-WASM.

<https://geolibre.app/user-guide/python-console/>

Python Console

The **Python Console** runs Python right inside the app, against the map you are looking at. Open it from **Processing** → **Python Console**. It docks as a resizable panel at the bottom of the window (drag its top edge to resize, and the × button to close it) — it does not block the rest of the app.

Python runs in your browser via [Pyodide](#) (CPython compiled to WebAssembly), so it works in both the web and desktop builds with nothing to install. The console exposes a single object, `geolibre`, that drives the live app — its methods mirror the [geolibre Python package](#), so what you learn here transfers to notebooks and back.



First run

The Python runtime downloads the first time you open the console (a one-time download of a few MB, shown as a progress message in the header). After that it stays warm, and your variables persist as long as the app is open — even if you close and reopen the panel.

Type code in the input box and press **Ctrl/Cmd + Enter** (or click **Run**) to execute it. Output, return values, and errors appear in the scrollbar above.

Editing shortcuts

Key	Action
Ctrl/Cmd + Enter	Run the current code
Enter	Newline (multi-line editing)
↑ / ↓	Recall previous / next command (when the caret is on the first / last line)
Tab or Ctrl + Space	Autocomplete the name or attribute at the caret

Autocomplete introspects the **live** runtime, so `geolibre.` lists its real methods, and your own variables and any imported modules complete too. When more than one candidate matches, a list appears — use **↑ / ↓** to choose and **Enter/Tab** to accept, or **Esc** to dismiss.

```
geolibre.get_center()          # [lng, lat] of the current view
geolibre.get_bounds()         # [west, south, east, north]
geolibre.fly_to(-122.4, 37.8, zoom=10)
```

Script editor

For multi-line scripts you want to keep, click **Show editor** (the panel icon in the header) to open a script editor beside the console — like QGIS's Python editor. Drag the divider to resize it, and click the icon again to hide it.

The editor **shares the console's interpreter**, so a variable or function you define in a script is immediately usable in the console, and vice-versa.

- **New / Open / Save / Save As** work with `.py` files. On the desktop app, Save writes back to the open file; in the browser it downloads (or uses the File System Access API). An unsaved file shows a `•` next to its name.
- **Run** (or **Ctrl/Cmd+Enter**) executes the whole script — or just the **selected** lines if you have a selection. Output and errors appear in the console on the right.
- **Tab** indents, **Ctrl+Space** autocompletes, and **Ctrl/Cmd+S** saves.

Driving the map

```
# Add data (a GeoJSON dict, a geometry, or anything with __geo_interface__)
layer_id = geolibre.add_geojson(
    {"type": "Point", "coordinates": [-122.4, 37.8]},
    name="Pin",
    fillColor="#facc15", # style overrides are applied in the same call
)

# Style, toggle, and inspect layers
layer = geolibre.get_layer(layer_id)
layer.opacity = 0.6
layer.visible = False
layer.set_style(circleRadius=8, fillColor="#ff0000")
```

```
for layer in geolibre.layers:
    print(layer.name, layer.type, layer.visible)

# Identify features at a point (like clicking the map)
hits = geolibre.identify(-122.4, 37.8)
```

Async operations

The console supports top-level `await`. A few operations are asynchronous and must be awaited:

```
# Fetch a remote GeoJSON URL and add it
await geolibre.load_geojson("https://example.com/data.geojson", name="Data")

# Run a processing algorithm; result layers are added to the map
geolibre.list_algorithms() # discover ids + parameters
result = await geolibre.run_algorithm("buffer", {"layer": layer_id, "distance": 1000})
print(result["logs"], result["resultLayerIds"])
```

Loading more packages

To keep startup fast, only the base runtime loads up front. Pull in additional packages on demand:

```
await geolibre.load_package("numpy")
import numpy as np
np.array([1, 2, 3]).mean()

await geolibre.load_package("geopandas") # also pulls shapely/pyproj/pandas
```

Any package in the [Pyodide distribution](#) is available this way.

geolibre API reference

Method	Description
<code>get_view()</code>	Live camera {center, zoom, bearing, pitch, bbox}.
<code>get_center()</code>	Live map center [lng, lat].
<code>get_bounds()</code>	Live viewport bounds [west, south, east, north].
<code>fly_to(lng, lat, zoom=, bearing=, pitch=, duration=)</code>	Animate the camera; only the given fields change.
<code>fit_bounds([w, s, e, n])</code>	Fit the camera to a bounding box.
<code>set_basemap(url)</code>	Set the basemap style (an http(s) or root-relative URL).
<code>identify(lng, lat, layer_id=None)</code>	Query rendered features at a point (like a click).
<code>add_geojson(data, name=, **style)</code>	Add a layer from a GeoJSON dict / geometry / <code>__geo_interface__</code> , with optional inline style overrides; returns the layer id.

Method	Description
<code>await load_geojson(url, name=, **style)</code>	Fetch a GeoJSON URL and add it, with the same optional inline style overrides; returns the layer id.
<code>layers</code>	List of <code>Layer</code> objects, in draw order.
<code>get_layer(layer_id)</code>	The <code>Layer</code> with that id (raises if absent).
<code>remove_layer(layer_id)</code>	Remove a layer by id.
<code>list_algorithms()</code>	Available processing algorithms (<code>id</code> , <code>name</code> , <code>group</code> , <code>parameters</code>).
<code>await run_algorithm(id, parameters=None)</code>	Run an algorithm; adds result layers and returns <code>{logs, resultLayerIds}</code> .
<code>to_image()</code>	Capture the current map as PNG bytes .
<code>await load_package(name)</code>	Load a Pyodide package on demand.

Layer

A handle returned by `geolibre.layers` / `geolibre.get_layer(...)`.

Member	Description
<code>id</code> , <code>name</code> , <code>type</code>	Identity (read-only).
<code>visible</code>	Get/set visibility.
<code>opacity</code>	Get/set opacity (0–1).
<code>set_style(**style)</code>	Merge style overrides (e.g. <code>fillColor="#ff0000"</code>).
<code>get_features()</code>	The layer's features as <code>Feature</code> objects.
<code>zoom_to()</code>	Fit the camera to the layer's extent.
<code>remove()</code>	Remove the layer from the map.

Feature

A GeoJSON feature returned by `Layer.get_features()`. It is a plain dict (so it serializes and feeds into `geopandas.GeoDataFrame.from_features()`), with convenience accessors `.geometry`, `.properties`, `.id`, and `__geo_interface__`.

Notes & limitations

- **Runs on the main thread**, by design, so Python can drive the live map synchronously. A long-running cell briefly pauses the UI — avoid tight loops and prefer `await geolibre.run_algorithm(...)` for heavy work.
- **Sandboxed**. Pyodide cannot read your computer's filesystem or reach the desktop app's local tools; network access follows the app's content-security policy. `to_image()` therefore returns bytes rather than writing a file.

- **Same API as notebooks.** The [geolibre Python package](#) exposes the same methods from a Jupyter `Map` object, so console snippets and notebook code are interchangeable.

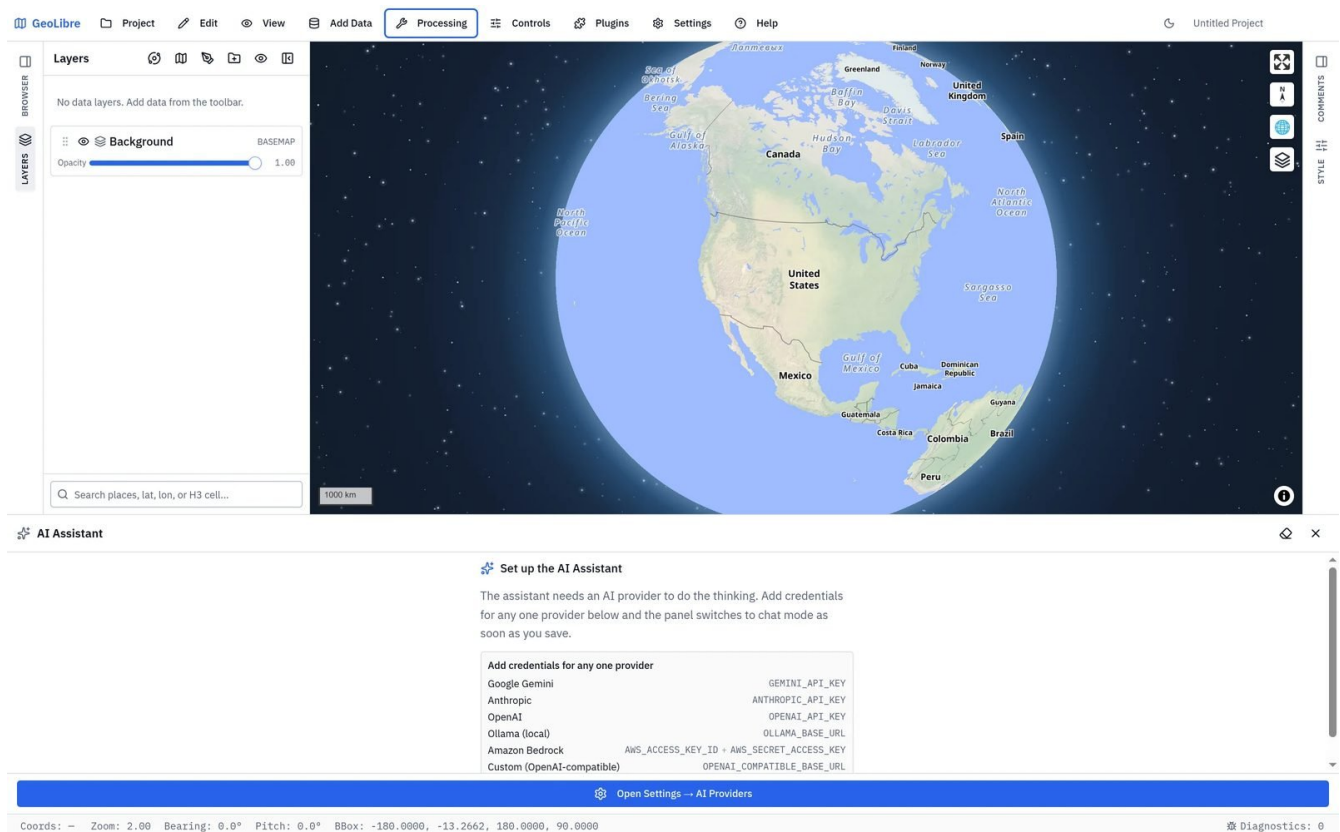
<https://geolibre.app/user-guide/ai-assistant/>

AI Assistant

The **AI Assistant** is a "chat with your data" panel that turns plain-English requests into GeoLibre's own operations — Spatial SQL, layer styling, map control, and more — and applies them **through the app**, the same way you would by hand. Open it from **Processing → AI Assistant** (top of the menu) or the command palette. It docks as a resizable panel at the bottom of the window (drag its top edge to resize, × to close).

Because the assistant acts through the store rather than poking the map directly, almost everything it does is **undoable** with **Ctrl/Cmd + Z**, and every tool call (including the SQL it generates) is shown in the transcript so you can see exactly what ran.

The assistant is **optional and disabled until you configure a provider or the deployment operator enables the managed AI proxy**. No data leaves your machine until you send a prompt to the configured provider.



Setup: choose an AI provider

The assistant is **provider-pluggable** — it uses the [Strands Agents](#) SDK. Configure one or more providers in **Settings → AI Providers**:

Provider	Environment variable(s)	Default model
Google Gemini	GEMINI_API_KEY or GOOGLE_API_KEY	gemini-3.6-flash
Anthropic	ANTHROPIC_API_KEY	claude-opus-5
OpenAI	OPENAI_API_KEY	gpt-5.6
OpenRouter	OPENROUTER_API_KEY (+ optional OPENROUTER_MODEL)	openai/gpt-5.6-luna
Ollama (local)	OLLAMA_BASE_URL (e.g. http://localhost:11434)	gemma4
Amazon Bedrock	AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY (+ AWS_REGION , optional AWS_SESSION_TOKEN)	global.anthropic.claude-opus-5
Custom (OpenAI-compatible)	OPENAI_COMPATIBLE_BASE_URL (+ optional OPENAI_COMPATIBLE_API_KEY) and OPENAI_COMPATIBLE_MODEL	—

- **Ollama** runs models on your own machine — no API key and nothing leaves your computer. Point `OLLAMA_BASE_URL` at your Ollama host (the `/v1` suffix is added automatically); set `OLLAMA_MODEL` to pick which pulled model to use.
- **Bedrock** calls AWS from the browser using your credentials (and the model id is an inference-profile id such as `global.anthropic.claude-sonnet-4-6` ; set `BEDROCK_MODEL` to choose another).
- **OpenRouter** calls its Chat Completions API from your browser. Its model picker fetches the public catalog without sending your key; search by model name or id, or enter an unlisted `provider/model` id. Set `OPENROUTER_MODEL` to choose a different default.
- **Custom** covers any OpenAI-compatible endpoint — LiteLLM, vLLM, Groq, Together, a local server, etc. — via its chat-completions API.

Hosted keys (and AWS credentials) are used **directly from your browser** to call the provider; they are never sent to GeoLibre's servers. Saving the setting enables the panel immediately — no reload needed.

Using local Ollama from web.geolibre.app

The hosted web app connects directly from your browser to Ollama on your computer. Prompts and responses do not pass through a GeoLibre server. Because the page origin is `https://web.geolibre.app` , Ollama must explicitly allow that origin before the browser can call its API.

Set this environment variable for the Ollama server, then restart Ollama:

```
OLLAMA_ORIGINS=https://web.geolibre.app
```

Then open **Settings → AI Providers**, add or edit an **Ollama** profile, and use this Base URL:

```
http://localhost:11434
```

Configure `OLLAMA_ORIGINS` for your operating system:

Linux (systemd)

Run `sudo systemctl edit ollama.service` and add:

```
[Service]
Environment="OLLAMA_ORIGINS=https://web.geolibre.app"
```

Save the file, then reload and restart the service:

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

macOS

Set the variable for the Ollama application, then quit and reopen Ollama:

```
launchctl setenv OLLAMA_ORIGINS "https://web.geolibre.app"
```

Windows

1. Quit Ollama from the taskbar.
2. Open **Edit environment variables for your account**.
3. Create a user variable named `OLLAMA_ORIGINS` with the value `https://web.geolibre.app`.
4. Start Ollama again from the Start menu.

Docker

Pass the allowed origin when starting the container:

```
docker run -d \
  -e OLLAMA_ORIGINS=https://web.geolibre.app \
  -p 11434:11434 \
  ollama/ollama
```

For a self-hosted GeoLibre URL, replace `https://web.geolibre.app` with the exact origin shown in GeoLibre's error message, including its scheme and port, for example `http://192.168.1.98:4000`. This is an Ollama server permission and cannot be enabled automatically by a web page. See Ollama's [official server configuration and web-origin instructions](#).

Managed AI in a password-protected Docker deployment

A deployment operator can provide AI without distributing provider API keys. In this configuration, the browser calls the same-origin `/ai` route. Docker's nginx checks the instance's HTTP Basic Auth username and password when the operator has configured it, then adds a server-only instance token and forwards the request to `ai.geolibre.app`. The browser receives neither the instance token nor the Cloudflare AI Gateway token.

Users only need to sign in to the GeoLibre instance; they do not configure an AI provider key. Requests sent directly to `ai.geolibre.app` without the server-only token receive `401 Unauthorized`.

The operator must start the container with all managed-AI variables:

```
docker run --rm -p 8080:80 \
  -e GEOLIBRE_AUTH_USER=admin \
  -e GEOLIBRE_AUTH_PASSWORD='change-me' \
  -e GEOLIBRE_AI_URL=/ai \
  -e GEOLIBRE_AI_MODEL=openai/gpt-5.6-luna \
  -e GEOLIBRE_AI_PROXY_URL=https://ai.geolibre.app \
  -e GEOLIBRE_AI_PROXY_TOKEN="$GEOLIBRE_AI_PROXY_TOKEN" \
  ghcr.io/opengeos/geolibre:latest
```

If `GEOLIBRE_AI_URL` is unset, the Docker entrypoint leaves the managed proxy disabled and does not inject any AI proxy URL into the application.

For news search in the NASA OPERA disaster workflow, add `TAVILY_API_KEY` as a secret on the `geolibre-ai-proxy` Worker. Do not pass that key to the container with `docker run -e`: the container never reads it. The browser uses the same authenticated `/ai` route, and nginx keeps both the Worker token and Tavily key out of frontend configuration. The desktop app reads a variable of the same name for [its own web search](#), which is a separate client-side mechanism unrelated to this one.

Optional variables:

Variable	Purpose
<code>GEOLIBRE_ASSISTANT_PROVIDER</code>	Force a provider (<code>google</code> / <code>anthropic</code> / <code>openai</code> / <code>openrouter</code>) when several keys are set.
<code>GEOLIBRE_ASSISTANT_MODEL</code>	Pin a specific model id, overriding the default and the picker.

When more than one provider key is configured, a **provider** dropdown appears in the panel header; a **model** dropdown lets you switch models for the selected provider. Your choice is remembered across sessions.

Reading keys from your system environment (desktop)

On the **desktop app**, GeoLibre also reads the assistant's keys directly from your operating system's environment variables — the ones you export from your shell profile or set in the Windows *Environment Variables* dialog. Set a supported variable in your OS environment, restart the app, and the matching provider is configured automatically: you never type the key into Settings, and because it is not entered there, it is **never written to the saved** `.geolibre.json` **project file**. This is the recommended way to keep API keys out of project files that you share or commit.

Only the following allowlisted names are read from your environment — GeoLibre never reads any other system variable (your `PATH`, `HOME`, and the like never reach the app), and this allowlist is enforced in the native backend, not just the UI:

Group	Variables read from the OS environment
Provider / model overrides	GEOLIBRE_ASSISTANT_PROVIDER , GEOLIBRE_ASSISTANT_MODEL
Google Gemini	GEMINI_API_KEY , GOOGLE_API_KEY , GOOGLE_GENAI_API_KEY
Anthropic	ANTHROPIC_API_KEY
OpenAI	OPENAI_API_KEY
OpenRouter	OPENROUTER_API_KEY , OPENROUTER_MODEL
Ollama	OLLAMA_BASE_URL , OLLAMA_MODEL
Custom (OpenAI-compatible)	OPENAI_COMPATIBLE_BASE_URL , OPENAI_COMPATIBLE_API_KEY , OPENAI_COMPATIBLE_MODEL
Web search	TAVILY_API_KEY
Fast path and tool search	JEV_API_KEY

Precedence: a value you enter in **Settings → Environment Variables** always wins; the OS environment only fills in the gaps. In the AI settings, any field supplied by the environment shows a note naming the variable backing it — leave that field blank to keep using the environment value.

Amazon Bedrock is not sourced from the OS environment

Bedrock's `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` credentials (and the ambient `OLLAMA_HOST`) are deliberately **excluded** from OS-environment reading: developers commonly have AWS credentials exported in their shell for unrelated work, and silently adopting them could auto-activate Bedrock and bill their AWS account for LLM calls they never intended. To use Bedrock, enter the credentials in **Settings → Environment Variables**.

Desktop only

Reading OS environment variables requires the native backend, so it applies to the **desktop app** only. The browser and Jupyter builds cannot read the system environment; there, enter keys in **Settings → Environment Variables** (or bake them in at build time). Changes to OS variables are picked up on the next app launch.

Using it

Type a request and press **Enter** (or click **Send**); **Shift + Enter** inserts a newline, and **Ctrl/Cmd + Enter** still sends. The assistant streams its reply, runs tools as needed, and reports what it did. While it is working, **Send** becomes **Stop** — click it to cancel. **Clear** (the eraser icon) starts a fresh conversation.

```
show me all parcels larger than 1 hectare within 500 m of a river
color the counties by population using a graduated red ramp
buffer the roads by 100 m, then clip them to the county boundary
load the latest Sentinel-2 scene over this view
```

```
zoom to Africa, then switch to a dark basemap  
add an OpenTopoMap basemap
```

Fast path for simple commands (optional)

A handful of requests are pure routing — "hide the rivers", "make the basemap dark", "zoom to the counties layer". They need no reasoning, no SQL and no code, just an intent and a layer id. Sending them through the full model still costs a round trip carrying the system prompt and every tool definition.

If a [TypeSafe](#) credential is configured, GeoLibre routes those requests with **Jev**, a model that returns typed judgments instead of generated text, and then runs the very same tool the model would have called. Measured against this repo's own prompt, that turns a **~3.8 s** wait on a hosted model (or **~0.8 s** on a local one) into **~190 ms**.

It covers exactly six commands:

Command	Example
Switch the basemap style	"make the basemap dark"
Add a named tile basemap	"add OpenTopoMap"
Show or hide a layer	"hide the rivers"
Change a layer's opacity	"set the cities layer to half transparent"
Zoom to a layer	"zoom to the counties layer"
Remove a layer	"drop the elevation raster"

Everything else — querying data, geoprocessing, styling by attribute, adding data, running code — is classified as complex and goes to your LLM exactly as before. So does any request the router is not confident about, any layer it cannot match, and any request at all if TypeSafe is slow, unreachable or not configured. The fast path can never be the reason a request fails.

It is not free for those requests, though: routing happens **before** the model request starts, so a complex request waits for the routing answer (typically under 200 ms, and abandoned after 1.5 s) before its LLM round trip begins. On a multi-second model turn that is a small share of the total, but it is a real cost — which is the other reason the fast path stays off unless you enable it.

Because every action runs through the same tool as the slow path, fast-path changes appear in the transcript and are **undoable** like any other.

Enabling it

Desktop

Set `JEV_API_KEY` in **Settings → Environment Variables**, or export it in your shell before launching the app (see [Reading keys from your system environment](#)). The desktop app reaches TypeSafe through Tauri's native HTTP client.

Web / Docker

`api.typesafe.ai` refuses browser origins, so a browser build **cannot** call it directly — a personal `JEV_API_KEY` has no effect there. Instead the deployment operator sets `JEV_API_KEY` on the [AI proxy Worker](#), which exposes the routing endpoint at `/systemone` with the credential held server-side. Deployments already using the managed AI proxy pick this up with no client configuration.

```
cd workers/ai-proxy
npx wrangler secret put JEV_API_KEY
```

Leaving the secret unset disables the route; the assistant keeps working through the model as usual. A routing endpoint on **another origin must use HTTPS** — a plain-HTTP one is refused, because the answer it returns decides which tool the assistant runs. Loopback and an endpoint on the app's own origin are the exceptions: there, HTTPS would protect nothing that an attacker in a position to read the request does not already have.

Local development

A browser dev server can reach neither endpoint on its own: TypeSafe refuses `http://localhost:5173` as an origin, and the Worker wants an instance token a page cannot supply. So `npm run dev proxies /systemone` itself when you give it a credential, and points the app at that route:

```
# through the deployed Worker, which holds the TypeSafe key (preferred –
# this is the same path production uses)
GEOLIBRE_AI_PROXY_TOKEN=... npm run dev

# or straight to TypeSafe with your own key
JEV_API_KEY=... npm run dev
```

The dev server prints which one it is using at startup, and neither value enters the client bundle. With neither set the route is not registered and the fast path stays off. `npm run tauri:dev` needs none of this: the desktop build reads `JEV_API_KEY` from your environment and reaches TypeSafe through Tauri's native HTTP client.

Note that enabling the fast path means the text of a prompt and your **layer names** are sent to TypeSafe for routing. On a local-Ollama setup, where nothing otherwise leaves your machine, that is a real trade-off — which is why the fast path is off unless you configure it.

Finding the right Whitebox tool (optional)

The same credential also changes how the assistant searches the **Whitebox catalog** — 775 raster, terrain, hydrology, LiDAR and imagery tools, far too many to put in front of a model at once.

Without it, the assistant filters that catalog by substring, which only works when it already guesses the word the catalog uses. Ask for something in ordinary language and the filter finds nothing: no tool is called "grainy", so "get rid of the grainy speckle in a radar image" matches none of the four speckle filters the catalog actually ships.

With a credential configured, the search is also answered by meaning. Two questions go to Jev — which part of the catalog, then which tool inside it — and the ranked answer is merged in front of the substring hits, which keep working unchanged. Measured over 20 raster requests:

The assistant searches for	Right tool found	Right tool listed first
a keyword, substring only (before)	19/20	8/20
a keyword, merged	19/20	15/20
the request in plain words, substring only (before)	0/20	0/20
the request in plain words, merged	19/20	19/20

The lookup adds about **400 ms** to a tool call the model is already waiting on, and like the fast path it can only add candidates: if TypeSafe is unreachable, slow or unconfigured, the search is exactly the substring filter it always was.

This sends the assistant's search text to TypeSafe. Unlike the fast path it does **not** send your layer names — the questions are asked against the tool catalog, which is public data shipped with the app.

Tool summaries

Both searches read each tool's `summary`, and 770 of the 775 in the bundled catalog have one. The five that do not — `assign_projection_lidar`, `assign_projection_raster`, `assign_projection_vector`, `reproject_lidar` and `reproject_raster` — are in the tool taxonomy but not in the Whitebox runtime catalog the summaries come from, so they are matched on their name and category alone.

Voice commands

The assistant also listens. When your browser supports speech recognition, the composer gains a **microphone** button, and you can talk to the map instead of typing. What you say runs through exactly the same agent and the same tools as a typed request — so it is still auditable in the transcript and still undoable.

There are two ways in, and they behave differently on purpose:

Gesture	Mode	How it ends
Click the microphone	Open mic — it keeps listening, and each finished sentence is sent on its own	Click it again
Hold Space for half a second	Push-to-talk — one turn	Release Space

Push-to-talk is deliberately a *hold*. A short tap on Space still activates whatever control has focus, and a space typed in the composer is always just a space, so the shortcut never gets in the way of ordinary use. The hold is only armed while the assistant panel is open.

Answers are read back aloud, and the **speaker** button next to the microphone turns that off (the choice is remembered). Only *spoken* questions get spoken answers — typing while the microphone happens to be open stays silent. Code blocks and URLs are skipped when reading, and long answers are trimmed to their first few sentences; the full reply is always in the transcript.

A few things worth knowing:

- **The microphone closes while an answer is being read**, so the assistant cannot transcribe itself and answer its own reply. To interrupt a push-to-talk answer, hold Space again — the new turn stops the playback at once. An open mic reopens by itself when the answer ends, and the button stops it sooner; Space deliberately leaves an open mic alone, so a stray key press can never cut off a session you started with the button.
- **Speaking again while it is still working cancels that run** and starts over with what you just said, the same way sending a new message would.
- **Recognition is your browser's, not GeoLibre's**. Chrome, Edge and Safari provide it; Firefox and the desktop (Tauri) builds do not, and the microphone button is simply not shown there. Note that Chrome's implementation sends audio to Google's speech service for transcription — separate from your configured AI provider, and subject to Google's terms. The text it returns is then sent to your provider like any typed prompt.
- **It listens in the app's language**, refined by your browser's regional variant when they agree (`pt` with a `pt-BR` browser listens as `pt-BR`).

What it can do

The assistant works by calling a fixed set of tools — it cannot invent operations, so its actions stay within GeoLibre's validated surface.

Capability	What it does
Inspect layers	Lists loaded layers, their geometry, attribute fields, and SQL table names (schema only — never your full data).

Capability	What it does
NL → Spatial SQL	Generates and runs a read-only DuckDB Spatial SQL query through the SQL Workspace , and can add the result as a layer.
Geoprocessing	Runs the registered processing algorithms (buffer, clip, dissolve, intersection, difference, union, spatial join, simplify, H3 grids, ...) and chains them into multi-step pipelines, adding each result as a layer.
Raster tools	Searches the Whitebox catalog and runs its tools in the browser via WASM — hydrology (fill depressions, flow accumulation, extract streams), terrain (slope, aspect, hillshade), LiDAR, image processing, raster↔vector conversion — adding each result as a layer.
Model Builder	Creates a validated, editable Model Builder workflow from a description, saves it with the project, and opens it for review before you run it. It draws on the same palette as the canvas — client-side vector tools plus the full Whitebox catalog — so a raster chain such as fill depressions → flow accumulation → extract streams is a valid model. If the canvas holds unsaved work or a run is still in flight, Model Builder asks before replacing it.
Symbology	Applies a graduated (numeric) or categorized (text) color ramp to a layer.
Add data	Adds a layer from a public GeoJSON URL, or an XYZ tile basemap by name (<code>osm</code> , <code>opentopomap</code> , <code>carto-dark</code>) or a custom <code>{z}/{x}/{y}</code> URL.
Earth observation	Searches the Microsoft Planetary Computer STAC catalog (Sentinel-2, Landsat, NAIP, DEMs, ...) and adds an item over the current view as a raster layer — tiles are signed server-side, so no credentials are needed.
Map control	Moves the camera (fit a layer or a bounding box), switches the basemap, toggles layer visibility/opacity, and removes layers.
Web search	Looks up current information online (best with <code>TAVILY_API_KEY</code>).
Code fallback	For tasks with no dedicated tool, runs a small JavaScript snippet against the live map (e.g. globe projection) or a Python snippet in the Pyodide runtime .

Sample prompts

Prompts are free-form — these are starting points, not fixed commands. Refer to layers by name; the assistant looks up the rest. You can also chain steps in one message ("buffer the roads by 100 m **and then** clip to the county boundary") or keep refining across turns ("now color it by area").

Explore & query

```
what layers are loaded, and what fields does the parcels layer have?  
how many parcels are larger than 1 hectare?  
list the 10 most populous counties with their population  
show parcels within 500 m of a river and add them as a layer  
count points in each polygon of the districts layer
```

Geoprocessing & analysis

```
buffer the roads by 100 meters  
buffer the roads by 100 m, then clip the buffer to the county boundary  
create a Model Builder model that buffers roads by 100 m, clips the result to counties, and  
names the output Road buffers  
dissolve the parcels by zoning type  
find where the floodplain overlaps the buildings (intersection)  
create an H3 hex grid at resolution 8 over the points and count points per cell  
compute centroids of the counties and add them as a layer
```

Symbology

```
color the counties by population with a graduated red ramp  
style the parcels categorized by land-use type  
shade the tracts by median income using a viridis ramp with 7 classes
```

Add data & imagery

```
load the latest Sentinel-2 scene over this view  
add the most recent cloud-free Landsat image for this area  
search the Planetary Computer for NAIP imagery here  
add an OpenTopoMap basemap  
add this GeoJSON: https://example.com/data.geojson
```

Map control & styling

```
zoom to the parcels layer  
fly to San Francisco  
switch to a dark basemap  
hide the buildings layer and set the parcels opacity to 0.5  
remove the temporary buffer layer
```

Advanced (code fallback)

```
switch the map to a 3D globe projection  
enable terrain with hillshade exaggeration of 1.5  
load a CSV from a URL with pandas and summarize its columns
```

Safety and privacy

- **Acts through the store.** Layer, style, basemap, and add/remove actions go through the same one-way data flow as the rest of the app, so they are reconciled consistently and covered by **undo/redo**.
- **Auditable.** The generated SQL and every tool call appear in the transcript.
- **Read-only SQL.** The `run_sql` tool rejects anything that isn't a `SELECT / WITH` query.

- **Scoped context.** Only layer/table **names**, attribute **field names**, and the current view are sent to the model — not your feature data.
- **What leaves your browser.** When you send a prompt, it (plus that scoped context) is sent to your chosen LLM provider using your own key. Don't enable the assistant on sensitive data you can't share with that provider. A TypeSafe credential adds a second destination for two things: the [fast path](#) sends the prompt and your **layer names** there for routing, and [tool search](#) sends the text the assistant is **searching the tool catalog for** (that one carries no layer names). Both are off unless you configure them, and GeoLibre refuses a routing endpoint that is plain HTTP on another origin.

Code-execution caveat

The JavaScript and Python fallbacks execute model-generated code in the app to cover requests no dedicated tool handles. Their direct map changes bypass the store and are **not undoable**. The code is shown in the transcript.

Limitations

- Requires a provider API key; offline use is not supported.
- Voice commands need a browser with the Web Speech API (Chrome, Edge, Safari); they are unavailable in Firefox and in the desktop app.
- Subject to each provider's cost, rate limits, and your network's CORS/CSP policy (browser-side calls).
- The unofficial Google Maps tile endpoints are intentionally **not** included; use the listed officially-supported basemaps or supply your own XYZ URL.

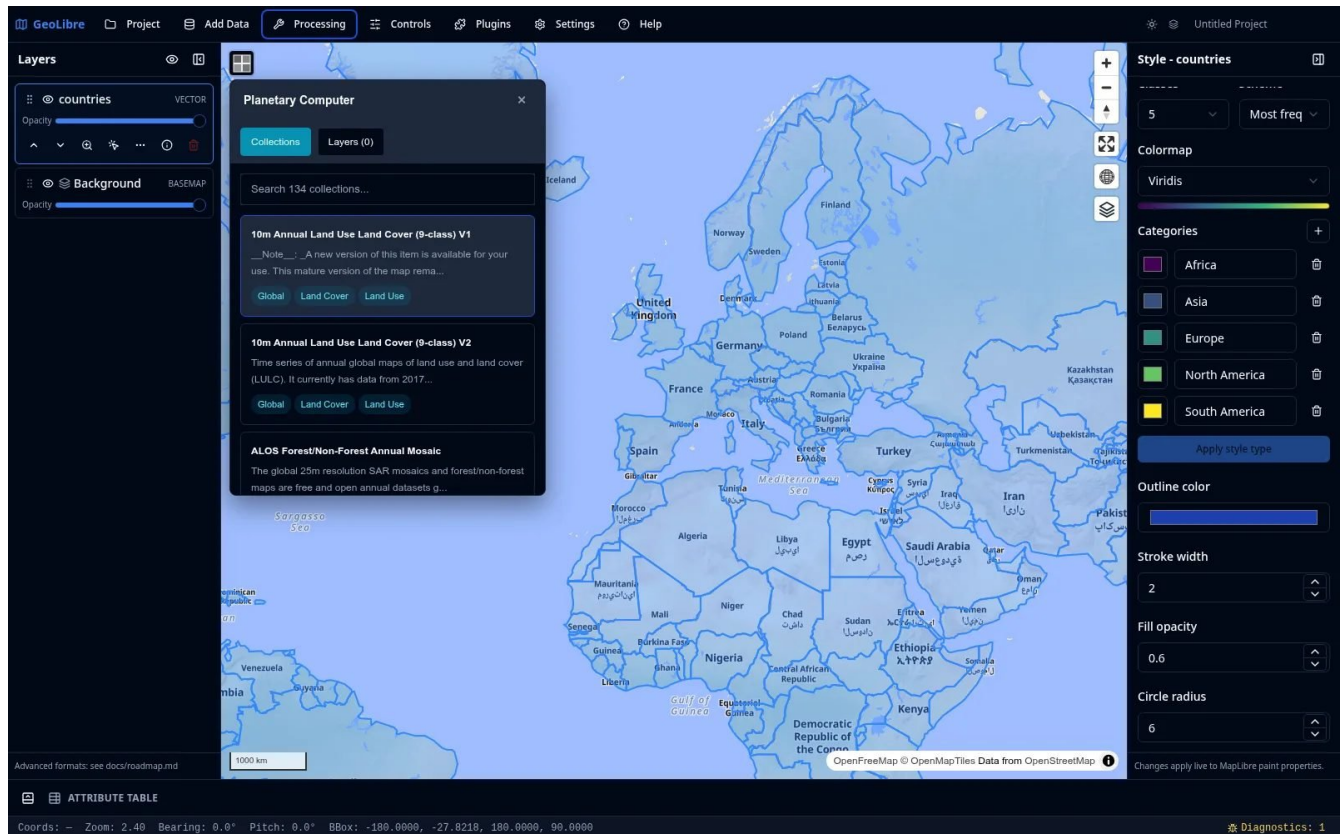
Not the same as the agent skill

The assistant runs **inside** GeoLibre and edits the map you are looking at. To have an AI agent *outside* GeoLibre — in a terminal, an editor, or a notebook — build a project file for you, see the [agent skill](#) and the [MCP server](#).

<https://geolibre.app/user-guide/data-integrations/>

Data Integrations

Beyond the [Add Data](#) menu, GeoLibre connects to several hosted catalogs and imagery providers through dedicated panels and plugins. This page is a map of what is available and where to find it.



Cloud catalogs

Integration	Where	What it does
Planetary Computer	Processing menu	Browse and load STAC data from Microsoft Planetary Computer (Sentinel, Landsat, and more).
Earth Engine	Processing menu	Browse and load Google Earth Engine datasets after authenticating.
Overture Maps	Plugins menu	Load Overture Maps data themes (such as buildings, places, and transportation).
STAC	Add Data menu	Search any STAC catalog and add matching raster items. See Adding Data .

Credentials

Earth Engine requires authentication, and some providers expect an API key or token. Set these in **Settings → Environment Variables**. See [Settings & Preferences](#).

Self-hosted catalogs

Integration	Where	What it does
GeoLens	Plugins menu	Connect to a self-hosted GeoLens catalog, search its datasets, and add them as signed vector tiles, OGC API Features GeoJSON, or server-rendered raster tiles.

GeoLens is an open-source spatial catalog (FastAPI + PostGIS) that you run on your own infrastructure, which makes it the recommended way to keep private data private while still working with it in GeoLibre. In the plugin panel, enter your server's base URL (for example `https://maps.example.org`) and, for private datasets, a GeoLens API key. Each result links back to its metadata page, vector tile tokens are refreshed automatically before they expire, and a dataset loaded as GeoJSON can be edited and written back to GeoLens feature by feature when the server allows it.

The API key is kept in memory for the session only: it is never written to the project file or to browser storage, so a saved project records just the server URL and dataset id. Layers from public datasets restore automatically; layers from private ones stay blank until the recipient reconnects with their own key.

See [Self-Hosting & Private Data](#) for the full deployment guide, including how to serve GeoLibre and GeoLens from one origin behind a single sign-on layer.

Web Services

The **Web Services** submenu of the [Plugins menu](#) bundles more than thirty catalog and service browsers, from the United States federal sources below to general-purpose STAC, ArcGIS Hub, Source Cooperative, and Hugging Face browsers. All of them are documented on the [Web Services](#) page.

Service	Data
FEMA NFHL	National Flood Hazard Layer (NFHL) flood data.
NASA Earthdata	NASA satellite and Earth science imagery.
EPA EnviroAtlas	Environmental and ecosystem data.
USGS National Map	The National Map topographic and geographic layers.
USGS NLDI	Flowline tracing, hydrolocation, basins, and network navigation. See USGS NLDI workflows .

Imagery and street-level

Integration	Where	What it does
Historical Imagery	Plugins menu	Browse historical Esri World Imagery snapshots.
Vantor Open Data	Plugins → Web Services	Search disaster-event satellite imagery, filter pre/post-event scenes by map extent, visualize Cloud-Optimized GeoTIFFs, and download selected scenes.
Planet Open Data	Plugins → Web Services	Browse Planet Labs PBC disaster data releases through the STAC Catalogs interface, with the public disaster catalog selected by default.
Street View	Plugins menu	View Google Street View or Mapillary street-level imagery at a clicked point. Needs provider credentials (see Getting Started).
Mapillary	Plugins menu	Browse Mapillary street-level imagery in its own viewer. Needs a Mapillary access token (see Getting Started).

Time series and comparison

Plugin	What it does
Time Slider	Animate time series raster and vector data (COG, XYZ/WMTS, WMS-Time, and time-filtered GeoJSON) through a docked timeline.
Layer Swipe	Compare two layers side by side with a swipe handle.

AI analysis

Plugin	What it does
GeoAgent	AI-assisted geospatial analysis.

The plugins under **Imagery and street-level**, **Time series and comparison**, and **AI analysis** are activated from the [Plugins menu](#) (the integrations further up this page are reached from the Processing and Add Data menus instead). Most of them, like the Web Services panels, open docked beside the Layers panel; only **Street View** and **Mapillary** place a control on the map, so only they offer a position choice in their Plugins submenu.

Geocoding

GeoLibre can turn addresses into points and points into addresses. Both run through a selectable provider; the public [Nominatim](#) service is the default and needs no key.

Tool	Where	What it does
Geocode Address	Processing → GeoLibre Toolbox	Pick a CSV with an address column and geocode each row into a point layer. Each matched row keeps its original columns plus <code>geocode_lat</code> , <code>geocode_lon</code> , <code>geocode_display_name</code> , and <code>geocode_importance</code> (a match score). A per-run provider picker lets you switch backend for that batch.
Delimited Text Layer → Address	Add Data	Geocode a CSV/TSV at import time instead of a separate step: choose "Addresses" as the import mode, pick one or more columns to concatenate into the address (e.g. street, city, state), and each row is geocoded through the project's configured provider. Matched rows become points; rows with no match are kept (not dropped) with <code>geocode_status</code> : "unmatched" so they stay visible and fixable in the attribute table.
Reverse Geocode	Control s menu	A toggle. While on, click anywhere on the map to look up the address at that point, shown in a popup with a copy button. The popup follows the app's light or dark theme.

All three send coordinates or addresses to a third-party service, so the first time you enable Reverse Geocode (and whenever you run a batch, including at CSV import time) your data leaves your device for those requests. Reverse Geocode shows a one-time notice before it is first enabled.

Providers

Choose a backend in **Settings → Geocoding**. The selection, per-provider API keys, optional endpoint overrides, and contact email are saved with the project.

Provider	API key	Notes
Nominatim (OpenStreetMap)	No	Default. Public endpoint is paced and row-capped (see below); point it at a self-hosted instance to relax both.
CartoCiudad (IGN España)	No	Official Spanish national geocoder (IGN/CNIG). Free public API, no key required.
Pelias	Optional	Hosted geocode.earth needs a key; a self-hosted Pelias does not.
ArcGIS World Geocoder	Yes	Esri token / API key.
Mapbox	Yes	Mapbox access token (pk....).
Google	Yes	Google Maps Geocoding API key. Google does not officially allow browser cross-origin requests to this API, so a same-origin proxy may be required.

API keys are stored in plain text in the `.geolibre.json` project file, so avoid sharing a project that carries them (the Project → Share flow can strip environment variables, but provider keys live under Geocoding settings).

Usage policy and limits

Requests to the public Nominatim endpoint are paced to one per second and a single batch run is capped at 1000 rows, in line with the [Nominatim usage policy](#). Browsers cannot set a `User-Agent`, so the app identifies itself through the page `Referer` and the optional `email` parameter.

The public CartoCiudad endpoint is paced to roughly five requests per second and is not row-capped. IGN publishes no rate limit, so this is a courtesy default rather than their policy: it keeps a large batch from bursting at a free public service without borrowing Nominatim's much stricter numbers.

Self-hosted Nominatim and the keyed providers (Mapbox, ArcGIS, Google, hosted Pelias) are not paced or capped by GeoLibre; their own quotas apply.

Configuring with environment variables

The Geocoding settings panel is the easiest way to configure a provider, but the same values can also be set as runtime environment variables (the same `VITE_`-prefixed mechanism used for [imagery credentials](#)). Explicit environment variables override the Settings panel.

Variable	Default	Purpose
<code>VITE_GEOCODER_PROVIDER</code>	<code>nominatim</code>	Provider id: <code>nominatim</code> , <code>cartociudad</code> , <code>pelias</code> , <code>arcgis</code> , <code>mapbox</code> , or <code>google</code> .
<code>VITE_GEOCODER_API_KEY</code>	<code>unset</code>	API key / access token for the selected provider.
<code>VITE_GEOCODER_ENDPOINT</code>	<code>provider default</code>	Forward (address to point) search endpoint override.
<code>VITE_GEOCODER_REVERSE_ENDPOINT</code>	<code>provider default</code>	Reverse (point to address) endpoint override.
<code>VITE_GEOCODER_EMAIL</code>	<code>unset</code>	Contact email sent as the <code>email</code> query parameter to identify your client to Nominatim.

<https://geolibre.app/user-guide/cloud-storage/>

Cloud Storage (Amazon S3)

GeoLibre reads data straight from Amazon S3 and S3-compatible object stores (MinIO, Cloudflare R2, Wasabi, Ceph). Public buckets need no setup; private buckets are read with an **S3 connection** you add under **Settings → Cloud Storage**.

Where S3 URLs work

Where	What to enter
Add Data → Raster Layer	<code>s3://bucket/path/image.tif</code> (COG / GeoTIFF)
Add Data → Vector Layer	The object URL, e.g. <code>https://bucket.s3.us-west-2.amazonaws.com/path/data.parquet</code>
Add Data → PMTiles	<code>s3://bucket/path/archive.pmtiles</code>
LiDAR (point clouds)	Through the S3 Browser, or <code>s3://bucket/path/cloud.copc.laz</code>
SQL Workspace	<code>read_parquet('s3://bucket/path/data.parquet')</code> , <code>ST_Read('s3://...')</code> , or a bare <code>FROM 's3://...'</code>
Plugins → Web Services → S3 Browser	Browse a bucket and add files with one click

Layers keep the `s3://` URI (or the object URL) you added, never a signed URL. GeoLibre signs each read when the layer loads, so a saved or shared project carries no credentials, and a signature never expires under an open project.

S3 connections

Open **Settings → Cloud Storage** and choose **Add connection**. Each connection has:

- **Credentials:** where its keys come from (see below).
- **Buckets:** the bucket names it signs for, comma-separated, with `*` as a wildcard (`data-*`). Leave it blank to use the connection for every bucket no other connection names. An exact bucket name beats a wildcard, and a wildcard beats a catch-all.
- **Region:** leave blank to auto-detect each bucket's region.
- **Endpoint** and **Path-style addressing:** for S3-compatible stores. Leave them blank for AWS.
- **Role to assume** and **External ID:** optional; see [IAM roles](#).

Test connection lists the first bucket named in the connection, or just checks that the credentials resolve when it names no bucket.

Credential sources

Source	Where	Notes
Access keys	Web and desktop	An access key ID, secret access key, and optional session token.
AWS profile	Desktop	A profile from <code>~/.aws/config</code> / <code>~/.aws/credentials</code> (or <code>AWS_CONFIG_FILE</code> / <code>AWS_SHARED_CREDENTIALS_FILE</code>). Blank uses <code>AWS_PROFILE</code> , else <code>default</code> .
Environment variables	Desktop	<code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code> , and <code>AWS_SESSION_TOKEN</code> from the environment GeoLibre was started with.
IAM role (instance/container)	Desktop	The role of the machine GeoLibre runs on. See IAM roles .
Anonymous (public)	Web and desktop	No signing, for public buckets on a custom endpoint.

AWS profiles can hold:

- static access keys;
- **IAM Identity Center (SSO)**, both `sso_session` and legacy `sso_start_url` profiles;
- `credential_process`, such as `aws-vault` or `granted`;
- IAM roles (`role_arn`).

Resolving any of these needs no AWS CLI.

Signing in with IAM Identity Center (SSO)

For an SSO profile, **Sign in with SSO** runs the same device sign-in as `aws sso login`. It opens your browser at the IAM Identity Center page and shows the code to confirm there. The token is cached in `~/.aws/sso/cache`, exactly where the AWS CLI keeps it, so signing in either way works for both. Tokens that carry a refresh token are refreshed automatically. When a session expires, reads fail with a message asking you to sign in again.

IAM roles

GeoLibre assumes IAM roles itself with AWS STS, so no AWS CLI is needed:

- **Role profiles:** a profile with `role_arn` and `source_profile` (chains included) or `credential_source` (`Environment`, `Ec2InstanceMetadata`, `EcsContainer`). `external_id`, `role_session_name`, and `duration_seconds` are honored. A profile with `web_identity_token_file` uses `AssumeRoleWithWebIdentity`. Profiles that set `mfa_serial` still need the AWS CLI (v2) on `PATH`, which prompts for the code.
- **Role to assume:** set a role ARN, plus an external ID if its trust policy requires one, on any connection. GeoLibre resolves the connection's credentials first (keys, a profile, the environment, or an instance role), then assumes the role, for example a cross-account read-only role.

- **IAM role (instance/container):** when GeoLibre runs on AWS, it uses the host's role in the order the AWS SDKs use:

1. web identity: `AWS_WEB_IDENTITY_TOKEN_FILE` with `AWS_ROLE_ARN` (EKS IRSA);
2. ECS and EKS Pod Identity container credentials;
3. the EC2 instance profile, through IMDSv2.

Temporary credentials are refreshed shortly before they expire.

Desktop only

AWS profiles, environment variables, instance roles, and assuming a role need the desktop app. Reading local files, the process environment, and instance metadata is impossible in a browser, and AWS STS does not answer browser (CORS) requests. The web app supports access keys (including temporary keys with a session token) and anonymous access.

Default S3 Browser location

Settings → Cloud Storage → Default S3 Browser location (e.g. `s3://my-bucket/data/`) is where the S3 Browser opens. You can also browse to a folder and click **Set as default** in the browser. The setting is saved with your app settings. When it is blank, the browser reopens the last location you browsed.

S3 Browser

Plugins → Web Services → S3 Browser lists a bucket's folders and files:

- Type `s3://bucket/prefix/` (or a bucket name, or an S3 HTTPS URL) and press **Go**.
- **List buckets** lists every bucket the selected connection's credentials can see. On the web this needs the S3 service endpoint to allow the page's origin, which AWS does not, so use the desktop app or type the bucket name.
- **Add** puts a COG/GeoTIFF, GeoParquet, GeoJSON, FlatGeobuf, GeoPackage, CSV, PMTiles, or COPC/LAZ/LAS point cloud file on the map through the same code paths as Add Data. Once a file is on the map its button reads **Added**; remove the layer and it turns back into **Add**. **Copy URI** copies the `s3://` URI for use elsewhere, such as the SQL Workspace.
- A `.geolibre` or `.geolibre.json` file shows **Open project** instead, which opens that project from the bucket.
- To add several files at once, tick their checkboxes (or **Select all**) and choose **Add selected**. They are added one after another, with progress shown under the buttons.
- **Up** goes to the parent folder; **Set as default** makes the current folder the one the browser opens at.
- The line under the location box shows whether the bucket is read with a connection's credentials or anonymously.

To have the S3 Browser open every time GeoLibre starts, turn on **Open the S3 Browser at startup** under [Settings](#) → [Startup Settings](#).

Where credentials are stored

Connections are device-local and never saved in a project file.

- **Desktop app:** the secret access key and session token are kept in the operating system's credential store (Keychain, Credential Manager, Secret Service). The rest of the connection is kept with the app settings.
- **Web app:** connections, including secrets, live in the browser's local storage, where any script on the same origin could read them. Prefer short-lived keys (a session token) on the web.

Signed URLs are redacted from the diagnostics log. Shared projects never carry them, since layers store only the unsigned URL.

CORS for private buckets (web app)

The web app, and the desktop webview for raster and PMTiles reads, fetch S3 objects directly, so the bucket's CORS configuration must allow the app's origin. A minimal rule:

```
[
  {
    "AllowedOrigins": ["https://web.geolibre.app", "tauri://localhost", "http://tauri.localhost"],
    "AllowedMethods": ["GET", "HEAD"],
    "AllowedHeaders": ["*"],
    "ExposeHeaders": ["Content-Length", "Content-Range", "ETag", "Last-Modified", "Accept-Ranges"],
    "MaxAgeSeconds": 3000
  }
]
```

Replace the first origin with your own deployment's origin. On the desktop app, listing and vector downloads go through the native HTTP client and need no CORS rule, but COG, PMTiles, and point cloud streaming still read from the webview and need the `tauri://localhost` / `http://tauri.localhost` origins.

When a read fails because the bucket's CORS rules block the app, GeoLibre says so — naming the bucket and the origin to allow — instead of a bare "Failed to fetch". It tells the two apart by repeating the request: if a normal request now succeeds, the failure was transient; if only a `no-cors` request (which CORS cannot block) gets an answer, the bucket is reachable and its CORS configuration is the cause.

How it works

A private object is read through a **SigV4 presigned URL**, minted in the app from the connection's credentials and valid for up to 12 hours (or until temporary credentials expire). Every reader GeoLibre uses already accepts an HTTPS URL: the COG renderers, PMTiles, DuckDB-WASM, and the vector loader. So signing the URL is the one change that makes all of them read private data. PMTiles archives re-sign on each range read once the previous URL nears expiry.

Limitations

- Zarr stores are not yet read from private buckets: a store is many objects, and the Zarr readers take no per-request signing hook.
- SQL globs (`read_parquet('s3://bucket/*.parquet')`) and Iceberg tables in private buckets are not supported; name individual files.
- `gs://` and `az://` URLs are still read anonymously.

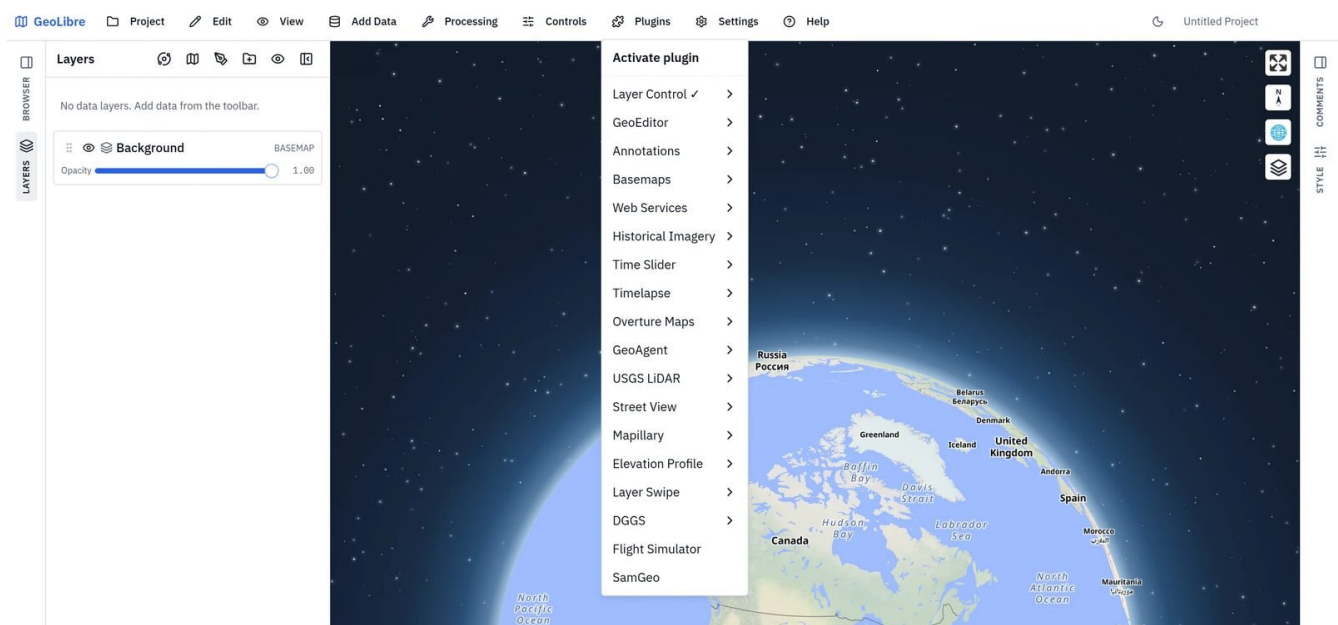
<https://geolibre.app/user-guide/plugins/>

Plugins & Marketplace

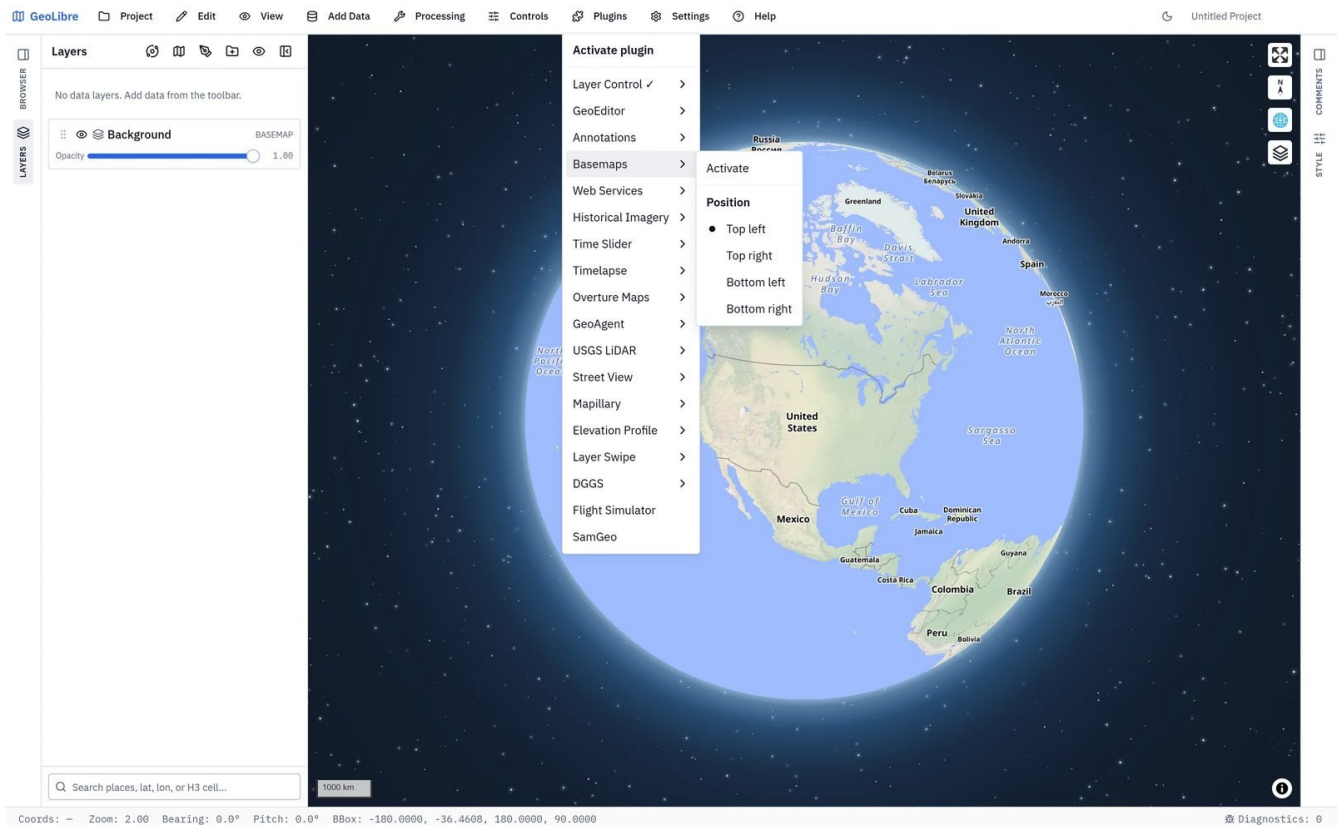
Much of GeoLibre's functionality ships as plugins. The **Plugins** menu activates built-in plugins, and the **Manage Plugins** dialog (under Settings) installs, updates, and removes external plugins from a curated registry.

The Plugins menu

The **Plugins** menu lists every available plugin under **Activate plugin**. Click a plugin to toggle it on or off; a check mark shows which are active.



Most entries toggle their plugin directly. The plugins that place a control on the map (**GeoEditor**, **Annotations**, **Dimensions**, **Street View**, and **Mapillary**) open a submenu instead, which **activates** the plugin and **positions** its on-map control in any corner: top left, top right, bottom left, or bottom right. **Web Services** and **DGGS** open a list of their sub-plugins, and plugins you install from the marketplace are listed under **Installed**, which also links to **Manage Plugins....**



Components live on the Controls menu

Measure, Bookmark, Legend, Colorbar, Minimap, View State, Search, and HTML are on-map component panels rather than Plugins-menu entries; toggle them from the [Controls menu](#). The Print composer is under [Project → Print Layout](#).

Built-in plugin reference

The table below is generated from the plugin registry, so it lists every built-in plugin the app registers. **Where** is the menu that toggles it; **Web Services** and **DGGS** are submenus of the Plugins menu, and a few plugins live on the Controls or Add Data menu instead. **Link name** is the value for a `?plugin= link`; "menu only" marks the plugins a link cannot open.

GeoLibre registers 68 built-in plugins.

Plugin	Where	Link name	What it adds
Layer Control	Controls → Layer Control	<code>layer-control</code>	The on-map layer list. On by default.
GeoEditor	Plugins	<code>geo-editor</code>	Drawing, vertex editing, snapping, and deletion tools for GeoJSON layers.
Annotations	Plugins	<code>annotations</code>	The map-annotation toolbar (text, arrows, highlights, pins, sticky notes, images) and the Elements panel. See Annotations .

Plugin	Where	Link name	What it adds
Dimensions	Plugins	<code>dimensions</code>	CAD-style linear and angular dimension features with vertex snapping, stored in the project as a GeoJSON layer.
FEMA NFHL	Plugins → Web Services	<code>fema-wms</code>	FEMA National Flood Hazard Layer WMS layers. See FEMA NFHL .
NASA Earthdata	Plugins → Web Services	<code>nasa-earthdata</code>	Over 1,100 pre-rendered global NASA GIBS imagery layers, by date. See NASA Earthdata .
US EPA EnviroAtlas	Plugins → Web Services	<code>enviroatlas</code>	US EPA environmental and ecosystem map services. See US EPA EnviroAtlas .
USGS National Map	Plugins → Web Services	<code>national-map</code>	USGS topo, imagery, hydrography, elevation, and index services. See USGS National Map .
USGS NLDI	Plugins → Web Services	<code>usgs-nldi</code>	Flowline tracing, hydrolocation, upstream basins, and network navigation. See USGS NLDI .
USGS 3DEP	Plugins → Web Services	<code>usgs-dem</code>	USGS 3DEP digital elevation models (1 m, 1/3 and 1 arc-second, and more). See USGS 3DEP .
USGS LiDAR	Plugins → Web Services	<code>usgs-lidar</code>	Clip a USGS 3DEP point cloud to an area of interest and download the result as COPC.
Vantor Open Data	Plugins → Web Services	<code>vantor</code>	Vantor disaster-event satellite imagery (COG). See Vantor Open Data .
Planet Open Data	Plugins → Web Services	<code>planet-open-data</code>	Planet's disaster data releases, through the STAC browser. See Planet Open Data .
Portolan	Plugins → Web Services	<code>portolan</code>	Browse the public Portolan Registry of catalogs, or connect to a publisher's catalog URL, and add assets through the STAC loaders.
Earthdata GIS	Plugins → Web Services	<code>earthdata-gis</code>	NASA EOSDIS ArcGIS image, map, and feature services, and published web maps. See Earthdata GIS .
OpenAerial Map	Plugins → Web Services	<code>openaerialmap</code>	Openly licensed drone and aerial imagery. See OpenAerialMap .
OSM Downloader	Plugins → Web Services	<code>osm-downloader</code>	Download OpenStreetMap buildings, roads, amenities, waterways, land use, or custom tags through Overpass. See OSM Downloader .
IGN LiDAR HD	Plugins → Web Services	<code>ign-lidar-hd</code>	Search France's IGN LiDAR HD tile coverage and download COPC point clouds.
ArcGIS Portal	Plugins → Web Services	<code>arcgis-portal</code>	Browse and add the content of your ArcGIS Online organization or ArcGIS Enterprise portal after signing in. See ArcGIS Portal .
ArcGIS Hub	Plugins → Web Services	<code>arcgis-hub</code>	Public datasets published to ArcGIS Hub. See ArcGIS Hub .
Tennessee GIS	Plugins → Web Services	<code>tennessee-gis</code>	The geodata.tn.gov open GIS data portal. See Tennessee GIS .

Plugin	Where	Link name	What it adds
US Federal GIS	Plugins → Web Services	us-federal-gis	The public GIS portals of 24 US federal agencies. See US Federal GIS .
US State GIS	Plugins → Web Services	us-state-gis	The public GIS data portals of all 50 states and DC. See US State GIS .
US Local GIS	Plugins → Web Services	us-local-gis	The GIS and open-data portals of about 150 large US cities, counties, and regional agencies. See US Local GIS .
Socrata	Plugins → Web Services	socrata	Search government Socrata open-data portals. See Socrata .
CKAN	Plugins → Web Services	ckan	Humanitarian Data Exchange (CKAN) resources. See CKAN .
STAC Catalogs	Plugins → Web Services	stac-catalogs	Any STAC API or static catalog, via STAC Index. See STAC Catalogs .
Source Cooperative	Plugins → Web Services	source-coop	Cloud-native products (PMTiles, GeoParquet, COG) on Source Cooperative. See Source Cooperative .
S3 Browser	Plugins → Web Services	s3-browser	Browse any Amazon S3 or S3-compatible bucket, public or private, and add its files. See S3 Browser .
Natural Earth	Plugins → Web Services	natural-earth	The Natural Earth vector and raster themes. See Natural Earth .
Hugging Face	Plugins → Web Services	huggingface	Geospatial files in Hugging Face dataset repos, and uploads to them. See Hugging Face .
Satellite Embeddings	Plugins → Web Services	satellite-embeddings	Pre-computed foundation-model embeddings such as AlphaEarth, Tessera, and Earth Index. See Satellite Embeddings .
Fields of the World	Plugins → Web Services	fields-of-the-world	Global agricultural field boundaries colored by model confidence. See Fields of the World .
Sentinel-2 Explorer	Plugins → Web Services	sentinel2-explorer	Sentinel-2 imagery search and viewer over static STAC-GeoParquet, with no API. See Sentinel-2 Explorer .
Ocean Data Platform	Plugins → Web Services	ocean-data-platform	HUB Ocean's public ocean datasets: habitats, protected areas, fisheries, and observations. See Ocean Data Platform .
GeoLens	Plugins → Web Services	geolens	A self-hosted GeoLens spatial catalog. See GeoLens .
Street View	Plugins	streetview	Google Street View panoramas at a clicked point.
Mapillary	Plugins	mapillary	Mapillary street-level imagery.
H3	Plugins → DGGS	h3-grid	The H3 hexagonal grid over the current view; identify a cell and export the grid or selection.
S2	Plugins → DGGS	s2-grid	The S2 cell grid over the current view; identify a cell and export the grid or selection.
A5	Plugins → DGGS	a5-grid	The A5 pentagonal grid over the current view; identify a cell and export the grid or selection.

Plugin	Where	Link name	What it adds
DGGRID	Plugins → DGGS	dggrid	DGGRID discrete global grids over the current view; identify a cell and export the grid or selection.
DGGAL	Plugins → DGGS	dggal	DGGAL discrete global grids over the current view; identify a cell and export the grid or selection.
OLC	Plugins → DGGS	olc	The Open Location Code (plus code) grid over the current view; identify a cell and export the grid or selection.
Geohash	Plugins → DGGS	geohash	The Geohash grid over the current view; identify a cell and export the grid or selection.
Tilecode	Plugins → DGGS	tilecode	The web-map tile grid (z/x/y tile codes) over the current view; identify a cell and export the grid or selection.
Basemaps	Plugins	basemap-control	A basemap gallery for switching the background map, from the same catalog as the Change background dialog.
Historical Imagery	Plugins	esri-wayback	Browse historical aerial and satellite imagery (Esri World Imagery Wayback) for a location.
Time Slider	Plugins	time-slider	Filter a temporal layer by a date or number field, or step a time-enabled raster or Zarr cube through its time axis.
Timelapse	Plugins	timelapse	Animate annual cloudless basemaps (EOX Sentinel-2, NASA GIBS Landsat/WELD and MODIS land cover) and record them to video.
Overture Maps	Plugins	overture-maps	Browse and add Overture Maps themes.
GeoAgent	Plugins	geoagent	An in-map AI agent panel.
Elevation Profile	Plugins	elevation-profile	A terrain profile along a drawn line, or along the selected line features of a layer.
Layer Swipe	Plugins	swipe	A swipe bar comparing two layers.
Gridlines	Controls → Gridlines	graticule	A coordinate grid with edge labels, including UTM easting/northing and MGRS/USNG modes.
Clouds	Controls → Weather	clouds	Near-realtime NASA cloud imagery, animated day by day.
Precipitation	Controls → Weather	precipitation	RainViewer precipitation radar, animated over roughly the last two hours.
Atmospheric Effects	Controls → Atmospheric Effects	atmosphere-effects	A deep-space backdrop, starfield, comets, and an atmospheric halo at low zoom.
Sun Simulation	Controls → Sun	sun	The sun's position and the day/night terminator for any date and time.
Route Animation	Controls → Route Animation	route-animation	Animate a marker along a line layer with play/pause, speed, a trail, and camera follow.

Plugin	Where	Link name	What it adds
Flight Simulator	Plugins; also Controls → Flight Simulator	flight-simulator	Fly over terrain and 3D layers with keyboard controls.
God's Eye View	Plugins	gods-eye-view	Live earthquakes, satellite orbits, flights, transit, public cameras, and more on the Cesium globe. See God's Eye View .
SamGeo	Plugins	samgeo	Segment imagery into vector features. See SamGeo .
Point Cloud Annotation	Plugins	point-cloud-annotation	Select LiDAR points with a box or lasso, assign ASPRS classes, and export LAS 1.4 or a Segments.ai label. See Point Cloud Annotation .
Directions	Controls → Directions	menu only	Click the map to add waypoints and get a route from the public OSRM demo server.
Reverse Geocode	Controls → Reverse Geocode	menu only	Click the map to look up the address at that point through the configured geocoding provider.
Deck.gl Layer	Add Data → Deck.gl Layer	deckgl-viz	The deck.gl overlay that renders layers built in the Deck.gl Layer dialog.
Components	Controls (Search, Legend, Measure, ...)	components	The component panels; each is toggled from the Controls menu rather than as one plugin. See Components .

God's Eye View

God's Eye View is inspired by the MIT-licensed [bilawalsidhu/gods-eye-view](#), including its public-feed selection and normalization approach. Each enabled layer adds its provider credit to Cesium's **Data attribution** control. The datacenter, dam, and current-view infrastructure feeds use OpenStreetMap data under ODbL; Radio Browser is public domain; USGS earthquake data is U.S. public domain; CelesTrak requests citation; live transit combines MBTA, CapMetro, Metro Transit, HSL, OVapi, Entur, and TransLink feeds under their respective public-data terms; and the live TeleGeography cable feed is read from GeoLibre's Source Cooperative mirror under CC BY-NC-SA 3.0, including its NonCommercial restriction. Public camera imagery comes from TfL, Austin, Calgary, Fintraffic, Ontario 511, DriveBC, Live Traffic NSW, and Caltrans under each provider's public-data terms.

Some layers have options under their own row, saved with the project. **Radio Stations** has a **Category** filter (News, Talk, Weather / Emergency, Public Safety, Aviation / Marine, Traffic / Transit, Music, or Other), matched from each station's Radio Browser tags. **Public CCTV Cameras** has **Snapshots on map**: by default, snapshots appear only once you zoom in past street level; **Always** shows them at any zoom, and **Off** keeps plain markers, with the snapshot still in each camera's popup. Either way, cameras load only for a view less than 5° across.

Two layers use a key of your own, entered under **API keys** at the bottom of the panel. **Live AIS Vessels** streams ship positions from [AISStream](#) for the current view (up to 30° across) and shows nothing without a key. **Simulated Street Traffic** works keyless; with a [TomTom](#) key it also draws live

congestion and paces its vehicles by it. Keys typed there stay in the browser and are never written to the project; `VITE_TOMTOM_API_KEY` or `AISSTREAM_API_KEY` under **Settings → Environment variables** works too.

Open a plugin from a link

Add `plugin=<link name>` to a GeoLibre web address to open the app with that plugin already active, as if you had picked it from the Plugins menu:

```
https://web.geolibre.app/?plugin=nasa-earthdata
```

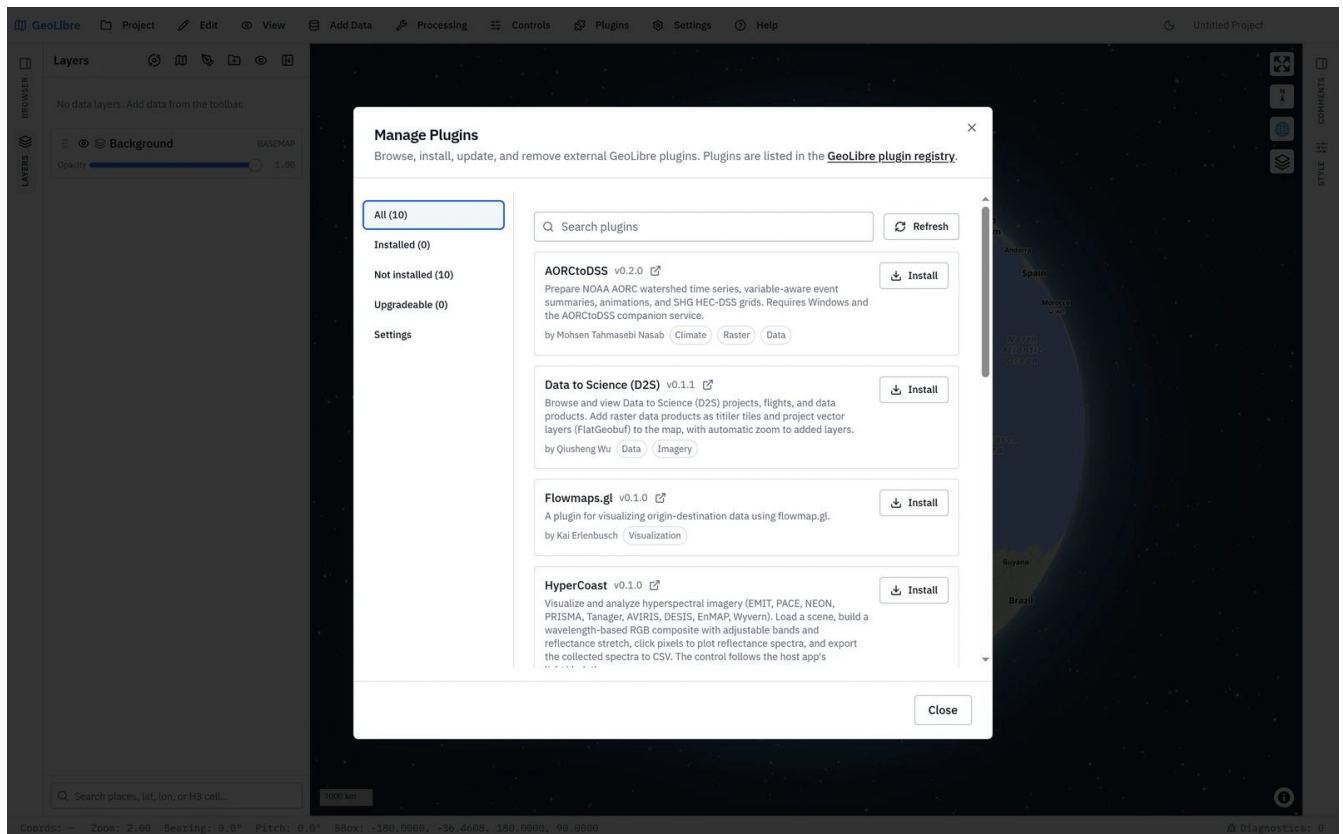
List several with commas (`?plugin=graticule,h3-grid`). The full plugin id, such as `maplibre-gl-nasa-earthdata`, works too. See [Deep-linking a plugin](#) for how it combines with a shared project and the read-only viewer.

The **Link name** column of the [plugin reference](#) gives the name for each built-in plugin.

Directions and Reverse Geocode are "menu only": they send what you click to a public server, so they only open from the menu, after their one-time notice. A plugin from the [plugin registry](#) opens from a link by its full registry id, for example `?plugin=openrindt-geolibre`. If it is already installed it activates like a built-in one. If not, GeoLibre may prompt to trust a compatible registry entry (showing its name, version, author, description, and homepage), then installs and activates it only when the current deployment plugin policy permits loading. Trust confirmation does not bypass policy; a denied plugin will not load or activate. In read-only `layout=viewer`, an installed registry plugin still opens, but the prompt is skipped and uninstalled ones are never installed. This is distinct from project manifest URL trust, which is suppressed by `sideload: false`. Permitted registry entries remain eligible with sideload disabled. See [Plugin policy](#).

Manage Plugins

Open **Settings → Manage Plugins** to browse the marketplace. The dialog is modeled on QGIS, with sections for **All**, **Installed**, **Not installed**, **Upgradeable**, and **Settings**.



- **Search** the registry and **Install** an entry with one click. Installation records the plugin's manifest URL and registers it immediately, with no restart.
- **Update** appears when a newer version is published; it re-fetches and re-registers the plugin in place, keeping the old version if the update fails.
- **Uninstall** (after a confirmation) unregisters the plugin at runtime and tears down any active control.
- The **Settings** section installs a packaged plugin (.zip) with **Install from file** and manages additional plugin sources: extra local directories (desktop only) and manual manifest URLs.

Compatibility is checked against each entry's `minGeoLibreVersion` , so incompatible plugins are flagged rather than installed.

Trust model

The registry is a curated source, manifests require HTTPS (or HTTP on localhost, 127.0.0.1, or `[::1]` for development), and installs require explicit consent. Deployment plugin policy also governs whether external plugins load; trust confirmation does not bypass it. This client policy is bypassable and provides neither signing nor sandboxing.

Where plugins come from

- **Curated registry:** the marketplace fetches a versioned JSON registry, hosted by default at `plugins.geolibre.app`. The registry and plugin bundles live in the [opengeos/geolibre-plugins](#) repository.
- **Manifest URL:** point the Settings section at any `plugin.json` manifest URL.
- **Local directory:** load a plugin from a local folder (desktop app).
- **Plugin archive:** install a packaged `.zip` from **Install from file**; in the browser it is stored locally and reloads on your next visit.
- **Bundled drop-ins:** plugins placed in `public/plugins/<id>/` load automatically in a build.

Writing your own plugin

To build a plugin, see [Reference → Plugin API](#) for the TypeScript interfaces, the `plugin.json` manifest contract, and the list of built-in plugins.

USGS NLDI

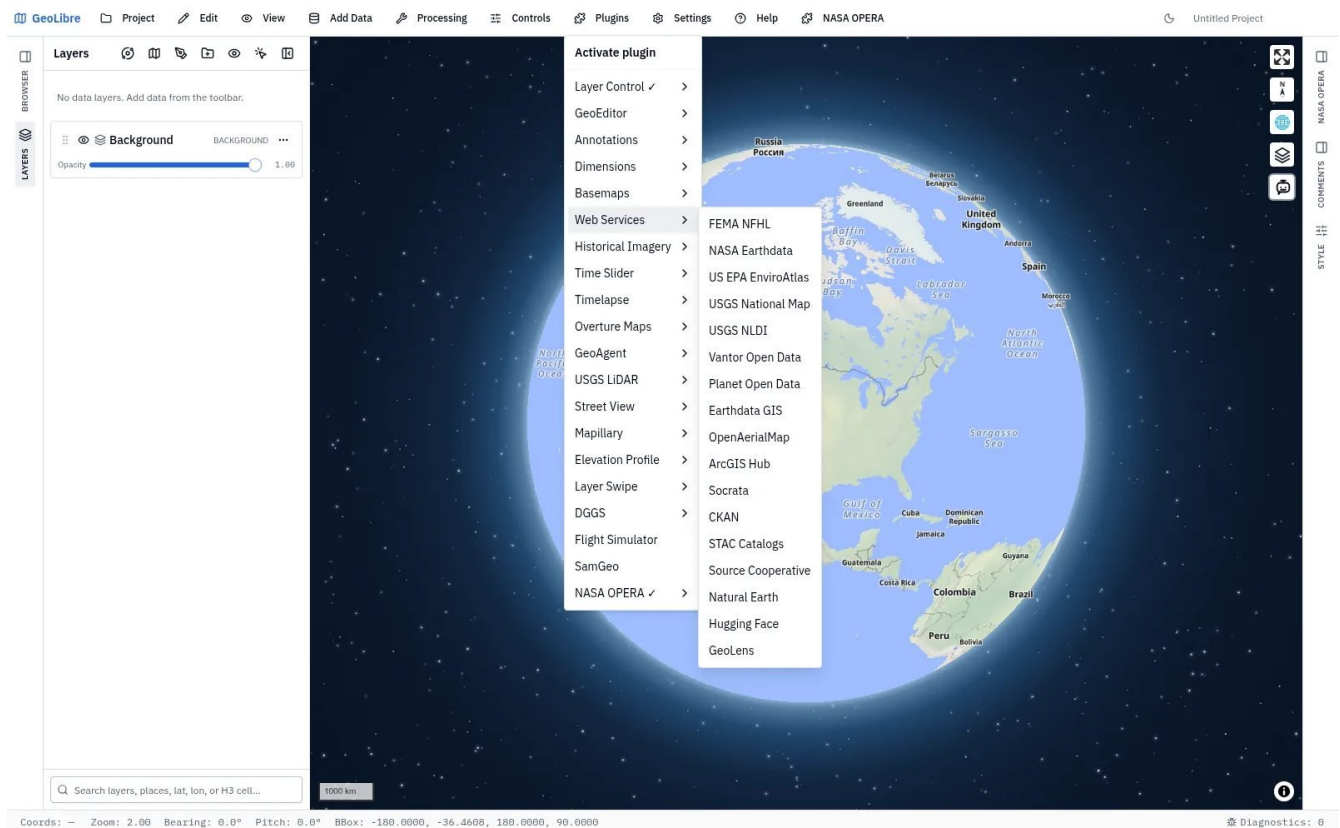
See [USGS NLDI workflows](#) for point-to-flowline tracing, hydrolocation, upstream basin, COMID navigation, and GeoJSON export. It is activated from the [Web Services](#) submenu.

<https://geolibre.app/user-guide/web-services/>

Web Services

Plugins → Web Services is a submenu of catalog and service browsers. Each entry connects to one public (or self-hosted) data provider, searches it, and adds what you pick to the map as a normal GeoLibre layer.

They are grouped together because they behave the same way, not because they share a data source: every one of them opens a **docked side panel** rather than a floating on-map control, so it sits alongside the Layers and Style panels, resizes with them, and can be collapsed. That is also why these entries have no "position" submenu: there is no on-map control to place in a corner.



How the panels behave

- **Activating** an entry opens its panel; closing the panel deactivates the plugin. A check mark next to **Web Services** in the Plugins menu means at least one of them is active.
- **Layers you add are real layers.** Whatever a panel puts on the map is mirrored into the GeoLibre layer store, so it appears in the **Layers panel**, can be reordered, hidden, restyled, and removed there, and is saved into the `.geolibre.json` **project file**. Reopening the project restores the layer and hands it back to its panel.
- **The catalog browsers are mutually exclusive.** STAC Catalogs, Planet Open Data, and Portolan share panel state, so activating one deactivates the other; if the switch fails, the plugin that was displaced comes back.

- **Nothing here needs an account** except ArcGIS Portal (an ArcGIS sign-in), private S3 buckets (an S3 connection), Hugging Face uploads (a user access token), and GeoLens private datasets (an API key).

At a glance

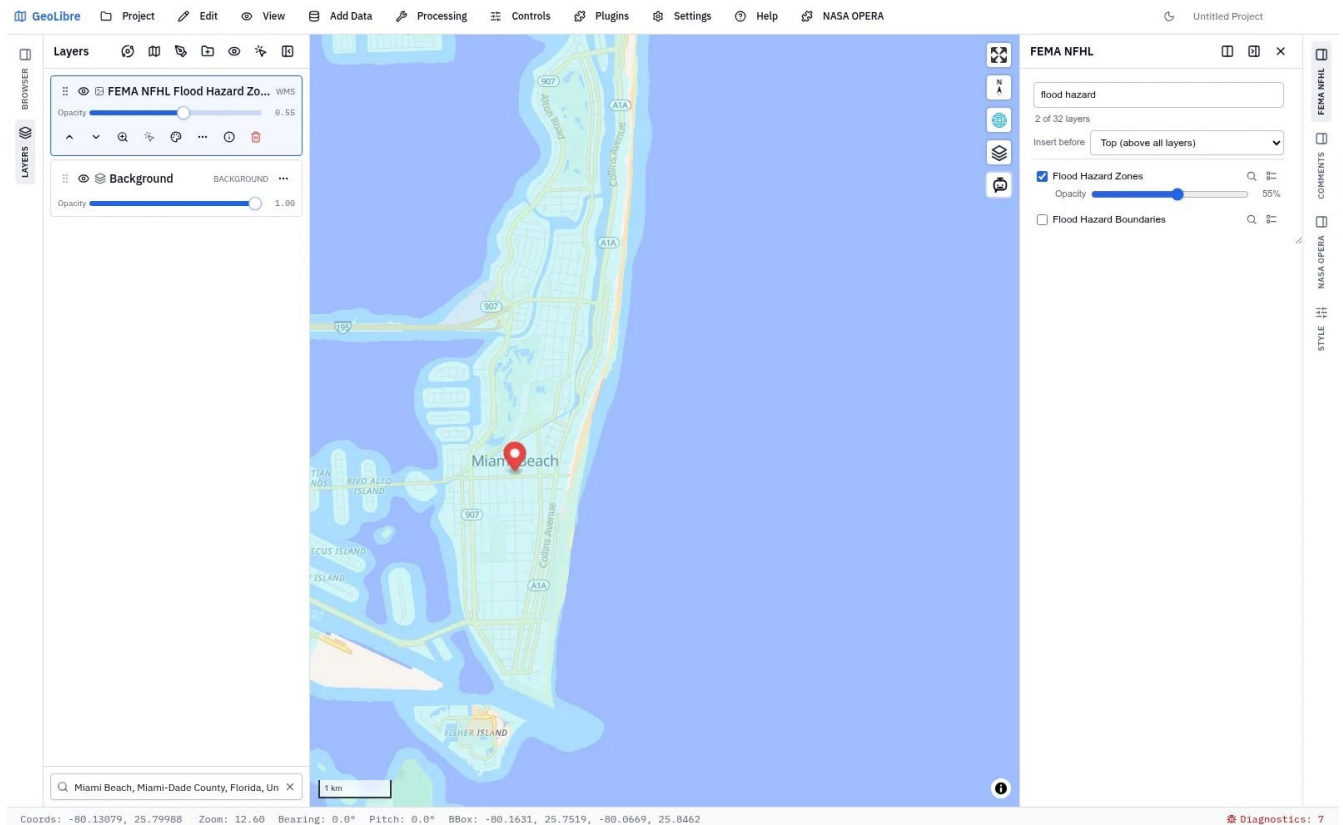
Panel	Provider	What you get
FEMA NFHL	FEMA	National Flood Hazard Layer WMS layers
NASA Earthdata	NASA GIBS	1,100+ pre-rendered global imagery layers, by date
US EPA EnviroAtlas	EPA	Environmental and ecosystem map services
USGS National Map	USGS	Topo, imagery, hydrography, elevation, and index services
USGS NLDI	USGS	Flowline tracing, hydrolocation, basins, and network navigation
USGS 3DEP	USGS	3DEP digital elevation models (1 m, 1/3 and 1 arc-second, and more)
USGS LiDAR	USGS	3DEP LiDAR point clouds clipped to an area of interest, as COPC
Vantor Open Data	Vantor	Disaster-event satellite imagery (COG)
Planet Open Data	Planet Labs	Planet's disaster data releases, through the STAC browser
Portolan	Portolan Registry	Registered geospatial catalogs, or a publisher's catalog URL, through the STAC browser
Earthdata GIS	NASA EOSDIS	ArcGIS image, map, and feature services, and published web maps
OpenAerialMap	OpenAerialMap	Openly licensed drone and aerial imagery
OSM Downloader	OpenStreetMap / Overpass	Buildings, roads, amenities, waterways, land use, or custom OSM tags
IGN LiDAR HD	IGN (France)	LiDAR HD tile coverage and COPC point clouds
ArcGIS Portal	Esri	The content of your ArcGIS Online organization or Enterprise portal, after signing in
ArcGIS Hub	Esri	Public datasets published to ArcGIS Hub
Tennessee GIS	State of Tennessee	The geodata.tn.gov open GIS data portal
US Federal GIS	US federal agencies	The public GIS portals of 24 federal agencies, from the Census Bureau and NOAA to USGS and FEMA
US State GIS	US state GIS offices	The public GIS data portals of all 50 states and DC
US Local GIS	US cities and counties	The GIS and open-data portals of about 150 large US cities, counties, and regional agencies
Socrata	Socrata	Government open-data portals
CKAN	HDX	Humanitarian Data Exchange resources

Panel	Provider	What you get
STAC Catalogs	any STAC	Any STAC API or static catalog, via STAC Index
Source Cooperative	Source.coop	Cloud-native products (PMTiles, GeoParquet, COG)
S3 Browser	Amazon S3 / S3-compatible	Browse any bucket, public or private, and add its files
Natural Earth	Natural Earth	The Natural Earth vector and raster themes
Hugging Face	Hugging Face	Geospatial files in dataset repos — and uploads
Satellite Embeddings	Source.coop, Tesseract	Pre-computed foundation-model embeddings (AlphaEarth, Tesseract, Earth Index, ...)
Fields of the World	Source.coop	Global agricultural field boundaries (2024, 2025)
Sentinel-2 Explorer	Source.coop, AWS	Every Sentinel-2 L2A scene since 2015, searched from static GeoParquet
Ocean Data Platform	HUB Ocean	Public ocean datasets: habitats, protected areas, fisheries, observations
GeoLens	your server	A self-hosted spatial catalog

FEMA NFHL

Searches the [FEMA National Flood Hazard Layer](#) WMS service and adds its layers — Flood Hazard Zones, FIRM Panels, LOMAs, Base Flood Elevations, and the rest — as raster layers.

- Filter the layer list by name and check layers on or off; each one becomes its own map layer.
- Per-layer opacity, and a legend fetched on demand through `GetLegendGraphic`.
- **Feature info**: click the map to query the active layers via `GetFeatureInfo` and read the attributes in a popup.
- **Zoom to layer extent**, taken from the service capabilities.
- **Insert before** places new layers beneath an existing map layer (for example, below basemap labels).



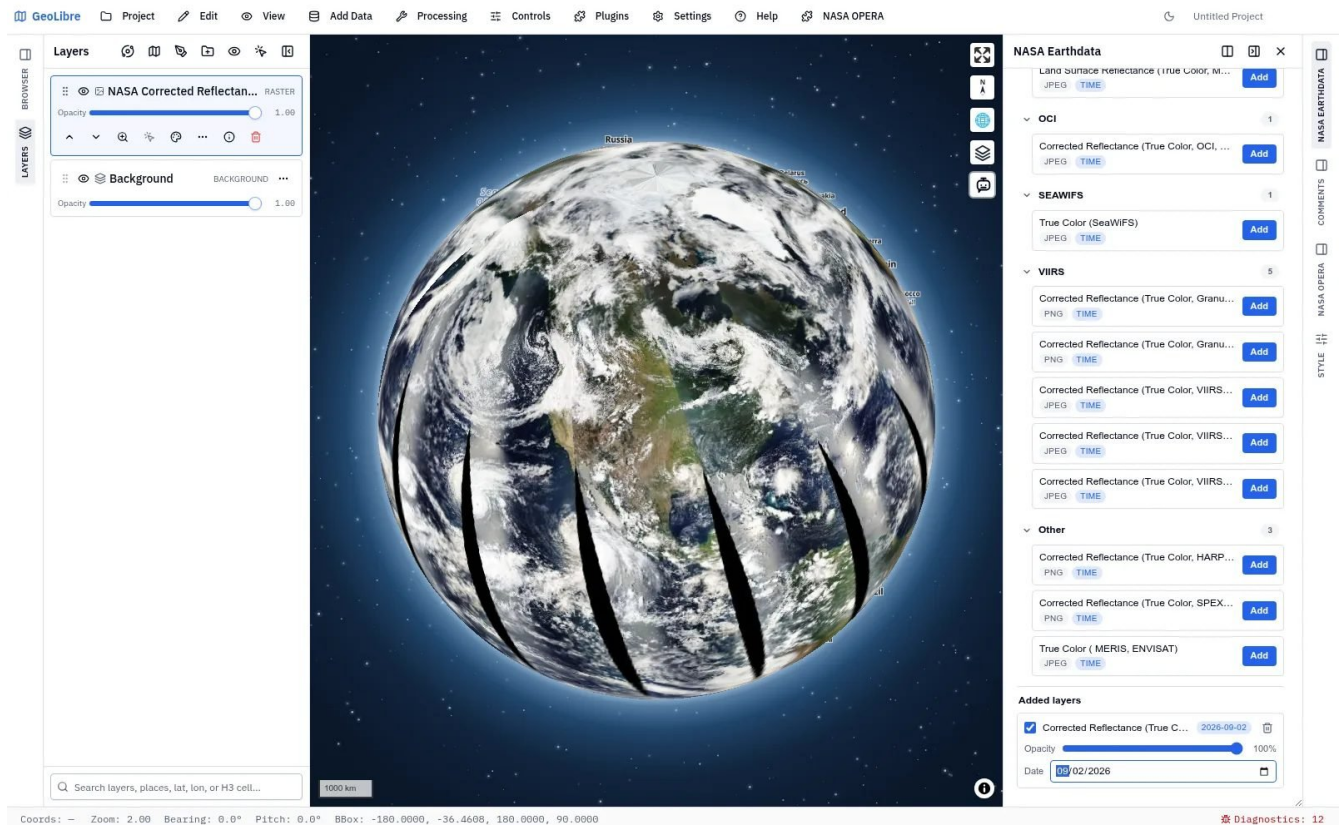
NASA Earthdata

Browses [NASA GIBS](#) (Global Imagery Browse Services), the pre-rendered global imagery tiles behind Worldview.

- Search 1,100+ raster layers by title or identifier, or browse them grouped by platform and instrument (MODIS, VIIRS, MERRA-2, ...).
- **Time-enabled layers get a date picker.** Add the same layer more than once with different dates to compare them side by side.
- Visibility toggle, legend, opacity slider, and removal for each added layer, plus an **Insert before** selector.

GIBS dates

Some GIBS products publish with a lag, so today's date can return empty tiles. Step back a day if a layer looks blank.



US EPA EnviroAtlas

Browses [EPA EnviroAtlas web services](#) — environmental, ecosystem, and community health data for the United States.

- A collapsible tree of folders, services, and individual sublayers, with a **deep search** that matches sublayer names too (searching "tree cover" or "asthma" finds the specific layers, not just the parent services).
- Adds ArcGIS **MapServer** and **ImageServer** services as MapLibre raster layers, reprojected to Web Mercator on the fly.
- Requests are clamped to each service's data extent, and the map zooms to a layer's extent as it is added.
- Visibility, opacity, legend, removal, and an **Insert before** selector per layer.

USGS National Map

Browses the [USGS National Map services](#) catalog, grouped into Basemaps, Hydrography, Elevation, Imagery, Cartography, Hazards, Other Data, and Indexes.

- Search by name, title, description, or category; matching categories expand automatically.
- The catalog is fetched live from the USGS ArcGIS REST endpoints, with a bundled static catalog as an instant fallback.
- Cached tile services, dynamic map exports, and ImageServer exports all arrive as raster layers.

- Visibility, opacity, removal, and an **Insert before** selector per layer.

No API key is required; every endpoint is CORS-enabled.

USGS NLDI

Traces a clicked point onto the National Hydrography Dataset network: flowline and raindrop path, hydrolocation and COMID, the upstream basin, and network navigation to streamgages, wells, HUC12 pour points, and other catalogs.

Unlike the other entries, this one is a click-driven analysis workflow rather than a catalog search. See [USGS NLDI workflows](#) for the full walkthrough, including exporting the traced results to GeoJSON or copying them into the Layers panel.

USGS 3DEP

Searches The National Map for [3D Elevation Program](#) digital elevation models.

- Search the **current view**, a **bounding box drawn on the map**, typed **coordinates**, or a **USGS 1:24,000 topographic quad** by name and state.
- Pick the **elevation datasets** (1 m DEM, 1/3 arc-second, 1 arc-second, Alaska 5 m, and others) and the file **format**, and optionally hide redundant partial tiles.
- Footprints render on the map and in the Layers panel; click one to select its result, or export them all to GeoJSON.
- **Load on Map** streams a GeoTIFF DEM through GeoLibre's raster path; **Download** saves the source file (IMG products are download-only).

USGS LiDAR

Clips a USGS [3D Elevation Program](#) LiDAR point cloud to an area of interest and downloads the result as COPC. While the panel is open, a 3DEP Elevation Index layer shows where point clouds exist, and the map switches to the Mercator projection (your previous projection is restored when you close it), since the streamed point cloud does not render on the globe.

Vantor Open Data

A STAC explorer for [Vantor's](#) open disaster imagery releases.

- Filter scenes by **event** and by **phase** (pre-event or post-event).
- Restrict the search to the current view or to a **bounding box drawn on the map**; footprints render on the map and highlight as you hover a result.

- Cloud-Optimized GeoTIFFs are added through GeoLibre's own raster path, so they become persistent layers you can restyle — and you can pick the **COG rendering engine** (GPU/deck.gl, the WebAssembly tiler, or a TiTiler server).
- Download the source scene.

Planet Open Data

The same panel as [STAC Catalogs](#), pinned to [Planet Labs PBC's](#) continuously updated disaster data releases so the catalog is already selected when it opens. Everything below about searching, filtering, and adding assets applies here too.

Portolan

The same panel as [STAC Catalogs](#), opened on the public [Portolan Registry](#) of catalogs. Pick a registered catalog, or enter a publisher's catalog URL to connect to it directly, then add assets as in [STAC Catalogs](#).

Earthdata GIS

Searches NASA's [Earthdata GIS portal](#) — the ArcGIS services EOSDIS publishes for its DAACs and disaster responses.

This is a different catalog from [NASA Earthdata](#) above: GIBS serves pre-rendered global imagery tiles, while Earthdata GIS serves analysis-ready ArcGIS services.

- **ImageServer** and **MapServer** items are added as raster layers rendered through their export endpoints.
- **FeatureServer** items are added as GeoJSON vector layers, arriving with attributes, styling, and export intact.
- Published **web maps** are expanded into their constituent layers.
- An ArcGIS service can be **exported to a Cloud-Optimized GeoTIFF**; GeoLibre re-encodes the plain GeoTIFF ArcGIS returns, since ArcGIS has no COG output of its own.

OpenAerialMap

Searches [OpenAerialMap](#), the open catalog of drone and aerial imagery.

- Search by the **current map view**, a **box drawn on the map**, or typed coordinates.
- Result footprints are drawn on the map as a single entry in the Layers panel, so you can hide or restyle them; the selected footprint is highlighted separately.
- Add a scene to the map, zoom to its footprint, inspect its metadata, or download the source GeoTIFF.

OSM Downloader

Downloads current OpenStreetMap vector data through the public Overpass API.

- Start from the current map extent or type west, south, east, and north coordinates.
- Choose buildings, roads, amenities, waterways, land use, all tagged features, or a custom OSM tag key and optional value.
- Add the result as a normal GeoJSON layer or save it as a `.geojson` file.
- OSM nodes become points, ways follow OSM's line/area conventions, and multipolygon relations retain their outer and inner rings.

Public Overpass instances are intended for bounded interactive queries. Zoom to the area you need before downloading. To prevent accidentally requesting an enormous result, **All tagged features** is limited to 0.25 square degrees and filtered downloads are limited to 4 square degrees.

The panel identifies the source as © OpenStreetMap contributors and notes the Open Database License (ODbL); keep the required attribution when publishing derived maps or data.

IGN LiDAR HD

Searches the tile coverage of France's [IGN LiDAR HD](#) program and adds its COPC point clouds.

- **Use map extent** and **Search tiles** list the LiDAR HD tiles in the area; their footprints are drawn as one entry in the Layers panel, and hovering a tile highlights its footprint.
- **Add to map** streams a tile as a point cloud layer (MapLibre, Mapbox, or ArcGIS renderer), and **Download** saves the COPC file. Check several tiles and choose **Add selected to map** to add them in one go.
- The data is © IGN, published under the Licence Ouverte 2.0.

ArcGIS Portal

Browses the content of your own ArcGIS Online organization or ArcGIS Enterprise portal, including items shared only with your organization or your groups, and adds it to the map.

- Sign in from the panel. Leave the portal URL blank for ArcGIS Online, or enter your organization URL (`https://yourorg.maps.arcgis.com`) to get its sign-in page (including SSO), or your Enterprise portal. You need an OAuth client ID registered on the portal; the hosted web app fills one in for ArcGIS Online. See [Signing in with ArcGIS](#).
- **Browse** picks the scope: **My content**, **My favorites**, **My groups** (then choose the group), **My organization**, or **All of the portal**. Narrow it with a keyword or an item type. Results are newest first, or by relevance once you type a keyword.

- **Add to map** loads feature, map, image, and vector tile services. A web map adds its feature, map service, and image service layers. Once added, the button reads **Remove from map** and removes them again; it turns back to **Add to map** if you remove the layers elsewhere, such as in the Layers panel. **Zoom** frames the item and **Details** opens its page on the portal.
- The session is the same one Add Data → ArcGIS uses, so signing in or out in either place applies to both. Layers you add renew their token while you stay signed in. **Another portal** signs in to a second portal, and the portal picker switches between them.
- Your sign-in token is sent only to services on the portal's own host, or on Esri's `arcgis.com` hosting for ArcGIS Online. A web map layer or item that points anywhere else is loaded without it, so only public services there load.

ArcGIS Hub

Searches public datasets published to [ArcGIS Hub](#).

- Search by keyword, or tick **Search the current map area** to restrict results to the view.
- Each card shows the description and links out to the dataset's Hub page.
- **Add to map** loads supported layers, **Zoom** frames them, and **Download** saves the data. A dataset with several layers downloads only the first, and the panel tells you so.
- Results are paged: the panel shows how many of the total you are looking at, and the next page loads as you scroll to the end of the list (**Load more** does the same by hand).

The screenshot displays the GeoLibre web application interface. The main map area shows a topographic map of Utah with various cities and geographical features. On the left, the 'Layers' panel is visible, showing 'Administrative Boundaries ... VECTOR' and 'Background' layers. The top menu bar includes options like 'Project', 'Edit', 'View', 'Add Data', 'Processing', 'Controls', 'Plugins', 'Settings', 'Help', and 'NASA OPERA'. On the right, the 'ArcGIS Hub' panel is open, displaying search results for 'national park boundaries'. The search bar contains the text 'national park boundaries'. Below the search bar, there are checkboxes for 'Search the current map area' and 'Added Administrative Boundaries of National Park System Units'. The results list includes three items: 'Administrative Boundaries of National Park System Units', 'National Parks of Canada', and 'National Park Service - Regional Boundaries'. Each item has a thumbnail, a description, and buttons for 'Add to map', 'Zoom', 'Download', and 'Details'. At the bottom of the results list, there is a 'Load more' button. The bottom status bar shows coordinates, zoom level, bearing, pitch, and a bounding box.

Tennessee GIS

Browses the State of Tennessee's [downloadable GIS data portal](#) — roads, boundaries, parcels, hydrography, wildlife management areas, state parks, and the other datasets state agencies publish there.

- The panel lists the whole catalog as soon as it opens, alphabetically; type a keyword to narrow it. **Search the current map area** starts off, since most layers are statewide.
- **Add to map** loads feature services as editable vector layers and map or image services (such as statewide imagery) as raster layers. **Zoom** and **Details** work as in ArcGIS Hub, and Details opens the dataset's page on geodata.tn.gov. **Download** saves feature services as GeoJSON; map and image services only render imagery, so they have nothing to download.
- The portal is an ArcGIS Hub site, so the panel searches the groups that make up its catalog. It reads that list from the site when it opens, so datasets the state adds show up without a GeoLibre update.

US Federal GIS

Browses the public GIS portals of US federal agencies, for the national layers a state or city portal does not hold: Census boundaries, NOAA radar and weather warnings, USGS land cover, LANDFIRE fuels and PAD-US protected areas, NRCS soils, Fish and Wildlife Service wetlands, national transportation networks (NTAD), FEMA flood data, and NAIP imagery.

- Choose a department, then an agency from the second menu; the panel lists that agency's catalog alphabetically, and a keyword narrows it. **Open portal** opens the agency's own site. The map-area filter starts off, since most federal layers are national; turn it on to keep only datasets that cover the current view.
- Feature services add as vector layers, map and image services (radar, land cover and fuel rasters, imagery) as raster layers, and Download saves feature services as GeoJSON, as in [US State GIS](#).
- Agencies with an ArcGIS Hub site (Forest Service, BLM, National Park Service, Fish and Wildlife Service, FEMA, BTS, and others) are searched through the site's catalog. Agencies without one (USGS, NOAA, EPA, NRCS, NASA, the Census Bureau, NGA) are searched across everything their ArcGIS Online organization shares publicly, which is larger and includes some non-GIS items.
- The Census Bureau's own organization publishes mostly statistics (community resilience estimates, business patterns) rather than its boundary files, so TIGER boundaries (tracts, counties, ZCTAs, congressional districts) are easiest to find under **Multi-agency → Esri U.S. Federal Datasets**, which Esri maintains from the federal sources. The same menu holds the federal GeoPlatform, home of the National Geospatial Data Assets.
- HIFLD Open is not listed, since its ArcGIS Hub site no longer exists. Every listed agency was checked to return datasets in September 2026; `npm run check:gis-portals -- federal` repeats the check.

US State GIS

Browses the public GIS data portal of any US state or the District of Columbia — the state list follows [Open Source GIS Data](#).

- Choose a state and the panel lists its catalog alphabetically; type a keyword to narrow it. Where a state has more than one portal (for example a statewide geoportal plus its transportation or natural-resources agency), a second menu picks between them. **Open portal** opens the portal's own website.
- **Add to map**, **Zoom**, **Download**, and **Details** work as in [Tennessee GIS](#): feature services add as vector layers, map and image services (such as statewide imagery) as raster layers, and Download saves feature services as GeoJSON.
- Every portal listed is an ArcGIS Hub site or ArcGIS Online organization, which is what lets one panel search them all. A Hub site's catalog is read from the site when you pick it, so datasets the state adds show up without a GeoLibre update. A few states (Louisiana, Montana, Nevada, Ohio, Oklahoma, South Dakota, among others) have no Hub catalog to read, so the panel searches everything the state's ArcGIS organization publishes; expect some non-GIS items such as dashboards' source tables there. States whose main clearinghouse is not built on ArcGIS (such as PASDA or TNIRIS) are represented by the agency portals that are.

US Local GIS

Browses the public GIS and open-data portals of large US cities and counties, plus a few regional agencies (such as the Atlanta Regional Commission and Oregon Metro), for the local data a statewide portal rarely holds: parcels, zoning, building footprints, address points, street trees, bike lanes, and incident logs.

- Choose a state, then a city or county from the second menu; the panel lists that portal's catalog alphabetically, and a keyword narrows it. **Open portal** opens the portal's own website. Some large places have two portals, such as a city's open-data site next to its GIS hub (Los Angeles Open Data and Los Angeles GeoHub, Seattle Open Data and Seattle GeoData); both are listed.
- About three quarters of the portals are ArcGIS Hub sites, which behave as in [US State GIS](#): feature services add as vector layers, map and image services as raster layers, and Download saves feature services as GeoJSON.
- The rest are [Socrata](#) portals, which many of the largest cities use (New York, Chicago, San Francisco, Seattle, Austin, Dallas). There the panel lists only datasets with a geometry column and adds each one as GeoJSON, zooming to what was loaded. A Socrata export is capped at 50,000 features, so a city-wide incident log arrives as its first 50,000 rows; filter it on the portal first for a complete subset. Socrata cannot search by extent, so **Search the current map area** is unavailable for those portals, and the result count is the number found so far rather than a total.

- Portals on other platforms (CKAN, OpenDataSoft, or custom sites such as Boston's or Pittsburgh's main portals) cannot be searched and are not listed, though their ArcGIS Hub sites are where one exists. Every listed portal was checked to return datasets in September 2026; `npm run check:gis-portals` repeats the check.

Socrata

Searches public [Socrata](#) open-data catalogs — the platform behind many city, county, and state data portals — and adds their GeoJSON datasets to the map. Keyword search, paged results, and **Load more**, the same as ArcGIS Hub.

CKAN

Searches the [Humanitarian Data Exchange](#) CKAN catalog and adds its available GeoJSON resources.

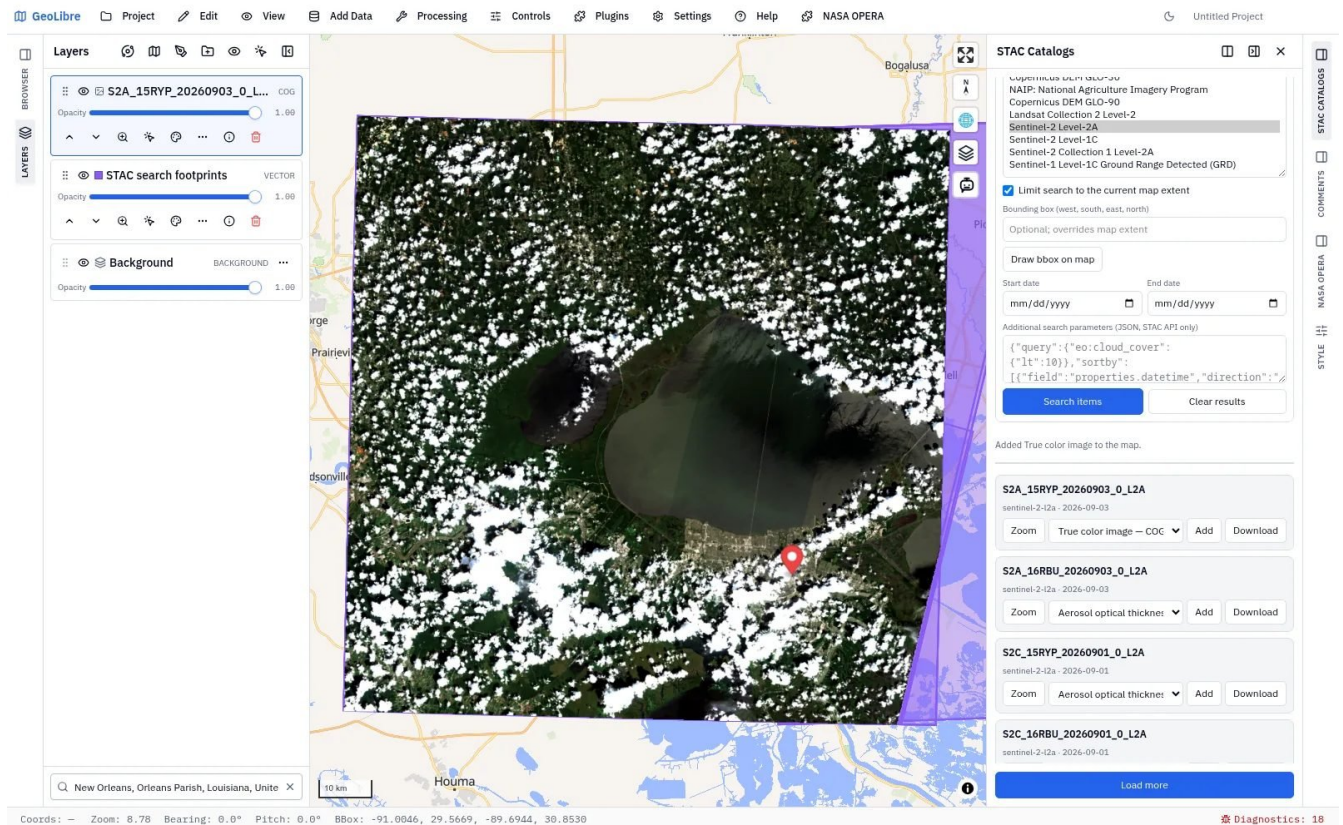
Why some searches route through a proxy

HDX does not send CORS headers to browsers, so the web build routes this search through GeoLibre's public tiles Worker. The desktop app queries the API directly over native HTTP. The same applies to the OpenAerialMap metadata API and the Source Cooperative catalog.

STAC Catalogs

A general-purpose [STAC](#) browser that works with any STAC API or static catalog.

- **Pick a catalog** from [STAC Index](#), or paste a catalog/API URL. Static catalogs are browsed as a tree; APIs are searched.
- **Filter** by collection (multi-select), date range, and area — the current map extent, a typed bounding box, or a box drawn on the map. STAC APIs also accept extra JSON search parameters.
- **Add an asset** to the map. GeoTIFF/COG, GeoJSON, GeoParquet, PMTiles, and Zarr (including Icechunk repositories) are supported; unsupported assets say so rather than failing silently.
- **Raster rendering options** — bands, colormap, min/max, and NoData — apply to assets added after you change them. The COG rendering engine picker is global: it applies to every raster on the map, including ones already added.
- Search-result footprints are drawn as their own layer, and each item can be zoomed to, added, or downloaded.



Source Cooperative

Browses [Source Cooperative](#) — a repository of large, cloud-native open datasets.

- Filter the catalog, or jump straight to an `account/product` reference.
- Browse a product's files with their sizes and formats, then add one to the map or download it.
- Adding delegates to the same code paths as [Add Data](#): PMTiles archives, GeoParquet and other vector formats, and COG rasters all land in the Layers panel exactly as if you had added the file by hand. Large GeoParquet can be **streamed** rather than fully downloaded.

S3 Browser

Browses Amazon S3 and S3-compatible buckets. Public buckets are read anonymously; private ones with the S3 connections in **Settings** → **Cloud Storage**, including AWS profiles, SSO, and IAM roles on the desktop app.

- Type `s3://bucket/prefix/` and press **Go**, or pick a connection and **List buckets**.
- **Add** puts COG, GeoParquet, GeoJSON, FlatGeobuf, GeoPackage, CSV, PMTiles, and COPC/LAZ/LAS point cloud files on the map through the same paths as [Add Data](#); layers keep the `s3://` URI, never a signed URL. Tick several files and choose **Add selected** to add them in one go.
- **Set as default** makes the current folder the one the browser opens at.

See [Cloud Storage](#) for connections, credential sources, and the CORS rule private buckets need on the web.

Natural Earth

The Source Cooperative panel pinned to the [Natural Earth](#) product, so it opens directly on the file list with no catalog or search step. Because the listing is live, it always reflects what is actually published rather than a copy that can drift.

Hugging Face

Browses geospatial data in [Hugging Face](#) dataset repositories, and is the one panel here that can also **write**.

- **Browse** — search the Hub or name an account, walk a repo's folders, and add its vector and raster files to the map (PMTiles, GeoParquet and friends, COG), through the same paths Add Data uses.
- **Upload** — with a user access token, create a dataset repo and push files into it.

The access token is stored in `localStorage` under your control, sent only as a bearer header to the Hugging Face API, and never written into a layer URL or a saved project.

Satellite Embeddings

A catalog of popular pre-computed **satellite embedding** datasets — per-pixel or per-patch vectors produced by geospatial foundation models — with search, on-map visualization, and download.

Dataset	Layout	Resolution	Search	Visualize	Download
AlphaEarth Foundations (Google Satellite Embedding V1), 2017–2025	64-band raster	10 m	✓	RGB composite of any three bands	Clipped GeoTIFF, source COG, VRT
Tessera , 2017–2025	128-band raster	10 m	✓	RGB composite of any three bands	Embeddings and scales (<code>.npy</code>)
Earth Index , 2024	Points (GeoParquet)	~320 m	✓	Points colored by principal components	Source GeoParquet
Clay, Major TOM, Copernicus-Embed	—	—	Links to the data source only		

- Pick a dataset to see its provider, model, resolution, dimensions, years, coverage, and license, with links to the data and the paper.

- Search by the **current map view** or a **box drawn on the map**, and (for annual datasets) a year. Result footprints are drawn as one entry in the Layers panel; hovering a result outlines it, and clicking a footprint scrolls to its result.
- **AlphaEarth** → **Visualize** adds the chosen bands (Earth Engine's `A01`, `A16`, `A09` by default) as an RGB layer, stretching de-quantized values across ± 0.3 by default. When the raster rendering engine is **cog-tiler-wasm (WASM)** (the default) the whole tile becomes a regular COG layer, zoomable to full 10 m detail, with its bands and range adjustable in the Style panel; the stretch is applied to the raw int8 values, so colors differ slightly from the de-quantized stretch. The GPU and TiTiler engines cannot read these files, so on those the panel instead renders a snapshot image of the search area (from an overview when the area is large; the status line says so) rather than switching the engine for every raster on the map.
- **AlphaEarth** → **GeoTIFF** saves all 64 bands over the search area as a north-up GeoTIFF in the tile's UTM zone, either de-quantized to float32 (unit-length vectors, NoData as NaN) or as the raw int8 values (NoData -128). The clip is built in memory and capped at 256 MB: about 100 km² of float32 values, or 400 km² of int8.
- **Tessera** → **Visualize** reads the chosen bands (0, 1, 2 by default) of the v1.1 embeddings released on [AWS Open Data](#) over the search area clipped to the 0.1° tile, de-quantizes them (int8 × the per-pixel scale), and adds them as a three-band float32 GeoTIFF raster layer in the tile's UTM zone, stretching each band to its own 2–98% range; the Style panel can re-stretch or reorder the bands, and Identify reads the embedding values. Unembedded pixels are NoData. The store has no overviews, so the read is always full 10 m resolution and costs about 1 MB per km² whichever bands you pick; it is capped at 1,300 chunks of 320 × 320 m (about 130 MB, enough for one whole tile). Chunks are fetched as a few merged byte ranges per shard rather than one request each. The status line shows the progress and the upper bound. The `.npy` downloads still fetch the v1 tiles.
- **Earth Index** → **Load points** adds the embeddings inside the search area as a point layer. Each point is colored by the top three principal components of the loaded vectors, so similar places get similar colors.

AlphaEarth files are stored bottom-up

The AlphaEarth COGs on Source Cooperative put their southern row first, which many GDAL workflows do not expect. The WASM engine (cog-tiler-wasm 0.3.8 and later) and the panel's own reader flip them; the GPU engine does not yet. The panel also offers each tile's companion `.vrt`, which GDAL reads north-up. The data is licensed CC-BY 4.0: *The AlphaEarth Foundations Satellite Embedding dataset is produced by Google and Google DeepMind.*

Where the Tessera v1.1 data is read from

The AWS release (`s3://tessera-embeddings/v1.1/dclimate.icechunk`) is an Icechunk repository whose `scales` array is PCodec-encoded, which no browser Zarr reader decodes. The panel reads the publishers' plain Zarr v3 mirror of the same snapshot on [Source Cooperative](#) instead. A 0.1° tile always lies in one UTM zone, so no reprojection across zones is needed.

Fields of the World

Browses [Fields of the World](#) (FTW), the global agricultural field boundaries predicted from Sentinel-2 imagery by the FTW PRUE model. It reads the same public files on [Source Cooperative](#) as the [FTW inference app](#).

- Pick a **year** (2025 or 2024).
- **Field boundaries** adds that year's global archive as a PMTiles layer, colored by the model's confidence from red (low) to green (high). The 2025 archive has tiles at every zoom; the 2024 alpha archive starts at zoom 10, so zoom in to see its fields.
- **Field density** adds the global 500 m field-density raster as a COG layer, for the zoomed-out picture. It keeps the current view and hides itself above zoom 12, where the field boundaries take over; both can be changed in the Style panel.
- The **confidence threshold** (70% by default, as in the FTW app) hides fields the model is less confident about. It applies to every FTW field layer on the map, is saved with the project as the layer's filter, and can be edited later in the Style panel. Downloads are never filtered.
- **Search tiles** lists the $1^\circ \times 1^\circ$ download tiles in the **current map view** or a **box drawn on the map** that have data for the year, most fields first, with each tile's field count and file size. Their outlines are drawn as one entry in the Layers panel; hovering a result outlines it, and clicking an outline selects its result.
- Each tile offers **Add to map** (its fields as an editable GeoJSON layer, styled and filtered like the archive), **GeoParquet** (the source file, unchanged), and **GeoJSON**. With **Only fields in the search area** checked (the default), Add to map and GeoJSON keep only the fields overlapping the search area. The file is read one row group at a time, so even the largest tiles (about 3 million fields) can be clipped to a small box; the result is limited to 250,000 fields on the map or 1,000,000 in a GeoJSON file. Download the GeoParquet for anything larger.

Running the FTW model

The plugin shows and downloads the published global predictions. To run the FTW model on your own area and Sentinel-2 scenes, use the [FTW inference app](#) or the [ftw-baselines](#) command-line tools, then add the result to GeoLibre. The data is licensed CC-BY-4.0.

Sentinel-2 Explorer

Finds and views Sentinel-2 L2A imagery anywhere on Earth, the way Taylor Geospatial's [reference explorer](#) does. It reads the [s2-stac-geoparquet](#) catalog on [Source Cooperative](#), which republishes every scene Earth Search indexes as partitioned STAC-GeoParquet. There is no API or server behind it: each search is a set of HTTP range reads against static files, and each image streams from the scene's Cloud-Optimized GeoTIFFs on AWS.

- Pick a **collection**: **Collection 1** (ESA's uniform reprocessing, from October 2015, the default) or the original **L2A** index (from November 2016).

- Set the **From** and **To** dates. The map colors every MGRS tile by its statistics over that window, read from the catalog's small monthly stats files: the **clearest scene's** cloud cover, the **scene count**, the **median cloud** cover, or the **coverage** (the most of the tile any one scene fills). Green is good and dark red is poor.
- The **Max cloud %**, **Min coverage %**, and **Min scenes per tile** filters grey out tiles that fail them. Max cloud and Min coverage also filter the scene list. **Show the tile grid** hides the grid without closing the panel.
- **Click a tile** to search its scenes. The panel reads only the row groups of the window's GeoParquet parts that can hold the tile, usually a few hundred KB, and reports how many range reads it took. The scenes can be sorted by least cloud, most coverage, or newest, and hovering a scene outlines its footprint.
- **Add scenes as** picks what **Add to map** streams:
 - the **true color** (TCI) image, or any **single band** (B01 to B12, B8A, AOT, WVP, the SCL scene classification, and for Collection 1 the cloud and snow probability masks), added as a COG layer and restyled in the Style panel like any raster;
 - a **composite** of several band files: **false color infrared** (B08, B04, B03), **agriculture** (B11, B08, B02), or **short-wave infrared** (B12, B8A, B04), or an **index**: **NDVI** (B08/B04) or **NDWI** (B03/B08) on a diverging ramp. Each map tile reads the bands' windows straight from their COGs at the matching overview, warps them from UTM, and paints them in the browser, so 10 m and 20 m bands combine at full resolution. Reflectance is corrected for the 1000 offset of processing baseline 04.00 and later. Composites need the MapLibre renderer.
- Once a scene is on the map in the chosen display, its button reads **Remove from map**; removing the layer in the Layers panel turns it back into **Add to map**. All of these layers are saved with the project. The selected tile is left unfilled so the scene shows through.
- **Download** lists the scene's files (TCI, every band, SCL, and the Collection 1 masks). Each opens the Cloud-Optimized GeoTIFF in your browser (the system browser on desktop), which saves it.
- **About this explorer** at the top of the panel collapses to save space.

Data and license

The imagery is Copernicus Sentinel-2 data processed by ESA, indexed by Element 84's Earth Search, and hosted on the AWS Registry of Open Data. The catalog is published by Taylor Geospatial under CC-BY-4.0.

Ocean Data Platform

Browses the public datasets on HUB Ocean's [Ocean Data Platform](#) (ODP), which shares ocean data from research institutions, governments, and industry: benthic habitats, coral reefs, protected areas, fisheries, seafloor features, observations, and more.

- The panel loads the whole public catalog (a few hundred datasets, grouped into collections) when it opens. Type in **Search** to match every word against the title, description, collection, and keywords; titles that match rank first. Pick a **Collection** to narrow the list, or check **Only datasets in the map view** to keep those whose extent overlaps the view (the list follows the map as you pan). Datasets the catalog gives no extent for only appear with that box unchecked.
- **Add to map** streams the whole dataset as a vector tile layer with every attribute, and zooms to its extent. It is saved with the project. Tiles can take a few seconds the first time a dataset is viewed.
- **Features in view** reads the features in the current map view through OGC API Features and adds them as an editable GeoJSON layer; **GeoJSON** saves the same features to a file. Both read at most 10,000 features, and say so when the view held more: zoom in, or use Add to map for the whole dataset.
- **Zoom** fits the map to the dataset's extent, and **Details** opens its page in the ODP catalog, with its full description, provenance, and download options.

Public datasets only

The panel lists and reads only datasets shared publicly on ODP; it does not sign in or take an API key. ODP's tile and features endpoints do not allow other websites to read them directly, so GeoLibre reads them through its own tile proxy (`tiles.geolibre.app`). Each dataset keeps its provider's license, shown in the result list; check it before reuse.

GeoLens

Connects to a self-hosted [GeoLens](#) catalog — an open-source spatial catalog (FastAPI + PostGIS) you run on your own infrastructure, which makes it the recommended way to work with private data in GeoLibre.

- Enter your server's base URL and, for private datasets, an API key. Search the catalog; each result links back to its metadata page.
- Datasets are added over the standards GeoLens already serves: **signed vector tiles** (the scalable default), **OGC API Features** GeoJSON, or server-rendered **raster tiles**.
- Vector-tile tokens are short-lived, so the plugin re-mints them automatically before they expire — which is why this is a plugin rather than a URL you paste into Add Data.
- A dataset loaded as GeoJSON can be edited and written back to GeoLens feature by feature, when the server allows it.

The API key is kept in memory for the session only. A saved project records just the server URL and dataset id, so public datasets restore automatically while private ones stay blank until the recipient reconnects with their own key. See [Self-Hosting & Private Data](#) for the deployment guide.

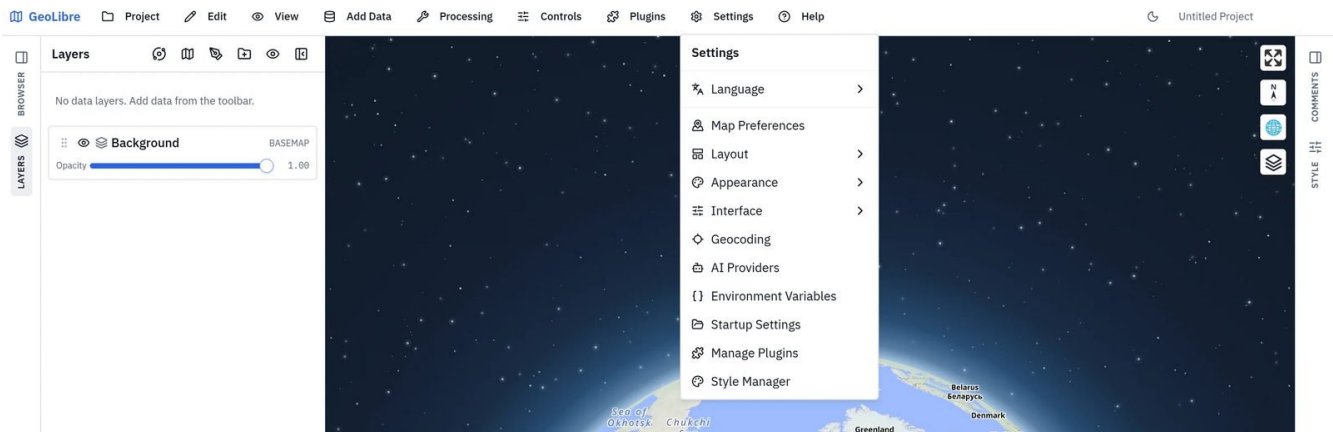
Related pages

- [Plugins & Marketplace](#) — the Plugins menu and installing external plugins
- [Data Integrations](#) — Planetary Computer, Earth Engine, Overture Maps, imagery, and geocoding
- [Adding Data](#) — the Add Data menu, including its own STAC, WMS, and WFS dialogs
- [Managing Layers](#) — what happens to a layer once a panel adds it

<https://geolibre.app/user-guide/settings/>

Settings & Preferences

The **Settings** menu holds the workspace preferences: how the map behaves, which panels are visible, the interface language and appearance, geocoding and AI providers, runtime environment variables, and the entry point to [Manage Plugins](#).



The Settings dialog is organized into these sections:

Section	What it covers
Language	The interface language and optional Whitebox language packs. See Language .
Map	Navigation constraints, celestial body, scale units, and coordinate format. See Map Preferences .
Layout	Which panels and toolbar labels are shown. See Layout .
Appearance	Light or dark mode and the accent color applied on top of it. See Appearance .
Interface	The UI profile — an experience level (Beginner, Intermediate, Advanced, or Custom) that simplifies the menus, data sources, and plugins on offer. Nothing is removed permanently; you can switch levels at any time. See Interface .
Geocoding	The address-search provider. See Geocoding .
AI Providers	Model and credentials for the AI Assistant . See AI Providers .
Environment	The share token and runtime key-value pairs. See Environment Variables .
Cloud Storage	S3 connections for private buckets, and the S3 Browser's default location. See Cloud Storage .
Updates	Update checks (desktop only). See Updates .

Section	What it covers
Startup	The untitled workspace's projection and view, and (desktop only) which project the app opens with. See Startup .
Style Manager	Your saved symbology presets, reachable here and from a layer's Layer actions → Styles → Saved styles . See Styling Layers .

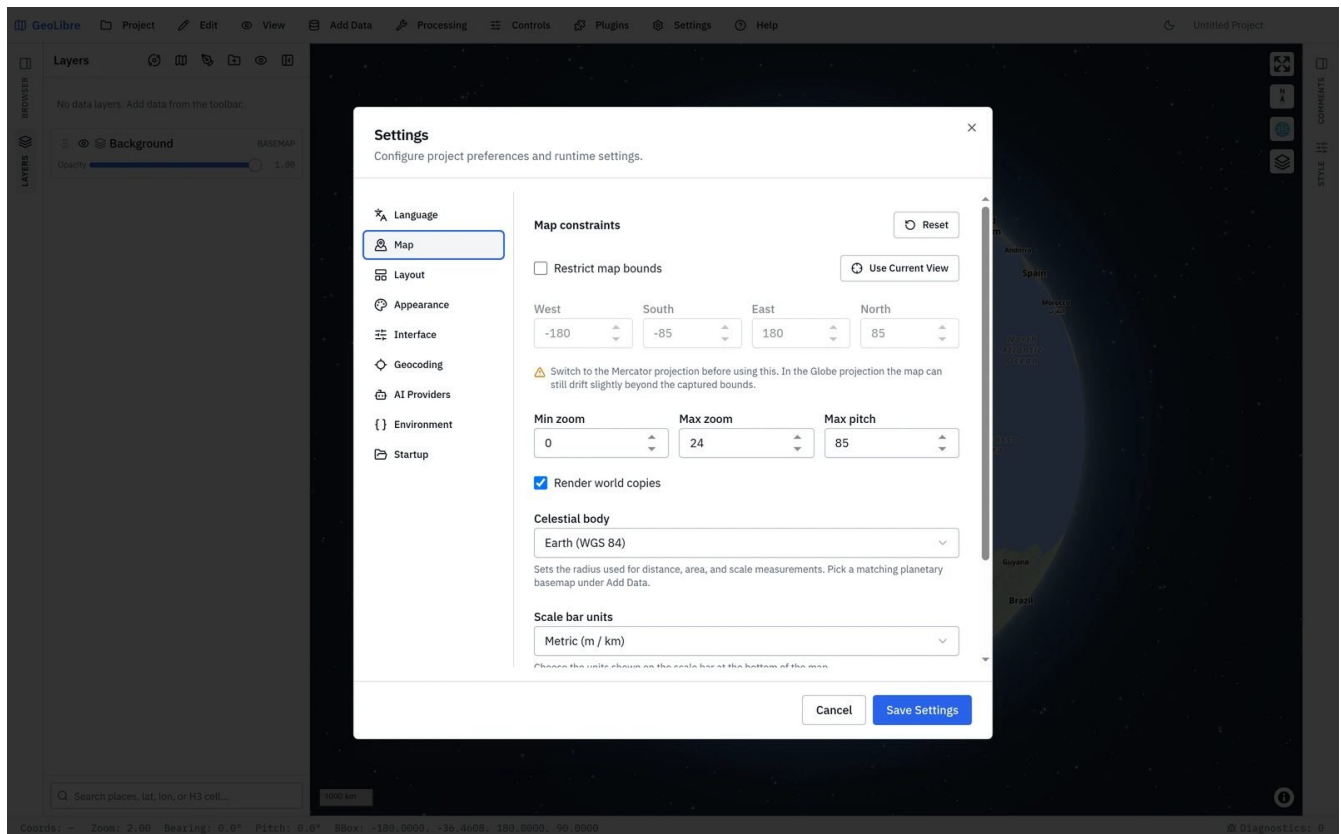
Language

Settings → **Language** sets the **Interface language**. Each option is shown in its own script, with the English name alongside. Right-to-left languages mirror the whole layout. See [Internationalization](#) for the supported languages.

The same section manages optional **Whitebox language packs**, which translate Whitebox tool names, parameters, and help text without making every installation larger. Choose **Download pack** to fetch the official pack for the selected language, or **Import file** to install a pack file. **Remove pack** returns the Whitebox metadata to English. Packs are stored only on the current device. Downloading one sends the locale code, but no project data, to the pack host. Builds that disable downloads offer only **Import file**.

Map Preferences

Settings → **Map Preferences** controls how the map can be navigated:



Setting	Description
Restrict map bounds	Limit panning to a bounding box.
Bounds	The west, south, east, and north limits of that box.
Min zoom / Max zoom	The allowed zoom range (0 to 24).
Max pitch	The maximum tilt angle (0 to 85 degrees).
Render world copies	Show repeated copies of the world when zoomed out.
Celestial body	The body whose radius drives distance, area, and scale measurements. Pick the one matching your planetary basemap under Add Data .
Scale bar units	Metric (m / km), Imperial (ft / mi), or Nautical (nmi). This also sets the units used by the status bar's Elev and Eye alt readouts and by the quick-analysis buffer presets.
Coordinate format	The notation the status bar reports the pointer coordinate in: decimal degrees, DMS, DDM, UTM, MGRS, USNG, or x/y in a projected CRS. Choosing the projected format shows an EPSG field for the CRS code. See the status bar .

Use **Use Current View** to set the bounds from where the map is now, or **Reset** to restore the defaults. These preferences are saved in the project file.

Capturing bounds on the globe

Use Current View is most accurate in the Mercator projection. In the Globe projection the map can still drift slightly beyond the captured bounds, and the dialog says so.

Layout

Settings → **Layout Settings** toggles the chrome around the map:

- **Show toolbar labels**: text labels next to toolbar buttons, or icon-only.
- **Show project info**: the project name and path in the toolbar.
- **Show Layers panel, Show Style panel, Show Browser panel, Show Comments panel**: per-panel visibility.

Panels also auto-hide on small screens for a responsive layout.

Appearance

Settings → **Appearance** picks the **Mode** (Light or Dark) and an **Accent color** applied on top of it: Blue, Violet, Emerald, Rose, Amber, or **Custom**, which takes any color from a picker or a hex code.

Interface

Settings → **Interface** chooses an **Experience level** that simplifies the interface: **Beginner**, **Intermediate**, and **Advanced** apply a curated layout at once. Below the levels you can switch individual **Data sources**, **Plugins**, and **Menus** on or off. Editing any of them switches the level to **Custom**. Nothing is removed permanently, and **Reset** restores the defaults. When an administrator manages the profile, this section is read-only. See [UI Profiles](#).

Geocoding

Settings → **Geocoding** chooses the **Provider** used by Geocode Addresses, address import, and Reverse Geocode. Nominatim is the default and needs no key. For providers that need one, enter an **API key**. You can also override the **Forward endpoint** and **Reverse endpoint**, for example to point at a self-hosted instance, and give Nominatim a **Contact email**. The section notes where keys are stored: in the system keychain when it is available, otherwise in the project file. Keys are sent as query parameters, so they appear in request URLs. See [Data Integrations](#) → [Geocoding](#) for the provider list.

AI Providers

Settings → AI Providers configures the credentials for the [AI Assistant](#). Each configuration is a named **profile**, such as *Work Gemini* or *Local Ollama*. Choose **Add profile**, pick a provider, fill in the fields it asks for, then **Save profile**. Mark one profile **Set as default** to use it when the assistant opens; one configured profile is enough to start chatting. Credentials are saved on the current device, never in the project file. When a system keychain is available, the secret fields go there. On the desktop app, a blank key field falls back to the matching operating-system environment variable. See [AI Assistant → Setup](#).

Environment Variables

Settings → Environment Variables (the **Environment** tab in the Settings dialog) holds the share account connection and the runtime key-value pairs that GeoLibre and its plugins read, such as API keys:

- **Share.GeoLibre account:** sign in to `share.geolibre.app` (a popup on the web, your system browser on the desktop app), or sign out. Used by **Project → Share** and the Project Gallery. See [Projects](#).
- **Share.GeoLibre API token:** a personal API token for **Project → Share**, the fallback when sign-in is not available (for example on mobile or in a notebook embed). See [Projects](#).
- **Environment variables:** named key-value pairs (for example, API keys for Earth Engine, Street View, and other integrations). You can enable or disable individual variables, and secret values are masked. Variable names must start with a letter or underscore and contain only letters, numbers, and underscores.

Where credentials go

Provider credentials for integrations like Earth Engine, Street View, Google Photorealistic 3D Tiles, or other keyed services belong here. See [Data Integrations](#) and [Getting Started](#).

Reading AI keys from your system environment (desktop)

On the desktop app, the [AI Assistant](#) also reads its own allowlisted keys (`OPENAI_API_KEY` , `ANTHROPIC_API_KEY` , `GEMINI_API_KEY` , and the other provider variables) straight from your operating system's environment variables — so you can keep API keys out of the saved project file entirely. A value entered here always takes precedence over the OS environment. See [AI Assistant → Reading keys from your system environment](#) for the full list.

Protomaps basemaps

To use the [Protomaps](#) basemaps in the **New project** dialog, add an environment variable named `VITE_PROTOMAPS_API_KEY` with your own Protomaps API key. The Protomaps options appear in the dialog as soon as the key is enabled — no restart needed. When no key is set, the Protomaps section is hidden. See [Getting Started](#) for setting the key at build time for a self-hosted deployment.

Cloud Storage

Settings → **Cloud Storage** (also in the **Settings** menu) holds the S3 connections that read private Amazon S3 and S3-compatible buckets (access keys, AWS profiles including SSO, environment variables, and IAM roles), and the S3 Browser's default location. See [Cloud Storage](#).

Project name and file

The project name is edited in place on the right of the toolbar, and it is saved into the `.geolibre.json` file. To also see the file path the project was opened from or last saved to, turn on **Show project info** under [Layout](#). See [Projects](#) for the rest of the project lifecycle and [Project Format](#) for what the file contains.

Startup

Settings → **Startup Settings** chooses how GeoLibre opens a new session. The installed desktop app also offers project restoration modes:

Mode	Behavior
Open the default workspace (default)	Start with a new, untitled project.
Reopen the last project	Open the most recently used <i>local</i> project.
Open a specific project	Always open one chosen project. Use Choose Project to pick the file; the mode stays unavailable until you have.

Enable 3D globe by default controls the projection of the new, untitled workspace shown when no project is provided. Turn it off to start that workspace in Mercator. **Default map view** sets the center longitude, center latitude, and zoom level for the same workspace. Choose **Use Current View** to copy the center and zoom from the map canvas. A restored project or project link always uses the projection and camera saved in that project instead.

Open the S3 Browser at startup shows the [S3 Browser](#) panel each time GeoLibre starts; it stays open when you open a project. **Default layer styles** takes a layer styles file (**Project → Export → Export Layer Styles**): each layer you add whose name matches a style in it is styled automatically, while layers in opened projects keep their saved styles. GeoLibre keeps a copy of the file, so choose it again after editing it.

If the startup project has been moved or deleted, GeoLibre opens the default workspace instead, says so in a banner, and drops the missing file from the recent-projects list.

Project restoration is desktop only

The browser build includes the empty-workspace projection option, but has no persistent local file to reopen, so the three project modes appear only in the installed desktop app.

Two deliberate limits are worth knowing:

- **Only local projects are reopened.** Opening a share link records it in your recent projects by its `https://` URL, so *Reopen the last project* skips remote entries rather than fetching a third-party host on every launch.
- **A URL always wins.** Launching with a project or `?data=` parameter in the URL skips the startup restore entirely, and so does opening your own project before the restore finishes.

Android reopens its own copy

Android identifies a project picked from device storage by a temporary reference that stops working once the app's process ends — which is exactly when the startup restore runs. So on Android GeoLibre keeps a copy of the startup project in its own private storage and reopens that copy, refreshing it every time you open or save the project. Two consequences worth knowing: a project edited in another app after you last saved it in GeoLibre reopens as GeoLibre last saw it (open it again from **Project → Open From → File...** to pick the newer contents back up), and a project deleted from the device still reopens from GeoLibre's copy rather than dropping out of the startup preference, because Android reports a deleted file and an expired reference the same way.

Saving a project you opened from device storage asks you where to save it, once — Android does not grant write access to a file you only picked to read. If your startup project is that project, the preference follows it to the file that save creates, so it keeps opening the copy you are actually working in.

Updates

Settings → Update Settings (desktop only) controls the update check: whether GeoLibre checks for a newer version at startup, and which kinds of releases raise a notification. Turn the check off for a fully offline workflow.

Style Manager

Settings → Style Manager opens your personal library of symbology presets. Save the style you have built on a layer, then apply it to any other layer in any project. The same library is reachable from a layer's **Layer actions → Styles → Saved styles (Style Manager)....** See [Styling Layers](#).

Manage Plugins

Settings → Manage Plugins opens the plugin marketplace. See [Plugins & Marketplace](#).

<https://geolibre.app/user-guide/storymaps/>

Story Maps

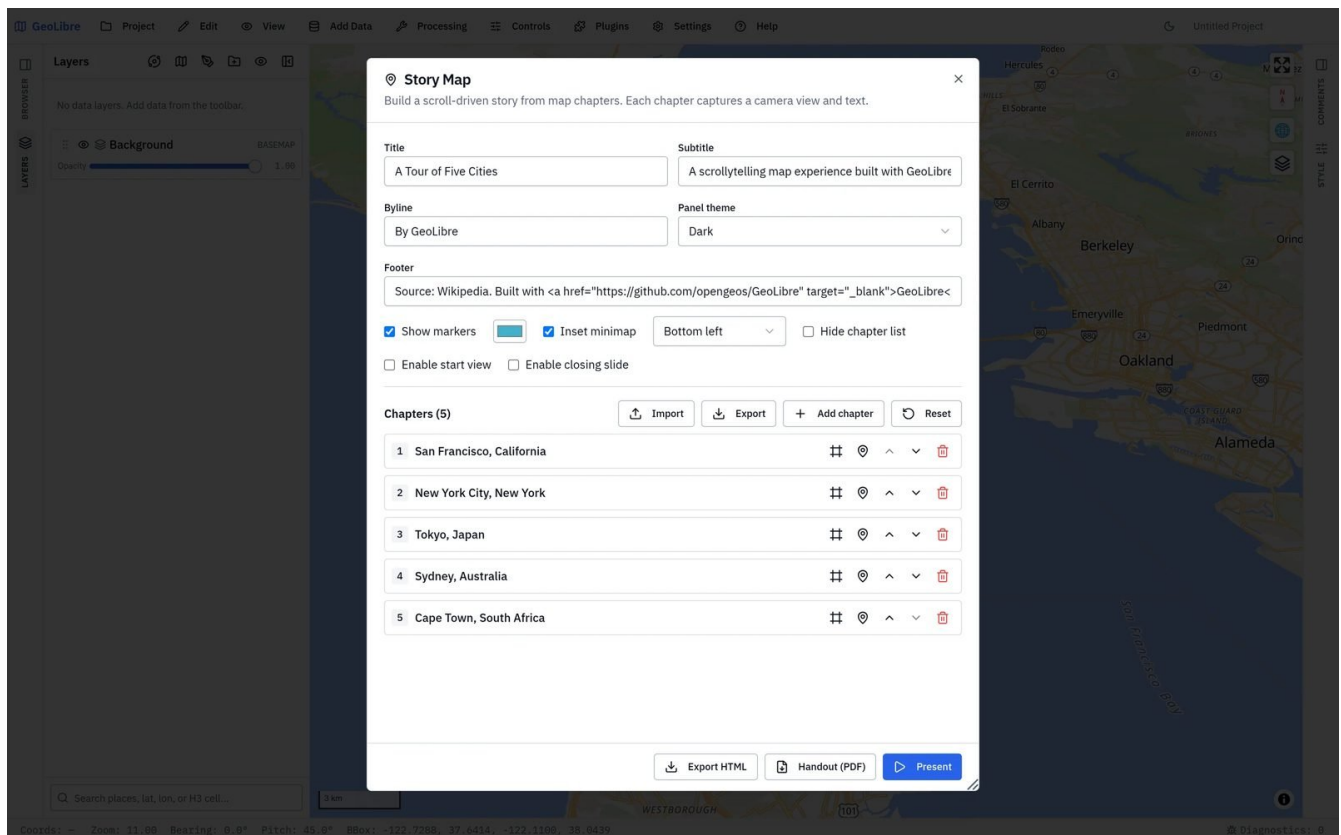
Story maps turn a project into a scroll-driven narrative. As the reader scrolls, the map flies between chapters, and project layers can fade in and out. The feature is inspired by the [maplibre-gl-storymaps](#) template. Authoring and the in-app presentation run locally, though basemap styles and any chapter images are still fetched from their URLs, and the standalone HTML export loads MapLibre and scrollama from a CDN.

Open the builder

Choose **Project** → **Story Map** to open the builder. The builder edits a story that is saved inside the `.geolibre.json` project file, so it travels with your project.

Try it instantly

When the story is empty, click **Load sample story** to populate a five-city world tour. Hit **Present** and scroll to see it in action, then edit or replace the chapters with your own.



Story settings

- **Title / Subtitle / Byline / Footer** appear in the presentation header and footer. The footer accepts inline HTML (for example, links and credits).
- **Panel theme** switches the chapter panels between light and dark.
- **Show markers** drops a marker at each chapter's center; pick its color.
- **Inset minimap** shows a small overview map in a chosen corner.
- **Hide chapter list** removes the navigation pane from the presentation, for a story meant to be read straight through.
- **Enable start view** adds an opening screen before the first chapter, carrying the story's title block. Choose what the map shows behind it: an **Empty screen**, a **Black screen**, a **Global view**, or a **Preview of first chapter**.
- **Enable closing slide** adds a final screen after the last chapter, for credits or a closing note. The same backdrop choices apply, plus **Hold on last chapter** to leave the camera where the story ended.

Chapters

Click **Add chapter** to capture the current map view as a new chapter. Each chapter has:

- **Title, Description** (inline HTML allowed), and an optional **Image URL**.
- **Panel alignment:** `left`, `center`, `right`, or `full`.
- **Map animation:** `flyTo`, `easeTo`, or `jumpTo`.
- **Hide panel** keeps the map transition but hides the text (useful for a pure map beat).
- **Rotate camera** slowly spins the view after the transition settles.

Use **Set to current view** to re-capture the camera after panning, zooming, or tilting the map, and the map-pin button to fly to a chapter while editing. Reorder chapters with the arrows and remove them with the trash icon.

Import and export chapters

To author content outside GeoLibre, use **Import** / **Export** next to **Add chapter**:

- **Export JSON** writes the whole story (settings and chapters); **Import JSON** reads it back (a full `.geolibre.json` project file also works).
- **Export CSV** writes one row per chapter (spreadsheet-friendly). **Import CSV** replaces the chapters while keeping your current story settings.

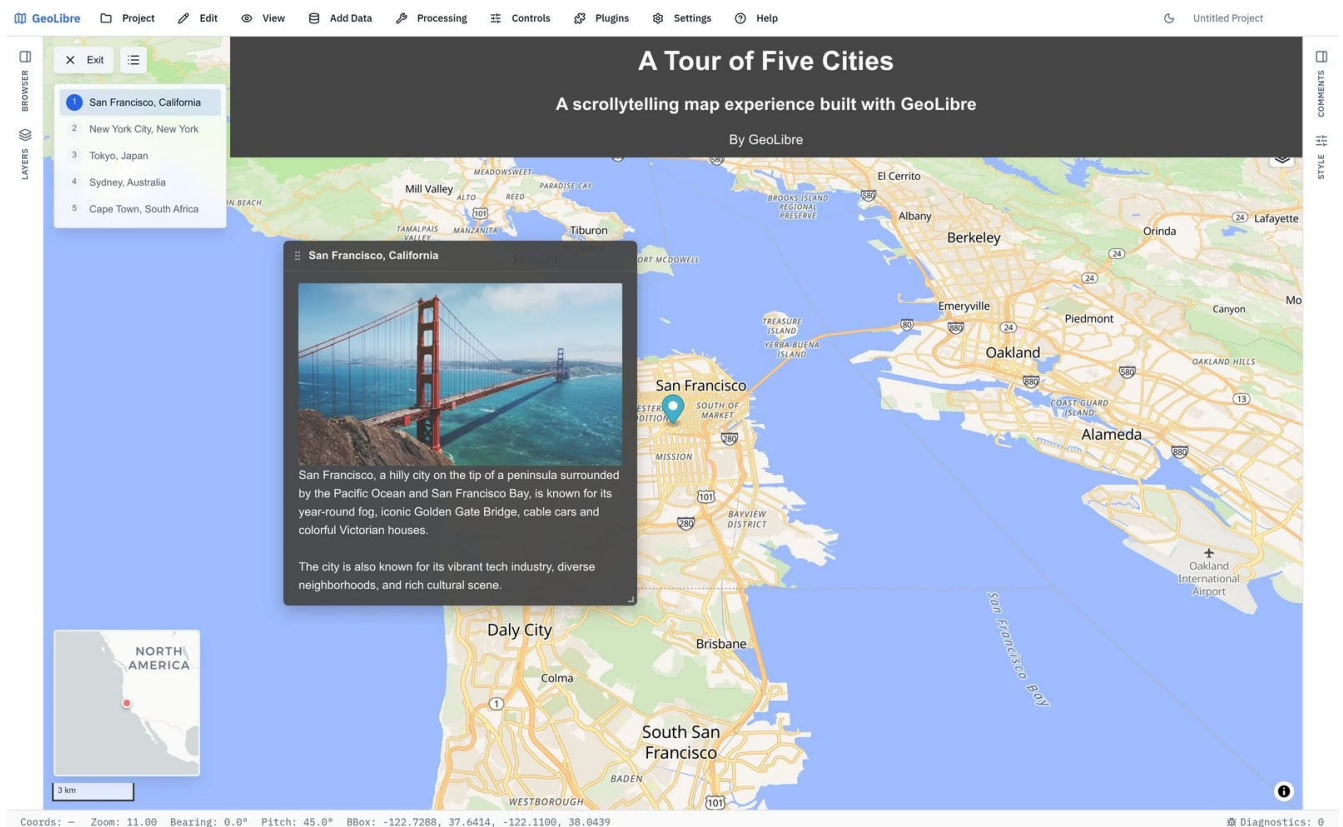
The CSV columns are `id`, `title`, `description`, `image`, `alignment`, `hidden`, `lng`, `lat`, `zoom`, `pitch`, `bearing`, `mapAnimation`, `rotateAnimation`, `onChapterEnter`, `onChapterExit`. Columns are matched by name, so order is flexible and you only need the ones you use; `lng` / `lat` set the chapter center, and missing ids are generated on import. The `onChapterEnter` / `onChapterExit` columns hold JSON and can be left blank when authoring by hand.

Layer effects

Under **On enter** and **On exit**, add layer fades that run as a chapter becomes active or is left behind. Pick a project layer and a target opacity (and optional transition duration). This is how you reveal or hide data as the story progresses.

Present

Click **Present** to start the scroll-driven presentation over the live map. Scroll to move between chapters; press **Esc** or **Exit** to return to editing.



A **navigation pane** on the left lists every chapter for a quick overview; click any entry to jump straight to it. Toggle the pane with the list button next to **Exit**. The map controls (navigation, fullscreen, globe, layer control) stay usable during the presentation.

Each chapter card is **movable and resizable**: drag its title bar to reposition it (double-click the bar to reset), or drag the bottom-right corner to resize, so you can move a card aside and explore the map beneath it.

Handout (PDF)

Click **Handout (PDF)** to export the story as a multi-page PDF, one page per screen. Pick which screens to include, the paper size and orientation, and the document title, subtitle, byline, and footer text drawn on every page. Each chapter page shows the captured map view, the chapter photo beside it when the chapter has one, and the chapter description.

Clickable location markers draws a marker at each chapter's location, its tip on the chapter coordinate. Opened in a PDF viewer, clicking a marker opens that coordinate in Google Maps, so a reader can navigate to the place. The marker takes the story's **Marker color**, and the option starts on when **Show markers** is enabled for the story.

Export to HTML

Click **Export HTML** to save a self-contained `.html` page that reproduces the story without GeoLibre. In-memory GeoJSON layers referenced by chapter effects are inlined so the exported story behaves like the in-app preview. The page is a single static file you can host anywhere (GitHub Pages, Netlify, S3, or any web server).

<https://geolibre.app/user-guide/embedding/>

Embedding & Sharing

GeoLibre's browser build can be embedded in any web page and configured through URL query parameters. This is how you turn a shared project into a live, focused map for a website, a report, or a dashboard.

The live viewer

The browser build is hosted at <https://web.geolibre.app/>. It is a static site deployed on GitHub Pages that runs entirely in your browser: there is no server account, and the data you load is processed client-side. The hosted site counts page visits with Google Analytics, which never sees the data you load (see the [Privacy Policy](#)); a copy you host yourself has no analytics at all. Data leaves your browser only when you add a remote URL or explicitly share a project.

Open a public project by passing its `.geolibre.json` URL with the `url` parameter:

```
https://web.geolibre.app/?url=https://share.geolibre.app/giswqs/3d-tiles.geolibre.json
```

A project URL like this comes from **Project** → **Share**. See [Projects](#).

A chrome-free `maponly` embed shows only the map, as in this shared 3D Tiles project:



URL parameters

Parameter	Example	Description
<code>url</code>	<code>url=https://share.geolibre.app/you/project.geolibre.json</code>	Loads a <code>.geolibre.json</code> project from a public URL.
<code>loading</code>	<code>loading=true</code>	Exposes screenshot readiness on the document element. Accepts a bare flag, <code>true</code> , <code>1</code> , <code>yes</code> , or <code>on</code> ; disabled by default. See below.
<code>data</code>	<code>data=https://assets.geolibre.app/data/places.geojson</code>	Loads public GeoJSON, GeoParquet, PMTiles, a COG, or a ZIP/REST response containing multiple GeoJSON files.
<code>style</code>	<code>style=https://assets.geolibre.app/data/sample.style.json</code>	Applies a GeoLibre/MapLibre vector style or raster-style JSON to the data loaded by <code>data</code> .
<code>dataType</code>	<code>dataType=lidar</code>	Marks a <code>data</code> URL whose path has no file extension as a LiDAR point cloud; pairs with <code>data</code> by position. See Open remote data .
<code>stac</code>	<code>stac=https://earth-search.aws.element84.com/v1/collections/naip</code>	Opens the STAC Catalogs browser connected to a STAC catalog, API, or API collection. A collection URL is searched on load, so its item footprints appear as a layer. See Open a STAC catalog .
<code>layout</code>	<code>layout=viewer</code>	<code>viewer</code> provides read-only chrome: Layers, View, Controls, basemaps, search/identify, Help, and any quick filters the project's layers carry, with authoring UI hidden. <code>compact</code> is the icon-only full-app layout; <code>embed</code> and <code>iframe</code> are aliases.
<code>toolbar</code>	<code>toolbar=none</code>	Hides the top toolbar while keeping panels and the status bar. Use <code>icons</code> for icon-only buttons; <code>icon</code> and <code>icon-only</code> are aliases. <code>hidden</code> , <code>hide</code> , and <code>off</code> are aliases for <code>none</code> .
<code>panels</code>	<code>panels=collapsed</code>	Starts Layers and Style collapsed to their icon rails. Use <code>none</code> to hide all panels; <code>hidden</code> , <code>hide</code> , and <code>off</code> are aliases.
<code>hidePanels</code>	<code>hidePanels=true</code>	Alternative way to hide those panels.
<code>maponly</code>	<code>maponly</code>	Hides all chrome (toolbar, panels, and status bar), leaving only the map. The bare flag or <code>true</code> , <code>1</code> , <code>yes</code> , or <code>on</code> enable it.
<code>welcome</code>	<code>welcome=0</code>	Hides the first-launch welcome wizard. Accepts <code>0</code> , <code>false</code> , <code>off</code> , or <code>no</code> . A <code>url=</code> or <code>data=</code> deep link already suppresses it automatically.
<code>theme</code>	<code>theme=dark</code>	Sets the initial color theme, overriding the OS preference. Accepts <code>dark</code> or <code>light</code> ; the in-app toggle still works afterward.

Parameter	Example	Description
settingsUrl	settingsUrl=https://example.com/desktop-settings.json	Loads shared presentation settings before the first render. Supports <code>language</code> , <code>layout</code> , <code>accent theme</code> , and <code>uiProfile</code> . The override lasts for this page only and does not replace locally saved settings. <code>settingUrl</code> is accepted as an alias.
tool	tool=adaptive_filter	Opens the Processing (Whitebox toolbox) dialog on a specific tool by its id. Unknown ids open the dialog without preselecting a tool.
plugin	plugin=swipe	Activates one or more built-in plugins, as if picked from the Plugins menu. See Activate a plugin .

Private projects and data

`url=` and `data=` are fetched by the browser with same-origin credentials, so a project or dataset gated by a session cookie loads only when GeoLibre is served from that same origin. Passing a login-protected URL to the hosted viewer at `web.geolibre.app` fails, because the cookie is not sent cross-origin. Serve the app from your own host, or use a signed, expiring URL. See [Self-Hosting & Private Data](#).

Parameters combine. For a narrow, chrome-free, dark embed of a shared project:

```
https://web.geolibre.app/?url=https://share.geolibre.app/you/project.geolibre.json&maponly&theme=dark
```

The settings document must be public or same-origin, return valid JSON, and allow cross-origin browser requests when hosted elsewhere. Its supported fields use the same shape as their counterparts in the `geolibre.desktopSettings` local-storage value. Credential-bearing fields, plugin sources, local startup paths, and update settings are ignored. Missing or invalid fields are replaced with GeoLibre defaults. If the document fails to load within ten seconds, GeoLibre starts with the visitor's local settings. The `locale` and `lang` URL parameters take precedence over a shared `language`. Other embed parameters, including `theme`, `layout`, and `maponly`, independently control the initial light/dark mode and chrome rather than the desktop accent and panel preferences.

Encode the settings URL with `encodeURIComponent` when it contains `&`, `+`, `%`, or `#`, just as for a nested `data` URL.

Deep-linking a Processing tool

`tool=<id>` opens the Processing (Whitebox toolbox) dialog preselected to a tool. Any **additional** query parameters pre-fill that tool's form, using the tool's own parameter names, so a link can arrive ready to run:

```
https://web.geolibre.app/?tool=extract_cog_subset&url=https%3A%2F%2Fdata.source.coop%2Fgiswqs%2Fopengeos%2Fdem.tif&bbox_crs=4326
```

When `tool=` is present the app is in **tool mode**: `url` names a tool input (here, the COG to subset) rather than a project to load, so the project loader stands down. App/embed parameters above (`theme`, `layout`, `panels`, `maponly`, `locale`, ...) keep their own meaning and are never passed to the tool. Preselection and parameter prefilling apply only to an id that matches the Processing menu: an id not in the menu still opens the dialog, but without preselecting a tool or applying any parameters. A known id the current engine doesn't expose (WASM in the browser, the Python sidecar on desktop) likewise isn't preselected. Tool ids match the Processing menu — the same ids used across the [Whitebox toolbox](#).

Deep-linking a plugin

Use `plugin` to open the app with a built-in plugin already active, as if you had picked it from the **Plugins** menu:

```
https://web.geolibre.app/?plugin=swipe
```

Name a plugin by its id (`maplibre-gl-time-slider`) or by its short name, the id without a `maplibre-gl-`, `maplibre-`, or `geolibre-` prefix (`time-slider`). Short names are case-insensitive. The [Plugins](#) page lists every link name. List several plugins with commas, or repeat the parameter:

```
https://web.geolibre.app/?plugin=graticule,h3-grid
```

It combines with `url`: the plugin opens once the shared project has loaded, on top of the plugins the project itself turns on.

```
https://web.geolibre.app/?url=https://share.geolibre.app/giswqs/3d-tiles.geolibre.json&plugin=swipe
```

A plugin from the official plugin registry works too, by its registry id (`?plugin=openrndt-geolibre`). An uninstalled compatible entry prompts for **Trust and load**; confirming trust does not bypass deployment plugin policy. A denied plugin will not load or activate. In read-only `layout=viewer` an installed registry plugin still opens, but the prompt is skipped and uninstalled ones are never installed. This is distinct from project manifest URL trust, whose prompt is suppressed when `sideload: false`. See [Plugin policy](#).

Unknown names are ignored. A plugin that does not support the current renderer does not activate. Directions and reverse geocoding send what you click to a public server, so they only open from the menu, after their one-time notice. The drawing and editing plugins stay off in `layout=viewer`.

Waiting for a screenshot

Add `&loading=true` to a project link to enable a machine-readable readiness signal without adding a visible overlay to the screenshot:

```
https://web.geolibre.app/?url=https://share.geolibre.app/giswqs/national-land-cover-database-nlcd.geolibre.json&maponly&loading=true
```

The `<html>` element exposes three attributes:

Attribute	Value
<code>data-geolibre-load-state</code>	<code>loading</code> , <code>ready</code> , or <code>error</code>
<code>data-geolibre-load-pending</code>	JSON array of pending layer names (or initialization work)
<code>data-geolibre-load-errors</code>	JSON array of failure messages

`ready` means the project/data URL has loaded, visible layers have attached, their current-viewport tiles have loaded, the camera has stopped, and browser fonts have loaded. These checks must remain satisfied for 500 ms across animation frames. Changing the view or layers returns the signal to `loading`. Hidden layers (including hidden groups), fully transparent layers, and layers outside their zoom range do not block readiness. This does not download an entire dataset or tiles outside the viewport.

The check supports native MapLibre layers, the raster control's COG layers, and deck.gl visualization layers, including their shared deck.gl renderer. Custom renderers without a readiness probe, such as Cesium, LiDAR, Zarr, splats, and video, report an explicit error instead of assuming they are ready. A map or tile failure the map recovers from does not block `ready`; one it never recovers from is reported alongside the timeout, as is any loading that does not settle within 120 seconds. Check the errors before capturing; neither the embed API's `ready` event nor a browser's `networkidle` state establishes this rendering readiness.

For example, with Playwright:

```
await page.setViewportSize({ width: 1600, height: 1000 });
await page.goto(projectLink + "&maponly&loading=true");
await page.waitForFunction(
  () => ["ready", "error"].includes(document.documentElement.dataset.geolibreLoadState),
  undefined,
  { timeout: 150_000 },
);
const result = await page.evaluate(() => ({
  state: document.documentElement.dataset.geolibreLoadState,
  errors: JSON.parse(document.documentElement.dataset.geolibreLoadErrors || "[]"),
}));
if (result.state !== "ready") throw new Error(result.errors.join("; "));
await page.screenshot({ path: "map.png" });
```

Embedding in a page

Drop the viewer into an `<iframe>`:

```
<iframe
  src="https://web.geolibre.app/?url=https://share.geolibre.app/you/
project.geolibre.json&maponly"
  title="Geolibre map"
  width="100%"
```

```
height="600"
style="border: 0;"
loading="lazy"
allow="fullscreen; geolocation"
></iframe>
```

Use `toolbar=none` to hide the top toolbar while retaining the configured side panels and status bar. Use `layout=viewer` for a read-only map with layer toggles, search/identify, and basemap switching. Its layer list mirrors the authoring Layers panel, folders and all, so group names carry over, and a layer that carries [quick filters](#) shows them below its row so the map can be questioned as well as read. The Controls menu is part of the viewer chrome, minus the two entries that write to the project (Field Collection and GPS Tracking); Record Tour and Record Video only read the map, so they stay. Read-only covers the keyboard too: the project shortcuts (Ctrl/Cmd+N, +O, +S) and the command palette (Ctrl/Cmd+K) go with the menus they belong to, while the View shortcuts (`[`, `]`, `n`, `u`, `r`) keep working since the View menu stays. Dropping a file onto the map imports nothing, and the plugins that write to the project — the geometry editor, Annotations, and GeoAgent — cannot be active, even if the loaded project saved them that way. So an embed cannot be steered into authoring by a key press, a drag, or a project file. Display plugins (layer control, basemaps, time slider, legend and colorbar components) keep working, so the project still looks as it was saved. Use `layout=compact` for the complete authoring toolbar in a smaller space, or `maponly` for a pure map.

Open remote data

Use `data` to open public GeoJSON, GeoParquet, PMTiles, Cloud-Optimized GeoTIFF (COG), a LiDAR point cloud (LAS, LAZ, COPC, or an EPT `ept.json`), or a ZIP archive containing one or more `.geojson` / `.json` FeatureCollections. Each GeoJSON file in a ZIP becomes a separate layer. An optional `style` URL applies Mapbox/MapLibre style JSON to vector data:

```
https://web.geolibre.app/?data=https://assets.geolibre.app/data/places.geojson&style=https://assets.geolibre.app/data/sample.style.json
```

Repeat `data` to load multiple independent datasets on the same map. Repeat `style` in the same order to style each dataset; use an empty `style=` as a placeholder when an earlier dataset should use its default style:

```
https://web.geolibre.app/?data=https://example.com/roads.geojson&data=https://example.com/dem.tif&style=&style=https://example.com/dem.style.json
```

GeoLibre loads every entry and fits the map once to their combined stored extents. A style applies only to the `data` value at the same position.

`data` may also point to a REST API endpoint that returns either a GeoJSON `FeatureCollection` or a ZIP containing multiple GeoJSON files. ZIP API responses are recognized from their `Content-Type` / `Content-Disposition` headers or their ZIP file signature, so the endpoint does not need a `.zip` suffix. An endpoint that takes its own query parameters is the case that does need percent-encoding, so its `&` separators are not read as GeoLibre's own:

```
https://web.geolibre.app/?
data=https%3A%2F%2Fapi.example.com%2Ffeatures%3Fcategory%3Dparks%26limit%3D100
```

The example API URL is illustrative. Use the hosted `places.geojson` example above for a directly runnable test.

A hosted ZIP with per-file styles can be tested directly:

```
https://web.geolibre.app/?data=https://assets.geolibre.app/data/multiple-
layers.zip&style=https://assets.geolibre.app/data/multiple-layers.style.json
```

GeoParquet is loaded through GeoLibre's DuckDB-backed vector reader. PMTiles is streamed with HTTP range requests and may contain either vector or raster tiles. These real vector examples also apply the hosted sample style:

```
https://web.geolibre.app/?data=https://data.source.coop/giswqs/opengeos/
building_count_h3.parquet&style=https://assets.geolibre.app/data/sample.style.json
```

```
https://web.geolibre.app/?data=https://data.source.coop/giswqs/opengeos/
building_count_h3.pmtiles&style=https://assets.geolibre.app/data/sample.style.json
```

For a vector PMTiles archive, the style is applied to the layer after the archive's source layers are discovered. A raster PMTiles archive can be loaded with `data`, but it does not accept a MapLibre vector style through `style`.

A public DEM COG can be tested directly:

```
https://web.geolibre.app/?data=https://data.source.coop/giswqs/opengeos/dem.tif
```

A LiDAR point cloud opens in the LiDAR layer control. A COPC file (`.copc.laz`) or an EPT dataset (`ept.json`) streams the points in view on demand; a plain LAS or LAZ file is downloaded whole, and is refused if its server reports it larger than 250 MB (convert a larger file to COPC to stream it). A point cloud does not take a `style`:

```
https://web.geolibre.app/?data=https://s3.amazonaws.com/hobu-lidar/autzen-classified.copc.laz
```

GeoLibre recognizes a point cloud from its URL path. An API endpoint whose path has no `.las` / `.laz` suffix, such as `https://api.example.com/download/42?token=...`, needs `dataType=lidar` to say what it returns. Like `style`, `dataType` pairs with `data` by position, so repeat it (leaving earlier values empty) in a batch. A hinted endpoint is downloaded whole, since COPC streaming is chosen only for a `.copc` URL. An access token passed in the query string is kept on every request, including the byte-range reads of a streamed COPC file; encode the whole `data` value when the endpoint has more than one query parameter:

```
https://web.geolibre.app/?
data=https%3A%2F%2Fapi.example.com%2Fdownload%2F42%3Ftoken%3Dabc%26expires%3D3600&dataType=lidar
```

A plain `https://...` URL can be passed as-is, as above: `:` and `/` are legal in a query value and need no escaping. Encode the nested data and style URLs with `encodeURIComponent` only when they contain a character that would be read as GeoLibre's own query syntax — `&`, `+`, `%`, or `#`. A bare `=` inside the value is fine, since only the first `=` in each `&`-delimited pair separates the name from the value. Remote servers must permit browser cross-origin requests (CORS). COG, GeoParquet, PMTiles, and COPC servers should also support HTTP byte-range requests.

For a COG, `style` may point to a raster style JSON object. Supported fields are `mode` (`single`, `rgb`, or `index`), 1-based `bands`, `rescale` ranges, `colormap`, `reversed`, `nodata`, `opacity`, `gamma`, `stretch` (`linear`, `log`, or `sqrt`), and the normalized-difference `index` preset. For example:

```
{
  "mode": "single",
  "bands": [1],
  "rescale": [[0, 1000]],
  "colormap": "viridis",
  "reversed": false,
  "nodata": "auto",
  "opacity": 0.85,
  "gamma": 1,
  "stretch": "linear"
}
```

After hosting that JSON as `dem.style.json`, pass both URLs:

```
https://web.geolibre.app/?data=https://data.source.coop/giswqs/opengeos/dem.tif&style=https://
assets.geolibre.app/data/dem.style.json
```

For a ZIP containing files of the same geometry type, assign different styles by setting each Mapbox style layer's `source` to the corresponding filename stem. For example, `source: "parks"` targets `parks.geojson`, while `source: "counties"` targets `counties.geojson`. Style layers without a `source` are shared by every imported file. GeoLibre validates all filename/style matches before adding any ZIP layers.

You do not need to author that JSON by hand. Open the vector layer's **Layer actions** → **Styles** → **Export GeoLibre URL style** menu. The downloaded `.geolibre.style.json` contains only symbology—not feature data—and its render-layer `source` is already set to the original GeoJSON filename stem. Host the file on a CORS-enabled server and pass its URL as `style` alongside the corresponding `data` URL. For a multi-file ZIP, export each layer's GeoLibre URL style and combine their `layers` and `sources` into one style document; layers without `source` can be used for rules shared by every ZIP member.

The same file can be applied interactively to an existing vector layer through **Layer actions** → **Styles** → **Import style from file (GeoLibre URL / Mapbox GL / SLD / QML)**.... Interactive import ignores the file's query-param `source` binding and applies its supported symbology to the layer you selected, so the data filename does not need to match.

Open a STAC catalog

Use `stac` to open the **STAC Catalogs** browser already connected to a catalog. It accepts a static catalog, the root of a STAC API, or one collection of a STAC API:

```
https://web.geolibre.app/?stac=https://earth-search.aws.element84.com/v1/collections/naip
```

A collection URL (`.../collections/{id}`) connects to the API it belongs to, found through the collection's `root` link, selects that collection, and runs a search on it. The map fits the collection's extent and the matching item footprints are added as a **STAC search footprints** layer. From the result list you can then add an item's assets (COG, GeoJSON, GeoParquet, PMTiles, or Zarr) as layers. A catalog or API root only connects, leaving the search to you.

Percent-encode the value with `encodeURIComponent` when the STAC URL carries its own query string. The server must allow cross-origin browser requests (CORS). The browser is a side panel, so `stac` has no effect in `layout=viewer` (which cannot add layers), and its panel is hidden by `maponly` or `panels=none`.

An "Open in GeoLibre" badge

If you publish a dataset — in a repository README, a data catalog, a paper's supplementary material — an **Open in GeoLibre** badge turns it into a one-click interactive map. Copy one of these and swap in your own URL:

Open in 

```
[[Open in GeoLibre](https://img.shields.io/badge/Open%20in-GeoLibre-green.svg)](https://web.geolibre.app/?data=https://assets.geolibre.app/data/places.geojson)
```

The same badge can open a shared project instead of a single file, using `url`:

```
[[Open in GeoLibre](https://img.shields.io/badge/Open%20in-GeoLibre-green.svg)](https://web.geolibre.app/?url=https://share.geolibre.app/you/project.geolibre.json)
```

In reStructuredText:

```
.. image:: https://img.shields.io/badge/Open%20in-GeoLibre-green.svg
   :target: https://web.geolibre.app/?data=https://assets.geolibre.app/data/places.geojson
   :alt: Open in GeoLibre
```

In HTML:

```
<a href="https://web.geolibre.app/?data=https://assets.geolibre.app/data/places.geojson">
  
</a>
```

Add any of the [URL parameters](#) to the link to control how the map opens — `&style=` for symbology, `&theme=dark`, or `&maponly` for a chrome-free view.

One caveat specific to badges: a badge lives in a `README.md` that GitHub, PyPI, and docs sites all render, and each of those rewrites relative links differently, so keep the target URL **absolute**. The encoding rule above applies unchanged — a data URL carrying its own query string still needs `encodeURIComponent`, because the `&` is read as GeoLibre's own separator well before the browser ever sees it.

Talking to the map at runtime

URL parameters configure the app once, at load. To keep talking to a **live** embed (fly to the record the user just clicked in your app, highlight it, open a processing tool) and to hear what the user does inside the map, use the embed `postMessage` API. The dependency-free `@geolibre/embed` package provides the recommended typed client and handles origin checks and request correlation:

```
import { connect } from "@geolibre/embed";

const map = await connect(document.querySelector("iframe"), {
  origin: "https://web.geolibre.app",
});
await map.setView({ center: [-95.7, 37.1], zoom: 5 });
await map.setLayerVisibility("roads", false);
const added = await map.addData("https://assets.geolibre.app/data/places.geojson");
const layers = await map.listLayers();
map.on("selectionChanged", ({ featureIds }) => console.log(featureIds));
```

Enabling it

The API is **off by default**. The primary configuration is `sharing.embedOrigins` in `deployment.json`. For example:

```
{
  "version": 1,
  "sharing": {
    "embedOrigins": ["https://portal.example.com", "https://erp.example.com"]
  }
}
```

For Docker, `GEOLIBRE_EMBED_ORIGINS` overrides the policy at container startup:

```
docker run --rm -p 8080:80 \
  -e GEOLIBRE_EMBED_ORIGINS="https://portal.example.com,https://erp.example.com" \
  ghcr.io/opengeos/geolibre:latest
```

For a static build, the legacy `VITE_GEOLIBRE_EMBED_ORIGINS` build setting is still honoured as a fallback. Policy origins are exact `scheme://host[:port]` values; the JSON schema does not accept trailing paths. The environment inputs retain their existing trailing-path normalization.

`sharing.embedOrigins` entries are exact origins (`scheme://host[:port]`) with no trailing path; `*` allows any origin and is only appropriate on a private network. The legacy environment inputs retain trailing-path normalization. The allowlist is enforced in both directions: a message from an unlisted origin is ignored, and every message the app sends is addressed to a listed origin. (With `*` configured, outbound messages are addressed to `*` until the host's first message identifies it, which is one more reason to name your origins.)

Setting the allowlist also narrows the `?embed=1` project/scripting bridges (used by the [Python package](#)) to the same origins. As extra hardening you can stop other sites from framing the app at all by adding `Content-Security-Policy: frame-ancestors <your origins>` at your reverse proxy.

The allowlist decides *who* may send commands. To narrow *which* commands exist at all — so a trusted host page still cannot turn the embed into a general-purpose data-fetching proxy — configure [deployment capabilities](#) in the runtime policy. A denied command rejects with `Missing <capability> capability` rather than silently doing nothing: `loadProject` needs `project:edit`, `addLayer` and `addData` need `data:add`, `openTool` needs `processing:run`, and `exportImage` needs `export:data`. The rest — `setView`, `highlight`, layer visibility — are unprivileged and always available.

The typed client

`@geolibre/embed` is a dependency-free ESM package published to npm from each GeoLibre release, so its version tracks the app version:

```
npm install @geolibre/embed
```

`connect(iframe, options)` returns a promise that resolves once the app sends `ready`, so there is no `ready` handshake to write yourself. It also stamps a `requestId` on every command and settles that command's promise from the matching `ack`, which is what lets several commands be in flight at once without their answers getting crossed.

```
import { connect } from "@geolibre/embed";

const map = await connect(iframe, {
  origin: "https://web.geolibre.app", // exact origin hosting the iframe
  timeoutMs: 15_000, // wait for `ready`; default 15s
  requestTimeoutMs: 15_000, // wait for each `ack`; default 15s
});
```

`origin` is required and must be an `http(s)` origin: it is both the target of every outbound message and the filter on inbound ones, so a message from any other frame or origin is ignored. Pass the *app's* origin, not your own.

Method	Resolves with	Notes
<code>loadProject(url)</code>	<code>void</code>	Swaps the project without reloading the iframe.
<code>setView(target)</code>	<code>void</code>	{ <code>bbox</code> }, or any of { <code>center</code> , <code>zoom</code> , <code>bearing</code> , <code>pitch</code> , <code>duration</code> }.

Method	Resolves with	Notes
<code>highlightFeature({ layerId, ... })</code>	<code>void</code>	<code>featureId</code> , <code>featureIds</code> , or <code>filter</code> ; <code>fit: true</code> zooms to the match.
<code>openTool(id, params?)</code>	<code>void</code>	Runtime twin of <code>?tool=</code> .
<code>setLayerVisibility(layerId, visible)</code>	<code>void</code>	Shows or hides a project layer.
<code>listLayers()</code>	<code>LayerSummary[]</code>	<code>{ id, name, type, visible, opacity }</code> per layer.
<code>setFilter(layerId, expression)</code>	<code>void</code>	A MapLibre filter expression, or <code>null</code> to clear it.
<code>setRenderer(renderer)</code>	<code>void</code>	See Switching renderers .
<code>getRenderer()</code>	the active renderer	<code>"maplibre"</code> , <code>"mapbox"</code> , <code>"cesium"</code> , or <code>"arcgis"</code> .
<code>getViewport()</code>	<code>Viewport</code>	<code>{ bbox, center, zoom, bearing, pitch }</code> .
<code>addLayer(spec)</code>	the new layer's <code>id</code>	Takes a project-format layer specification.
<code>addData(url, options?)</code>	the new layer <code>id</code> s	Loads remote data like <code>?data=</code> ; options are <code>{ styleUrl, fit }</code> .
<code>exportImage()</code>	a PNG <code>data: URL</code>	The map as currently rendered.
<code>on(event, listener)</code>	an unsubscribe function	Not a promise; events are the set the app posts (see below).
<code>disconnect()</code>	not a promise	Removes the listener and rejects anything still in flight.

Every command rejects rather than resolving falsely: an `ack` carrying `ok: false` rejects with the app's own error message, and a command with no answer inside `requestTimeoutMs` rejects with a timeout. `connect` itself rejects if `ready` does not arrive inside `timeoutMs` — most often because the deployment has not allowlisted your origin, or because `origin` does not match the iframe.

Call `disconnect()` when you tear the iframe down (a React effect cleanup, a route change). It stops the `message` listener and rejects every pending command, so a promise cannot hang for the life of the page.

`on` returns its own unsubscribe function and accepts every event in the table [below](#), typed by name. Subscribing to `ack` is only worth it to observe the traffic, since each command's promise is already settled from it.

Hosts that cannot take a dependency can speak the protocol directly; the rest of this page documents it.

Message shape

Every message, in both directions, is versioned:

```
{ "v": 2, "type": "setView", "payload": { "center": [-95.7, 37.1], "zoom": 5 } }
```

Messages the **app** sends also carry `"source": "geolibre"`, so you can filter them out of the other `postMessage` traffic on your page.

Host to GeoLibre

Type	Payload	Effect
<code>loadProject</code>	<code>{ url }</code>	Loads a <code>.geolibre.json</code> project without reloading the iframe.
<code>setView</code>	<code>{ bbox }</code> or <code>{ center, zoom, bearing, pitch, duration }</code>	Fits a bounding box, or flies the camera to the properties you send.
<code>highlightFeature</code>	<code>{ layerId, featureId \ featureIds \ filter, fit }</code>	Selects and highlights features; <code>filter</code> matches properties. <code>fit</code> zooms to them.
<code>openTool</code>	<code>{ id, params }</code>	Opens the Processing dialog on a tool, pre-filling <code>params</code> . Runtime twin of <code>?tool=</code> .
<code>setLayerVisibility</code>	<code>{ layerId, visible }</code>	Shows or hides a project layer.
<code>listLayers</code>	<code>{}</code>	Returns layer summaries in the acknowledgement's <code>result</code> .
<code>setFilter</code>	<code>{ layerId, expression }</code>	Applies a MapLibre filter expression; send <code>null</code> to clear it.
<code>getViewport</code>	<code>{}</code>	Returns the current camera and bounds in <code>result</code> .
<code>setRenderer</code>	<code>{ renderer }</code>	Switches the map renderer. See Switching renderers .
<code>getRenderer</code>	<code>{}</code>	Returns the active renderer in <code>result</code> .
<code>addLayer</code>	<code>{ spec }</code>	Adds a project-format layer specification at runtime.
<code>addData</code>	<code>{ url, styleUrl?, fit? }</code>	Loads GeoJSON/API, ZIP, GeoParquet, PMTiles, or COG data without reloading the iframe.
<code>exportImage</code>	<code>{}</code>	Returns the rendered map as a PNG data URL in <code>result</code> .

Send `{ layerId }` alone to `highlightFeature` to clear the highlight. A request that names features (or a filter) but matches none is rejected rather than treated as a clear, so a mistyped id does not silently wipe the user's selection. Highlighting reads the layer's features from the project, so it applies to vector layers GeoLibre holds as GeoJSON, not to ones whose features live only in a tile source.

`setFilter` compiles the expression through the MapLibre style spec before storing it, and `addLayer` requires a source the map can actually read: a `url` or a non-empty `tiles`, or — for the two layer types drawn from inline features, `geojson` and `deckgl-viz` — an inline `geojson`. Both report the problem in the `ack` rather than reporting success and rendering nothing. `addLayer` also refuses `javascript:`, `vbscript:`, `data:`, `file:`, and `blob:` URLs on a source; a custom map protocol such as `pmtiles://` is fine.

`addData` uses the same format detection, CORS requirements, safety limits, and optional Mapbox/MapLibre style import as the `data` URL parameter. It fits the newly loaded data by default; pass `fit: false` to preserve the current camera. Its acknowledgement returns every created layer id, including all layers extracted from a ZIP archive.

Add a `requestId` to any message and the app answers with an `ack` (below) reporting whether it worked.

GeoLibre to host

Type	Payload	Fires when
<code>ready</code>	<code>{ version }</code>	The app has mounted and is listening.
<code>ack</code>	<code>{ requestId, ok, error, result }</code>	A message you sent with a <code>requestId</code> was applied (or rejected).
<code>projectLoaded</code>	<code>{ url, name, layerIds }</code>	A project finished loading, whoever started it.
<code>selectionChanged</code>	<code>{ layerId, featureIds }</code>	The user (or your <code>highlightFeature</code>) changed the selection.
<code>rendererchange</code>	<code>{ renderer }</code>	The map renderer changed.
<code>viewChanged</code>	<code>{ bbox, center, zoom, bearing, pitch }</code>	The camera moved (throttled to about four events a second).
<code>toolCompleted</code>	<code>{ id, name, status, engine, durationMs, outputLayerNames }</code>	A processing run finished, successfully or not.
<code>serverFileWritten</code>	<code>{ path, toolId }</code>	A file-based tool wrote an output (conversion and raster tools).

A host page

```
<iframe
  id="map"
  src="https://gis.example.com/?url=https://erp.example.com/fields.geolibre.json&maponly"
  title="GeoLibre map"
  width="100%"
  height="600"
  style="border: 0"
></iframe>

<script>
  const frame = document.getElementById("map");
  const APP_ORIGIN = "https://gis.example.com";

  const send = (type, payload) =>
    frame.contentWindow.postMessage({ v: 2, type, payload }, APP_ORIGIN);

  window.addEventListener("message", (event) => {
    if (event.origin !== APP_ORIGIN) return;
    const message = event.data;
    if (message?.source !== "geolibre" || message.v !== 2) return;

    if (message.type === "ready") {
```

```
// Safe to start sending commands.
} else if (message.type === "selectionChanged") {
  showRecordFor(message.payload.featureIds[0]);
}
});

// Click a record in your own UI: fly to it and highlight it, no reload.
function focusField(field) {
  send("setView", { bbox: field.bbox });
  send("highlightFeature", {
    layerId: "fields",
    filter: { parcel_id: field.id },
    fit: true,
  });
}
}
</script>
```

Wait for `ready` before sending: messages that arrive before the app has mounted are not queued. Treat `ready` as idempotent, since it is re-sent whenever the app remounts (a navigation inside the frame, a development hot reload).

Protocol v2 is current. GeoLibre continues to accept v1 request envelopes and answers a v1 host with v1 events, so existing hand-written integrations remain compatible.

What works in an embed

The browser build supports map navigation, browser-selected and URL-based data, styling, the SQL Workspace, and most plugins. Desktop-only features (local file dialogs, local MBTiles and raster reads, project save/open, and the Python sidecar tools) are not available in an embed. See [Getting Started](#).

See the [Sharing & Embedding tutorial](#) for a full walkthrough.

Switching renderers

With the [typed client](#):

```
map.on("rendererchange", ({ renderer }) => console.log(renderer));
await map.setRenderer("cesium");
const renderer = await map.getRenderer();
```

Both methods accept or return `"maplibre"`, `"mapbox"`, `"cesium"`, or `"arcgis"`. The change event reports the selected renderer. `setRenderer` acknowledges the selection; the new canvas mounts asynchronously. Wait for the next `ready` event before issuing camera or capture commands.

`exportImage()` supports all four renderers and waits for the visible layers to settle before returning a PNG data URL.

<https://geolibre.app/user-guide/troubleshooting/>

Troubleshooting

This page collects fixes for common problems in GeoLibre, and shows how to find out what went wrong and report it. Most display glitches in the browser after an update come down to a stale browser cache, so start there.

Stale browser cache

GeoLibre Web is a single-page app that browsers cache aggressively. After a new version ships, an outdated cached copy can leave you with display errors, missing or non-responsive controls, plugins that appear not to load, or stale data. The fix is to clear the cache and reload so the browser fetches the latest build.

Try these in order:

- **Hard refresh** the page to bypass the cache once:
 - **Windows / Linux:** `Ctrl + Shift + R` (Chrome, Edge, Firefox).
 - **macOS Chrome / Edge / Firefox:** `Cmd + Shift + R`.
 - **macOS Safari:** `Cmd + Option + R`.
- **Open the app in a private / incognito window.** If it works there, the problem lies in your browser profile, most often the cache; clearing the site's data (next step) confirms it.
- **Clear the cached files** for the GeoLibre site if a hard refresh is not enough, then reload.

Safari users

Safari caches web GIS apps aggressively, so it is the most common source of these issues. To empty Safari's cache, choose **Develop → Empty Caches** (`Cmd + Option + E`), or **Safari → Clear History**, then reload the page. If the **Develop** menu is hidden, enable it in **Safari → Settings → Advanced → Show features for web developers**.

A fresh page load against the latest build resolves the large majority of "the app looks broken after updating" reports.

The Diagnostics dialog

GeoLibre records what goes wrong while it runs. Open the record with **Help → Diagnostics**, the **Diagnostics: N** button at the right end of the status bar, or "Diagnostics" in the command palette (`Ctrl / Cmd + K`). The status bar button counts errors and warnings, and turns red when there are errors and amber when there are only warnings.

Each entry has a level (error, warning, or info), a time, and one of five categories:

Cate gory	What lands there
app	Every error notification the app shows you (see Notifications).
conso le	Errors and warnings written to the browser console, by GeoLibre or by a library or plugin.
map	Errors raised by the map engine: a tile, source, style, or 3D layer that failed.
netwo rk	Failed requests: an HTTP error status such as <code>GET 404 Not Found</code> , or a request that never completed (<code>request failed (network/TLS/CORS)</code> or <code>timed out</code>), with a hint about the likely cause.
runti me	Uncaught exceptions and promise rejections, a part of the interface that crashed, and plugin failures.

The filter buttons along the top show all entries, errors, warnings, or network entries. By default only failed requests are logged; tick **Log all network requests** to record successful and aborted requests too from that point on (it slows the app down, so leave it off otherwise).

- **Copy JSON** copies the entries currently shown, to paste into an issue or a message.
- **Clear** empties the log.

The log keeps the newest 500 entries, lives in memory only (it starts empty on every launch), and strips access tokens, API keys, and similar query parameters from the URLs it stores.

Reporting a problem

Error notifications, and error rows in the Diagnostics dialog, carry a **Report issue** button. It opens GeoLibre's bug report form on GitHub in your browser, filled in with:

- the error message as the title,
- the app version and whether it is the desktop app or the web app,
- your platform (the browser's user agent) and the active rendering engine,
- the diagnostics entry, as JSON.

Nothing is sent until you review the form and submit it yourself. Before it is filled in, the report is scrubbed: URL query strings and fragments, user names and passwords in URLs, embedded `data:` URIs, credential-like `key=value` pairs, recognizable tokens (bearer tokens, Mapbox, GitHub, AWS, and Google keys, JWTs), and your home-directory user name are all replaced. Still read it over before submitting, and add what you were doing when the error appeared.

For a problem with no error behind it, use **Help → Give Feedback**, which opens the GitHub issue tracker.

Common errors

"Could not reach the service"

A remote layer or service that fails with "Could not reach the service...", or a `network` diagnostic ending in `request failed (network/TLS/CORS)`, usually means one of:

- **CORS.** In the browser, a server must allow GeoLibre's origin with an `Access-Control-Allow-Origin` header, or the browser blocks the response. The desktop app is not subject to CORS, so loading the same URL there is the quickest test. For an S3 bucket, GeoLibre says so directly when the bucket's CORS configuration does not list the app's origin; see [Cloud Storage](#).
- **Mixed content.** An `https://` page cannot load `http://` data. Use an HTTPS URL, or the desktop app.
- **TLS or reachability.** An expired or self-signed certificate, a firewall or proxy, or a host that is down.

A layer that fails to load is reported once per session with a notification naming it. Tile requests that return `204` or `404` are not reported, because sparse tile sets return those for empty areas.

A GeoTIFF that will not display

A plain (striped) GeoTIFF cannot be streamed tile by tile. GeoLibre recognizes this and offers to convert it to a Cloud-Optimized GeoTIFF and load that instead. Conversion runs in the browser, so very large files are refused as too large to convert safely; convert those yourself with `gdal_translate -of COG` or `rio cogeo create`, then load the result. See [Supported Data Formats](#).

"This project is too large to save"

A project that embeds a lot of vector data can outgrow what can be saved. Convert the embedded data to PMTiles or FlatGeobuf and add those as remote layers rather than embedding the features. A related notice, **Autosave paused** in the status bar, means the project has grown past the autosave limit, so no new history snapshots are being kept; save it to disk to keep your work. See [Project history and crash recovery](#).

Sidecar messages

Raster tools, format conversions, and the GeoPandas engine for vector tools use the optional Python sidecar. Messages such as "Raster tools need the GeoLibre desktop app with a running sidecar", "Could not connect to sidecar", or "The GeoPandas sidecar is not available" mean it is not running or lacks the extra a tool needs. Vector tools fall back to the in-browser engine. See [The Python sidecar](#) for how to start it and which extras each tool needs.

Missing tiles in an offline map

Project → Offline Basemap... can only cache what the basemap's servers allow:

- "Offline caching needs the installed web app" means the browser build is not running as an installed app, so tiles are not stored for offline use.
- Some basemap sources do not allow offline downloading; GeoLibre lists them before the download, and their tiles will be missing.
- A large area at many zoom levels can exceed the offline cache limit, and the oldest tiles are then evicted as new ones download. Pick a smaller area or fewer detail levels.

Manage Offline Areas shows how much storage the cached areas use.

"Something went wrong"

If part of the interface crashes, GeoLibre shows "{name} failed to render. The rest of GeoLibre is still available." and keeps running. If the whole app crashes, a full-window "Something went wrong" panel offers **Try again** and **Reload app**. Try again first: it keeps your work. Your project is also kept in [project history](#), so it can be recovered after a reload. The crash is recorded in Diagnostics under `runtime`, ready to report.

The map is blank or renders incorrectly

The map is drawn with WebGL on your graphics hardware. If it stays blank, goes black, flickers, or 3D layers are missing:

- **Check that hardware acceleration is on** in the browser (Chrome and Edge: **Settings → System → Use graphics acceleration when available**; then check `chrome://gpu` for "WebGL: Hardware accelerated"). Without it, WebGL falls back to slow software rendering or is unavailable.
- **Update the graphics driver**, and on a laptop with two GPUs try running the browser on the other one.
- **Try another rendering engine** from **View → Rendering engine**. See [Rendering Engines](#).
- **Look in Diagnostics** for `map` and `console` entries that name the failing layer or shader.

When a map canvas loses its WebGL context (the graphics driver resets, or the GPU runs out of memory), GeoLibre shows a **The map lost its graphics context** warning with a **Reload** button. If the map does not recover on its own, reload the page. The warning goes away on its own if the context comes back. Save your project before reloading if you can. A recent autosave may also be in [project history](#), but autosave runs on a delay and skips projects above the snapshot size limit, so it may not have your latest changes.

Linux desktop app

The Linux desktop app runs in WebKitGTK, which has its own GPU quirks. The app sets the WebKitGTK rendering options known to work for your WebKitGTK version and GPU at startup (including the DMA-BUF renderer settings on NVIDIA), and works around a WebAssembly crash on CPUs without AVX. A value you set yourself in the environment, such as

`WEBKIT_DISABLE_DMABUF_RENDERER=1`, always wins, so that is the first thing to try if the window is blank or the map flickers. See [Linux desktop troubleshooting](#).

Plugins do not appear to activate

If a plugin's **Activate** action seems to do nothing, first rule out the stale-cache cause above with a hard refresh or a private window. Once you are on the latest build, an active plugin shows a checkmark next to its entry in the **Plugins** menu, and its panel opens in the side panel, or, for the few plugins with an on-map control (such as GeoEditor or Annotations), the control appears at the configured corner of the map. A greyed-out plugin does not support the current rendering engine; hover over it for the reason. A plugin that fails to start is reported as a notification and in Diagnostics.

The Processing menu shows the same category twice

`Processing → Vector` and `Processing → GeoLibre Toolbox → Vector` are different things, and so are the two `Conversion`, `Network`, and `Raster` entries. The Processing menu carries two independent toolboxes: the Whitebox Toolbox, whose nine category submenus sit at the top of the menu, and the GeoLibre Toolbox, which holds GeoLibre's own built-in tools. Neither list is a subset of the other. [Two toolboxes in one menu](#) explains which is which and when to use each.

Desktop-only features in the browser

Some capabilities require the desktop app and are unavailable in the browser build: local filesystem dialogs, local MBTiles, local raster file reads, and project save/open. If one of these is missing, you are likely running GeoLibre Web rather than the desktop app. See [Getting Started](#) for how the web, desktop, and Jupyter builds differ.

Self-hosted deployments

When you host GeoLibre yourself, most "it works on web.geolibre.app but not here" problems come from the deployment's security settings rather than the app:

- **Content Security Policy.** The Docker image's nginx configuration allows `https:` data and tile hosts, but blocks plain `http://` remote data and WebSocket (`wss://`) hosts other than the configured collaboration server. A request blocked by CSP appears in the browser console and in Diagnostics under `console`. The desktop app allows any `https:` or `http:` data host, but scripts and WebSockets only from a fixed list.
- **The sidecar proxy.** The browser build reaches the sidecar at `/sidecar` on the same origin, so it needs no CORS, and it can only read files under `GEOLIBRE_CONVERSION_ROOTS` (`/data` by default). A `403` from `/sidecar` means the sidecar is disabled (`GEOLIBRE_DISABLE_SIDE CAR`) or the deployment policy does not grant the capability.
- **Deployment policy.** A `deployment.json` can switch features, services, and plugins off, so a missing menu item may be intentional. See [Deployment Policy](#).

See [Self-Hosting](#) for why same-origin hosting matters, and its [deployment checklist](#).